

Blockchain Scalability Book

A mechanism-first guide to blockchain scalability, security, trust,
and recovery

Neil Han

Version 1.1.1 · Print Edition Candidate

Blockchain Scalability Book

Version 1.1.1, print edition candidate

© 2026 Neil Han. Book prose, exercises, tables, and original figures are licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0, <https://creativecommons.org/licenses/by-sa/4.0/>). Build software under scripts/, theme/, and .git-hub/ is licensed under the MIT License. Third-party quotations, trademarks, and credited materials retain their respective rights. Figure sources and credits appear in the Figure Credits chapter.

Free digital editions (web, PDF, EPUB): <https://github.com/neilydhan/Blockchain-Scalability-Book/releases>

Independently published. Print ISBN: to be assigned through Amazon KDP.

10 9 8 7 6 5 4 3 2 1

This book is educational material, not financial, legal, or investment advice. Protocol details change; every time-sensitive claim carries a source and a date, and readers should verify the current specification.

Cite this edition: Han, Neil. *Blockchain Scalability Book*. Version 1.1.1, 2026.

Contents

Preface	13
Purpose of This Book	13
Who This Book Is For	13
Origin	13
How to Read the Worked Examples	14
Reader Pathways	15
How to Use This Book	16
What This Book Measures	17
Conventions	17
Contributions and Corrections	17
Acknowledgments	18

CORE CHAPTERS

Chapter 1: Understanding Blockchain Scalability Challenges	19
Introduction	19
A Minimal Blockchain Model	19
Why Naive Comparisons Fail	22
Gas and the Price of Shared Computation	22
Case Study: Congestion and Application Design	23
Named Case Study: From CryptoKitties Congestion to a Production Rollup Workload	23
From Database Benchmarks to Blockchain Benchmarks	26
A Scalability Model for This Book	27
A Transaction as a Resource Vector	28
Worked Example: Benchmarking Two Chains	28
The Four-Layer Evaluation Framework	29
Security and Decentralization Measurements	29
How to Read Performance Claims	30
From Transaction Submission to Finality	30
Queueing and the Capacity Knee	30
State, History, and Working Sets	31
A Reproducible Benchmark Procedure	31
Scalability Claims as Falsifiable Statements	32
Worked Capacity Envelope	32
Chapter Summary	34
References	35

Chapter 2: The Blockchain Trilemma	37
Introduction	37
A Beginner's Way to Use the Trilemma	37
What Is the Blockchain Trilemma?	38
Breaking Down the Trilemma	39
Architecture Examples	40
Named Case Study: Bitcoin, Ethereum, and Solana on One Fixed Workload	42
The Trilemma Is a Constraint, Not a Theorem	44
Quantifying the Three Axes	45
Worked Example: Increasing the Block Size	45
Worked Example: A Rollup's Trade	46
Security Budgets and Participation Costs	46
Decentralization Across Roles	46
Asymmetric Verification	47
Failure Domains, Not Key Counts	47
Worked Comparison: Scaling an Exchange	47
Governance as a Fourth Axis	48
Worked Failure-Domain Audit	48
Conclusion	50
References	50
Chapter 3: Layer 1 vs Layer 2 - Comparing Different Approaches to Blockchain Scaling	53
Introduction	53
First Intuition: Move the Work or Change the Base	53
What Is Layer 1 Scaling?	54
What Is Layer 2 Scaling?	55
Comparing Layer 1 vs Layer 2	56
Real-World Trade-Offs	57
Why Both Layers Matter	57
A Transaction's Path Through Layer 1 and Layer 2	58
The Scaling Bottleneck Moves	58
A Better Comparison Framework	59
Worked Design Choice: An On-Chain Game	60
Forced Transactions and Censorship Recovery	60
Deposits, Withdrawals, and Finality	60
Fee Anatomy Across Layers	61
L3s and Recursive Layering	61
Migration and Upgrade Strategy	61
Architecture Decision Record	61
Worked Migration: L1 Application to Rollup	61
Conclusion	64

References	65
Chapter 4: Layer 1 On-Chain Scalability	67
Introduction	67
A Map of Layer 1 Work	67
The Four Jobs of a Base Layer	69
Vertical Scaling: Making One Chain Faster	69
Horizontal Scaling Through Sharding	70
Interoperability as a Form of Sharding	71
Named Case Study: NEAR Receipts and the IBC Packet Lifecycle	72
Ethereum's Rollup-Centric Data Sharding	75
Trade-Offs	76
Worked Example: A Cross-Shard Transfer	76
Committee Security by Calculation	77
Implementing a Sharded State Machine	77
Sharding Failure Modes	79
L1 Engineering Checklist	79
State Sharding Design Choices	79
Stateless Validation and Witnesses	80
Cross-Shard Contract Pattern: Request and Callback	80
Sharding and Composability Economics	81
Stateless Validation: Witness Construction and Update	81
State Expiry and Revival Operations	84
History Expiry, Archives, and Verifiable Queries	87
Mempool Admission and Denial-of-Service Control	90
Node Synchronization and Snapshot Recovery	93
Worked Resharding Trace: Moving a Hot Account Range	96
Conclusion	99
References	99
Chapter 5: Layer 2 Off-Chain Scalability	101
Introduction	101
Intuition: Signed Receipts Instead of Global Updates	101
The General Off-Chain Pattern	103
Payment Channels	104
Generalized State Channels	104
Sidechains and Commit-Chains	105
Named Case Study: Polygon Chain Has Independent Consensus	105
Plasma	107
Rollups for General-Purpose Layer 2 Execution	108
Comparing Off-Chain Approaches	108
Security Is a Spectrum	109
Worked Example: Closing a Channel Against an Old State	109

Routing Is a Liquidity Problem	109
Implementing a State Channel	110
Implementing HTLC Routing	111
Plasma Exit Games in More Detail	111
L2 Classification Checklist	111
Channel Factories and Virtual Channels	112
Channel Privacy and Metadata	112
Sidechain Bridge Verification Models	112
Mass-Exit Capacity	113
Payment-Channel Rebalancing	113
Worked Lightning Route: Amounts, Fees, and Timelocks	114
Channel Backup, Restore, and Disaster Recovery	115
Watchtower Data and Privacy	118
Conclusion	119
References	119
Chapter 6: Rollups	121
Introduction	121
Rollups From First Principles	121
The Rollup Lifecycle	123
Named Case Study:Arbitrum Nitro andBoLD	124
Named Case Study:ZKsync Era's Validity Pipeline	127
Production Rollup Comparison on One Payment-and-Withdrawal Workload	130
Optimistic Rollups	132
Validity Rollups	133
Where the Data Lives	134
Compression and the Cost Model	134
Bridges and Forced Exits	134
Optimistic vs Validity Rollups	135
From Rollups to Appchains	135
Worked Example: From Transaction to Withdrawal	136
Sequencer Failure and the Escape Hatch	136
Rollup State Commitments and Inboxes	136
Interactive Fault Proofs	137
Validity-Proof Pipeline	137
Data Encoding and Fee Estimation	138
Rollup Operations Checklist	139
Deriving a Canonical Withdrawal	139
Fee Estimation Under Volatility and Reorganizations	139
Fault-Proof Game Economics	143
Prover Performance Engineering	143
Prover Market Scheduling and Failure Recovery	143

Sequencer Architecture	147
Decentralized Sequencing Trade-Offs	148
Sequencer Decentralization and Failover Protocol	148
Rollup Upgrade and Escape Testing	152
Fault-Proof Pipeline: From Assertion to One-Step Dispute	152
Circuit Versioning, Trusted Setup, and Verifier Migration	155
Proving Pipeline: From Execution Trace to L1 Verification	158
Cross-Rollup Withdrawal and Liquidity Operations	162
Rollup Fee Market and Batch-Packing Trace	165
End-to-End Implementation Example: A Rollup Payment	168
Conclusion	172
References	172
Chapter 7: Modular vs Monolithic Blockchains	173
Introduction	173
From One Machine to a Stack of Services	173
The Monolithic Model	174
The Modular Stack	175
Why Specialization Helps	176
The Hidden Costs of Modularity	176
Celestia and Data Availability Specialization	177
Named Architecture Trace: AnOP Stack Chain With Celestia DA	177
Sovereign and Settled Rollups	180
Monolithic vs Modular	180
A Framework for Evaluating an Architecture	181
End-to-End Transaction Walkthrough	181
Security Composition	182
Designing the Interfaces Between Modules	182
Sovereign Rollup Fork Choice	183
Modular Liveness Matrix	183
Implementation Checklist for a Modular Stack	184
Cross-Layer Message Envelopes	184
Relayers Are Replaceable, Not Trusted	185
Cross-Layer Reorganizations	185
Light-Client Bootstrap and Checkpoint Trust	185
Light-Client Bridge Verification Trace	188
Bridge Accounting, Limits, and Circuit Breakers	191
Operating a Modular Stack	195
Modular Cost Accounting	196
Worked Failure Trace: A Cross-Rollup Swap	196
Conclusion	199
References	199

Chapter 8: Data Availability Scaling	201
Introduction	201
Intuition: A Correct Answer Needs Recoverable Inputs	201
Validity Is Not Availability	203
The Withholding Attack	204
Erasure Coding	204
Data Availability Sampling	204
Celestia's Approach	205
Ethereum Blobs and PeerDAS	205
External Data Availability Layers	206
Named Case Study: One Rollup Batch on Celestia, EigenDA, and Avail	206
Availability, Retrievability, and Permanence	210
Trade-Offs	211
Sampling Probability by Example	211
A Data-Withholding Failure	211
Two-Dimensional Reed-Solomon Encoding	212
Namespaced Data and Selective Retrieval	212
Blob Commitments and Retention	213
Implementing a DA Client	213
DA Threat Model Checklist	213
Availability Certificates and Their Limits	214
Selective Disclosure and Eclipse Attacks	214
Reconstruction and Repair	214
Sampling Networks, Peer Diversity, and Adversarial Serving	215
DA Capacity Planning	218
DA Integration Test	218
Worked Integration Trace: Publishing a Rollup Batch	218
Conclusion	221
References	221
Chapter 9: Parallel Execution	223
Introduction	223
Intuition: Which Transactions Can Run Together?	223
Why the EVM Is Commonly Sequential	225
Conflict Graphs	226
Declared Access Lists	226
Optimistic Parallel Execution	226
Determinism and Consensus	227
The Hot-State Problem	227
Parallel Execution vs Sharding	228
Named Transaction Traces: Solana, Sui, and Aptos	228

Benchmarking Parallel Engines	232
Worked Example: Three Transactions, Two Cores	232
Designing Contracts for Concurrency	232
Multi-Version State and Validation	233
Static Scheduling With Access Lists	233
Deterministic State Commitment	234
Parallel Execution Failure Modes	234
Benchmark Matrix	234
Scheduling Algorithms	235
Parallel Execution Memory, I/O, and NUMA Effects	235
Fee Markets for Contention	238
Parallel Smart-Contract Patterns	238
Testing Determinism	239
End-to-End Speedup Limits	239
Worked Scheduler Trace: Speculate, Validate, Commit	239
Contention and Admission Control	241
Conclusion	242
References	242
Chapter 10: Consensus Scaling	243
Introduction	243
Consensus From First Principles	243
Safety, Liveness, and the Finality Boundary	245
Nakamoto and BFT-Style Consensus	246
HotStuff	246
Sync HotStuff	247
Reducing Communication	247
Block Propagation Is Part of Consensus	248
Consensus and Execution Separation	248
Named Case Studies: How Consensus Families Reached Production	249
Comparison	253
Worked Example: Why a Quorum Certificate Matters	254
The Liveness Path	254
Throughput Is Not Finality Latency	254
HotStuff State and Voting Rules	255
Pacemaker and View Synchronization	255
Threshold Signatures and Accountability	256
DAG Mempools	256
Consensus Implementation Checklist	256
Epoch and Validator-Set Changes	257
Slashing Safety and Validator Key Operations	257
Slashing and Equivocation Evidence	260

Fork Choice and Finality Gadgets	261
Consensus Safety Testing	261
Consensus Network Partitions and Recovery	261
Consensus Operations	264
Worked Protocol Trace: Four Replicas and One Fault	264
Quorum Arithmetic With Weighted Validators	266
Conclusion	267
References	267
Chapter 11: Future Directions	269
Introduction	269
How to Read This Future-Facing Chapter	269
Real-Time Validity Proving	271
Shared Sequencing and Cross-RollupComposability	271
Proof Aggregation	272
More Data Through Sampling	272
Statelessness and State Expiry	272
Parallel and Specialized Virtual Machines	273
Account Abstraction, Bundlers, and Paymasters	273
Chain Abstraction	276
Decentralizing the Full Stack	276
MEV and Proposer-Builder Separation	277
Proposer-Builder Separation in Practice	277
Named Implementation Atlas: What Runs, What Is Piloted, What Is Proposed	281
Mechanism-to-Implementation Map	286
How to Judge Future Claims	286
A Plausible End-State Architecture	287
Research Problems Still Open	287
Based and Shared Sequencing Protocols	287
Proof Markets and Aggregation Trees	288
Intents and Solver Safety	288
Intent Protocol and Solver Settlement Trace	289
Stateless Validation Pipeline	293
Research Evaluation Milestones	293
Decentralized Prover Networks	293
Preconfirmations	294
Encrypted Mempools and Threshold Decryption	294
Threshold-Encrypted Mempool Protocol Trace	294
Cross-Domain MEV	297
Hardware Acceleration and Decentralization	298
Post-Quantum Considerations	298
What "Finished" Means for a Scaling System	298

Research-to-Production Gate	298
Worked Decision: Should an Exchange Use Preconfirmations?	300
Technology Watch Template	301
Conclusion	301
References	302
 REFERENCE AND PRACTICE	
Glossary	303
Start Here: Core Blockchain Terms	303
Architecture and State	305
Layer 2 Systems	306
Data Availability and Proofs	307
Messaging, Operations, and Governance	309
Performance and Reliability	310
Execution and Consensus	312
Review Questions and Design Exercises	315
Basic-Knowledge Warm-Up	315
Chapters 1-2: Measuring Scalability and the Trilemma	316
Chapters 3-5: Layers, Sharding, and Channels	317
Chapters 6-8: Rollups, Modularity, and Data Availability	317
Chapters 9-10: Parallel Execution and Consensus	318
Chapter 11: Future Architecture	318
Capstone Design Exercise	319
Quantitative Laboratory: Capacity and Finality	319
Fault-Injection Laboratory	320
Design Review Rubric	320
Instructor Notes and Solution Sketches	320
Practitioner's Evaluation Handbook	325
How a Basic Reader Can Evaluate a System	325
1. Define the Workload Before the Architecture	327
2. Draw the Transaction and Trust Paths	327
3. Identify the Capacity Equation	328
4. Benchmark in Layers	329
5. Report Distributions, Not One Number	329
6. Evaluate State Growth	330
7. Evaluate Data Availability Separately	330
8. Evaluate Sequencing and MEV	330
9. Evaluate Proof Systems	330
10. Evaluate Consensus Under Its Real Model	331
11. Evaluate Bridges as Security-Critical Systems	331

12. Evaluate Upgrades and Governance	332
13. Make Recovery Affordable	332
14. Produce a Comparable Result	332
Worked Example: Evaluating a Rollup Exchange	333
Evaluation Template	334
Incident Readiness and Postmortems	335
Worked Adoption Decision	336
Incident Command for Scaling Systems	338
Upgrade Operations and User Exit Windows	342
Conclusion	345
Figure Credits	347
Original Figures for This Book	347
Rights Checklist	347
Threat-Model Worksheets	349
Start With Assets and Outcomes	349
Role Worksheet	349
Trust Boundary Worksheet	350
Adversary Worksheet	351
Economic Worksheet	351
Availability and Deadline Worksheet	352
Upgrade Worksheet	352
Test-to-Claim Matrix	352
Review Output	353
Benchmark Reporting Template	355
1. Claim	355
2. System Under Test	355
3. Topology and Control	356
4. Hardware and Network	356
5. Initial State	356
6. Workload	357
7. Offered-Load Schedule	357
8. Metrics	357
9. Fault Schedule	358
10. Results Table	358
11. Resource Envelope	358
12. Finality and User Journey	359
13. Recovery Results	359
14. Reproduction Package	359
15. Limitations and Conclusion	360

Preface

Blockchain scalability is often presented as a list of projects or a race for the largest throughput number. This book takes a different approach. It follows the work a system must perform, the assumptions that make the result trustworthy, and the recovery path when the fast path fails.

Purpose of This Book

The goal is to connect research mechanisms with implementation and evaluation. Readers should finish able to trace a transaction from submission through execution, data publication, consensus, settlement, and finality; identify the first saturated resource; and explain what a user can do when an operator, prover, relayer, or validator set fails.

The book covers Layer 1 optimization and sharding, channels and Plasma, optimistic and validity rollups, modular architectures, data availability, parallel execution, and consensus scaling. It treats bridges, upgrades, observability, state growth, and mass recovery as part of scalability rather than operational details outside the design.

Who This Book Is For

The primary readers are:

- developers implementing blockchain protocols, rollups, bridges, wallets, and applications;
- architects comparing monolithic, modular, sharded, and layered systems;
- researchers and students connecting papers to deployed mechanisms;
- operators and reviewers testing performance, safety, liveness, and recovery claims.

A reader should be comfortable with hashes, signatures, transactions, smart contracts, and basic distributed-systems vocabulary. The glossary defines the specialized terms used throughout the book.

Origin

I developed this material from blockchain courses taught at Nanyang Technological University in Singapore, including CZ4153/CE4153 Blockchain Technology and SC6019 Blockchain Privacy & Scalability. The lectures covered data

sharding, rollups, zero-knowledge proofs, modular systems, parallel execution, and consensus.

The book grew from a teaching need: students could find papers, documentation, and product descriptions, but few resources connected the full scaling stack and its failure modes. The figures and examples retain that classroom objective while adding implementation checklists, calculations, test plans, and primary references.

How to Read the Worked Examples

The book uses worked examples to make protocol limits concrete. They are models with stated assumptions, not live fee quotes or performance promises.

Each calculation should be read in four steps:

1. **Name the quantity.** Throughput is work per second; latency is time per action; bandwidth is bytes or bits per second; probability is dimensionless.
2. **Write the assumptions.** Transaction mix, hardware, finality rule, compression, and failure conditions determine the result.
3. **Carry the units.** Dividing bytes by bytes per second produces seconds. Multiplying gas by price per gas produces a fee. Units expose many mistakes.
4. **Interpret the boundary.** The smallest resource ceiling is the current bottleneck; it is not a universal maximum after the workload changes.

Common notation

Letters are local labels, defined near each example:

- n commonly means a total count, such as validators or encoded shares;
- k commonly means a required subset or number of shards;
- f commonly means faulty participants or a fraction;
- λ (lambda) means an arrival rate;
- μ (mu) means a service or processing rate;
- p_{50} , p_{95} , and p_{99} are percentiles: 50, 95, or 99 percent of observations complete at or below that value;
- R_0 and R_1 label a prior and next state root;
- T_1 , T_2 , and so on label transactions in one trace.

A percentile describes a distribution, not an average. If p_{99} latency is ten seconds, 99 percent of measured actions finished within ten seconds and one percent took longer. The test duration and sample count still matter.

Decimal and binary units

- kB, MB, and GB use powers of 1,000 in this book unless a source says otherwise.
- KiB, MiB, and GiB use powers of 1,024.
- Network rates written Mbps are megabits per second; divide by eight to obtain ideal megabytes per second before overhead.
- ms means milliseconds; 1,000 ms equals one second.
- Ethereum fees may use gwei, where one gwei is one billionth of an ETH.

Probability language

A model probability is conditional on its assumptions. Independent samples must truly be hard for an attacker to predict or correlate. A one-in-a-million result per event may still occur often when a system runs billions of events. Always pair a probability with the event rate, exposure time, and consequence.

Pseudocode and data structures

Code blocks such as `Packet { ... }` are often pseudocode. They expose fields that must be bound or checked but are not a copy-paste implementation. A production encoding must additionally define byte order, field lengths, canonical forms, versioning, bounds, and error behavior.

When a section becomes difficult, return to five questions: who acts, what data they use, what evidence the next party checks, when the result becomes final, and how failure is recovered.

Reader Pathways

The book supports three routes.

Basic blockchain background

Read the Preface, then Chapters 1-3 in order. Chapter 1 supplies the mental model and Chapter 2 teaches how to reason about trade-offs. In later chapters, read the opening intuition section before the detailed protocol traces. Use the core glossary and basic-knowledge warm-up whenever a term is unfamiliar.

A first reading can skip implementation data structures and return to them after answering:

- What problem does the mechanism solve?
- Who performs the work?
- What evidence does another party check?
- When is the result final?

- What happens when the normal service fails?

Builder or operator

Read Chapters 1 and 3, then follow the mechanism you operate: Chapter 4 for L1/state, Chapters 5-6 for L2, Chapters 7-8 for modular/DA, Chapters 9-10 for execution/consensus, and Chapter 11 for emerging designs. Use the evaluation handbook, threat-model worksheets, and benchmark template alongside the chapter.

Course or research study

Read Chapters 1-11 in sequence, complete the warm-up and chapter questions, then run the quantitative and fault-injection laboratories. Treat references as starting points for source verification, not substitutes for reproducing an argument.

Signals in the text

- **Intuition sections** explain the mental model and vocabulary.
- **Worked examples** calculate a bounded scenario and state assumptions.
- **Protocol traces** show message and state order.
- **Failure matrices** separate safe behavior from recovery.
- **Production assertions** turn explanations into tests.
- **References** identify primary specifications and papers.

No route removes the need to state assumptions. The book becomes more formal gradually, but each deep section should remain connected to an actor, message, evidence check, deadline, and recovery path.

How to Use This Book

Chapters 1-3 establish the measurement model and architecture vocabulary. Chapters 4-10 examine the main mechanisms. Chapter 11 looks at developing directions without treating research proposals as deployed facts.

The glossary supports reference use. Review questions test reasoning rather than recall. The practitioner handbook turns the book's methods into an evaluation procedure. Readers designing a system should work through the capstone and failure-injection exercises, not only the descriptive chapters.

Each chapter can be consulted independently, but later chapters assume the distinction among execution, settlement, consensus, and data availability introduced in Chapter 1.

What This Book Measures

A performance claim is incomplete unless it names:

- the workload and state distribution;
- offered load, sustained throughput, and tail latency;
- the start and completion boundary;
- hardware, client version, network topology, and duration;
- validator, sequencer, prover, and data assumptions;
- behavior during faults and the cost of recovery.

The unit of analysis is the end-to-end user action. Faster execution does not help when data publication is saturated. A cheap normal transaction does not prove that mass exit is affordable. A validity proof establishes only the statement encoded by its program and public inputs.

Conventions

Layer 1 (L1) is the base chain whose consensus defines canonical history. **Layer 2 (L2)** performs work outside the base chain while using it for settlement or enforcement. **Data availability (DA)** means the information needed to verify or reconstruct state was published and obtainable. **Finality** is always relative to a stated protocol and fault model.

Example numbers are illustrative unless a source and measurement context are given. Protocol rules and roadmaps change. Current claims should be checked against the linked specification or primary paper before they drive a production decision.

Code-like examples emphasize invariants and interfaces rather than one programming language. Terms such as *must* describe requirements for the design under discussion, not standards-language obligations unless a specification is cited.

Contributions and Corrections

This is a living technical book. Corrections, reproducible measurements, protocol updates, implementation lessons, and original figures are welcome through the repository's contribution process.

A useful contribution states its source and scope. Performance updates should include workload and hardware. Security corrections should name the violated assumption or invariant. Figures should have editable source and clear rights. Time-sensitive product claims should be replaced with protocol mechanisms or pinned to a dated primary source.

Acknowledgments

This book builds on the work of protocol researchers, client engineers, auditors, operators, educators, and students. Classroom questions and implementation failures are especially valuable: they reveal where a paper-level description is not enough for someone who must build, operate, or rely on the system.

Chapter 1: Understanding Blockchain Scalability Challenges

Introduction

Blockchain scalability is the ability to support increasing useful demand while keeping verification, participation, and failure recovery within acceptable resource and latency bounds. The definition deliberately includes more than transactions per second (TPS). It asks what work completed, when it became final, which machines and operators were required, and what happens when a component fails.

Public blockchains combine replicated execution and storage with adversarial consensus. That redundancy lets users verify shared state without trusting one database operator, but it also makes capacity expensive. The techniques in this book divide, compress, schedule, or prove the work while trying to preserve that independent check.

A Minimal Blockchain Model

Before discussing scalability, it helps to picture the smallest useful blockchain.

A **transaction** is a signed instruction, such as "send one token to Maya" or "exchange these two assets." A digital signature lets anyone verify that the holder of a private key authorized the instruction without revealing the private key.

Transactions are grouped into **blocks**. Each block points to the preceding block by including its cryptographic hash, a short fingerprint that changes if the earlier data changes. These links create an ordered history. Changing an old block would change its fingerprint and every later link.

A blockchain also maintains **state**: the latest balances, contract storage, ownership records, and other values applications use now. A transaction changes the old state into new state under deterministic rules. "Deterministic" means honest computers starting from the same inputs calculate the same result.

Several independent computers, called **nodes**, exchange transactions and blocks. Some nodes participate in **consensus**, the protocol for choosing one canonical order when messages arrive at different times or a participant lies. A **validator** is a consensus participant that checks proposed blocks and votes, attests, or otherwise helps the network accept them. A **block producer**

proposes an ordered block. One machine can fill several roles, but the roles are conceptually different.

The word **canonical** means "the version the protocol currently accepts." This matters because two valid-looking blocks can briefly compete. A wallet may first show a transaction as included, then wait for **finality**, the point at which the protocol's assumptions make reversal sufficiently unlikely or forbidden. Finality is not instant by definition; later chapters distinguish probabilistic and voting-based forms.

Why replicate the work?

A normal online service can keep one authoritative database. Users trust its operator to preserve balances and apply rules. A public blockchain instead lets many parties hold and check the same history. Replication is intentionally inefficient: it prevents one database owner from silently rewriting the result.

Imagine a shared notebook copied to hundreds of desks. Everyone checks each new page before adding it. The copies make tampering visible, but every page must travel to many desks and be checked many times. Blockchain scalability asks how to handle more useful work without abandoning the checks that make the notebook trustworthy.

Accounts, contracts, and virtual machines

An **account** identifies an owner or program and may hold assets or data. A **smart contract** is a program stored and executed under blockchain rules. It does not understand legal intent; it applies code to inputs and current state.

A **virtual machine (VM)** defines the contract instruction set and execution rules. Ethereum's VM is the **Ethereum Virtual Machine (EVM)**. Every EVM validator must calculate the same result for the same ordered transactions. This shared execution enables applications to interact, but it also makes computation a replicated resource.

Hashes, trees, and commitments

A cryptographic hash maps any input to a fixed-length fingerprint. Finding two useful inputs with the same fingerprint should be infeasible. Protocols combine hashes into a **Merkle tree**, where leaf fingerprints are repeatedly paired and hashed until one **root** remains.

The root is a compact **commitment** to all leaves. A **Merkle proof** supplies the few neighboring hashes needed to show that one leaf belongs under that root. Think of the root as a tamper-evident seal on a large filing cabinet: the proof opens one drawer while still letting the verifier check the seal.

State roots, transaction roots, data commitments, and proof systems build on this idea. A commitment proves what data was bound only when the verifier also has a valid proof and the protocol specifies how the data was encoded.

The basic user journey

A transfer usually follows these steps:

1. a wallet constructs and signs a transaction;
2. a node receives it and shares it with peers;
3. a producer selects and orders it in a block;
4. validators check signatures, rules, and the resulting state;
5. consensus accepts the block as canonical;
6. later blocks or votes strengthen finality;
7. wallets and applications update what they show the user.

The same action therefore has several completion points: submitted, received, included, executed, accepted, and final. Much confusion about block-chain performance comes from timing one point and comparing it with another.

The adversarial setting

Ordinary distributed systems expect crashes and network delay. Blockchains also consider **Byzantine faults**: participants may send conflicting messages, fabricate data, censor users, or coordinate attacks. A system's security claim must say how many or how much weight may be Byzantine and which network conditions are assumed.

Safety means honest participants do not accept incompatible outcomes. **Liveness** means valid work eventually progresses under the stated conditions. A network partition may preserve safety by halting, or preserve availability by letting sides continue and reconciling later. Asset systems usually make that choice explicit because accepting two conflicting spends creates loss.

Where scaling enters

The basic design repeats computation, storage, data transfer, and consensus. Scaling techniques change that repetition:

- **sharding** assigns different work to different groups;
- **channels** keep repeated interactions between participants off the shared chain;
- **rollups** execute batches elsewhere and publish data plus a proof or challenge path;

- **data availability sampling** lets nodes test that large published data can be recovered without downloading all of it;
- **parallel execution** runs independent transactions at the same time;
- **succinct proofs** let a verifier check a large computation with a much smaller proof.

Every shortcut must answer the beginner's most important question: if fewer parties do the original work, what evidence lets everyone else trust the result, and what can a user do when that evidence or service is missing?

Why Naive Comparisons Fail

A payment network, an exchange database, and a public blockchain perform different work under different trust assumptions. A card network may authorize a payment quickly while settlement, fraud handling, and bank reconciliation occur later. A public blockchain validates signatures, executes shared state, propagates data, and reaches Byzantine consensus before offering its strongest finality.

This does not make performance comparison useless. It means the comparison must hold the workload and completion boundary constant. A transfer-only TPS figure cannot be compared with arbitrary smart-contract execution. A sequencer acknowledgement cannot be compared with L1 finality. A laboratory cluster cannot be compared with a geographically distributed validator network without reporting the difference.

The immediate symptoms of insufficient capacity are familiar: transactions wait, fee markets rise, users retry, and applications become unreliable. Historic events such as the CryptoKitties congestion episode made those symptoms visible. Their precise fees and shares of network traffic are less important than the mechanism: demand exceeded scarce blockspace, so inclusion latency and price rose together.

Gas and the Price of Shared Computation

Ethereum meters execution in **gas**. Every operation consumes a defined gas amount, and a transaction supplies a gas limit that bounds its work. This prevents a Turing-complete program from consuming validator resources forever.

Since EIP-1559, an Ethereum transaction pays a protocol-determined **base fee** that is burned and may add a **priority fee** to reward inclusion. The base fee changes with block utilization. The transaction's execution payment is approximately:

gas used × (base fee + priority fee)

Blob-carrying transactions use a separate blob fee market. This separation matters for scaling: rollup data demand can change without pricing every EVM operation identically.

Gas is both metering and congestion pricing. It approximates execution and state cost well enough for the protocol to ration resources, but it is not a perfect hardware benchmark. An opcode's gas schedule is a governance and security parameter, while actual client performance changes with software and hardware.

Case Study: Congestion and Application Design

When one popular application fills blocks, every application sharing the fee market competes for the same capacity. Users who need immediate inclusion bid more; users with low-value actions wait or leave. This is the practical consequence of synchronous composability: applications share one state and can interact atomically, but they also share congestion.

Scaling designs respond differently. A larger L1 block admits more shared activity but raises validator load. An application-specific chain isolates congestion but fragments state and security. A rollup batches application execution and shares the cost of data publication. A state channel avoids publishing repeated interactions but works only for a constrained participant set.

The right design depends on whether the application needs global synchronous state, how valuable its assets are, what latency users need, and how they recover from operator failure.

Named Case Study: From CryptoKitties Congestion to a Production Rollup Workload

Deploy-

ment labels: the CryptoKitties episode is historical production; Arbitrum One is a current production rollup. Together they show that scaling is not merely increasing one transactions-per-second number. The 2017 episode exposed a capacity knee on Ethereum's shared execution layer. A modern rollup moves much application execution into another pipeline, but adds sequencing, batch publication, proof and bridge boundaries that users must understand.

Ethereum in December 2017: one application meets shared blockspace

CryptoKitties let users buy, breed and transfer unique on-chain cats. Breeding was not one cheap database update. The application's own incident post

explained that `giveBirth()` combined genetic data in a contract call using more than 250,000 gas, over ten times the 21,000 gas of a simple ETH transfer. When demand surged, the team said Ethereum was "completely full," raised its suggested gas price to 25 gwei, and increased the birthing fee from 0.002 ETH to 0.015 ETH so independent callers would again have an incentive to execute births.¹

Trace one congested birth. A user submitted the breeding transaction and waited through the gestation rule. A later `giveBirth()` call had to enter Ethereum's public transaction pool and compete for block gas with unrelated payments, token sales and contract calls. Miners selected transactions partly by fee. A previously reasonable gas price could become uncompetitive while the transaction waited. If the user's account then sent a higher-nonce transaction, that later work could wait behind the underpriced nonce.

The application had also created a time-sensitive external incentive. Anyone could call `giveBirth()` and receive the birthing fee. Under normal fees, community-operated bots performed this work. When gas cost exceeded the fixed reward, those operators stopped and the CryptoKitties team paid the difference itself. Congestion therefore changed more than latency: it changed who could economically perform a safety- and fairness-related application action.

MetaMask reported user confusion over long-pending and failed transactions, expanded infrastructure capacity, and added a way to resubmit with a higher gas price.² Consensys's retrospective describes rising pending queues, overloaded read infrastructure, fees that could exceed the item being purchased, and application changes that moved nonessential activity away from on-chain transactions.³ These are observable consequences of offered load exceeding several capacities at once: block gas, fee estimation, mempool management, remote procedure call (RPC) service, user nonce management, and application support.

Failure path: the user saw no completion and resent without understanding nonce replacement. The wallet could create another pending transaction rather than more capacity. An underpriced earlier nonce could hold later actions. A centralized birth bot could keep the game moving but change the application's operational trust. The correct short-term tools were status, fee replacement, queue visibility and reducing unnecessary writes. The long-term lesson was to separate which actions require global consensus from browsing, offers, indexing and other work that can remain off-chain.

Arbitrum One: the same demand enters a layered pipeline

Now place a non-fungible token (NFT) marketplace workload on Arbitrum One. Users submit transfers, listings and contract calls to the Arbitrum sequencer. Nitro executes EVM-compatible transactions and the sequencer feed gives applications fast soft confirmations. A batch poster compresses ordered

transactions and publishes them to Ethereum through blobs or calldata. Validators reproduce execution, state assertions are governed by the Bounded Liquidity Delay (BoLD) dispute protocol, and canonical withdrawals wait for the accepted state path described in Chapter 6.^{4 5}

At the user interface, this feels faster because the application need not wait for every individual call to win space in an Ethereum L1 block. Many L2 transactions share one compressed data-publication cost. But the production workload now consumes a resource vector:

```
sequencer admission and execution
+ L2 state reads and writes
+ compressed bytes in an Ethereum batch
+ blob/calldata publication and settlement gas
+ validator replay and assertion/dispute capacity
+ bridge and withdrawal processing
```

Congestion can move rather than disappear. A popular mint can saturate sequencer execution or create one hot contract state. Ethereum blob prices can raise the shared publication component. A batch backlog can make soft confirmations diverge in age from L1-published data. A sequencer outage can stop the fast path, after which users rely on the delayed inbox and pay L1 cost. A proof or assertion dispute can delay canonical withdrawals even while the L2 interface remains responsive.

The production dashboard should therefore report at least sequencer acceptance latency, L2 inclusion, oldest unpublished batch age, data-publication cost and bytes, assertion status, forced-inclusion queue, withdrawal age, and Ethereum settlement finality. L2BEAT's Arbitrum One page is an example of an independent view that separates activity, data posted, liveness, state validation, operator, sequencing, withdrawals, permissions and upgrades rather than compressing the system into TPS.⁶

Failure path: the sequencer accepts a user's purchase and then stops before publication. The user has a soft receipt but no canonical Ethereum batch evidence yet. The wallet should say this and offer the delayed-inbox route when warranted, not call the purchase settled. If Ethereum blob space becomes expensive, the rollup can amortize cost across a batch, but users may still see higher L2 data fees or delayed posting. If a disputed assertion is false, BoLD needs an effective honest validator with data and the ability to act. If an emergency upgrade replaces critical code, ordinary proof assumptions do not answer whether governance used that power safely.

What actually improved

Question	2017 Ethereum application	Production rollup application
User's fast path	Public L1 mempool and miner inclusion	Rollup sequencer and L2 execution
Shared scarce resource	L1 block gas for each contract call	L2 execution plus compressed shared L1 data and settlement
Early acknowledgment	Mempool visibility	Sequencer receipt or L2 block
Stronger completion	Included history under Ethereum confirmation policy	Batch published, assertion accepted, Ethereum finality; withdrawal has another bridge boundary
Congestion symptom	Pending nonces, gas bidding, RPC load, unaffordable application calls	Sequencer queue, hot L2 state, publication backlog, blob price, withdrawal delay
Escape/recovery	Replace or wait for L1 transaction; application redesign	Forced L1 inbox, independent replay/challenge, canonical bridge path, plus governance controls

The rollup increases useful capacity by compressing data and avoiding repeated L1 execution. It also creates more completion states. That is an acceptable engineering trade when each state is measurable and recovery is real. The wrong comparison takes CryptoKitties-era L1 transactions per second and places it beside sequencer acknowledgements. The correct comparison fixes the contract workload, reports hardware and data cost, and follows each transaction to the security boundary the application actually needs.

From Database Benchmarks to Blockchain Benchmarks

Database benchmarks show why a workload must be specified. TPC-C, analytical queries, and key-value workloads exercise different resources. A result becomes meaningful through a transaction mix, dataset, concurrency, duration, hardware, tuning policy, and latency objective.

Blockchain evaluation keeps those controls and adds consensus topology, state replication, signature work, adversarial faults, finality, and recovery. BlockBench helped separate consensus, data, and execution behavior in permissioned blockchains.⁷ Gas per second is useful for comparing EVM execution because transactions consume different gas, but it remains only one resource metric: gas schedules approximate work and omit network consensus, data availability, proof generation, and long-run state costs.⁸

The right output is not one universal score. It is an operating envelope showing which workloads a system sustains, on which resources, at which latency and finality boundary, and under which faults.

A Scalability Model for This Book

The word *scalability* has long lacked one universally accepted systems definition.⁹ In blockchain discussions, it is often used when a team really means peak throughput. This book uses a stricter definition:

A blockchain scales when it can support increasing useful demand while keeping verification, participation, and failure recovery within acceptable cost and latency bounds.

This definition has four consequences. First, the workload must be specified. Ten thousand independent transfers are not equivalent to ten thousand swaps that all modify one liquidity pool. Second, resources must be specified: CPU model, core count, memory, storage, network bandwidth, validator count, and geographic distribution. Third, security must remain comparable. A system that replaces 1,000 independent validators with one database has increased capacity but has not demonstrated blockchain scalability. Fourth, the result must survive failure. Normal-path TPS says little about a sequencer outage, leader change, data withholding attack, or mass exit.

Throughput, Latency, and Capacity

Throughput is completed work per unit time. *Latency* is the time one request takes. *Capacity* is the maximum sustainable load before latency or failure rate becomes unacceptable. They interact but are not interchangeable.

A batching system illustrates the distinction. Waiting to collect 1,000 transactions may increase throughput and reduce cost per transaction, but the first user in the batch waits longer. A pipelined consensus protocol may commit one block per round after warming up even though each block needs several rounds to reach finality. Always ask which quantity improved.

Vertical and Horizontal Scaling

Vertical scaling extracts more work from one machine by using faster hardware or better software. Horizontal scaling divides work among machines. Blockchains need both, but horizontal scaling is harder because machines may be faulty or adversarial and must agree on shared state.

A useful mental model is a replicated state machine. If every validator executes every transaction, adding validators increases redundancy and security but does not add execution capacity. Sharding, rollups, and parallel execution change which work is repeated, where it is performed, or how independent work is scheduled.

A Transaction as a Resource Vector

TPS treats every transaction as one unit. In reality, a transaction consumes several resources:

- signature verification and EVM computation;
- state reads and writes;
- bytes propagated across the network;
- bytes retained temporarily or permanently;
- consensus votes and block space;
- proof generation or verification in a validity system.

Represent a workload as a vector rather than a count. A calldata-heavy rollup batch stresses data bandwidth. A zero-knowledge application may stress proving. A hot DeFi contract stresses sequential state access. The sustainable transaction rate is bounded by the first exhausted resource.

Ethereum gas captures part of computation and storage cost, which makes gas per second more informative than transfer TPS. It is still not a universal metric because gas schedules approximate resource usage and exclude consensus and off-chain proving.

Worked Example: Benchmarking Two Chains

Chain A reports 20,000 TPS using simple transfers on eight validator machines in one data center. Chain B reports 4,000 TPS using contract calls across 200 geographically distributed validators. Chain A is faster for the measured workload, but the headline does not establish that its architecture is more scalable.

A fair experiment fixes or reports the transaction mix, validator hardware, network topology, state size, block size, duration, client version, and tolerated failure rate. It measures p50 and p99 latency, not only the average. It runs long enough to expose state growth and database compaction. It introduces a faulty leader or network delay and records recovery.

The output should be a curve. At low load, latency is stable. As offered load approaches capacity, queues form and tail latency rises. Beyond the knee, throughput may flatten while failures increase. The knee of that curve under a realistic workload is more useful than one peak number.

The Four-Layer Evaluation Framework

Four functions behind blockchain scalability

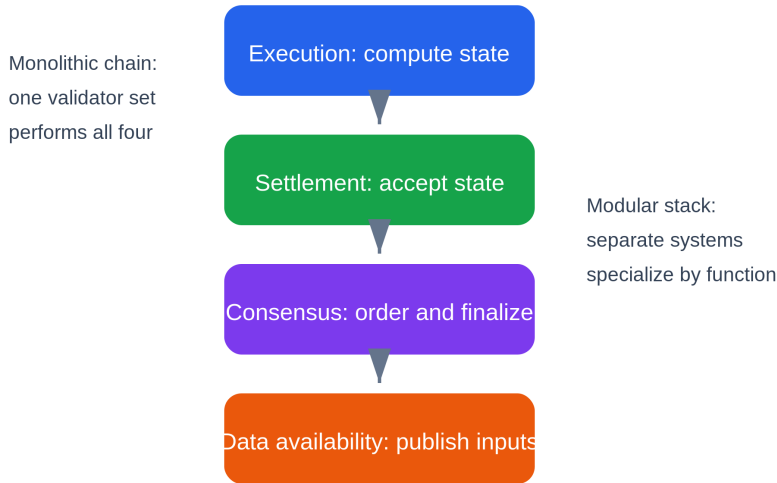


Figure 1.1: A blockchain transaction depends on execution, settlement, consensus, and data availability. Monolithic chains combine them; modular systems separate some roles. Original figure for this book.

The chapters ahead repeatedly separate four functions:

1. **Execution** computes state transitions.
2. **Settlement** decides which transition is accepted and resolves disputes.
3. **Consensus** orders data and finalizes a history.
4. **Data availability** ensures that the information needed for verification can be obtained.

A monolithic chain performs all four together. A rollup may execute elsewhere while Ethereum supplies settlement, consensus, and data. A modular DA layer may order and publish data without knowing an application's execution rules. Keeping these functions separate prevents category errors, such as assuming a validity proof also proves data availability or assuming a bridge gives a sidechain Layer 1 security.

Security and Decentralization Measurements

Decentralization is not a raw node count. Nodes operated by one company, hosted in one cloud, or controlled by one key do not provide independent failure domains. Relevant measures include stake or hash-power concentration, client diversity, hosting and geographic distribution, hardware require-

ments, governance power, and the ability of an ordinary user to verify and exit.

Security should be expressed as an adversary model. What fraction of validators can be Byzantine? Can the attacker delay the network, corrupt participants adaptively, or withhold data? Which guarantees fail first: liveness, safety, censorship resistance, or data retrieval? A protocol is secure only relative to those assumptions.

How to Read Performance Claims

When a project claims a large scalability gain, ask:

- What transactions were executed?
- Was the number measured, simulated, or projected?
- Which hardware and network were used?
- How many independent validators participated?
- Which finality boundary ended the timer?
- Was data publication included?
- Did the test include proof generation, state storage, and failures?
- Can users recover if the normal operator disappears?

These questions do not dismiss performance work. They make results reproducible and comparable.

From Transaction Submission to Finality

A blockchain performance measurement needs a precise start and finish. A transaction passes through construction and signing, RPC admission, peer-to-peer propagation, block inclusion, execution and fork choice, then finality. "Transaction latency" may stop at any one of these boundaries.

A wallet often shows block inclusion because it is fast and useful. A bridge waits for stronger finality because releasing assets against a reverted source transaction can create an unbacked claim. A rollup adds further milestones: sequencer acknowledgement, L2 inclusion, data publication, proof acceptance, and settlement finality.

This path explains why VM throughput is not chain throughput. The result still has to be encoded, propagated, agreed upon, committed to state, indexed, and served back to users.

Queueing and the Capacity Knee

When transactions arrive more slowly than the system processes them, queues remain short. Near capacity, ordinary variance produces bursts faster

than blocks or batches can absorb. Waiting time rises even before average demand exceeds average service.

Let work arrive at rate λ and be processed at sustainable rate μ . A basic queue becomes unstable when $\lambda \geq \mu$. Blockchains add batches, heterogeneous transactions, and fee priority, but the lesson holds: running continuously at advertised peak throughput creates poor tail latency.

A sound capacity plan reserves headroom for larger witnesses, failed leaders, prover retries, database compaction, and recovery traffic. Fee markets act as admission control. When demand exceeds scarce blockspace, users bid for priority while low-value work waits. The fee spike is the mechanism allocating congestion, not an unrelated symptom.

State, History, and Working Sets

History is the ordered record of blocks and transactions. **State** is the latest value of live accounts, contracts, or objects. The **working set** is the portion current execution touches. Pruning history does not shrink live state. Stateless validation reduces a validator's need to store state by supplying witnesses, but a builder or state provider still needs data to produce those witnesses.

Benchmark duration matters because caches hide storage cost. A short test may keep its working set in memory. A long-running chain reads cold state, compacts databases, creates snapshots, and serves synchronization. Sustainable performance includes these tasks.

A Reproducible Benchmark Procedure

1. Define completion: execution, inclusion, optimistic confirmation, proof acceptance, or consensus finality.
2. Publish transaction templates, state size, locality, and conflict rate.
3. Record client commits, protocol parameters, validator count, hardware, geography, and network shaping.
4. Warm representative state under a published rule.
5. Sweep offered load while measuring queue depth, failures, resource usage, fees, and p50/p95/p99 latency.
6. Inject a faulty leader, sequencer outage, delayed region, data withholding event, and prover crash.
7. Report the highest load meeting the latency and error objective, not the largest transient burst.

Worked Benchmark: A Rollup Exchange

Suppose an exchange sees 60% limit-order placements, 30% cancellations, and 10% market orders. Placements write an order and one price level. Market orders can touch many levels and accounts. A transfer benchmark misses this shared state.

A realistic generator samples prices and sizes, creates bursts around market moves, and records sequencer acknowledgement, L2 inclusion, publication, proof completion, and settlement. The first limit may be a hot price level. After partitioning markets, blob publication may dominate. After compression, proof generation may queue. Each optimization requires another end-to-end test.

Scalability Claims as Falsifiable Statements

A useful claim states workload, duration, validator topology, hardware, latency, finality, and failure behavior. "This system sustains the published mixed workload for one hour across 100 distributed validators while p99 finality stays below ten seconds and one leader failure recovers within thirty seconds" can be tested. "Up to 100,000 TPS" cannot.

Worked Capacity Envelope

Assume a chain targets a mixed workload of 70 percent transfers, 20 percent swaps, and 10 percent contract deployments. Measurements on disclosed hardware produce these per-transaction averages:

Transaction	Execution gas	Published bytes	Net state growth
Transfer	21,000	110	16 B
Swap	140,000	260	80 B
Deployment	1,200,000	8,000	6,000 B

The weighted average execution demand is:

$$0.70 \times 21,000 + 0.20 \times 140,000 + 0.10 \times 1,200,000$$
$$= 162,700 \text{ gas/transaction}$$

The weighted published data is:

$$0.70 \times 110 + 0.20 \times 260 + 0.10 \times 8,000$$
$$= 929 \text{ bytes/transaction}$$

The weighted state growth is:

$$0.70 \times 16 + 0.20 \times 80 + 0.10 \times 6,000 \\ = 627.2 \text{ bytes/transaction}$$

Suppose the measured system sustains 60 million execution gas per second, 500 kB/s of canonical data, and 25 kB/s of acceptable long-run state growth. Each resource implies a different transaction ceiling:

$$\begin{array}{lll} \text{execution: } 60,000,000 / 162,700 & \approx & 369 \text{ tx/s} \\ \text{data: } 500,000 / 929 & \approx & 538 \text{ tx/s} \\ \text{state: } 25,000 / 627.2 & \approx & 40 \text{ tx/s} \end{array}$$

Under these assumptions, long-run state growth is the tightest policy limit even though CPU and block data could support much more short-term throughput. If the state-growth budget is a governance objective rather than a hard protocol limit, the report should say so and project the resulting database size.

At 40 transactions per second, annual net state growth is approximately:

$$40 \times 627.2 \text{ B} \times 31,536,000 \text{ s} \approx 791 \text{ GB/year}$$

That result should trigger questions: can inactive state expire, do updates overwrite existing values rather than add new ones, how large are witnesses, who serves snapshots, and is the 10 percent deployment mix realistic? A surprising calculation is a reason to inspect the workload, not hide the number.

Add latency and headroom

Capacity is not the same as a safe operating target. If the state database, block propagation, and leader changes become unstable above 70 percent of the measured limit, target load should remain below that knee. Use the minimum across resources after applying their own headroom policies:

```
safe rate = min(
    execution capacity × execution headroom,
    data capacity × data headroom,
    state budget × state headroom,
    consensus capacity × consensus headroom,
    proving capacity × proving headroom
)
```

Headroom is resource-specific. Data publication may need burst capacity after an outage. Consensus needs time for a failed leader. A prover fleet needs capacity after one worker fails. State growth is cumulative and may need a policy margin rather than an operational utilization target.

Workload sensitivity

If deployments fall from 10 percent to 1 percent while swaps increase to 29 percent, both average gas and state growth change sharply. Report a sensitivity table instead of one mix:

Scenario	Transfer	Swap	Deployment	Purpose
Typical	70%	20%	10%	Expected mean
Mature application	70%	29%	1%	Fewer deployments
Launch burst	45%	35%	20%	Contract-heavy stress
Hot market	20%	80%	0%	Shared-state contention

A weighted average also hides variance. One maximum-size deployment can delay many transfers even when the mean fits. Run burst and adversarial scenarios, and publish p50, p95, and p99 completion latency at each offered load.

Reproducibility record

A reviewer should be able to reconstruct every number from:

```
client commit and protocol parameters
transaction generator and random seed
initial state snapshot and working-set distribution
hardware, storage, operating system, and compiler
validator count, regions, latency, loss, and bandwidth
run duration, warm-up, offered-load schedule, and errors
raw block, queue, resource, and finality measurements
calculation script and unit conventions
```

Keep decimal and binary byte units distinct. State whether kB means 1,000 bytes and KiB means 1,024. Define whether throughput counts submitted, included, executed, successful, or finalized transactions. Unit discipline catches many performance claims that are numerically correct but semantically incomparable.

Chapter Summary

Blockchain scalability is sustained useful capacity under explicit workload, resource, security, and recovery assumptions. TPS alone omits transaction complexity, latency, hardware, decentralization, and data. The central difficulty comes from globally replicated execution and storage plus the communication required for Byzantine consensus.

The rest of the book studies ways to divide or compress that work: sharding at Layer 1, channels and rollups at Layer 2, modular data availability, parallel execution, and more efficient consensus.

References

1. CryptoKitties. "CryptoKitties birthing fees increases in order to accommodate demand." <https://medium.com/cryptokitties/cryptokitties-birthing-fees-increases-in-order-to-accommodate-demand-acc314fcadf5>. ↵
2. MetaMask. "CryptoKitty Performance Update." <https://medium.com/metamask/metamask-cryptokitty-performance-update-83d851af0147>. ↵
3. Consensys. "The Inside Story of the CryptoKitties Congestion Crisis." <https://consensys.io/blog/the-inside-story-of-the-cryptokitties-congestion-crisis>. ↵
4. Arbitrum Docs. "Transaction lifecycle on Arbitrum." <https://docs.arbitrum.io/how-arbitrum-works/deep-dives/transaction-lifecycle>. ↵
5. Arbitrum Docs. "The Sequencer and Censorship Resistance." <https://docs.arbitrum.io/how-arbitrum-works/deep-dives/sequencer>. ↵
6. L2BEAT. "Arbitrum One." <https://l2beat.com/layer2s/projects/arbitrum>. ↵
7. Dinh, Tien Tuan Anh, et al. "BLOCKBENCH: A Framework for Analyzing Private Blockchains." SIGMOD 2017. <https://www.comp.nus.edu.sg/~ooibc/blockbench.pdf>. ↵
8. Konstantopoulos, Georgios. "Reth's path to 1 gigagas per second, and beyond." Paradigm, 2024. <https://www.paradigm.xyz/writing/reth-perf>. ↵
9. Hill, Mark D. "What is Scalability?" *ACM SIGARCH Computer Architecture News* 18, no. 4 (1990). https://minds.wisconsin.edu/bitstream/handle/1793/9676/file_1.pdf?sequence=1&isAllowed=y. ↵

Chapter 2: The Blockchain Trilemma

Introduction

The blockchain trilemma is a design heuristic: increasing capacity, broad independent participation, and adversarial security draw on the same finite bandwidth, computation, storage, capital, and coordination. It is often summarized as a tension among **decentralization**, **security**, and **scalability**.

1

It is not a theorem saying a blockchain can choose exactly two properties. Better cryptography, networking, and software can improve all three. The heuristic becomes useful when it forces a proposal to name which resource burden, trust assumption, or recovery cost changed. This chapter defines each axis, tests the framework against several architectures, and turns it into measurable questions.

A Beginner's Way to Use the Trilemma

The trilemma is easiest to understand as a budget, not a triangle that forces a choice of two labels.

A blockchain spends bandwidth, computation, storage, capital, and human coordination to provide useful work. It also asks independent people to run nodes and withstand attackers. A scaling proposal changes how those finite resources are spent.

Imagine a town keeping a public record. Sending every update to every resident makes independent checking easy but slow. Letting three clerks handle updates is faster, but residents now depend more on those clerks. Giving everyone better copying machines may improve speed without changing authority, although the machines still cost money. The trilemma asks what changed in each proposal.

- **Scalability:** How much useful work completes, and with what delay and operating cost?
- **Decentralization:** How many genuinely independent parties can verify, produce, govern, and recover the system?
- **Security:** Which attackers and failures can the system tolerate, and what loss or halt occurs beyond that boundary?

These are not percentages to add into one score. A chain can have decentralized validation but centralized block building,

strong consensus safety but a weak bridge, or high normal throughput but poor recovery.

Trade-off versus improvement

An efficiency improvement performs the same verified work using fewer resources. Better signature verification is an example. A trade-off changes the service or assumption: larger blocks may exclude slower validators; a committee checks work instead of everyone; a rollup moves execution to a sequencer and restores verifiability with data and proofs.

Both can be reasonable. The reader should ask whether the result still protects the same workload, assets, finality boundary, and recovery path.

How to read percentages

Security thresholds often use fractions of voting weight, stake, or hash power. A claim such as "fewer than one-third Byzantine" does not mean an attacker has a 33 percent chance of success. It means the proof assumes adversarial voting weight remains below that boundary. At or beyond it, different properties may fail.

Concentration percentages can overlap. If 40 percent of validators use one cloud and 30 percent are in one region, adding them to get 70 percent is wrong when many validators belong to both groups. Failure-domain analysis needs validator-level overlap.

Cost to attack versus value at risk

A large token market capitalization is not automatically the amount an attacker must spend or lose. Ask which stake can be borrowed, hedged, withdrawn, or slashed; who holds signing keys; how fast defenders react; and how much value a bridge or application releases before finality.

This chapter's tables and scenarios turn the three broad labels into those concrete questions.

What Is the Blockchain Trilemma?

The three labels are shorthand for measurable system properties:

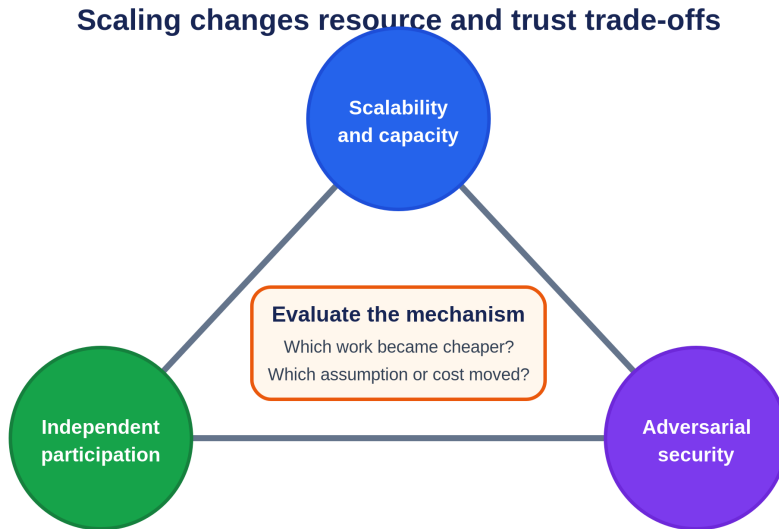


Figure 2.1: Scaling proposals should show which resource burden, participation cost, security assumption, or recovery cost changes. Original figure for this book.

1. **Decentralization:** how widely verification, production, governance, and recovery power are distributed, including the cost of entering each role.
2. **Security:** the faults and adversaries the system tolerates, the value required to attack it, and the consequences and recoverability of failure.
3. **Scalability:** whether useful throughput grows under a specified workload while latency, cost, state growth, and validator burden remain within an operating envelope.

A design may improve capacity without sacrificing either other axis when it performs the same work more efficiently. A trade-off appears when larger blocks exclude slower validators, a small committee replaces broad verification, or another layer moves safety or liveness into a bridge, sequencer, proof, or data assumption.²

Breaking Down the Trilemma

Each axis needs more than a slogan.

Decentralization

Decentralization is role-specific. A network can have many validating keys while a few custodians control stake, one client dominates execution, one relay carries order flow, or an upgrade multisignature can replace the rules.

Count independent failure domains and dangerous coalitions, not only nodes.

Broad verification deliberately replicates work. Ethereum validators reproduce canonical EVM execution so users need not trust one operator.³ This limits per-node execution capacity, but cheap verification and permissionless entry can keep the check on producers widely available.

Security

Security is a set of conditional guarantees. Consensus safety may hold below a Byzantine threshold while liveness also depends on network timing. Application safety may depend on contract correctness, key custody, data availability, and bridge finality.

Bitcoin makes history costly to rewrite through proof of work, but attack analysis must distinguish majority hash power, censorship, eclipse attacks, software bugs, and custody failure.⁴ A smaller committee can reduce communication latency while making capture or correlated failure easier. The relevant question is which actor can cause which loss under which condition.

Scalability

Scalability is the response to increasing offered load. Measure completed throughput, queue growth, tail latency, fees, state growth, and recovery while holding workload and finality boundary constant. A transfer benchmark does not establish smart-contract capacity, and a sequencer acknowledgement is not settlement finality.

Larger or faster blocks may raise throughput while increasing orphan risk and validator bandwidth. Delegating work to a subset can raise aggregate capacity while concentrating production or introducing committee security. The result is a new operating point, not free capacity.

Architecture Examples

Conservative Replicated Execution

A chain with conservative block limits and broad independent verification accepts lower base-layer throughput to keep propagation and validation within reach of more operators. This can strengthen censorship resistance and fault detection, but users compete for scarce blockspace when demand rises.

High-Performance Monolithic Execution

A chain can raise capacity with optimized networking, parallel execution, explicit account access, and higher validator hardware. The relevant trade is

not a brand-level label. Measure validator and RPC requirements, client and hosting concentration, network recovery, and performance under contentious state access.

Sharded or Committee-Based Execution

Assigning different work to subsets of validators increases aggregate capacity when committees operate in parallel. Security then depends on assignment randomness, committee size, adversarial concentration, cross-shard messaging, data reconstruction, and reshuffling. The whole validator set may be large while one transaction is protected by a smaller sample.

Rollup-Centric Execution

Rollups let a base layer specialize in settlement and data availability while separate systems execute applications. Cheap base-layer verification can support far more computation, but users encounter sequencers, proof or dispute systems, bridges, data retention, and upgrade controls. Each role has its own decentralization and failure boundary.

Exchanges as an Architectural Comparison

A centralized exchange can update an internal order book quickly because one operator chooses the database, matching policy, custody system, and recovery process. Users gain low-latency trading but must trust the operator for solvency, withdrawals, and rule enforcement.

An on-chain automated market maker uses consensus and contract rules for custody and execution. Every validating node reproduces the transition, which makes the result independently checkable but competes for scarce blockspace. A rollup exchange batches execution and settles commitments to a base layer, reducing per-trade cost while adding sequencer, data, proof or challenge, bridge, and upgrade dependencies.

These architectures should not be reduced to "centralized versus decentralized." Compare who holds assets, who orders trades, who verifies state, when a withdrawal is final, and how a user recovers during operator failure.

The Trilemma in Context: Lessons from Traditional Systems

The CAP theorem and blockchain trilemma are not interchangeable. CAP concerns consistency and availability when a network partition occurs.⁵ A blockchain additionally deals with Byzantine actors, Sybil resistance, economic incentives, public verifiability, and irreversible asset movement. CAP vocabulary can clarify partition behavior, but it does not prove a trilemma claim.

Named Case Study: Bitcoin, Ethereum, and Solana on One Fixed Workload

Deployment label: all three are production networks. Compare one workload: 10,000 users each authorize a simple native-asset payment to a distinct recipient, with no smart-contract call beyond the chain's normal transfer rules. Measure four boundaries separately: node admission, block inclusion, the chain's strongest commonly exposed finality or confirmation policy, and independent verification from protocol data. This is not a live throughput benchmark; transaction sizes, fees, congestion, hardware and software versions change. It is an architectural trace that keeps the work constant.

Bitcoin: UTXOs, mempool policy, and probabilistic confirmation

Each payer constructs a transaction consuming one or more unspent transaction outputs (UTXOs), creates a recipient output and usually a change output, and signs the relevant spending conditions. A node checks syntax, scripts, input availability and local relay policy before admitting it to its mempool. Mempool acceptance is not consensus: another node can apply different policy, and miners choose transactions under fee and block constraints.^{4 6}

A miner includes a subset in a proof-of-work block. Nodes validate the block and update the UTXO set. A payment has one confirmation when its block is accepted on the node's best-work chain; each later block increases the work that would need to be replaced by a competing history. There is no Byzantine-finality certificate after a fixed number of rounds. An application chooses a confirmation policy based on value and reorganization risk.

The 10,000 payments can spend distinct UTXOs and be individually verified, but Bitcoin's canonical state transition is still applied in block order. The workload consumes serialized bytes, signature checks, script execution, UTXO reads and writes, propagation bandwidth, and long-term history. Scaling the block limit would admit more payments but increases propagation, validation and storage load for every fully validating participant.

Failure path: two payments spend the same UTXO. Nodes may relay one under policy, but a valid chain can include at most one. A low-fee transaction may remain in some mempools or be evicted without confirmation. A shallow reorganization can remove an included payment, so an exchange should not equate "seen" or one block with its high-value completion rule.

Ethereum: account nonces, gas, execution, and proof-of-stake finality

Each payer signs a typed Ethereum transaction naming chain, nonce, recipient, value, gas limit, fee caps and signature. Nodes check format, signature, account nonce, balance and fee conditions for admission. A block builder orders eligible transactions. Ethereum's execution-layer state-transition func-

tion applies them against account state and charges gas. The EVM executes recipient code when the destination is a contract; a plain transfer to an externally owned account performs the protocol's balance and nonce updates without application bytecode.^{7 8}

The 10,000 users have distinct sender accounts, so they do not contend on one nonce. They still share block gas, builder ordering, state access, propagation and consensus. Inclusion changes balances and increments each sender nonce. Proof-of-stake validators attest to blocks; Ethereum's consensus exposes justified and finalized checkpoints under its Gasper design rather than asking applications to count proof-of-work confirmations indefinitely.⁹

The workload shows why "one transfer" is not identical across chains. Ethereum carries an account and gas model designed for general stateful computation. That flexibility lets the same transaction envelope call contracts, but also makes shared execution and state access a base-layer resource. A higher gas limit can fit more transfers while raising worst-case block processing and propagation requirements.

Failure path: the user submits nonce 8 before nonce 7 is available. The later transaction waits or is rejected under node policy. A base-fee rise can make the fee cap insufficient. A transaction included before finality can move to a noncanonical branch. A builder can reorder transactions, though a native transfer with distinct accounts has less application-level ordering sensitivity than an exchange call.

Solana: declared accounts, compute budget, and slot commitment

Each payer signs a Solana transaction whose message includes a recent blockhash, instructions, and every account the runtime needs. Writable and read-only flags expose dependencies before execution. The pipeline receives and deserializes the transaction, verifies signatures, sanitizes it, checks compute budget and age, validates nonce and fee payer, loads accounts, executes instructions, and commits or rolls back.^{10 11}

For 10,000 transfers over distinct sender and recipient accounts, the writable sets are mostly disjoint. Solana's runtime can schedule independent transactions across cores. If all payments also update one writable global counter, that account lock serializes the workload. The declared-access model converts application state layout directly into available parallelism.

A leader includes transactions in slots, validators replay them, and RPC commitment levels describe progressively stronger observation of the cluster's voted history. Applications must name the commitment level they treat as complete. A recent blockhash also gives an ordinary transaction a finite validity window; a client that cannot find a result must distinguish expiry from a transaction that landed under a signature it is about to resend.

Failure path: two payments write the same sender account or another hot account and encounter lock contention. A transaction with an expired block-hash cannot be newly processed through the ordinary path. A leader or RPC acknowledgement is not finality, and a skipped or reorganized slot can change an early observation. High parallel execution does not remove networking, leader, state-storage or vote-processing bottlenecks.

Architectural result

Boundary	Bitcoin	Ethereum	Solana
Payment state model	Consume and create UTXOs	Update sender/recipient accounts with nonce and gas	Execute instruction over declared accounts
Admission evidence	Local mempool policy and input/script checks	Local pool policy, nonce, balance and fee checks	Pipeline checks, recent blockhash, signatures, account loading
Parallelism exposed by workload	Independent validation, but one canonical block transition	Distinct accounts reduce nonce conflict; block execution and state remain shared	Disjoint writable account sets are schedulable in parallel
Strong completion notion	Application-selected depth on best-work chain	Consensus finality checkpoint for containing history	Named RPC/consensus commitment and cluster finality policy
Main duplicate/conflict guard	UTXO can be spent once	Sender nonce orders transactions	Signatures, recent block-hash/nonces, and writable-account state
Scale cost paid by verifiers	Bytes, scripts, UTXO/state growth, bandwidth	Gas execution, state access/growth, bytes, attestations	Signatures, account locks, compute, high-throughput networking/storage

The trilemma appears in the resources required to reproduce and defend the result. Bitcoin spends latency and limited block capacity to keep validation rules and hardware demands conservative. Ethereum supports general synchronous state and economic finality while budgeting gas and validator load. Solana exposes parallel account access and uses higher-performance validator pipelines to reduce latency and increase throughput. None "solves" the workload without cost. The costs move among hardware, bandwidth, state, consensus timing, application design and the population able to verify independently.

The Trilemma Is a Constraint, Not a Theorem

The blockchain trilemma is a design heuristic, not a mathematical impossibility result with one universal definition. A protocol can improve all three dimensions through better cryptography, networking, or hardware. The con-

straint appears when it scales one dimension by increasing the burden on another.

For example, larger blocks improve transaction capacity but raise bandwidth and storage requirements, which may reduce the population able to validate independently. Smaller committees improve communication latency but concentrate consensus power. A rollup improves execution capacity while introducing a sequencer, bridge, proof system, and data-publication path that each need separate analysis.

The proper question is not whether a project has "solved" the trilemma. It is which costs moved, which assumptions changed, and whether users can verify those assumptions.

Quantifying the Three Axes

No single metric captures an axis, but a measurement set makes comparisons less subjective.

Scalability includes sustained throughput for a stated workload, finality latency, fee behavior under demand, state growth, and recovery performance.

Decentralization includes the concentration of stake or hash power, number of independent operators, client and hosting diversity, entry cost, governance and upgrade control, and whether users can verify with ordinary hardware.

Security includes the cost or fraction required to violate safety, the network model, resistance to censorship and data withholding, bridge and smart-contract risk, and the time and cost of recovery.

The measurements can conflict. Reducing validator hardware may improve participation but lower capacity. Adding a committee may improve latency but add a bribery target. The purpose of the framework is to expose those choices.

Worked Example: Increasing the Block Size

Assume a chain doubles its block payload from 2 MB to 4 MB while keeping the block interval fixed. If transaction size and execution cost remain constant, nominal throughput may nearly double. But each validator now receives, verifies, and stores twice the block data.

A validator with a 20 Mbps connection needs at least 1.6 seconds just to receive 4 MB under ideal conditions, before gossip overhead and verification. Validators with slower links see blocks later and are more likely to build on stale state. Block producers with private high-speed links gain an advantage. Archive growth also doubles, increasing long-run operating cost.

The change may still be worthwhile. The point is that it occupies a different trilemma position. A complete proposal includes propagation measurements, stale-block behavior, hardware distribution, state-growth projections, and the effect on home validators.

Worked Example: A Rollup's Trade

A rollup moves execution away from every Ethereum validator. One sequencer may execute thousands of transactions and Ethereum verifies a compressed commitment. This improves throughput and fees without asking every L1 node to replay the work.

The trade is architectural rather than a simple sacrifice of security. State validity may inherit Ethereum through fraud or validity proofs, while liveness initially depends on the sequencer. Data may be published to Ethereum blobs, inherited from another DA layer, or held by a committee. Upgrade keys may override contracts. The rollup occupies several points on the trilemma at once, depending on the property examined.

This is why later chapters use a layered threat model rather than labeling a whole system decentralized or secure.

Security Budgets and Participation Costs

Decentralization and security are connected through participation cost. If validation requires rare hardware or privileged networking, fewer independent operators can check the chain. If production requires large stake, specialized proving hardware, or exclusive order flow, control concentrates even when entry is formally permissionless.

Every design has a security budget. Proof-of-work miners spend energy and hardware. Proof-of-stake validators lock capital and risk penalties. Rollups pay for data and challengers or provers. Bridges pay committees, relayers, or light-client verification. A system claiming dramatically lower cost should identify which security work became cheaper and which work was delegated.

The relevant budget includes stake liquidity, concentration among custodians, correlated software, cloud dependence, governance capture, and response time, not token price alone.

Decentralization Across Roles

Modern stacks divide power among RPC providers, sequencers, builders, validators, provers or challengers, DA nodes, relayers, bridge contracts, and governance. A system can be decentralized in one role and centralized in another.

er. Permissionless validators do not compensate for one instant upgrade key. Multiple provers do not help if one sequencer can censor the fallback path.

Build a role matrix. For each role, ask what it can do alone, what coalition is dangerous, how users detect abuse, and whether the system can recover without it.

Asymmetric Verification

One way to improve the trilemma frontier is to make verification much cheaper than production. Digital signatures make authorization cheap to check. Succinct proofs compress large computations. Data sampling gives confidence about a large block after downloading random pieces.

Asymmetry lets many ordinary participants verify work produced by stronger machines. It does not remove assumptions. A proof depends on its encoded program. Sampling depends on correct encoding and dissemination. Cheap verification can coexist with centralized production. The target is competitive production, cheap independent verification, and permissionless recovery.

Failure Domains, Not Key Counts

One hundred validator keys do not create one hundred independent replicas if eighty run one client, seventy share one cloud region, or most stake is controlled by two custodians. Client diversity, hosting, operator control, network paths, jurisdiction, and signing infrastructure form different correlation graphs.

A decentralization report should show these distributions and the protocol actions each concentration enables. This is more useful than a single node count.

Worked Comparison: Scaling an Exchange

A larger monolithic L1 preserves synchronous shared state but asks every validator to process more data. An appchain isolates exchange traffic and can use a small fast committee, but its assets inherit that committee and bridge. A rollup batches execution and uses an L1 for settlement, while adding sequencer, data, proof, and bridge dependencies.

None has one trilemma score. The correct comparison lists roles, participation cost, assets at risk, finality, and recovery for the exchange's workload.

Governance as a Fourth Axis

The trilemma usually treats rules as fixed, but deployed systems change. Governance can replace a verifier, pause a bridge, increase block limits, or rotate a committee. Fast emergency action can protect security while concentrating power; slow action improves predictability but delays fixes.

For every upgradeable system, evaluate current code and the process that can replace it. The latter is part of the security boundary from launch.

Worked Failure-Domain Audit

Suppose a proof-of-stake network advertises 1,200 active validator keys. Key count alone does not reveal whether faults are independent. Collect control and infrastructure data:

Dimension	Largest observed concentration
Beneficial owner or custodian	31% of stake
Validator client	68% of stake
Cloud provider	46% of stake
Geographic region	39% of stake
Maximum extractable value (MEV) relay or block-builder path	72% of proposed blocks
Governance delegation bloc	37% of votes

These percentages cannot be added: one validator may belong to every category. Instead, construct correlation scenarios.

Scenario 1: client bug

If one client controls 68 percent and a deterministic bug makes it accept an invalid transition, the effect depends on consensus rules and what minority clients do. Diversity counted by the number of available clients is irrelevant when deployment share remains concentrated.

Test the actual failure: feed all clients the divergent block, observe voting and fork choice, and rehearse coordinated but non-simultaneous recovery. An emergency social response may preserve the intended history, but that is a recovery assumption outside ordinary consensus.

Scenario 2: cloud and region outage

A cloud provider holds 46 percent of stake, with 30 percentage points in one region. A provider-wide outage may threaten liveness; a regional outage overlaps but is not independent. Model the union from validator-level data rather than summing 46 and 39.

Operators should spread replicas only when the protocol permits redundant signing safely. Running two active signers for one key can create equivocation during a partition. High availability needs fencing or remote-signing controls that ensure only one valid signing state.

Scenario 3: custodian compromise

A custodian controls 31 percent, below a one-third Byzantine boundary but close enough that additional correlated stake can cross it. Determine whether the custodian controls withdrawal keys, validator signers, governance votes, or only delegates stake. Each permission produces a different attack.

Economic security cannot be reduced to 31 percent of market capitalization. Measure stake that can actually be slashed, time to exit, borrowability, derivative hedges, and whether governance can cancel penalties.

Scenario 4: block-building concentration

A builder or relay path touching 72 percent of blocks may censor or reorder transactions without controlling consensus finality. Inclusion lists, alternate relays, local building, and forced paths address this role. Counting consensus validators does not measure ordering decentralization.

Independence scorecard

For each role, record:

```
entities and weights
shared software and version
hosting provider, region, and network
signing and custody infrastructure
governance and upgrade authority
fallback path and tested recovery time
```

Then calculate the smallest plausible correlated set that can:

- halt finality;
- finalize conflicting state;
- censor a transaction past its deadline;
- withhold enough data to prevent recovery;
- replace protocol or bridge code;
- stop users from exiting.

The answer can differ for every action. One coalition may halt consensus, another may censor ordering, and a single upgrade key may replace a verifier.

Home-validator budget

Decentralization also depends on the lower end of participation. Publish bandwidth, CPU, memory, storage IOPS, state size, history growth, synchronization time, and operational attention for a validating node. Measure p95 requirements under load and recovery, not minimum idle specifications.

If a capacity increase raises annual state by 2 TB and requires 50 Mbps sustained ingress, estimate how many current operators and regions can still participate. A proposal may improve throughput and keep the same consensus threshold while reducing the population able to verify independently.

Audit conclusion

A defensible report should not say "1,200 validators means decentralized." It should say which roles were measured, the largest control and infrastructure concentrations, the smallest dangerous coalitions, the participation budget, and the observed recovery from correlated faults.

This converts the decentralization axis from a brand judgment into a set of failure hypotheses that can be tested.

Conclusion

The trilemma is useful when it exposes where a scaling design spends resources and trust. It does not assign one score to a chain, prove that only two properties are possible, or make decentralization a validator count.

A defensible comparison names workload, completion boundary, validator burden, dangerous coalitions, control keys, data and bridge assumptions, and recovery path. Chapter 3 applies that discipline to Layer 1 and Layer 2 architectures.

References

-
1. Buterin, Vitalik. "The Blockchain Trilemma." *Ethereum Blog* (2017). Available at: https://vitalik.eth.link/general/2017/12/31/sharding_faq.html. ↩
 2. Hill, Mark D. "What is Scalability?" *ACM SIGARCH Computer Architecture News* (1990). Referenced in Chapter 1. ↩
 3. Wood, Gavin. "Ethereum: A Secure Decentralised Generalised Transaction Ledger." *Ethereum Yellow Paper* (2014). Available at: <https://ethereum.github.io/yellowpaper/paper.pdf>. ↩
 4. Nakamoto, Satoshi. "Bitcoin: A Peer-to-Peer Electronic Cash System" (2008). Available at: <https://bitcoin.org/bitcoin.pdf>. ↩ ↩2
 5. Brewer, Eric A. "Towards Robust Distributed Systems." *PODC Keynote* (2000). <https://doi.org/10.1145/343477.343502>. ↩
 6. Bitcoin Developer Reference. "Transactions." <https://developer.bitcoin.org/reference/transactions.html>. ↩

7. Ethereum.org. "Transactions." <https://ethereum.org/developers/docs/transactions/>. ↵
8. Ethereum.org. "Ethereum gas and fees." <https://ethereum.org/developers/docs/gas/>. ↵
9. Ethereum.org. "Gasper." <https://ethereum.org/developers/docs/consensus-mechanisms/pos/gasper/>. ↵
10. Solana Documentation. "Transactions." <https://solana.com/docs/core/transactions>. ↵
11. Solana Documentation. "Transaction processing pipeline." <https://solana.com/docs/core/transactions/transaction-pipeline>. ↵

Chapter 3: Layer 1 vs Layer 2 - Comparing Different Approaches to Blockchain Scaling

Introduction

Layer 1 and Layer 2 are different places to spend a system's resource and trust budget. Layer 1 changes the base protocol that validators execute and agree on. Layer 2 moves repeated work into another protocol while retaining an enforcement or settlement relationship with the base chain.

The labels do not rank security or performance. A larger L1 block, a payment channel, a rollup, and a sidechain improve different workloads and fail in different ways. This chapter compares them by following execution, data, finality, cost, and recovery rather than by treating "L2" as a synonym for fast or cheap.

First Intuition: Move the Work or Change the Base

A beginner can think of a blockchain as a public court plus a shared computer. The base chain receives evidence, applies rules, and records the accepted result. Scaling can change the court itself or let another system do routine work while the court remains available for enforcement.

Layer 1 (L1) is the base chain and its native rules. An L1 change affects every node that follows the chain. Increasing the block limit, changing consensus, or adding sharded data capacity are L1 changes.

Layer 2 (L2) is a protocol built above an L1 that uses the L1 for a meaningful security function. That function might be settling disputes, checking proofs, publishing recovery data, or enforcing exits. Calling a network "Layer 2" does not by itself say which function it inherits.

Three everyday analogies help separate the designs:

- An **L1 upgrade** is like widening the main road. Everyone uses the new road, and every road operator must handle its new width.
- A **payment or state channel** is like opening a tab. Participants exchange signed updates privately and use the base chain only to open, close, or settle a disagreement.

- A **rollup** is like a clerk processing many receipts and submitting a compressed report plus evidence that the total is correct. The base chain does less work per receipt but retains a rule for rejecting a false report.

A **sidechain** is a neighboring road with its own traffic authority. A bridge connects it to the base chain, but the bridge does not automatically give the sidechain the base chain's validators or security.

Questions to ask of any layer

Before comparing performance, follow one user action and ask:

1. Who orders it?
2. Who executes it?
3. Where can another party obtain the data?
4. What proof, challenge, or validator vote makes the result acceptable?
5. When can the result still be reversed?
6. Who can change the rules?
7. How does the user recover if the normal operator disappears?

These questions turn a layer label into a concrete mechanism. The rest of the chapter answers them for common L1 and L2 designs.

What Is Layer 1 Scaling?

Layer 1 refers to the **base blockchain protocol**, such as Ethereum, Bitcoin, or Solana. Scaling Layer 1 involves changing the underlying consensus mechanism, data structure, or execution environment to boost performance.

Sharding partitions some combination of data, state, execution, or validator work. It can increase aggregate capacity without asking every node to process every item, but cross-shard messages, committee security, state movement, and data availability become protocol responsibilities.

Common Layer 1 Scaling Techniques

- **Increasing Block Size or Block Frequency**
More transactions per block or faster blocks can increase throughput. However, this increases the resource requirements for running full nodes, possibly reducing decentralization.
- **Optimizing Execution Environments**
Better interpreters, compilers, storage layouts, state commitments, and parallel runtimes reduce the resource cost of executing or verifying a unit of work. A VM choice alone does not guarantee parallelism; the state-access model and workload determine conflicts.

- **Consensus Optimization**

Pipelining, signature aggregation, efficient block propagation, and linear-message Byzantine fault tolerant (BFT) protocols can reduce latency or communication. Changing Sybil resistance from proof of work to proof of stake changes energy use and security economics, but does not by itself create execution capacity.

- **Sharding**

Assigning different data or execution work to shards so that not every validator performs every task. Aggregate capacity can increase when committees, cross-shard communication, data availability, and state movement do not become the new bottlenecks. Whether node requirements remain manageable is a measured result, not a consequence of the label.

What Is Layer 2 Scaling?

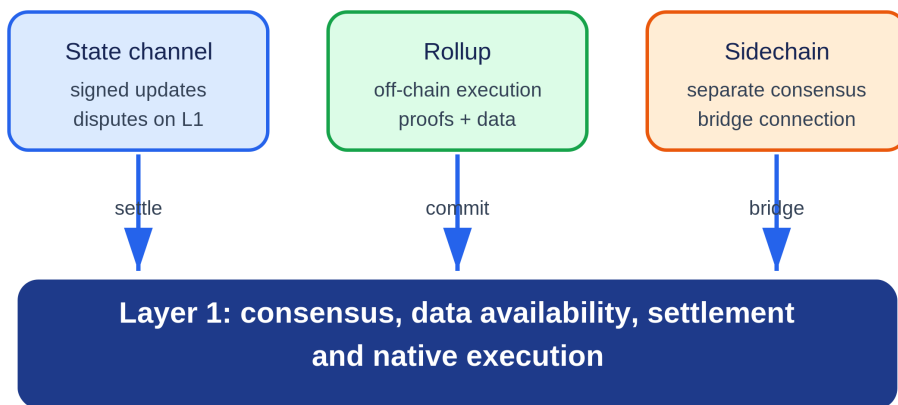


Figure 3.1: Channels and rollups use Layer 1 for enforcement or settlement, while a sidechain connects through a bridge but runs separate consensus. Original figure for this book.

A Layer 2 protocol performs some work outside Layer 1 while using Layer 1 to enforce a result, resolve a dispute, or let users recover. The inheritance is specific, not total. A rollup may inherit settlement consensus while depending on a sequencer for timely inclusion and an upgrade process for contract integrity.

Optimistic rollups use a dispute window and at least one honest party able to reconstruct and challenge invalid state. Validity rollups require a proof accepted by an L1 verifier. Neither design makes sequencer acknowledgement

equivalent to L1 finality, and neither removes data-availability or upgrade risk.

Common Layer 2 Techniques

- **State Channels**
A fixed participant set locks assets or state under an L1 adjudicator, exchanges successively newer signed states off-chain, and can submit evidence on-chain to close or resolve a dispute. Channels can have more than two participants, but membership, online monitoring, liquidity, and dispute deadlines constrain the workloads they fit.¹
- **Plasma**
A family of child-chain constructions that commits roots to L1 while requiring users to retain data and use exit games when the operator withholds data or proposes invalid history. Plasma does not provide the general data-publication and arbitrary-state guarantees of a rollup.²
- **Optimistic Rollups**
An operator executes transactions outside L1 and publishes the data and commitments required by the rollup protocol. The L1 contract provisionally accepts state commitments and rejects an invalid transition when a valid fault proof succeeds during the challenge window. Sequencer confirmation and L1 data inclusion can be fast, but uncontested state and ordinary withdrawals reach their strongest protocol status only after the dispute window.
- **Validity Rollups, often called ZK Rollups** An operator executes a batch outside L1 and submits a succinct proof whose public inputs bind the claimed transition to the verifier program. The proof need not hide transaction data, so "validity" is the more precise term. A valid proof establishes only the statement encoded by the circuit and verifier; settlement still depends on verifier correctness, data availability, contract upgrades, and L1 inclusion and finality.³

Comparing Layer 1 vs Layer 2

Question	Layer 1 change	Rollup or channel	Sidechain
Who executes?	Base-layer validators	L2 operators or participants	Sidechain validators
Where is canonical data?	Base layer	L1, a DA layer, or participants, depending on design	Sidechain and archival services

Question	Layer 1 change	Rollup or channel	Sidechain
What establishes correctness?	Base consensus and execution rules	L1 contract plus dispute, proof, or signed-state rules	Sidechain consensus and bridge verifier
What gives fast confirmation?	Block producer and fork choice	Sequencer or participant signature	Sidechain consensus
What is the recovery path?	Reorganization/finality and client recovery	Force inclusion, dispute, or exit	Sidechain governance and bridge recovery
Main scaling cost	Higher validator load or protocol complexity	New operators, contracts, proofs, data, and exit paths	Independent security budget and bridge risk

Real-World Trade-Offs

An L1 capacity increase benefits every application sharing that protocol, but requires broad coordination and may raise validator cost. An L2 can specialize and upgrade faster, but users must cross a bridge and reason about an additional operator, contract, data, and finality path.

Ethereum's proof-of-stake transition changed consensus economics and energy use; it was not by itself a throughput multiplier. EIP-4844 is a clearer example of L1 scaling for L2: separate blob capacity lowers rollup data cost while preserving a base-layer availability commitment.⁴ Rollups then amortize publication and verification across batches.

In an optimistic rollup, the security condition is not that every user watches the chain. The condition is that at least one independent challenger can access the data and successfully use the fault-proof path before the deadline.⁵ A validity rollup replaces that challenge assumption with proof-system, circuit, verifier, and prover-liveness dependencies.

Why Both Layers Matter

Layer 1 and Layer 2 capacity are complementary when their interfaces are explicit. Base-layer data, settlement, and forced-inclusion capacity bound what rollups can safely process and recover. L2 execution lets applications specialize without forcing every L1 validator to execute every user action.

Adding more layers does not automatically add security. Each layer introduces another finality, data, upgrade, and recovery dependency. The architecture is useful when the lower layer can enforce the property the upper layer claims to inherit.

A Transaction's Path Through Layer 1 and Layer 2

The difference between L1 and L2 becomes clearer by following the same token transfer through each system.

On Layer 1, a user signs a transaction and sends it to the peer-to-peer network. A block producer chooses its position, every validating node executes it, consensus selects the canonical block, and the resulting state becomes final under the chain's rules. Execution, data publication, and consensus all happen in the same security domain.

On a rollup, the user normally sends the transaction to an L2 sequencer. The sequencer orders and executes it and may return a soft confirmation immediately. Later, the rollup publishes transaction data and a state commitment. Ethereum consensus finalizes that publication. The rollup contract then accepts the state after either a fraud-proof window or verification of a validity proof. One user action therefore has several milestones: receipt by the sequencer, inclusion in an L2 block, publication to L1, proof acceptance, and L1 finality.

A sidechain follows a different path. Its own validators execute and finalize the transaction. Ethereum sees nothing unless a bridge message is later submitted. The sidechain may be fast, but its correctness comes from its validator set and bridge, not from Ethereum merely because assets can move between them.

These paths explain why the label "Layer 2" should describe a security relationship rather than a position in a diagram. A system is meaningfully layered when the base chain can enforce state correctness or user exits under stated assumptions.

The Scaling Bottleneck Moves

A system rarely has one permanent bottleneck. Raising an L1 gas limit can move the limit from execution to block propagation. A parallel VM can move it from CPU to state-database access. A rollup can make execution cheap enough that data publication dominates fees. Cheap blob space can make proving or sequencing the next constraint.

This is why capacity planning must be end to end. Let:

- E be sustainable execution capacity in gas per second;
- D be data capacity in bytes per second;
- C be consensus capacity in blocks or votes per second;
- P be proof generation capacity for a validity system.

Application throughput is bounded by the first saturated resource. Adding capacity to E has little effect when each transaction consumes enough

published bytes to saturate D. An optimization should identify which resource is limiting under the target workload and show that the next resource can absorb the displaced demand.

A Better Comparison Framework

The earlier table gives a useful overview, but a production choice needs more detail.

Security and Recovery

Ask who can make invalid state final, who can withhold data, and what a user can do when the normal operator disappears. An L2 with a centralized sequencer may still preserve fund safety if users can force inclusion and exit. A sidechain with many validators may still be exposed if its bridge is controlled by a small multisig.

Finality

Separate user-perceived confirmation from economic and cryptographic finality. L2 confirmations are often fast because a sequencer promises an order. A bridge receiving a large deposit may wait for L1 settlement and proof acceptance instead.

Cost

Layer 1 users pay for globally replicated execution and data. Rollup users share publication and verification across a batch. Sidechain users pay that chain's validators. Channel users pay to open and close while bearing the opportunity cost of locked liquidity. Fee comparisons should include the cost of withdrawal and failure recovery, not only the normal transaction.

Composability

Contracts on one synchronous state machine can call each other atomically. Moving execution across rollups or shards turns those calls into messages. The result may scale better, but developers must handle delay, replay protection, and partial completion.

Decentralization of Operations

Validator count is only one dimension. Sequencers, provers, relayers, bridge administrators, RPC providers, and upgrade signers can each become a control point. A complete architecture diagram names these roles.

Worked Design Choice: An On-Chain Game

Suppose a game generates 500 player actions per second, needs sub-second feedback, and has occasional high-value asset withdrawals.

Putting every action on Ethereum L1 gives strong settlement but poor cost and latency. A payment channel is insufficient because players interact with shared game state. A high-throughput sidechain can meet latency, but the game's valuable assets inherit the sidechain bridge's trust model. A rollup can execute frequent actions cheaply and settle asset ownership to Ethereum, while a centralized sequencer initially provides fast feedback.

The rollup is not automatically the answer. If the game state is too large to publish economically, a validium or external DA layer may be considered. That lowers cost but adds a withholding assumption. If the game requires every player action to synchronously interact with DeFi on Ethereum, asynchronous messaging may make the experience unacceptable. The design decision follows the workload and recovery requirements, not a universal ranking of technologies.

Forced Transactions and Censorship Recovery

A layered system should let users bypass the normal operator. An L1 inbox can accept canonically encoded L2 transactions. The sequencer must include them within a bounded period; otherwise another party advances state, the protocol changes mode, or the user exits. In the OP Stack, for example, L1 deposits form part of the derivation inputs used to construct the L2 chain.⁶

The deadline balances responsiveness and cost. A short timeout turns temporary outages into expensive L1 recovery. A long timeout gives a censor more power. Test the mechanism under L1 congestion, when many users may invoke it together.

Deposits, Withdrawals, and Finality

An L2 should wait for sufficient L1 finality before crediting a deposit. Otherwise an L1 reorganization can remove collateral after its L2 representation has moved. In the other direction, a canonical bridge verifies an accepted L2 withdrawal message and replay protection before releasing L1 assets.

Optimistic withdrawals wait for the challenge rule; validity withdrawals wait for proof acceptance. A concrete optimistic withdrawal specification separates initiation on L2, proof on L1, finalization delay, and execution.⁷ Liquidity bridges can pay earlier but introduce separate solvency and message risk. A full fee and latency estimate follows the user's entire deposit-action-withdrawal journey.

Fee Anatomy Across Layers

```
L2 fee = execution charge
        + allocated publication charge
        + proving or challenging and operating charge
        + margin or congestion premium
```

Batching spreads publication over many transactions. An appchain may subsidize validators or DA through inflation or treasury spending. "Low fee" should identify who pays the remainder. Recovery and canonical withdrawal costs matter for one-time users.

L3s and Recursive Layering

An L3 executes above an L2 and uses that L2 for settlement, data, or both. A withdrawal may wait for L3 proof, L2 publication, L2 proof or challenge, and Ethereum finality. If the L2 operator is offline, the L3 must state whether its fallback can reach L1 directly.

Layer numbers do not define security. Apply the same transaction-path and recovery-path analysis at every boundary.

Migration and Upgrade Strategy

Applications may move from L1 to a shared rollup and then an appchain. A safe migration announces source and destination versions, freezes or snapshots finalized source state, publishes a commitment to migrated balances, allows verification or challenge, activates destination state, and preserves an exit for users who decline migration.

Without a verifiable state mapping, migration becomes administrator custody.

Architecture Decision Record

Record workload, latency objective, assets at risk, composability needs, execution and DA requirements, sequencing, proof or challenge rules, normal and forced message paths, fee subsidies, upgrade authority, and shutdown plan. Revisit the record as demand changes.

Worked Migration: L1 Application to Rollup

Moving an application from L1 to a rollup is a state transition across security domains, not a deployment script. Users need a verifiable mapping from old assets and state to new state, and a choice not to follow the migration.

Consider a game with fungible balances, unique items, open marketplace orders, and pending withdrawals. The migration team selects an L1 block that is final under the published policy and records:

```
MigrationManifest {
  source_chain,
  source_contracts[],
  source_final_block,
  source_state_root,
  extraction_code_hash,
  destination_rollup,
  destination_contracts[],
  destination_genesis_root,
  mapping_version,
  challenge_deadline
}
```

Inventory and freeze policy

Inventory every source state category and decide whether it migrates, settles, cancels, or remains claimable on L1. An open order should not silently become executable in both places. A pending withdrawal should not disappear from one queue and reappear with a fresh nonce.

A full freeze simplifies the snapshot but stops the application. A rolling migration can reduce downtime while creating dual-write and replay complexity. If both systems remain active, define which actions are canonical and how one-way messages prevent double spending.

Deterministic extraction

The extraction program reads source state at the finalized block and emits canonical destination records. Publish the program, compiler/build information, input block, and output commitment. Independent parties should reproduce the same destination genesis root.

The mapping handles:

- address and signature-scheme differences;
- token decimals, metadata, and native-versus-wrapped identity;
- contract storage layout and default values;
- ownership and approval state;
- consumed nonces and pending messages;
- rounding and dust;
- paused, frozen, or blacklisted records if applicable.

Do not discard zero balances or empty records unless the source semantics make them irrelevant. Presence itself can affect authorization or future storage behavior.

Claim versus push

A **push migration** initializes every account on the destination. It offers a complete genesis but may be expensive. A **claim migration** commits to a Merkle root; each user later supplies a proof and initializes only needed state.

Claim migration needs a permanent or sufficiently long proof-data service. Correctness can be cryptographically verified while availability still depends on service providers. Several independent hosts should retain the leaf set and tree construction code. Claims need domain binding and a consumed key so the same source asset cannot initialize twice.

User choice and exit

Publish destination contracts, sequencer and DA assumptions, bridge, upgrade keys, expected fees, and finality before the opt-in deadline. Users who decline need an L1 withdrawal, sale, redemption, or continued old-version path. A choice is not meaningful when the escape transaction costs more than the asset or the window closes during congestion.

Activation trace

1. finalize the source snapshot or claim root;
2. complete the challenge or reproduction period;
3. deploy destination code with pinned versions;
4. initialize the destination root or claim contract;
5. test deposit, action, forced inclusion, and withdrawal on the final deployment;
6. activate user routing and indexers;
7. keep source recovery and proof data available through the promised window;
8. reconcile total supply, ownership counts, pending messages, and claims.

Failure matrix

Failure	Required outcome
Source block reorganizes	snapshot is invalidated unless finality policy still holds
Extraction code omits state	challenge or independent reproduction blocks activation

Failure	Required outcome
Destination root differs	deployment does not activate
User claims twice	consumed source identifier rejects the duplicate
Marketplace order exists on both systems	canonical cancellation/migration rule permits only one execution
Bridge or sequencer fails at launch	forced path and exit work on the deployed contracts
Claim data host disappears	independent hosts and reproducible tree restore service
Upgrade occurs during migration	manifest version remains fixed or migration restarts transparently

Reconciliation

For fungible assets, prove:

```
source locked or burned
= destination issued
+ source refunds
+ explicitly documented remainder
```

For unique items, compare the set of identifiers and owners. For messages, compare source pending, destination consumed, expired, and refunded sets. A total supply match alone misses ownership swaps or duplicate non-fungible identifiers.

Run reconciliation before activation, after the first claims, at the migration deadline, and after source cleanup. Publish machine-readable results. Migration is complete when users can verify the mapping and recovery, not when the new front end points at the rollout.

Conclusion

Layer 1 scaling changes the shared protocol and its validator resource envelope. Layer 2 scaling reduces the base-layer work per user action while retaining a defined enforcement or settlement path. Sidechains add independent capacity rather than inheriting correctness merely because they have a bridge.

A sound choice starts with workload and follows one transaction through ordering, execution, data publication, proof or dispute, finality, fees, and failure recovery. Chapter 4 now examines the base-layer mechanisms in detail.

References

1. Poon, Joseph, and Thaddeus Dryja. "The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments" (2016). <https://lightning.network/lightning-network-paper.pdf>. ↵
2. Poon, Joseph, and Vitalik Buterin. "Plasma: Scalable Autonomous Smart Contracts" (2017). <https://plasma.io/plasma.pdf>. ↵
3. Ethereum.org. "Zero-knowledge rollups." <https://ethereum.org/developers/docs/scaling/zk-rollups/>. ↵
4. Buterin, Vitalik, et al. "EIP-4844: Shard Blob Transactions." <https://eips.ethereum.org/EIPS/eip-4844>. ↵
5. Ethereum.org. "Optimistic Rollups." <https://ethereum.org/developers/docs/scaling/optimistic-rollups/>. ↵
6. Optimism. "Derivation." *OP Stack Specification*. <https://specs.optimism.io/protocol/derivation.html>. ↵
7. Optimism. "Withdrawals." *OP Stack Specification*. <https://specs.optimism.io/protocol/withdrawals.html>. ↵

Chapter 4: Layer 1 On-Chain Scalability

Introduction

Layer 1 scaling changes the base blockchain itself. Instead of moving work to a separate protocol, it asks how the network can process more computation, store more state, publish more data, and reach agreement faster while preserving open verification.

The simplest ideas are larger blocks and shorter block times. Both can raise headline throughput, but they increase bandwidth, storage, and CPU requirements. If fewer people can validate the chain independently, scalability has been purchased by weakening decentralization. Good Layer 1 design aims for more capacity per unit of validator resource, or divides work so that no validator must process everything.

The course frames the problem around replicated computation, replicated storage, and consensus communication. Layer 1 techniques attack these bottlenecks through protocol optimization, sharding, and interoperability.

A Map of Layer 1 Work

A base-layer transaction passes through several resources. Separating them makes the later techniques easier to understand.

1. **Propagation:** transaction and block bytes travel between nodes.
2. **Execution:** each validator applies program instructions to current state.
3. **State access:** the client reads and writes its database.
4. **Consensus:** validators decide which proposed block is canonical.
5. **Storage and synchronization:** nodes retain enough data and help new or recovering nodes catch up.

Raising a limit in one stage can move the bottleneck to another. A faster virtual machine does not help if blocks cannot reach validators before the next round. More bandwidth does not help if every transaction contends for one state entry.

A useful analogy is a warehouse. Trucks deliver orders, workers pick items, a ledger records inventory, supervisors approve each batch, and archives let a replacement warehouse reconstruct the ledger. Buying faster forklifts improves only the picking stage. Layer 1 scaling measures the entire path.

Blocks and propagation

A **block interval** is the target time between blocks. A **block payload** is the transactions and other data inside a block. Increasing payload or reducing interval sends useful work more often, but gives nodes less time to download and verify it.

Nodes usually use a **gossip network**: each node forwards new data to several peers, which forward it again. Gossip avoids one central broadcaster and survives peer failure, but repeats network traffic. A late block can cause honest producers to build on different tips temporarily, increasing stale work or reorganization risk.

World state and state roots

The **world state** is the current mapping from accounts or object identifiers to values. A full node stores that mapping in a database. The block header includes a compact state root, so another node can verify that execution produced exactly the committed result.

A **state witness** contains values and authentication paths needed for particular reads and writes. It is like giving a checker the few relevant pages of a huge ledger plus seals that connect them to the ledger's signed cover. Stateless validation reduces what the checker stores locally, but a builder or provider must still hold or reconstruct the pages.

Shards and cross-shard messages

A **shard** processes one partition of work. If accounts A and B belong to different shards, their transfer cannot be one ordinary local database update. The source shard records a debit and emits an authenticated message; the destination verifies it before crediting.

This resembles transferring between two banks' ledgers. The receiving bank needs proof that the sending bank finalized the debit, a unique transfer identifier, and a rule preventing the same receipt from being deposited twice. The message is therefore asynchronous: finality and delivery take time.

Committees

A sharded system may assign a subset of validators, called a **committee**, to each shard. Smaller committees process in parallel but give each transaction fewer independent checkers. Random assignment and periodic reshuffling make it harder for an attacker to concentrate validators in one shard.

Committee safety is probabilistic when members are sampled. Later calculations estimate the chance that an attacker controlling a fraction of the total population receives enough seats to control one committee. The formula is

less important than the intuition: larger committees cost more communication but make extreme bad samples rarer.

How to read the formulas

This chapter uses letters as labels for quantities. For example, k shards means the system has k parallel partitions; it does not mean exactly k times useful throughput. Cross-shard traffic, uneven demand, and committee communication reduce the ideal gain.

Units travel through every calculation. MB/s is data divided by time; gas per second is metered computation divided by time. If unlike units are added or a result drops its unit, the calculation is incomplete.

The Four Jobs of a Base Layer

A general-purpose blockchain performs four related jobs:

1. **Execution** - applying transactions to the current state.
2. **Settlement** - deciding which state transition is canonical and resolving disputes.
3. **Consensus** - ordering blocks and finalizing them despite faulty participants.
4. **Data availability** - ensuring that the data needed to verify a block can be obtained.

A monolithic chain performs all four within one validator network. Every full validator repeats much of the same work. Scaling one job can expose another bottleneck: faster execution is of limited value if block propagation or consensus cannot keep up.

Vertical Scaling: Making One Chain Faster

Vertical scaling improves the capacity of a single chain through:

- more efficient clients and databases;
- pipelined block production and execution;
- signature aggregation;
- faster peer-to-peer block propagation;
- a more efficient virtual machine;
- larger blocks or higher gas limits.

These changes matter, but are bounded by validator hardware and network capacity. Larger blocks take longer to propagate and verify. Higher state growth makes it harder for a new node to synchronize. Performance should

therefore be reported with latency, hardware, state growth, and validator distribution, not TPS alone.

Gas per second is useful for EVM systems because it measures computational work rather than treating a simple transfer and a complex smart-contract call as identical. It still does not capture data availability or consensus capacity.

Horizontal Scaling Through Sharding

A cross-shard transfer is an authenticated message

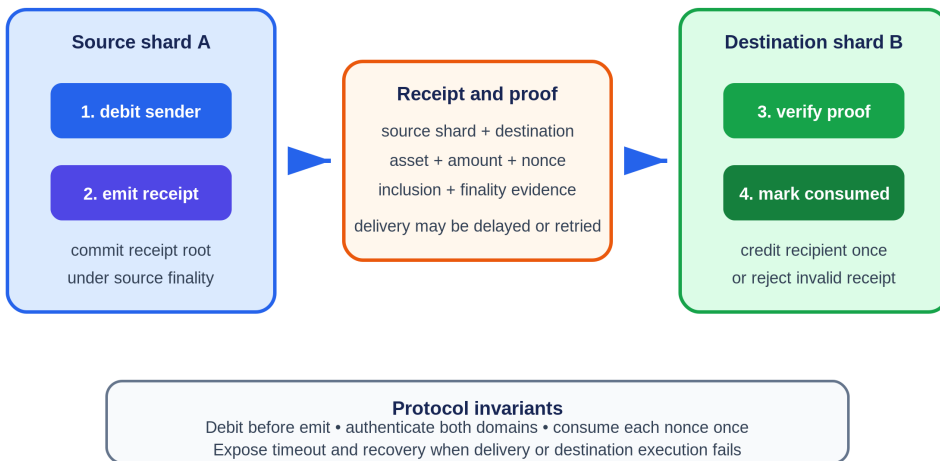


Figure 4.1: A source shard debits funds and emits a receipt; the destination verifies finality and inclusion, marks the nonce consumed, and credits once. Original figure for this book, based on the sharding workflow in SC6019 Lecture 02.

Sharding divides the system into groups that process different portions of the workload. Rather than every node storing and executing every transaction, nodes are assigned to subsets called **shards**.

- **Network sharding** divides validators into committees.
- **Transaction sharding** assigns transactions to committees.
- **State sharding** divides the world state.
- **Execution sharding** processes independent transitions in parallel.

If there are k shards operating concurrently, aggregate throughput can grow with k . The benefit is horizontal: adding validators can create more processing capacity instead of more replicas of the same work.

Random Validator Assignment

A shard committee is easier to attack than the full network because it is smaller. Random assignment makes it difficult for an adversary to choose the shard it controls. Periodic reshuffling limits long-lived capture.

The security question becomes probabilistic. More, smaller shards improve parallelism but reduce each committee's margin. Sharding changes the point at which the scalability trilemma is managed; it does not eliminate it.

Cross-Shard Transactions

Transactions contained within one shard are simpler. A transaction that reads or writes state across shards requires coordination. A common design uses asynchronous messages:

1. the source shard executes the first part;
2. it emits a receipt;
3. the destination verifies and applies the receipt later.

This preserves parallelism but changes application semantics. Developers must account for delayed completion, partial failure, and asynchronous calls. Atomic composability is harder across shards.

State Validity and Data Availability

Validators outside a shard need confidence that its transition is valid and that the underlying data exists. Techniques include fraud proofs, validity proofs, erasure coding, data availability sampling, and stateless validation with witnesses.

NEAR's Nightshade design represents shards as chunks within one logical chain. Cross-shard actions create receipts, while validators rotate responsibilities for chunk production and validation.¹

Interoperability as a Form of Sharding

The course compares sharding with an ecosystem of independent chains. Cosmos zones process separate state and communicate through Inter-Blockchain Communication (IBC). IBC uses light-client verification and packet commitments to authenticate messages between chains.²

Both models divide computation and state. Their security differs:

- protocol shards normally inherit one validator set;
- sovereign chains choose their own security and governance;
- shared-security systems sit between them.

Cross-chain communication introduces more assumptions. A bridge is only as safe as its verification mechanism, validator set, upgrade keys, and the chains on both sides.

Named Case Study: NEAR Receipts and the IBC Packet Lifecycle

Deployment labels: NEAR sharding and Inter-Blockchain Communication (IBC) are production. Both turn a user action into asynchronous work, but at different boundaries. NEAR receipts move work between shards inside one protocol. IBC packets move authenticated application data between sovereign chains through light-client proofs and relayers. In neither system does "sent" mean the remote state has already changed.

NEAR: a transfer becomes receipts across shards

NEAR divides state and execution across shards while blocks contain shard chunks. A signed transaction is routed to the shard holding the signer's account. Executing that transaction can create **receipts**, which are protocol objects carrying actions or data to another account and possibly another shard. NEAR's official transaction lifecycle explicitly separates the block where a transaction arrives, a later block where a receipt is processed, further function-call receipts, and a final refund receipt.³

Trace Alice calling a contract on a different shard. Alice signs a transaction with her account, public-key context, nonce, receiver, actions, recent block hash, and fee-related fields. The transaction reaches the shard responsible for Alice. That shard validates authorization and nonce, charges the attached resources under the protocol rules, and converts the cross-shard action into an outgoing receipt addressed to the receiver. The source chunk commits to the outgoing work. The destination shard later receives the receipt through NEAR's routed receipt mechanism and executes the contract call against destination state.

The destination call may produce another receipt. For example, a marketplace contract on shard B can call a token contract on shard C and then receive a callback. Gas or deposit refunds are also receipts. The user's one transaction hash can therefore lead to a tree or chain of outcomes. NEAR documentation warns that transaction finality and receipt completion are not the same event: a transaction can be final while its generated receipts are still being processed.^{3 4}

Observable evidence should include the transaction outcome, receipt IDs, predecessor and receiver accounts, block/chunk locations, execution status for each receipt, logs, generated child receipts, and final refund. A wallet that

shows only Alice's top-level transaction as successful can hide a failed remote function call. The safe status model is **transaction accepted, source execution complete, cross-shard receipt routed, destination execution complete, callbacks complete, and refund complete**.

Failure path: the destination contract panics. The source transaction and receipt creation can remain final while the destination execution records failure. The protocol does not roll back an already committed cross-shard history as if it were one database transaction. Application code must use callback results and idempotent state transitions. If a destination shard is temporarily congested, receipts queue and completion latency rises. If a resharding boundary moves accounts, routing and pending receipts must migrate together; otherwise a receipt can be lost or executed twice. If Alice resubmits because the interface still says pending, nonce and receipt identifiers must prevent accidental duplicate action.

The trust assumptions are those of one NEAR protocol: validator consensus, correct chunk validation and availability, deterministic runtime execution, routing, and shard-state handoff. Users do not choose an external relayer for an ordinary cross-shard receipt. This tighter integration improves uniformity, but a bug in the shared runtime, routing rules, or resharding logic can affect all applications using the mechanism.

IBC: send, relay, receive, acknowledge, or time out

IBC connects chains through on-chain light clients, connections, channels, packet commitments, and permissionless relayers. The packet lifecycle documentation names four application-visible stages: send on the source, receive on the destination, acknowledgement on the source, and timeout on the source.^{5 6}

Trace a fungible-token transfer from Chain A to Chain B. The application on A escrows or burns the source representation under the token-transfer rules and calls the IBC channel to send a packet. The packet binds source and destination ports and channels, a sequence, payload, and a timeout height or timestamp. Chain A stores a commitment to the packet. This commitment is the fact a relayer later proves; no trusted courier signature is required.

A relayer observes the committed packet, waits for the source state needed by the destination's light client, and submits a receive message to Chain B with a proof. The IBC handler on B verifies that proof against its client of A, checks channel and sequence rules, prevents duplicate receipt, and invokes the destination application. The application writes an acknowledgement indicating success or an encoded error. Chain B commits that acknowledgement.

A relay then submits the acknowledgement and proof back to Chain A. A verifies it through its light client of B, deletes or completes the packet commitment under the protocol rule, and calls the source application acknowledgement callback. The token application can finalize accounting and emit a user-visible result. Relayers may be paid or operated by third parties, but they do not gain authority to forge a packet that fails light-client, channel, sequence, and commitment checks.

If no valid receive occurs before the packet's timeout, a relay or user submits a timeout proof to Chain A. The proof establishes that the destination passed the timeout condition without receiving the packet, under the channel rules. The source application runs its timeout callback and can unescrow or restore the user's asset. Receive and timeout race across two chains, so the proof rules and packet-receipt state ensure only one terminal outcome succeeds.

Observable evidence includes the source transaction, packet sequence and commitment, client heights, relay transactions, destination receipt, acknowledgement bytes, source acknowledgement callback, and any timeout proof. "Relayed" is ambiguous: it must say whether a receive proof was submitted, verified, executed, acknowledged, and processed back at the source. The current channel specification defines three order modes. A strictly ordered channel requires delivery in sequence and closes after a packet times out. An `ordered_allow_timeout` channel requires each packet to be received or timed out before the next packet advances, but keeps the channel open. An unordered channel permits independent delivery while still preventing replay.⁶

Failure path: every relay goes offline. Funds are not automatically stolen, but progress stops until someone relays receive or timeout evidence. If Chain B halts before the timeout, Chain A may need a proof tied to a usable client state; client expiry or a frozen client can complicate recovery. If the destination application returns an error acknowledgement, the source callback must refund or unwind according to application rules rather than calling the packet a success. If a chain's consensus safety fails, its light-client security assumption fails too. IBC authenticates the state of the connected chains; it does not make a compromised source or destination chain honest.

What the comparison teaches

Boundary	NEAR cross-shard receipt	IBC packet
Security domain	One sharded protocol and validator system	Two sovereign chains plus on-chain light clients
Transport actor	Protocol routes receipts between shard chunks	Permissionless relayers carry proofs and messages

Boundary	NEAR cross-shard receipt	IBC packet
Unique work item	Receipt ID and receipt dependency graph	Port/channel pair and packet sequence
Completion	Destination receipt and all required callbacks/refunds execute	Receive plus acknowledgement processed, or valid timeout executes
Replay defense	Runtime receipt identity, nonce and execution state	Packet commitment, receipt/ack state, sequence and channel rules
Main liveness failure	Congested or unavailable shard/chunk path	No relayer, halted chain, expired/frozen client, unavailable proof
Application obligation	Handle asynchronous promise result and callback failure	Handle acknowledgement error and mutually exclusive timeout/refund

The shared programming rule is to record intent before sending asynchronous work, make the remote handler idempotent, bind callbacks to the original request, and expose a terminal error or timeout path. The major difference is proof scope. NEAR consensus knows the source and destination shards. IBC's destination verifies a proof of source-chain state through a client, then the source verifies destination acknowledgement or timeout evidence in return.

Ethereum's Rollup-Centric Data Sharding

Early Ethereum roadmaps emphasized execution shards. The rollup-centric roadmap shifted priority toward data capacity. Rollups execute outside Ethereum but publish data needed for reconstruction and verification. Increasing cheap data capacity can scale many rollups at once.

EIP-4844 introduced blob-carrying transactions, or proto-danksharding. Blob data is committed to by consensus but unavailable to EVM execution and retained only for a defined period. It provides a separate data market for rollups.⁷

Ethereum activated PeerDAS in the Fusaka upgrade on December 3, 2025. Under EIP-7594, blob data is erasure-extended into cells and columns, nodes custody a deterministic subset, and nodes sample columns from peers rather than every node downloading every blob. A node can reconstruct the data matrix from at least half of the columns under the specified coding scheme.⁸

⁹ Full danksharding remains a roadmap direction for expanding data capacity further.¹⁰ This is Layer 1 scaling designed primarily to support Layer 2 execution.

Trade-Offs

Technique	Main Gain	Main Risk
Larger or faster blocks	More transactions per unit time	Higher node requirements and propagation pressure
Client or VM optimization	More work from the same hardware	Network or storage becomes the next bottleneck
Sharding	Parallel execution and storage	Smaller security domains and cross-shard complexity
Appchains	Isolation and sovereignty	Fragmented liquidity and separate security
Data sharding	More capacity for rollups	Sampling, coding, and networking complexity

A production protocol must handle committee corruption, data withholding, partitions, validator churn, and state synchronization. The recovery path often determines real security.

Worked Example: A Cross-Shard Transfer

Consider Alice on shard A paying Bob on shard B. Shard B must not credit Bob unless shard A has irrevocably debited Alice. A practical design uses an authenticated receipt. Shard A verifies Alice's signature and balance, debits ten tokens, and places a receipt in an outgoing queue. The receipt names the destination, Bob's account, the amount, a nonce, and proof that shard A finalized the debit. Shard B later verifies the proof and consumes the receipt exactly once. The nonce prevents replay.

The transfer is not atomic in the database sense. Between debit and credit, the payment is in flight. Economic conservation requires a protocol-defined terminal outcome such as exactly-once credit or an authenticated cancellation and refund. If the destination can reject a receipt, the design must say which evidence authorizes the refund and why a late receipt cannot also credit Bob. Applications must expose the intermediate state. Congestion on shard B can delay delivery, and a contract on shard A cannot assume an immediate callback from shard B. Cross-shard calls therefore look more like authenticated message passing than like calls inside one EVM transaction.

The protocol must price receipt queues, bound their growth, handle reorganizations, and define what happens when a destination rejects a message. Aggregate shard throughput alone hides these costs.

Committee Security by Calculation

Suppose 1,000 validators include 250 controlled by an adversary. The protocol samples a committee of 100 uniformly without replacement and loses the relevant BFT safety condition if at least 34 committee members are Byzantine. If x is the number of adversarial members, then x is hypergeometric:

```
P[X >= 34]
= sum from x=34 to 100 of
  C(250, x) C(750, 100-x) / C(1000, 100)
approx 0.02144, or 2.14%
```

The expected adversarial count is only 25, so an average alone conceals a roughly one-in-47 committee-capture probability under these assumptions. The calculation does not cover stake-weighted sampling, several committees per epoch, repeated epochs, adaptive corruption, or correlated control. For m independent committees with per-committee capture probability p , the probability of at least one capture is $1 - (1-p)^m$; real assignments can be dependent, so the protocol must derive the joint probability from its actual sampler.

Smaller committees create more parallel shards but widen this tail risk. Rotation limits persistent targeting only under an explicit corruption-delay model, and it costs bandwidth because validators need new shard state. Committee size, reshuffle frequency, validator or stake distribution, correlation, and the exact safety or liveness threshold must be evaluated together.

Implementing a Sharded State Machine

A sharding protocol needs more than a partition function. It must specify which shard owns each state item, how validators learn their assignment, how blocks commit to every shard, and how messages survive validator rotation.

A simple account partition might define $\text{shard}(\text{account}) = \text{hash}(\text{account}) \bmod k$. This balances random account addresses reasonably well, but it cannot balance activity. A popular contract can make one shard hot while others remain idle. More advanced designs move ranges or split state dynamically, but migration itself consumes capacity and changes which committee can authenticate a receipt.

A shard block normally commits to its input messages, transactions, resulting state, and outgoing messages. One conceptual header is:

```
ShardHeader {
  shard_id
  height
```

```

    previous_shard_root
    input_receipts_root
    transactions_root
    new_state_root
    output_receipts_root
    committee_signature
}

```

The roots bind execution to the exact inputs and outputs. The committee signature certifies the header under the shard's fault rule. A beacon chain or common consensus layer can then order shard headers and make their receipts final.

Receipt Processing

A destination shard should treat an incoming receipt like an authenticated, exactly-once message:

```

process(receipt, proof):
    require verify_source_finality(receipt.source_header, proof)
    require receipt.destination_shard == this_shard
    require not consumed(receipt.id)
    require receipt.expiry >= current_height

    mark_consumed(receipt.id)
    result = execute(receipt.payload)
    emit acknowledgement(receipt.id, result)

```

The consumed marker prevents replay. Marking before external execution prevents reentrancy-like duplicate consumption. An expiry bounds queue lifetime, but requires a refund or cancellation rule on the source shard. If acknowledgements can themselves fail, the application needs an idempotent retry path.

Validator Rotation and State Handover

Randomly rotating validators frustrates adaptive corruption, but a validator joining a shard needs its state. Downloading full state at every epoch can dominate bandwidth. Checkpoints, snapshots, state-sync proofs, and witnesses reduce this cost.

The handover boundary is security-sensitive. The old committee must not sign two final checkpoints, and the new committee must not process receipts against an unfinalized snapshot. Protocols often overlap committees or anchor the transition in a common beacon block.

Sharding Failure Modes

Single-shard capture. An adversary gains enough committee seats to certify an invalid transition. Random assignment, adequate committee size, slashing, and cross-shard fraud or validity proofs reduce this risk.

Data withholding. A committee certifies a header but withholds the body. Other shards cannot verify receipts. Erasure coding and availability attestations make withholding detectable.

Receipt flooding. One shard creates more cross-shard messages than another can process. Per-destination queues, fees, rate limits, and backpressure prevent unbounded memory.

Hot shard. A popular application concentrates load. Dynamic resharding, application-level partitioning, and asynchronous subcomponents can redistribute work, but cannot make a single sequential state variable parallel.

Correlated validators. Random keys are not independent operators. If many validators share hosting, software, or control, committee probability computed from keys overstates resilience.

L1 Engineering Checklist

Before claiming horizontal scale, a design should answer:

1. Which state and transactions belong to each shard?
2. What is the committee-selection distribution and corruption model?
3. How are cross-shard receipts authenticated, replay-protected, and expired?
4. What happens to an in-flight receipt across reorganization or resharding?
5. How does a new validator obtain state and prove its correctness?
6. How is unavailable shard data detected and recovered?
7. Which workloads create hot shards?
8. Does aggregate throughput include cross-shard communication and state sync?

State Sharding Design Choices

A state-sharded chain must choose a partition key. Hash partitioning spreads addresses evenly but ignores application locality. Range partitioning keeps related keys together but can create hot ranges. Application-aware partitioning can reduce cross-shard calls but gives protocol designers or developers more responsibility.

Contracts complicate ownership. If a contract's code lives on one shard but its users' balances live elsewhere, calls become messages. If the complete

contract state stays together, a popular application becomes a hot shard. If storage slots are split, one contract invocation may need several asynchronous reads.

There is no partition function that makes every workload local. A protocol can expose placement hints, support dynamic resharding, or encourage applications to model state as independent objects. Each choice moves complexity between the protocol and application.

Dynamic Resharding

Dynamic resharding splits a busy shard or combines quiet shards. A safe split needs a finalized boundary:

1. choose a source checkpoint;
2. partition its state into child commitments;
3. assign new committees;
4. route new transactions by the new mapping;
5. forward or transform receipts created under the old mapping;
6. retain proofs linking child roots to the source root.

During transition, clients may hold stale routing information. Gateways can forward requests, but signatures and receipts should bind logical destination state rather than a transient network endpoint. Migrations also need back-pressure; moving a large state database while processing peak load can worsen the congestion that triggered the split.

Stateless Validation and Witnesses

A stateless validator receives the values and authentication paths needed for a block's reads. Starting from the prior state root, it verifies each witness, executes the block, applies writes, and obtains the new root.

If an account proof contains $O(\log n)$ hashes, a block touching many unrelated accounts carries many paths. Multiproofs share common branches. Verkle trees use vector commitments to reduce witness size for wide state. Smaller witnesses lower validator storage needs but increase builder duties and cryptographic verification.

Witness availability becomes part of block validity. A proposer that announces a header without witnesses can stall validators even when transaction data is available. Builders need full or distributed state access to construct witnesses, creating a risk that cheap validation centralizes production.

Cross-Shard Contract Pattern: Request and Callback

Synchronous code might write:


```
price = oracle.read(pair)
settle_trade(price)
```

Across shards, the caller sends a request and returns. The oracle later sends a callback:

```
request_id = send oracle.read(pair)
store PendingTrade(request_id, user, limits, expiry)

on_oracle_reply(request_id, price):
    trade = load PendingTrade(request_id)
    require not trade.completed
    require now <= trade.expiry
    require price satisfies trade.limits
    mark trade.completed
    settle_trade(price)
```

The application must handle duplicate, late, missing, and reordered replies. It should not lock unrelated global state while waiting. This pattern resembles distributed services, with the added need for authenticated receipts and deterministic execution.

Sharding and Composability Economics

Cross-shard calls consume source execution, consensus on the source receipt, network delivery, destination verification, and destination execution. Pricing only the source call subsidizes remote work and invites flooding. A fee can reserve destination capacity or let the receipt carry a budget refunded when unused.

Developers then face locality economics. Contracts interacting frequently have an incentive to share a shard; popular clusters become hot. Dynamic placement may improve throughput but changes latency and fees. Tooling should profile cross-shard call graphs before deployment and show developers which state edges dominate cost.

Stateless Validation: Witness Construction and Update

Stateless validation replaces a validator's full local state lookup with a witness proving the values a block reads and changes. It reduces validation storage requirements, but moves witness production and bandwidth into the critical path.

Assume the prior state is committed by root R_0 . A transaction reads account A, checks its nonce and balance, debits it, and credits account B. The block builder supplies values plus authentication paths from each touched key to R_0 .

```

StateWitness {
    pre_state_root,
    keys[],
    pre_values[],
    proof_nodes[],
    access_order,
    commitment_version
}

```

The validator verifies each pre-value under R_0 , executes the canonical transaction, replaces the changed leaves, and recomputes root R_1 . The block is valid only if R_1 equals its declared post-state root.

Multiproofs

Independent Merkle proofs repeat nodes near the root. A multiproof shares common branches across many keys. Witness size depends on how accesses cluster: 1,000 adjacent keys can share more path material than 1,000 uniformly random keys.

Canonical multiproof encoding matters. The verifier needs an unambiguous rule for node order, omitted siblings, duplicate keys, empty values, and tree depth. Two encodings that prove the same leaves may be acceptable, but they must calculate one root and have bounded decoding work.

Reads after writes

A block can read a value written by an earlier transaction in the same block. The witness authenticates the value at the block's pre-state; the executor's in-block state overlay supplies later reads. Builders should not provide a second contradictory "pre-state" proof for a value already changed in the block.

Access order and transaction order determine this overlay. Parallel execution may speculate, but final witness verification must reproduce canonical semantics.

Missing and extra witness data

A missing proof node makes the block unverifiable and should reject it. Extra unused nodes consume bandwidth and parsing work; charge or cap them to prevent denial of service. Duplicate or cyclic references in a compressed witness must not cause unbounded memory or CPU.

The runtime can discover an undeclared key through a dynamic contract call. The block builder must include its witness. Access lists can help prefetching, but consensus validity follows actual execution, not a sender's incomplete prediction.

State providers and builder concentration

Validators can be stateless while someone still stores state to build blocks and witnesses. If one state provider serves every builder, storage centralization moves rather than disappears. A production design supports multiple providers, snapshot reconstruction, authenticated queries, and a way for a new builder to acquire current state.

A provider cannot forge a value when proofs are checked, but it can censor or selectively delay keys. Builders need redundant queries and enough time to assemble witnesses before proposal deadlines.

Witness bandwidth calculation

Suppose a block executes 2,000 transactions, touching 3 unique state keys each after deduplication. An average multiproof contribution of 180 bytes per unique key gives:

$$2,000 \times 3 \times 180 \text{ B} = 1.08 \text{ MB}$$

If blocks arrive every 2 seconds, witness payload alone averages 540 kB/s before transaction data, signatures, networking overhead, and bursts. A hot workload may reduce unique keys and witness bytes while increasing execution contention; a random workload does the opposite.

Do not assume witness size scales linearly from one transaction. Measure deduplication, path sharing, code proofs, and the current tree shape. Report compressed and uncompressed size plus verification CPU.

Witness generation pipeline

A builder:

1. selects ordered transactions;
2. simulates or executes to discover actual accesses;
3. fetches pre-state values and proof nodes;
4. deduplicates keys and builds a canonical multiproof;
5. re-executes against the witness as a validator would;
6. checks the resulting root;
7. publishes transactions and witness before the proposal deadline.

Late access discovery can force another state query and delay the block. Builders can prefetch from access lists or recent traces, but they must validate fetched proofs against the current root.

Witness test matrix

Test:

- absent accounts versus zero-valued accounts;
- repeated read and write of one key;
- contract creation and deletion semantics;
- dynamic code and storage access;
- a read after an earlier in-block write;
- duplicate, reordered, extra, and missing proof nodes;
- maximum depth and largest code witness;
- prior-root mismatch after a reorganization;
- parallel execution followed by canonical root update;
- builder restart after state selection but before publication.

Differentially compare full-state execution with witness-only execution for generated blocks. Assert identical receipts, gas, logs, and post-state root. Then delete the full state from the validator environment and confirm that every required input is present in the published witness.

Statelessness succeeds when ordinary validators can verify and synchronize cheaply while block building and state service remain competitive, observable, and recoverable.

State Expiry and Revival Operations

State expiry bounds the active state by removing data that has not been touched for a defined horizon. It is different from history expiry: history concerns old blocks and receipts, while state determines what the next transaction can read and change.

An expiry design needs four precise rules:

1. which objects expire: accounts, storage slots, contract code, or tree segments;
2. which event refreshes them: read, write, proof submission, or fee payment;
3. which commitment preserves expired values;
4. how a transaction revives data and who supplies the proof.

If these rules are vague, clients can disagree about whether a key is active and derive different state roots.

Epoch transition

Consider 90-day expiry epochs. At the transition from epoch e to $e+1$, untouched leaves are removed from the active tree, but their values remain committed by an expiry accumulator root E_e . Active state root A_e and expiry roots jointly define the valid state universe.

```
BlockState {
    active_root,
    expiry_roots[],
    expiry_policy_version,
    current_epoch
}
```

The number and retention of expiry roots must be bounded. Otherwise a client trades one ever-growing state structure for an ever-growing list of commitments.

Revival transaction

Suppose Alice's account expired with nonce 17 and balance 4 ETH. A transaction that needs it carries:

- the historical value;
- a proof to the applicable expiry root;
- enough active-tree path data to insert the revived leaf;
- a transaction whose signature and nonce are checked against the revived value;
- any revival fee required by protocol rules.

The validator verifies the expiry proof, confirms the leaf has not already been revived or superseded, inserts it into active state, and then executes the transaction. An attacker must not be able to revive an older version of a key after a newer version exists.

Worked revival cost

Assume a revival witness includes a 1,250-byte account proof, 600 bytes of code metadata, and 2,400 bytes for four storage slots. If calldata-equivalent accounting prices witness bytes at 8 gas each, the data component is:

$$(1,250 + 600 + 2,400) \text{ B} \times 8 \text{ gas/B} = 34,000 \text{ gas}$$

Add 28,000 gas for proof verification and tree insertion:

$$34,000 + 28,000 = 62,000 \text{ gas}$$

At 20 gwei, that is 0.00124 ETH, before the transaction's ordinary execution. This example is a cost model, not a recommendation: real pricing must track verification CPU, bandwidth, retained-state burden, and denial-of-service limits.

Who retains expired data?

A commitment does not make values available. Users need archival nodes, wallets, applications, state-service markets, or their own backups to recover values and proofs. Protocol designers should state the recovery assumption explicitly.

A wallet that creates an account or contract can retain a compact recovery package: key identifiers, values, code, relevant expiry epoch, and proof-service hints. But proofs can become stale as commitment schemes migrate, so recovery tooling needs versioning and conversion paths.

Applications with many users cannot assume each user will preserve every storage value. They may sponsor state rent, refresh required keys, maintain proof services, or redesign storage so critical claims can be reconstructed from durable logs or user-held receipts.

Incentives and abuse resistance

If touching a key refreshes its lifetime for free, an attacker can cheaply keep a large state set alive. Charge refresh according to the burden imposed, cap witness decoding, and decide whether a read refreshes state or only a write/payment does.

Revival also creates burst risk. A popular dormant application may revive thousands of keys after a news event. Benchmark epoch boundaries and revival storms, not just steady-state transactions.

State providers can extort users through availability rather than invalid data. Mitigations include redundant archives, standardized proof APIs, erasure-coded snapshots, periodic recovery drills, and application-level export tools.

Reorganizations and epoch boundaries

A reorganization across an expiry boundary can change which keys expired and which root authenticates them. Clients must retain rollback information for the maximum reorganization window and avoid deleting values immediately at transition. Finalization should gate irreversible pruning.

A transaction prepared against one expiry root may become stale after a reorganization. Its failure should be explicit and safely retryable; a wallet should fetch a fresh proof rather than repeatedly rebroadcasting invalid data.

Migration and release checklist

Before enabling expiry:

- publish object-level refresh and expiry semantics;
- prove that old values cannot overwrite newer ones;
- provide at least two independent archival/revival implementations;
- test recovery without the original full node;
- measure normal witnesses, revival storms, and epoch transitions;
- specify reorganization rollback and pruning delays;
- version commitment and proof formats;
- expose wallet warnings before state becomes costly to revive;
- document which data availability promises are protocol guarantees and which depend on services;
- rehearse commitment migration and archive-provider failure.

State expiry is operationally credible only when a user can return after the horizon, obtain the right data from more than one source, verify it against consensus commitments, and safely resume activity.

History Expiry, Archives, and Verifiable Queries

Live consensus does not require every validator to retain every old block forever. **History expiry** lets ordinary nodes discard old bodies, receipts, or auxiliary indexes after a retention window while preserving commitments needed to authenticate the canonical past.

History expiry differs from state expiry. History explains how state arrived; state contains values execution may need next. A node can prune old receipts while retaining current balances, or expire inactive state while archives retain history.

Data classes

Specify retention separately for:

- block headers and finality evidence;
- block bodies and transactions;
- receipts, logs, and events;
- state snapshots and diffs;
- consensus votes and slashing evidence;
- blobs or external DA payloads;
- indexes derived from canonical data;
- debugging traces that may not be consensus data.

Applications often depend on logs and indexes even when consensus does not. "The chain retains history" is incomplete without class and duration.

Commitments and proofs

A retained header may commit to transaction and receipt roots. An archive can answer a query with the item plus a Merkle proof to the finalized header. The client also needs an authenticated header chain or checkpoint.

An indexer response such as "all transfers by Alice" is harder. A Merkle proof can authenticate returned events but does not prove no matching event was omitted unless the protocol commits to a suitable index. Completeness may require scanning every relevant block or using a verifiable indexed structure.

Distinguish:

- **membership:** this item is in committed history;
- **non-membership:** this key is absent from a committed set;
- **completeness:** these are all items matching a query;
- **canonicity:** the containing header is on finalized history.

Archive providers

Archive service is operationally replaceable only when data formats, proofs, and request APIs are open and several providers retain the same history. A single public endpoint backed by one hidden database is a centralized dependency even if responses are authenticated.

Providers can omit or delay data without forging proofs. Clients use redundant sources, content-addressed snapshots, peer exchange, and local verification. Publish retention commitments and measure whether old random ranges remain retrievable.

Portal and peer networks

A distributed history network can partition data across peers and retrieve by content key. Availability depends on replication, incentives, routing, and repair. Cryptographic hashes authenticate content but do not ensure a peer stores it.

Measure unique providers by failure domain, replica count by age, lookup latency, failed ranges, repair time, and survival after high-capacity peers leave.

Pruning safety

A node should prune only after finality and the maximum reorganization window required by policy. Delete in resumable batches and preserve enough metadata to distinguish intentionally pruned data from corruption.

Snapshots used for sync need a verified successor path. Do not delete the only local state needed to reconstruct or roll back a snapshot activation still in progress.

Pruning can race RPC queries and indexers. Return an explicit "pruned before height H" response rather than null, which users may misread as proof that no event exists.

Worked storage budget

Suppose canonical history grows by 3 MB/s. Raw annual growth is:

$$3 \text{ MB/s} \times 31,536,000 \text{ s} \approx 94.6 \text{ TB/year}$$

Ten independent full replicas require about 946 TB before encoding, indexes, backups, and overhead. If ordinary nodes retain 30 days:

$$3 \text{ MB/s} \times 2,592,000 \text{ s} \approx 7.78 \text{ TB}$$

That is still substantial. Compression, data-class separation, and lower-rate historical serving matter. Report measured rather than theoretical compression.

Retrieval economics

Uploading data once does not fund indefinite serving. Archive models include protocol rewards, storage contracts, application payments, institutional archives, and voluntary replication.

Price retrieval separately from retention. A provider paid to store data may still throttle egress during mass exits. Recovery capacity must cover correlated demand, not ordinary query traffic.

If users need old data to withdraw, retention duration and retrieval throughput are part of asset safety. A proof window longer than data retention creates an impossible recovery path.

Migration and format changes

Old history may use earlier transaction, receipt, commitment, or compression formats. Archive software needs versioned decoders and test vectors. Converting data into a new container must preserve the old canonical hash and proof relationship.

Retain original bytes when signatures or hashes cover exact encoding. Semantic reserialization can change hashes even if fields look equal.

Privacy and legal operations

Public history may contain personal or unlawful content that cannot be removed from commitments. Service operators can limit indexing or serving under law, but should not claim the protocol erased what remains reconstructible elsewhere.

Avoid collecting unnecessary off-chain metadata in archive logs. Access patterns can reveal user interests even when chain data is public.

Recovery drill

From a new machine and no privileged database:

1. authenticate a finalized old header;
2. retrieve a random block body, receipt, and blob from independent providers;
3. verify each commitment and canonicity;
4. reconstruct an application query across a range and test completeness;
5. remove the largest provider and repeat under burst load;
6. identify a deliberately unavailable range and exercise escalation or repair.

Release assertions

Publish retention by data class, provider and region diversity, oldest retrievable height, random-range success rate, proof-verification tooling, pruning boundary, format migration policy, and recovery throughput.

History expiry is responsible when ordinary validation becomes cheaper while old evidence remains verifiably recoverable from a plural archive ecosystem for every protocol and user deadline that depends on it.

Mempool Admission and Denial-of-Service Control

Before a transaction reaches a block, nodes receive, validate, store, and relay it in a **mempool**. Mempool capacity is not consensus capacity: attackers can consume CPU, memory, bandwidth, and database lookups with transactions that never become valid blocks.

Admission pipeline

Apply checks from cheapest to most expensive:

1. bound message length and decode canonical fields;
2. check chain ID, version, and basic structure;

3. reject known duplicates and expired transactions;
4. verify fee floor and intrinsic resource bounds;
5. verify signature or authorization;
6. check nonce and obvious balance conditions against current state;
7. run bounded simulation when policy requires it;
8. store and relay under local limits.

Consensus validation still rechecks the transaction in a block. Mempool admission is local resource policy and must not become an undocumented consensus rule that prevents valid forced inclusion.

Cheap-to-send, expensive-to-check

An attacker seeks **asymmetry**: a small request causing large work. Examples include malformed encodings that parse deeply, signatures using expensive paths, contract-account validation, state reads over cold keys, and simulations that allocate memory before reverting.

Cap nesting, lengths, decompression, signatures per envelope, validation gas, state accesses, and concurrent simulations. Cache valid and invalid results only with state/version keys that prevent stale decisions.

Nonces and replacement

Account nonces normally impose order. If nonce 10 is missing, nonces 11-100 may wait. Attackers can create long gaps or repeatedly replace one transaction.

Limit queued future nonces per account and globally. A replacement must increase the relevant fee by a clear rule and preserve nonce and sender. Rate-limit replacement churn so tiny fee increments cannot force peers to repeatedly validate and gossip.

Structured nonce lanes need per-lane caps. Thousands of lanes can bypass a per-account sequential limit unless the account has an aggregate budget.

Fee floors

A local fee floor filters transactions unlikely to pay for scarce resources. It should reflect encoded bytes, execution, state access, and validation cost. One gas scalar may underprice large signatures or complex account validation.

Dynamic floors respond to congestion, but peers can have different policies. Wallets need rejection reasons and current minimums. A low-fee valid transaction may remain eligible for direct block inclusion even if many mempools drop it.

Eviction

When full, evict by an auditable policy using fee value, age, dependencies, and resource shape. Avoid one global fee-per-byte score that lets compute-heavy transactions crowd out bandwidth-efficient work or vice versa.

Evict dependent future-nonce transactions when their required predecessor disappears, or mark them parked with bounded storage. Do not gossip repeated evictions endlessly between peers with incompatible policy.

Per-sender and per-peer limits

One sender can generate many keys, so identity-based limits alone do not stop Sybils. Combine sender, peer, IP/network, resource, and global limits cautiously. Network limits should not permanently exclude shared gateways or privacy networks.

Peers earn relay capacity through useful behavior, but reputation must decay and resist poisoning. A malicious peer should not cause an honest transaction to be globally blacklisted merely by sending it first.

Gossip amplification

In a network of N nodes with average fanout d , naive forwarding can create many duplicate deliveries. Nodes advertise transaction hashes, request unknown bodies, deduplicate, and batch announcements.

Suppose 10,000 nodes each receive 2,000 new transactions per second and an average encoded transaction is 300 bytes. One full copy per node is already:

$$10,000 \times 2,000 \times 300 \text{ B} = 6 \text{ GB/s ecosystem ingress}$$

Duplicates, inventory messages, and signatures add overhead. Measure per-node bandwidth and duplicate ratio under burst, not only average transaction rate.

Reorganizations

Transactions from reverted blocks may return to the mempool if still valid. Recheck nonce, balance, fee, expiry, and conflicts against the new canonical state. Do not blindly restore a prior mempool snapshot.

Large reorganizations can cause a reinsertion storm while nodes also execute the new branch. Prioritize consensus catch-up and bound revalidation work.

Privacy

Public mempools reveal sender, calls, amounts, and fee willingness before inclusion. Private endpoints reduce broadcast but give operators information

and censorship power. Encrypted mempools change payload visibility while retaining size, timing, and ingress metadata.

Document retention and sharing. Mempool logs can contain sensitive pending actions that never appear on-chain.

Observability

Track admitted, rejected, parked, replaced, evicted, expired, and included transactions by reason; validation CPU; state-read latency; memory/bytes; sender and peer concentration; duplicate gossip; oldest eligible age; and inclusion outcomes.

A growing mempool can mean demand exceeds capacity, a producer censors, fees are mispriced, or nonce dependencies are missing. Metrics need these dimensions.

Adversarial tests

Flood malformed maximum-size envelopes, invalid signatures, cold-state checks, nonce gaps, replacement churn, many account-abstraction lanes, low-fee Sybils, conflicting transactions, peer reconnect loops, and reorganization reinsertion.

Assert bounded memory and validation work, continued admission for honest target traffic, no crash or unbounded queue, specific rejection reasons, and consensus catch-up priority.

The mempool is an untrusted network-facing scheduler. Scaling blocks without scaling and defending admission merely moves failure to the system's front door.

Node Synchronization and Snapshot Recovery

A chain has not scaled sustainably if existing validators keep up but a replacement node cannot join, recover, or audit state within an acceptable window. Synchronization is part of the protocol's availability and decentralization budget.

A new node can obtain state in several ways:

- **genesis replay:** download and execute every block from genesis;
- **checkpoint sync:** start from a trusted or consensus-verified finalized checkpoint;
- **snapshot sync:** download a state snapshot committed by a finalized header, verify its chunks, then replay later blocks;
- **state sync:** request authenticated state ranges from peers while following headers;

- **witness sync:** verify new blocks from witnesses without first storing the entire state.

Genesis replay provides the strongest independent historical reconstruction but becomes slow as history and state grow. Snapshot sync reduces time to participation, but it must not turn a convenient file into an unexamined trusted database.

Authenticated snapshot format

A snapshot manifest should bind:

```
SnapshotManifest {
  chain_id,
  protocol_version,
  finalized_height,
  block_hash,
  state_root,
  chunk_count,
  chunk_hashes_root,
  state_encoding_version,
  compression,
  created_at
}
```

Each chunk has an index, uncompressed length, compressed length, content hash, and proof to `chunk_hashes_root`. After decoding all chunks, the client reconstructs the canonical state commitment and checks it against `state_root` from the finalized block.

Chunk hashes detect transfer corruption; the final state-root check detects a malicious but internally consistent snapshot. Both are needed. The decoder must reject duplicate keys, non-canonical encodings, decompression bombs, out-of-range lengths, overlapping ranges, and trailing data.

Checkpoint trust

A hard-coded checkpoint is an explicit trust decision. A weak-subjectivity checkpoint may be needed when former validators can sign an alternative old history after exiting. Operators must know who distributes the checkpoint, its age limit, how it is authenticated, and how to obtain the same value through independent channels.

A BFT finality certificate can authenticate a checkpoint if the client already trusts the applicable validator set and verifies every set transition. A proof-of-work header chain needs its own accumulated-work and eclipse-resistance assumptions. "Fast sync" is not one security model.

Worked catch-up budget

Suppose the active snapshot is 1.2 TB compressed and the node has 400 Mbps usable ingress. Ignoring overhead, transfer time is:

$$1.2 \text{ TB} \times 8 / 400 \text{ Mb/s} = 24,000 \text{ s} \approx 6.7 \text{ hours}$$

If verification and insertion sustain only 25 MB/s, local processing takes:

$$1,200,000 \text{ MB} / 25 \text{ MB/s} = 48,000 \text{ s} \approx 13.3 \text{ hours}$$

Processing, not bandwidth, is the first bottleneck. Meanwhile the chain continues producing blocks. At 3 MB/s of new data, a 13.3-hour sync creates another roughly 144 GB of catch-up traffic. The node must process historical catch-up faster than the chain's live growth or it never converges.

Report time to header verification, snapshot acquisition, state-root verification, head catch-up, and validator readiness separately. "Synced" should mean the node can safely perform its intended role, not merely that it opened a peer connection.

Peer and provider diversity

Downloading chunks from many addresses does not provide diversity if they share one cloud bucket or snapshot producer. Record snapshot producer, serving provider, region, autonomous system, and software implementation. Randomize requests so one peer cannot selectively shape all ranges.

A malicious peer can send valid but useless old chunks, stall the last range, or repeatedly trigger expensive decoding. Use per-peer deadlines, authenticated range selection, bounded concurrent work, resumable progress, and penalties that do not let an attacker evict honest peers cheaply.

Crash consistency

Persist the verified manifest before chunks, record each completed chunk atomically, and make state installation idempotent. On restart, recheck stored chunk hashes and continue; do not silently mix chunks from different manifests or protocol versions.

Install the reconstructed database with an atomic directory or generation switch. If the process crashes during activation, it must restart from either the old database or the complete new one, never a partial mixture. Retain rollback state until the post-snapshot block replay and state-root checks succeed.

Snapshot production

Producing a snapshot can contend with block execution for disk bandwidth and cache. Generate from a consistent finalized database view, not a live collection changing beneath the exporter. Bound memory, stream chunks, and verify the finished artifact using a separate importer before publishing it.

Multiple independent operators should produce snapshots for the same height. Byte-for-byte files may differ because of compression or chunking, but all must reconstruct the same canonical state root.

Recovery drill

A release drill should:

1. remove the node's state database;
2. authenticate a finalized checkpoint through the documented rule;
3. download a snapshot with one corrupt, one delayed, and one unavailable serving peer;
4. crash during chunk verification and again during database activation;
5. resume without redownloading verified data;
6. reconstruct the exact committed state root;
7. replay blocks to the live head while load continues;
8. join validation without missing its operational readiness target.

Measure bytes, CPU, peak memory, disk amplification, random I/O, elapsed time, peer failures, and catch-up margin over live growth. Repeat from at least two geographic regions and on the minimum supported hardware.

Snapshot sync is credible when a skeptical operator can authenticate the checkpoint, reconstruct state from interchangeable providers, survive interruption, and converge faster than the chain grows without copying an incumbent's trust assumptions.

Worked Resharding Trace: Moving a Hot Account Range

Static shards eventually become unbalanced. A popular application can saturate one shard while others remain idle. Dynamic resharding changes the partition map without losing, duplicating, or accepting transactions against two owners of the same state.

Assume shard A owns keys in range $[m, z]$. The protocol will move $[t, z]$ to new shard B at epoch $E+1$. The transition needs an authenticated handoff point.

Prepare

Before the boundary, consensus finalizes a resharding plan:


```
ReshardPlan {
  plan_id,
  source_shard = A,
  destination_shard = B,
  key_range = [t, z],
  freeze_height,
  activation_epoch = E + 1,
  source_state_root,
  protocol_version
}
```

The plan is part of canonical state. Nodes reject a local operator command that changes ownership without this decision. Clients learn the future map early enough to route transactions and update proofs.

At `freeze_height`, shard A stops accepting new writes to the moving range under the old routing version. It finishes earlier transactions and outbound receipts, then commits a root for the frozen range. Reads may continue if the API labels the snapshot and prevents stale writes.

Transfer

Shard A exports state leaves, contract code, storage, pending asynchronous messages, consumed-message nonces, and any rent or metadata required to interpret the range. A snapshot manifest binds chunks to the finalized range root:

```
SnapshotManifest {
  plan_id,
  range_root,
  chunk_commitments[],
  pending_message_root,
  consumed_nonce_root,
  total_bytes
}
```

Shard B retrieves chunks from several peers, verifies every commitment, reconstructs the range, and replays or imports pending message state under a deterministic rule. Importing account balances without replay-protection state would let an old cross-shard receipt execute again after migration.

Activate

At epoch $E+1$, validators agree on a partition map assigning $[t, z]$ to B and an activation commitment produced by the import. Transactions carry or derive the routing version. Shard A rejects new-version writes to the range; shard B rejects old-version writes except explicitly authenticated forwarding messages.

There must be no interval where both shards can finalize ordinary writes to the same key. A forwarding period can improve availability, but forwarding is a message path, not dual ownership. It needs a nonce, expiry, destination binding, and idempotent handling.

Failure cases

Failure	Risk	Required behavior
Snapshot chunk missing	Activation with incomplete state	Delay activation or reconstruct from erasure/replica sources
Source reorganizes before freeze finality	Exported root no longer canonical	Discard snapshot and rebuild from canonical freeze point
Message arrives during freeze	Lost or double-applied callback	Include it before range root or queue under a bound handoff rule
Destination imports wrong code version	Divergent execution after activation	Bind protocol/code hashes and verify import vectors
Both shards accept a write	Conflicting ownership and asset creation	Routing-version and epoch checks reject one path
Validator set changes simultaneously	Handoff certificate ambiguity	Authenticate both set transition and range activation in one canonical boundary
Destination fails after source prunes	Unrecoverable state	Retain source snapshot until activation and recovery window finalizes
Client uses stale partition map	Submission delay or wrong-shard replay	Return authenticated redirect; never accept under ambiguous ownership

Capacity and state transfer

If the moving range contains 400 GB and must be copied within a 45-minute maintenance window, the minimum payload rate is:

$$400 \text{ GB} / 2,700 \text{ s} \approx 148 \text{ MB/s}$$

This excludes encoding expansion, proofs, retransmission, concurrent state changes outside the frozen range, and peer overhead. At a 60 percent planning utilization, provision roughly $148 / 0.6 \approx 247 \text{ MB/s}$ of usable transfer capacity. If that requirement is unrealistic, reduce range size, lengthen the window, pre-copy immutable data, or use incremental snapshots before the final freeze.

The freeze pauses writes, so migration planning must also bound user-visible downtime. A protocol can pre-copy most state while live, then transfer a final delta after freeze. The delta algorithm becomes safety-critical: it must prove that the base snapshot plus ordered delta equals the finalized handoff root.

Resharding assertions

A test harness should generate transactions and cross-shard callbacks continuously while triggering the handoff. It should crash source and destination nodes at every manifest and activation boundary, delay random chunks, reorganize the pre-final freeze block, and restart with stale routing caches.

Assert conservation of balances, one owner per key and epoch, exact message-once semantics, identical imported roots across clients, bounded redirect loops, and the ability to recover before the source prunes. Resharding is complete only when state, pending work, replay protection, validator authentication, routing, and operational recovery move together.

Conclusion

Layer 1 scaling does not mean only making blocks larger. It redesigns execution, storage, networking, data availability, and consensus so the system can grow without excluding independent validators. Sharding provides horizontal capacity; client and protocol optimization push vertical limits.

The emerging architecture is layered: the base layer supplies settlement and verifiable data under its consensus and availability assumptions while execution is parallelized across shards, rollups, and application-specific chains. The next chapter examines moving repeated interaction off the base chain.

References

1. Skidanov, Alex, Illia Polosukhin, and Bowen Wang. "Nightshade: NEAR Protocol Sharding Design." <https://near.org/papers/nightshade>. ↵
2. Cosmos. "IBC Protocol Overview." <https://docs.cosmos.network/ibc/latest/intro>. ↵
3. NEAR Documentation. "Lifecycle of a Transaction." <https://docs.near.org/protocol/transactions/transaction-execution>. ↵ ↵2
4. NEAR Documentation. "Token transfer flow." <https://docs.near.org/protocol/data-flow/token-transfer-flow>. ↵
5. Cosmos IBC Documentation. "IBC Lifecycle." <https://docs.cosmos.network/ibc/next/learn/ibc-lifecycle>. ↵
6. Cosmos IBC Specification. "Channel & Packet Semantics." <https://docs.cosmos.network/ibc/latest/spec/core/ics-004-channel-and-packet-semantics/README>. ↵ ↵2
7. Buterin, Vitalik, et al. "EIP-4844: Shard Blob Transactions." <https://eips.ethereum.org/EIPS/eip-4844>. ↵
8. Ethereum Improvement Proposals. "EIP-7594: PeerDAS, Peer Data Availability Sampling." <https://eips.ethereum.org/EIPS/eip-7594>. ↵
9. Ethereum Foundation. "Fusaka Mainnet Announcement" (November 6, 2025). <https://blog.ethereum.org/2025/11/06/fusaka-mainnet-announcement>. ↵
10. Ethereum.org. "Danksharding." <https://ethereum.org/roadmap/danksharding/>. ↵

Chapter 5: Layer 2 Off-Chain Scalability

Introduction

Layer 2 scaling moves repeated work away from the base blockchain while retaining a defined relationship with it. The Layer 1 chain becomes a court and settlement system rather than the place where every interaction must occur.

The important question is not merely whether transactions happen "off-chain." Centralized exchanges also process activity off-chain. A Layer 2 protocol must explain how users recover assets, how incorrect state is rejected, which data must be available, and which security properties are inherited from Layer 1.

The course groups the main approaches into channels, sidechains and commit-chains, Plasma, and rollups. They differ in participants, data location, dispute mechanisms, and trust assumptions.

Intuition: Signed Receipts Instead of Global Updates

A base chain is expensive because every update is broadcast, checked, and stored by many nodes. Off-chain protocols ask whether participants can exchange evidence privately and involve the chain only for a checkpoint or dispute.

Imagine Alice and Bob playing many rounds of a game. Instead of paying a court to record every score, they both sign the latest score sheet. If they agree at the end, they submit only the final sheet. If Bob submits an older sheet, Alice shows the newer signature during a dispute window.

This simple idea introduces the chapter's core terms:

- A **channel** locks assets or state in an on-chain contract and lets named participants exchange signed updates.
- A **commitment transaction** is a pre-authorized on-chain outcome representing one channel state.
- A **dispute window** is the time allowed to challenge stale or invalid evidence.
- A **timelock** prevents an action until a time or block height, giving another party time to respond.

- A **revocation secret** is evidence that an older Lightning commitment should no longer be used.
- A **watchtower** monitors the chain and responds for a user who is offline.

The chain is the judge of last resort. Safety depends on the contract recognizing the newest valid evidence and users being able to reach it before deadlines.

Payments through intermediaries

A payment channel connects only its participants. To pay someone without a direct channel, a sender can route through intermediaries. Each intermediary needs a guarantee: it should pay the next hop only if it can claim from the previous hop.

A **hash time-locked contract (HTLC)** combines two conditions. The receiver must reveal a secret whose hash matches a known value, and must do so before a deadline. Revealing the secret lets claims propagate backward along the route. Staggered deadlines give each intermediary time to react on-chain if the next hop waits.

Think of several locked boxes in a row that open with the same code. The receiver opens the last box, revealing the code; each intermediary then uses it to open the preceding box. The boxes also have closing times, ordered so an intermediary does not pay out after losing its own chance to claim.

Liquidity, not only connectivity

A route can exist in the network graph and still fail. A channel with 10 units total may have all 10 on the wrong side. **Liquidity** is spendable balance in the needed direction, not total channel capacity.

Routing therefore resembles finding roads that have both a connection and enough remaining cargo allowance. Fees, timelocks, private channels, concurrent payments, and failed attempts make the map imperfect.

Plasma, sidechains, and rollups

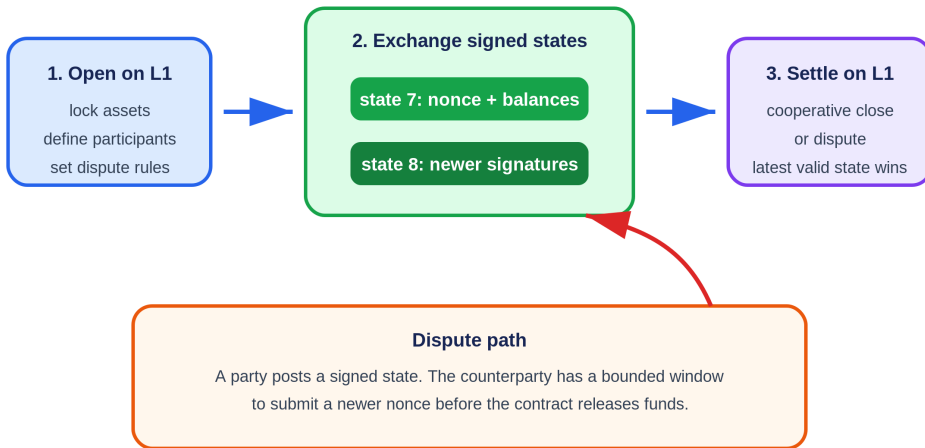
These systems all move activity away from ordinary L1 execution but return different evidence:

- A **sidechain** has its own consensus. A bridge decides when to accept its messages.
- A **Plasma chain** posts commitments while users retain data needed to challenge and exit. Missing data can force many users to leave together.
- A **rollup** publishes transaction data, or enough recoverable data under its model, and lets the base layer reject invalid state through a fault proof or validity proof.

The beginner's test is: if the off-chain operator disappears today, what evidence does the user already have, what data can they obtain, which contract can they call, and how long do they have?

The General Off-Chain Pattern

A state channel moves repeated updates off-chain



Security depends on authenticated state, monotonic nonces, monitoring, and affordable L1 access.

Figure 5.1: A channel locks assets on Layer 1, advances through newer jointly signed states, and returns to Layer 1 for cooperative settlement or dispute. Original figure for this book, based on the state-channel protocol described in SC6019 Lecture 03.

Most Layer 2 systems follow three steps:

- 1. Lock or represent assets on Layer 1.** A contract defines the rules and custody boundary.
- 2. Update state elsewhere.** Participants exchange signed messages or an operator publishes batches.
- 3. Settle or exit on Layer 1.** The latest valid state is enforced, or an invalid proposal can be challenged.

Moving work off-chain reduces base-layer computation. The challenge is making the cheaper path safe when another participant disappears, censors a transaction, or submits dishonest state.

Payment Channels

A payment channel allows two parties to transact repeatedly without placing every payment on-chain. They fund a shared output or contract. Each off-chain payment creates a newly signed allocation. Only opening, cooperative closing, or a dispute needs Layer 1.

The Bitcoin Lightning Network connects bilateral channels. A payment can be routed through intermediate nodes using hashed timelock contracts. The receiver reveals a secret to claim the payment, and the same secret lets each upstream hop settle. Timelocks protect participants if the route does not complete.¹

Strengths

- low marginal cost for repeated payments;
- fast confirmation between online participants;
- limited data burden on the base chain;
- more privacy than publishing every update on-chain.

Limitations

- Payment channels require capital to be locked in advance;
- participants or watchtowers must monitor the chain during dispute windows;
- routing depends on available liquidity along an entire path;
- channels work best for repeated interactions, not arbitrary one-off users.

Generalized State Channels

Ethereum-style state channels extend the idea beyond payments. Participants can update the state of a game, exchange, or application by signing messages. If everyone cooperates, only the final result reaches Layer 1. If they disagree, the contract examines signed evidence and resolves the dispute.

Application-specific channels encode one state machine. Generalized channels let participants install different applications inside a persistent channel. Sprites showed that a global contract can reduce the collateral lock-up time of multi-hop payments compared with purely local timelocks.²

Channels offer excellent performance inside a stable participant set. They are less suitable when membership changes frequently or when an application requires shared global state.

Sidechains and Commit-Chains

A sidechain is a separate blockchain connected to the base chain by a bridge. It has its own consensus and normally its own security budget. Users may gain low fees and fast blocks, but they do not automatically inherit Layer 1 security. If the sidechain validator set or bridge is compromised, the base chain may be unable to distinguish honest and dishonest withdrawals.

A commit-chain uses an operator or committee to order off-chain transactions and periodically commits a state root to Layer 1. Users retain signed evidence and may be able to exit through the base contract. The exact guarantee depends on whether transaction data is published and whether invalid commitments can be challenged.

These categories describe architecture, not a marketing label. A system should be evaluated by asking:

- Who orders transactions?
- Who can censor users?
- Where is transaction data stored?
- Can any user reconstruct the state?
- What must a user monitor to exit safely?
- Which keys can upgrade the contracts?

Named Case Study: Polygon Chain Has Independent Consensus

Deployment label: production. Naming checked September 2026. Current documentation calls the network **Polygon Chain**, while many bridge interfaces and documentation paths retain the **Polygon PoS** name. It provides EVM-compatible execution through its own validator and block-production system, connected to Ethereum by staking, checkpoint, and bridge contracts. The documented chain architecture has two operating layers: Heimdall-v2 for consensus and finality, and Bor for block production and EVM execution.³

⁴ This makes it a useful contrast with the rollups in Chapter 6.

Trace Maya depositing ETH, making a Polygon payment, and withdrawing. On Ethereum, Maya calls the PoS bridge and locks the asset. The bridge event is observed and processed through Polygon's state-sync path, which delivers a corresponding message to Polygon Chain. After that message executes, Maya receives the mapped representation on Polygon. This first leg already crosses two security domains: Ethereum finalized the escrow event, while Polygon validators and bridge logic must recognize and apply it exactly once.

Maya sends the payment to a Polygon RPC endpoint. Bor block producers order and execute EVM transactions. Heimdall-v2 validators participate in the network's validation and checkpoint process. A checkpoint summarizes a span of Bor blocks with a Merkle commitment and is proposed, validated in Heimdall, and submitted to the checkpoint contract on Ethereum. Ethereum records the checkpoint acknowledgement. The checkpoint lets a later bridge proof refer to Polygon events without asking Ethereum to execute every Polygon transaction.⁴

For the return path, Maya initiates a withdrawal on Polygon by burning or otherwise invoking the mapped-token exit logic. She waits for the Bor block containing the exit event to be covered by an accepted Ethereum checkpoint. She then submits the required Merkle proof to the Ethereum bridge so it can verify event inclusion and release the escrowed asset. The bridge marks the exit as consumed to prevent replay.⁶

The important contrast is correctness. A rollup posts data and a state claim that Ethereum can accept or reject through a validity proof or fault-proof rule. Polygon Chain reaches transaction consensus with its own validator set; Ethereum checkpoint contracts authenticate commitments produced through that system, but do not run a rollup proof that every Bor state transition followed EVM rules. Ethereum custody and checkpoints strengthen the bridge boundary. They do not make Polygon execution inherit Ethereum's rollup security model.

The user-visible stages should therefore be explicit: Ethereum deposit included; deposit final enough for policy; Polygon state-sync message observed; Polygon balance minted; Polygon payment included; payment checkpointed to Ethereum; withdrawal burn included; containing checkpoint accepted; exit proof submitted; Ethereum asset released. "On Ethereum" can describe the staking and checkpoint contracts while still leaving Polygon consensus as a separate assumption.

Failure path: Bor stops producing blocks. Polygon transactions and withdrawals stop advancing even if Ethereum continues normally. If Heimdall cannot validate or submit a checkpoint, Polygon may continue producing some local history while new exits cannot obtain the Ethereum checkpoint evidence required by the bridge. Polygon's checkpoint documentation includes an acknowledgement path and a missing-acknowledgement path precisely because submission and recognition are separate operational events.⁴ If enough Polygon validator power violates safety, a fraudulent or conflicting history can threaten the bridge according to its verification and governance rules. If bridge contracts contain a bug or an upgrade authority is compromised, correct Polygon consensus does not protect Ethereum escrow.

Now compare three designs on operator failure. In a payment channel, Maya can use signed states to close on-chain under the channel contract. In an optimistic rollup, she can force data through L1 and rely on an honest challenge against invalid state. In Polygon Chain, she depends on Polygon validators, checkpoint production, bridge proof machinery, and the bridge's pause and upgrade controls. A checkpoint delay is mainly liveness; acceptance of a bad authenticated commitment or a bridge-verification bug can be a safety failure.

Property	Polygon Chain	Optimistic rollup	Validity rollup
Transaction consensus	Polygon validator/ Bor-Heimdall system	Rollup ordering plus Ethereum-enforced dispute rule	Rollup ordering plus Ethereum-verified validity proof
Data needed to reconstruct state	Served by Polygon network and archives under its protocol	Published to the spe- cified DA path for chal- lengers and users	Published to the spe- cified DA path for users and future state
Ethereum evi- dence	Checkpoints and bridge proofs	Batch data, assertions and fault-proof out- come	Batch data/commit- ments and validity proof
Canonical exit depends on	Polygon checkpoint, valid event proof, bridge contracts	Confirmed assertion after challenge rule, bridge contracts	Verified proof/accep- ted state, bridge con- tracts
Main independ- ent security as- sumption	Polygon validator and bridge governance system	At least one effective honest challenger plus correct contracts/data	Sound proof system, verifier, data and cor- rect contracts

The classification is not an insult or a quality ranking. It tells Maya which failure she must survive. A production sidechain can be fast, inexpensive, and widely used. The engineering mistake is to call its Ethereum checkpoints "rollup settlement" and then omit the validator-set and bridge assumptions from a risk review.

Plasma

Plasma creates child chains whose operators periodically commit Merkle roots to Ethereum. Users can prove ownership of outputs and withdraw to the root contract. If an operator publishes an invalid spend or withholds data, users enter an exit process and challenge dishonest claims.⁷

Plasma Cash assigns each deposit a unique position. A user follows the history of that coin rather than the entire child chain. This improves verification, but mass exits and data withholding remain difficult. Complex smart-contract state is also harder to represent than simple payments or non-fungible outputs.

Plasma made a major conceptual contribution: computation can happen elsewhere while Layer 1 enforces exits. Rollups improved the model by publishing transaction data on-chain, allowing anyone to reconstruct the state instead of forcing each user to retain a personal history.

Rollups for General-Purpose Layer 2 Execution

A rollup executes transactions outside Layer 1 and posts compressed transaction data plus a state commitment to Layer 1. Because the data is available, independent nodes can reconstruct and verify the rollup state.

- **Optimistic rollups** accept state commitments by default and use fraud proofs to reject invalid transitions.
- **Validity rollups** submit cryptographic proofs that the new state follows from the previous state and the published inputs.

Rollups reduce cost by amortizing Layer 1 data and verification across a batch. They support general-purpose smart contracts more naturally than channels or Plasma. Chapter 6 studies their sequencers, bridges, proving systems, and security models in detail.

Comparing Off-Chain Approaches

Approach	Best Fit	Data Location	Main Security Requirement	Main Limitation
Payment channel	Repeated payments	Participants	Timely dispute and channel liquidity	Fixed capital and routing
State channel	Repeated interactions among known users	Participants	Signed updates and dispute contract	Changing participants/shared state
Sidechain	Independent high-throughput chain	Sidechain	Sidechain consensus and bridge	Does not inherit L1 security
Plasma	Payments and owned outputs	Operator/users	Users retain proofs and can exit	Data withholding and mass exits
Rollup	General smart contracts	Published to L1 or approved DA layer	Fraud or validity proof system	Sequencing, proving, and bridge risk

Security Is a Spectrum

The phrase "secured by Ethereum" can hide important differences. A mature assessment separates:

1. **State validity** - can an invalid transition be finalized?
2. **Data availability** - can users obtain the inputs needed to verify or exit?
3. **Censorship resistance** - can users force a transaction or withdrawal through Layer 1?
4. **Finality** - when is a transaction economically and cryptographically irreversible?
5. **Upgrade control** - can a small group change the contracts or proof rules?

A protocol may use Ethereum for settlement while relying on a centralized sequencer for liveness, an external committee for data, or a multisig for upgrades. These do not make the system useless, but they must be visible in the trust model.

Worked Example: Closing a Channel Against an Old State

Alice and Bob deposit five tokens each. After several updates, their latest signed state pays Alice two and Bob eight. Bob disappears, so Alice submits the latest state to the adjudicator contract.

Suppose Bob returns and submits an older state paying him nine. The contract cannot identify freshness from balances. Every update therefore carries a monotonically increasing sequence number. Alice presents both signatures and the higher sequence. The contract replaces Bob's claim and finalizes the two/eight allocation after the dispute window.

That window creates the channel's online assumption. Alice, or a watchtower acting for her, must notice the old-state attempt in time. A longer window protects occasionally offline users but delays settlement. A shorter window improves capital velocity but requires more reliable monitoring. Signed messages also need a chain ID, channel ID, participants, sequence, and expiry. Otherwise a valid signature may be replayed in another context.

Routing Is a Liquidity Problem

A Lightning path can exist topologically and still fail economically. Every hop needs outbound liquidity in the correct direction for the amount and fees. Hashed timelock contracts link the route: the receiver reveals a preimage, which propagates backward so every intermediary can claim its incoming payment. Timelocks decrease along the route, giving each intermediary time to react if the next hop stops.

This safety margin locks capital. Longer routes and a congested base layer require more conservative expiries. Channel scalability is therefore a graph-routing problem, a monitoring problem, and a capital-efficiency problem at the same time.

Implementing a State Channel

A channel contract can be small, but its signed message format must be exact. One state object might contain:

```
ChannelState {
  chain_id
  channel_contract
  channel_id
  participants[]
  sequence
  balances_or_application_state
  final
}
```

Every participant signs a domain-separated hash of this object. `chain_id` and `channel_contract` prevent a signature from being replayed on another chain or adjudicator. `channel_id` separates simultaneous channels among the same users. `sequence` gives states a total freshness order. The `final` flag permits cooperative closure without waiting through a dispute window.

The adjudicator stores the highest sequence it has seen:

```
challenge(state, signatures):
  require all_required_signatures(state, signatures)
  require state.channel_id == expected_channel
  require state.sequence > best_sequence
  best_state = state
  best_sequence = state.sequence
  deadline = now + challenge_period
```

After the deadline, settlement applies `best_state`. A generalized channel may instead execute an application-specific transition on-chain when participants disagree. That fallback must be deterministic and bounded, or an adversary can make disputes too expensive to resolve.

Watchtowers and Delegated Monitoring

A watchtower does not need custody of funds. The user gives it encrypted or pre-authorized evidence sufficient to respond to a stale close. A good design limits the tower's power to presenting a newer state. It also defines incentives: how the tower is paid, how missed responses are detected, and whether several towers can monitor the same channel.

Monitoring assumptions should be measured against base-layer conditions. A one-hour challenge window is weak if the L1 can remain congested for longer or if confirmation finality takes much of that hour.

Implementing HTLC Routing

For a three-hop payment, the receiver chooses secret r and sends the payer $H(r)$. Each channel update creates a conditional payment:

```
pay amount to next_hop if preimage(H) is revealed before timeout
otherwise refund amount to current_hop after timeout
```

Timeouts decrease toward the receiver. If Carol's output expires at height 100, Bob's incoming output must expire later, perhaps at 110, so Bob has time to learn the preimage and claim on-chain. Each hop adds a safety delta, increasing the total collateral lock time.

Atomic multipath payments split a large payment into smaller routes and reveal the claim condition only when all parts arrive. This improves routing around limited channels but creates coordination and probing concerns.

Plasma Exit Games in More Detail

A Plasma operator publishes roots of child-chain blocks. To exit, a user presents a coin's position, latest transaction, and Merkle proof. Other users can challenge with evidence that the coin was spent later or that the exiting history is invalid.

The parent contract cannot cheaply reconstruct the whole child chain. Safety depends on users retaining data and watching exits. If the operator withholds a block, many users may attempt to leave at once. The root chain then becomes the bottleneck precisely during failure. Exit priority, bonds, and challenge ordering must prevent the operator from stealing with an old history.

Rollups change this by publishing enough batch data for any observer to reconstruct all balances. Users no longer need a private copy of every coin history, though they still rely on correct contracts and a functioning challenge or proof system.

L2 Classification Checklist

Before calling a system Layer 2, identify:

- the contract or rule that holds canonical assets;
- the data each user must retain;
- who can propose new state;

- how invalid state is rejected;
- how censorship is bypassed;
- the longest period a user may safely remain offline;
- the cost of a disputed or mass exit;
- the keys that can replace these rules.

These answers classify the system more accurately than branding.

Channel Factories and Virtual Channels

Opening one channel per user pair consumes L1 transactions. A channel factory lets several users lock funds in one contract and create many internal channels off-chain. The factory's participants sign updates allocating its total collateral among subchannels.

A virtual channel connects two users through intermediaries without opening a new on-chain channel. Intermediaries lock collateral in underlying channels and agree to enforce the virtual relationship. This saves setup cost but adds participants whose liquidity and availability matter.

Factories increase capital reuse while enlarging the failure group. A dispute may involve the factory state, subchannel state, and application state. Sequence numbers and challenge rules at each level must compose without allowing an old factory allocation to invalidate a newer subchannel.

Channel Privacy and Metadata

Off-chain updates avoid public execution, but channel openings, capacities, routes, timing, and closures can reveal relationships. Routed-payment nodes observe neighboring hops and amounts. Probing can infer liquidity by testing which payments fail.

Privacy techniques include onion routing, route blinding, rendezvous routing, multipath splitting, and private channels. They reduce visibility but do not erase timing and liquidity signals. Watchtowers also need enough encrypted information to recognize a punishable close without learning every application update.

A privacy claim should state the observer: a routing intermediary, blockchain analyst, watchtower, counterparty, or global network adversary sees different metadata.

Sidechain Bridge Verification Models

A sidechain bridge can verify messages in several ways.

Multisignature or committee. A threshold signs withdrawals. This is simple and cheap but makes the signer set a custody boundary.

Light client. The destination verifies source consensus headers and inclusion proofs. Security follows the source consensus more closely, but on-chain verification may be expensive and validator-set changes must be tracked.

Optimistic bridge. Relayers assert messages and watchers can challenge invalid assertions during a delay. This reduces routine verification cost but needs available source data and one honest challenger.

Validity-proof bridge. A proof attests to source consensus or state transition. Verification is compact, while proving the source protocol and keeping circuits current is complex.

Bridge labels such as "trustless" hide these mechanisms. The verification rule, upgrade authority, finality delay, and recovery behavior are the meaningful facts.

Mass-Exit Capacity

Exit designs often prove one user can recover. The stronger question is whether many users can recover simultaneously.

If an L2 holds one million accounts and the parent chain can process only thousands of exit transactions per day, an emergency window can close before everyone exits. Aggregated exits, claim trees, priority queues, and validity proofs improve capacity. Rate limits can bound theft but delay honest withdrawals.

A mass-exit test should publish parent-chain gas per exit, maximum exits per block, challenge load, data required by each user, and the outcome when the operator withholds the final state. Safety for the fastest users is not system-wide safety.

Payment-Channel Rebalancing

Successful payments move liquidity in one direction. A channel can remain fully funded yet unable to send further along the depleted side. Rebalancing sends a circular payment that returns funds to the same owner through other channels, or performs an on-chain splice that changes channel capacity without a full close.

Rebalancing costs routing fees and may fail because the required cycle lacks liquidity. Automated policies choose target balances and fee limits. They can leak demand information or compete with user payments. Network throughput therefore depends on liquidity distribution and rebalancing efficiency, not only total locked value.

Worked Lightning Route: Amounts, Fees, and Timelocks

Consider a payment from Alice to Dave through Bob and Carol:

```
Alice -> Bob -> Carol -> Dave
```

Dave must receive 100,000 millisatoshis. Carol charges a 1,000 msat base fee plus 0.1 percent of the forwarded amount. Bob charges 500 msat plus 0.05 percent. Ignoring integer-rounding details for the illustration, compute backward from the receiver.

Carol must receive enough to forward 100,000 msat and retain:

$$1,000 + 0.001 \times 100,000 = 1,100 \text{ msat}$$

So Bob offers Carol 101,100 msat. Bob's fee is approximately:

$$500 + 0.0005 \times 101,100 = 550.55 \text{ msat}$$

Alice therefore offers Bob about 101,651 msat after applying the protocol's integer rounding. Route construction works backward because each incoming HTLC must cover the next hop's outgoing amount plus fee.

Timelock ladder

The outgoing HTLC near Dave expires first. Each upstream HTLC expires later by a safety delta:

```
Alice-Bob expiry: 700
Bob-Carol expiry: 660
Carol-Dave expiry: 620
```

Dave reveals the payment preimage to claim Carol's HTLC. Carol uses the same preimage to claim from Bob, who uses it to claim from Alice. The decreasing expiries give an intermediary time to learn the preimage downstream and enforce its incoming claim on-chain before that claim expires.

A delta that is too small exposes an intermediary during congestion or a counterparty delay. A delta that is too large locks liquidity longer. Implementations account for confirmation depth, block-time variance, fee spikes, and the time required to publish a commitment transaction and second-stage claim.

Failure traces

Insufficient directional liquidity. A channel can have enough total capacity while lacking balance in the required direction. The sender tries another route or amount. Repeated probes can leak payment intent.

Dave never reveals the preimage. Every HTLC times out in reverse order. Funds remain temporarily locked, reducing routing capacity, but no intermediary should lose principal if deadlines and on-chain actions are correct.

Carol learns the preimage but goes offline. Bob may enforce the outgoing or incoming contract on-chain according to the channel design. Watchtowers can react to revoked states, but they do not create missing liquidity or eliminate base-layer congestion.

An old channel state is published. The counterparty uses the revocation or newer-state mechanism during the dispute window. Security depends on monitoring and affordable chain access before deadline.

The base layer is congested. Many expiring HTLCs compete for blockspace. Fee reserves and deadline margins are part of channel safety. A theoretical claim path that cannot be included before expiry is not an effective remedy.

Multipath payments

If no one route can carry 100,000 msat, the sender may split the payment. Atomic multipath designs bind parts to one payment condition so the receiver claims only when the required total arrives. Splitting improves liquidity use but increases routing attempts, fees, privacy leakage, and the number of temporary locks.

Route-level accounting

A useful payment trace records:

- amount delivered and total amount sent;
- fee at each hop and rounding rule;
- outgoing and incoming expiry at each hop;
- channel balance before and after;
- attempt count and failure reason;
- time to receiver claim and full upstream settlement;
- whether any hop required an on-chain transaction.

Network TPS is not the useful metric. Capacity depends on directional liquidity, route length, fee policy, failure rate, lock duration, rebalancing, and base-layer dispute capacity.

Channel Backup, Restore, and Disaster Recovery

A channel wallet cannot recover from a seed phrase alone if current channel state, revocation data, or counterparty commitments are missing. Ordinary on-chain wallets reconstruct funds by scanning history; channels may require private, frequently changing state.

Recovery data

Back up:

```
ChannelBackup {
    chain_id,
    channel_id,
    funding_outpoint,
    counterparty,
    latest_commitment_number,
    encrypted_static_recovery_data,
    watchtower_appointment_receipts,
    close_and_sweep_descriptors,
    wallet_and_protocol_version
}
```

A **static channel backup** may contain enough information to find the counterparty and request a safe close, but not enough to continue operating the channel or unilaterally reconstruct every latest balance. State what the backup guarantees.

Never copy live database files without a consistent snapshot rule. A backup captured between related writes can restore a commitment number without corresponding secrets or signatures.

Revocation safety

In penalty-based channels, broadcasting an old commitment can let the counterparty claim a penalty. Restoring stale local state is therefore dangerous even when the file is authentic.

On startup after restore, mark channels recovery-only, contact counterparties and watchtowers, compare commitment points through the protocol, and avoid signing or broadcasting until synchronization proves the safe state. A user should not guess which backup is newest from filenames.

Data-loss protection

Peers can exchange bounded recovery information that helps detect stale state without giving one peer power to fabricate balances. The protocol must resist a malicious counterparty falsely claiming that the user is behind and forcing an unfavorable close.

Authenticated monotonic commitment numbers identify ordering but do not by themselves reveal missing signed transactions. Define the safe response for each mismatch: continue, cooperative close, force close, or manual recovery.

Watchtower receipts

A client needs durable evidence that a tower accepted the appointment covering its newest revocable state. Store tower identity, channel hint, covered commitment range, receipt signature, expiry, and fee policy.

A backup containing appointments that were queued but never acknowledged creates false confidence. Verify receipts after restore and reappoint when tower retention or chain horizon may have expired.

Seed and channel database separation

The seed derives keys but not necessarily counterparty signatures, revocation history, pending HTLCs, or watchtower acknowledgements. Document distinct backup materials and their confidentiality.

Channel backups may reveal counterparties, funding transactions, balances, or routing activity. Encrypt with authenticated encryption, version the format, test wrong-password behavior, and avoid cloud filenames that leak identifiers.

Pending HTLCs

Recovery during pending routed payments is time-sensitive. The node may need preimages, onion-routing secrets, incoming/outgoing HTLC mappings, and deadlines to claim or refund safely.

Prioritize channels by earliest on-chain expiry. A general restore process that takes hours can lose funds when one HTLC has minutes of safety margin. Persist critical forwarding state before acknowledging the upstream message.

Worked recovery point

Suppose backups upload every 10 minutes, but channel updates occur once per second. Up to 600 updates can occur between backups:

$$10 \text{ minutes} \times 60 \text{ updates/minute} = 600 \text{ updates}$$

A 10-minute recovery point objective is meaningless for continuing a penalty channel safely. Use synchronous or near-synchronous durable state, protocol data-loss protection, and watchtowers; periodic bulk backup is supplemental.

Force-close recovery

A disaster runbook identifies funding outputs, latest safe commitments, closing transaction, sweep paths, CSV or absolute timelocks, fee reserves, and monitoring duration. **CheckSequenceVerify (CSV)**-style relative delays commonly require waiting a number of blocks after confirmation before a sweep path becomes valid.

Fee spikes can make pre-signed or fixed-fee transactions unusable. Test fee bumping or child-pays-for-parent paths under the exact channel format.

A force close can create several outputs with different delays and conditions. "Close confirmed" does not mean every balance is spendable. Wallet status should show each output and earliest action.

Multi-device hazards

Two active devices controlling one channel can create divergent commitments. Cloud sync is not distributed consensus. Use one active writer, cryptographic fencing, or a protocol designed for replicated signing.

A device believed lost may later reconnect with stale state. Rotate credentials and prevent it from signing before restore is considered complete.

Recovery drill

1. destroy the primary node and local channel database;
2. restore seed and documented backup artifacts on a new host;
3. identify all channels and funding outputs;
4. handle one stale backup, one unavailable counterparty, and one pending HTLC;
5. verify watchtower coverage;
6. recover through cooperative or force close;
7. sweep every delayed output under a fee spike and chain reorganization;
8. reconcile recovered balances and fees.

Measure time to detect, first protective transaction, final spendability, user loss, and manual decisions. Repeat without the original cloud account or hardware.

Production assertions

A channel system should state what seed-only, static-backup, full-database, peer-assisted, and watchtower recovery each guarantee. Test every format version and migration.

Off-chain scalability is credible when reducing on-chain writes does not make private state an untested single point of fund loss.

Watchtower Data and Privacy

A watchtower needs enough information to recognize a revoked commitment and publish the appropriate remedy, but users should not disclose their entire channel history. Designs can send encrypted penalty material indexed

by a transaction-derived hint. The tower learns little until a matching breach appears on-chain.

The client must durably confirm that a tower accepted the latest appointment before treating protection as active. Towers need fee strategy, chain monitoring, data retention, and redundancy. If several apparent towers share one operator or infrastructure region, they are one failure domain.

Test breach response while the client is offline, the tower restarts, the fee market spikes, a chain reorganization removes the first remedy, and an appointment is duplicated or delivered out of order. Measure from breach publication to irrevocable remedy, not only webhook detection.

Conclusion

Layer 2 systems scale by avoiding global replication of every interaction. Channels are efficient for stable participants, sidechains provide independent capacity, Plasma uses exit games, and rollups combine off-chain execution with verifiable state commitments and available data.

No Layer 2 is free of trade-offs. Its safety depends on its exit path, data model, proof system, sequencer, and governance. The next chapter focuses on rollups, a general-purpose design in Ethereum's rollup-centric roadmap.

References

-
1. Poon, Joseph, and Thaddeus Dryja. "The Bitcoin Lightning Network." <https://lightning.network/lightning-network-paper.pdf>. ↵
 2. Miller, Andrew, et al. "Sprites and State Channels." <https://arxiv.org/abs/1702.05812>. ↵
 3. Polygon Documentation. "Polygon Chain overview." <https://docs.polygon.technology/pos/overview>. ↵
 4. Polygon Documentation. "Checkpoints." https://docs.polygon.technology/pos/architecture/heimdall_v2/checkpoints. ↵ ↵2 ↵3
 5. Polygon Documentation. "Ethereum to PoS." <https://docs.polygon.technology/pos/how-to/bridging/ethereum-polygon/ethereum-to-matic>. ↵
 6. Polygon Documentation. "PoS to Ethereum." <https://docs.polygon.technology/pos/how-to/bridging/ethereum-polygon/matic-to-ethereum>. ↵
 7. Poon, Joseph, and Vitalik Buterin. "Plasma: Scalable Autonomous Smart Contracts." <http://plasma.io/plasma.pdf>. ↵

Chapter 6: Rollups

Introduction

Rollups move transaction execution away from the base chain while using it for settlement and data. Hundreds or thousands of transactions are compressed into a batch. The rollup publishes a new state commitment and enough information for the base layer to enforce correctness.

This design separates three questions: who orders transactions, who executes them, and how the result is proven. Understanding that separation is more useful than memorizing project names.

Rollups From First Principles

A rollup moves transaction execution away from the base chain while keeping a base-chain contract able to check and enforce the result. Many transactions are collected into a **batch**. The rollup executes them, calculates a new state root, and posts data plus a claim about that new state.

Picture an accountant processing a box of receipts. Re-entering every receipt in the public ledger is expensive. Instead, the accountant publishes the receipts in a compact form, a new total, and evidence that the arithmetic follows the agreed rules. Anyone can reconstruct the work or check the evidence. The base chain acts as the final court and asset custodian.

The roles

- The **sequencer** receives transactions and chooses their order. It gives fast responses but may be able to delay or censor users.
- The **batcher** compresses and publishes ordered transaction data.
- An **executor** runs the rollup program and calculates state changes.
- A **prover** creates cryptographic evidence for a validity rollup.
- A **challenger** checks optimistic claims and starts a dispute when one is invalid.
- The **settlement contract** stores accepted commitments and applies proof or dispute rules on L1.
- The **canonical bridge** locks assets on one layer and releases or represents them on the other according to the settlement contract.

One operator can initially run several roles. Keeping the names separate shows which power must later be decentralized and which failure caused a delay.

Optimistic and validity proofs

An **optimistic rollup** treats a posted state claim as acceptable if nobody successfully challenges it before a deadline. "Optimistic" means the normal path assumes the claim will not need a dispute; it does not mean users trust the operator without evidence.

A **fault proof** demonstrates that a claimed transition was invalid. Interactive systems narrow a disagreement over a long execution trace until the base chain checks one small step. This resembles two people disputing a long spreadsheet by repeatedly identifying which half contains the first wrong row.

A **validity rollup** requires a cryptographic proof before accepting the new state. A **validity proof** convinces a verifier that the encoded program transformed the committed input state into the claimed output state. A **zero-knowledge proof** can additionally hide private witness data, but not every validity proof uses privacy.

Data is separate from correctness

A proof may show that a calculation was correct without giving users the inputs needed to reconstruct balances or make future transactions. **Data availability** asks whether those inputs can actually be obtained.

Rollups commonly publish compressed transaction data to L1 or a specified DA layer. A **blob** is a temporary, separately priced L1 data container designed for rollup batches. The chain commits to blob contents and makes them available for a protocol retention period; long-term archives are a separate service.

Deposits and withdrawals

A deposit locks an L1 asset in the bridge and creates a message the rollup must process. A withdrawal begins on L2, becomes part of an accepted state root, and then proves to the L1 bridge that the message is valid and unused.

Optimistic withdrawals wait through the challenge period because releasing an asset too early could honor an invalid state claim. Validity withdrawals do not need that optimistic challenge window, but they still wait for proof generation and acceptance, the deployment's settlement-contract stages and security delays, and the required L1 finality. Both designs need replay protection: each withdrawal identifier can be consumed only once.

Forced inclusion and escape

If a sequencer ignores a user, a **forced-inclusion inbox** lets the user submit through L1. The protocol specifies how soon the rollup must process that message. An **escape hatch** lets users recover or advance state without the normal operator under defined failure conditions.

These mechanisms matter only when tested. A contract entry point that requires unavailable data, unaffordable gas, or an active operator is not an effective escape.

Reading rollup status

A wallet can truthfully show several stages:

1. received by the sequencer;
2. ordered in an L2 block;
3. batch data published;
4. state claim submitted;
5. proof accepted or challenge period complete;
6. containing L1 block final;
7. withdrawal executed.

Later sections use "unsafe," "safe," "accepted," and "final" for these boundaries. The exact labels vary by implementation; the evidence behind the label is what matters.

The Rollup Lifecycle

A rollup transaction has several completion boundaries

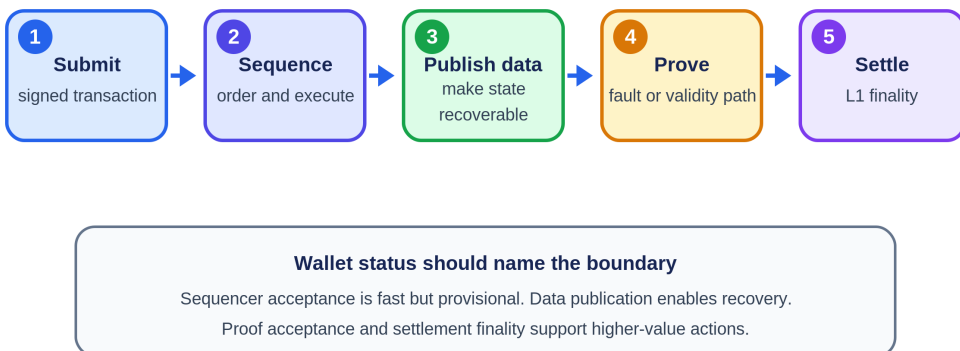


Figure 6.1: A rollup transaction moves through sequencing, data publication,

proof, and settlement. Applications should name which boundary they treat as complete. Original figure for this book.

A typical transaction follows this path:

1. a user signs and submits a transaction to a sequencer;
2. the sequencer orders transactions and produces an L2 block;
3. an executor computes the new rollup state;
4. batch data or a data commitment is posted;
5. a state root is submitted to the settlement contract;
6. a validity proof is accepted, or an optimistic assertion survives the challenge process under its honest-challenger and data assumptions;
7. the settlement transaction reaches the required L1 finality.

The sequencer provides fast inclusion and a useful user experience, but its acknowledgement is normally **soft confirmation**. Settlement finality comes later. A centralized sequencer can censor or reorder transactions even when it cannot steal funds. Force-inclusion mechanisms and sequencer decentralization address this liveness risk.

Named Case Study: Arbitrum Nitro and BoLD

Deployment label: production. Arbitrum One runs the Nitro stack as an optimistic rollup. Its official documentation separates the fast path through a sequencer from the enforceable path through Ethereum, and describes BoLD as the dispute protocol used to resolve competing state claims.¹ [^4]² The distinction is the useful lesson: an optimistic rollup is not secured by optimism. It is secured by data that validators can replay, a state-transition program they agree to run, and a bounded route for rejecting a false claim.

Architecture: four boundaries, not one chain

Nitro can be read as four connected systems. The **sequencer** accepts transactions, chooses an order, executes that order, and broadcasts a feed that gives applications a fast soft confirmation. The **batch poster** compresses the ordered data and sends it to Ethereum, normally using EIP-4844 blobs and with calldata as an alternative path described by the protocol documentation. Ethereum's **Sequencer Inbox** establishes the canonical data sequence. Nitro nodes read that sequence and execute the same state-transition function. Finally, validators make and check assertions about the resulting state; BoLD resolves a disagreement by reducing it until Ethereum can judge the disputed execution.

This architecture creates several clocks. A wallet may see a sequencer receipt in seconds, while the batch is not yet on Ethereum. After publication, the

transaction has stronger data availability, but a state assertion may still be disputable. A canonical L2-to-L1 withdrawal becomes executable only after the assertion supporting it is confirmed under the dispute rules and the required Ethereum conditions are met. Calling every one of these states "final" hides the exact risk the user is taking.

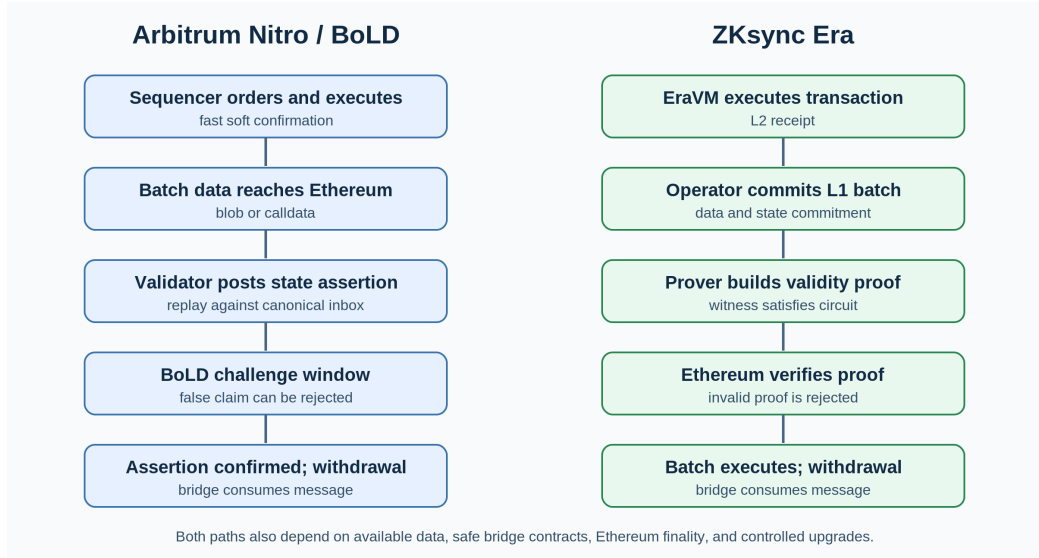


Figure 6.2: Named production rollups use different correctness paths after sequencing and data publication. Original figure for this book.

Trace: deposit, transfer, publication, dispute, withdrawal

Suppose Lina moves 1 ETH from Ethereum to Arbitrum One, pays a merchant on Arbitrum, and later withdraws the remaining ETH.

1. Deposit. Lina's Ethereum transaction calls the canonical bridge. The L1 contract escrows the asset and places an L1-to-L2 message into Arbitrum's delayed message path. This is not an ordinary transfer to a second custodian. The L1 event becomes an input that Nitro must consume in canonical order. After the message is included and executed on L2, the corresponding balance is created for Lina under the bridge rules. The L1 transaction can be reorganized before Ethereum finality, so software should retain the source transaction and report whether the deposit is merely observed, included, or final enough for its policy.

2. Sequencing and execution. Lina signs the merchant payment and sends it to an Arbitrum RPC endpoint. The sequencer checks the transaction, orders it, executes it with the Nitro state-transition function, and emits a sequencer-feed result. The merchant can treat that result as a low-latency promise, but it is not an Ethereum settlement guarantee. A sequencer can delay or reorder a transaction, and a feed result can precede canonical batch publication.^{1 3}

3. Data publication. The batch poster compresses the chosen sequence and submits it to the Sequencer Inbox on Ethereum. Nitro documentation describes blob publication through `addSequencerL2BatchFromBlobs` and calldata publication through `addSequencerL2Batch`.⁴ Once the batch is in the canonical inbox, an independent Nitro node can recover the same ordered input and reproduce execution. Publication therefore changes what observers can verify: before it, they mainly have the sequencer's promise; after it, they have Ethereum-ordered data from which to derive the L2 state.

4. Assertion and BoLD. A proposer posts an assertion about a state reached after executing the inbox. An honest validator replays the input and compares the result. If a conflicting or false assertion appears, BoLD lets participants challenge it. The dispute does not ask Ethereum to re-execute the whole rollup history. Parties commit to execution history, narrow the disagreement, and ultimately reach a small step that the on-chain verifier can decide. BoLD is designed so that an attacker cannot extend dispute resolution without bound merely by creating more conflicting claims; honest parties still need the data, software, capital for required bonds, and operational ability to act within protocol deadlines.²

5. Withdrawal. Lina initiates an L2-to-L1 message that burns or locks the L2 representation and names the L1 recipient and amount. That message is included in L2 state and supported by a state assertion. The canonical bridge must wait until the assertion is confirmed under the rollup's dispute rule before releasing escrowed ETH on Ethereum. A liquidity bridge may pay Lina sooner, but that is a separate market transaction with liquidity, pricing, and counterparty assumptions. It does not shorten the canonical security path.

Trust and upgrade assumptions

Nitro limits what a faulty sequencer can do, but it does not make the sequencer irrelevant. The sequencer controls the fast ordering path and can temporarily censor or delay users. Arbitrum exposes a **Delayed Inbox** path on Ethereum. Official documentation states that a user can bypass the sequencer and that delayed messages can eventually be force-included after the protocol delay.³ This is censorship resistance with latency and L1 gas cost, not instantaneous inclusion.

Correctness also depends on at least one capable honest validator monitoring assertions and being able to complete a dispute. Users depend on Ethereum for ordering, contract execution, and finality, and on available batch data for replay. They also depend on the deployed contracts, the Nitro program identified by the dispute machinery, and the governance and upgrade controls that can change those components. BoLD reduces one class of permission and delay risk. It does not remove contract bugs, implementation bugs, comprom-

ised upgrade keys, Ethereum failure, or the operational risk that nobody runs an effective validator.

The practical audit question is therefore broader than "does Arbitrum have fraud proofs?" It is: which contracts and program versions define a valid state, who can replace them, what delay precedes a replacement, can users exit during that delay, and which emergency body can intervene? Those facts can change, so a deployment review should read the live contracts and current governance documentation rather than copy a static decentralization label from this book.

Observable consequences and failure path

The layered design is visible. A block explorer or node can distinguish a transaction seen in the sequencer feed from one whose batch data is on Ethereum. An operator can measure the age of the oldest unpublished sequence, the delayed-inbox cursor, assertion status, and dispute activity. A wallet can tell Lina why a withdrawal is waiting instead of displaying one spinner.

Now let the sequencer stop after giving Lina a receipt but before publishing her payment. The payment may be visible on the feed yet absent from the canonical inbox. Lina or her wallet resubmits through the L1 delayed path. She pays Ethereum gas and waits through the inclusion delay, but the sequencer cannot turn its outage into a permanent veto. If, instead, the batch is published and a proposer asserts a state that omits or misexecutes the payment, an honest validator reconstructs the state and challenges the assertion through BoLD. The bad assertion must not become the basis for Lina's canonical withdrawal. If no capable honest validator acts before the applicable deadline, the optimistic safety assumption has failed even though the protocol had a dispute contract.

That last distinction is the transferable lesson. A sequencer failure is primarily a liveness and user-experience event when forced inclusion works. An unchallenged invalid assertion is a correctness failure. Missing data can make a challenge impossible. A malicious upgrade can change the rules by which every later state is judged. Each failure reaches a different boundary and needs a different alarm and recovery procedure.

Named Case Study: ZKsync Era's Validity Pipeline

Deployment label: production. ZKsync Era is a validity rollup whose protocol documentation describes a three-stage batch lifecycle on Ethereum: a batch is committed, proved, and executed. Its L1-to-L2 communication path treats deposits and other priority operations as messages that the L2 boot-loader must process, while its proof system lets Ethereum verify a batch transition without replaying EraVM execution.^{5 6}

Architecture: execution, data, proof, and bridge

A ZKsync transaction first runs under **EraVM**, with system contracts and the bootloader enforcing protocol rules. A sequencer forms L2 blocks and groups them into L1 batches. For rollup-mode operation, the operator publishes the information required by the protocol's data-availability rule and commits the batch metadata on Ethereum. A prover constructs a validity proof for the batch transition. Ethereum contracts verify the proof and later execute the batch, which makes its L2-to-L1 messages available to the bridge contracts under the protocol rules.

"Execute" is easy to misunderstand here. It does not mean Ethereum reruns every user transaction. User computation already ran on L2 and was covered by the proof. The L1 execution stage advances the settlement contracts after the commitment and proof checks, and permits effects such as finalized withdrawals. Separating **commit**, **prove**, and **execute** makes failure diagnosis much sharper than one "pending" status.

Trace: priority deposit, Era payment, proof, and exit

Suppose Lina deposits 1 ETH into ZKsync Era, makes an EraVM payment, then withdraws what remains.

1. L1 priority operation. Lina sends an Ethereum transaction to the canonical bridge. The L1 contract escrows ETH and creates an L1-to-L2 transaction, also called a priority operation. The request carries the target, value, gas-related parameters, refund recipient, and data needed for L2 execution. ZKsync's documented communication flow passes the operation into the L2 bootloader rather than letting the sequencer silently invent the deposit.⁶ The same channel is used for protocol upgrade transactions, which is a reminder that system changes and user deposits both need explicit cross-layer ordering.

2. L2 execution. The bootloader processes the priority operation and the deposit is reflected in Era state. Lina then signs the merchant payment and submits it through an RPC endpoint. A successful sequencer response tells her the transaction ran in an L2 block. It does not yet prove that Ethereum has accepted the containing L1 batch. Applications should expose at least the distinction between L2 execution, batch commitment, proof verification, and L1 execution.

3. Batch commitment and data publication. The operator collects L2 blocks into an L1 batch and submits a commitment transaction to Ethereum. The commitment binds the previous and new state information and the public data needed by the protocol. An independent observer can associate Lina's transaction with its L2 block and L1 batch, then identify the Ethereum com-

mitment transaction. If the operator stops before commitment, Lina has an L2 receipt without the strongest settlement evidence.

4. Proof. The proving pipeline builds a witness for the EraVM transition and produces a validity proof. Ethereum's verifier checks the proof against public inputs tied to the batch commitment. If the witness does not satisfy the circuit or the public inputs do not match, a sound verifier rejects the proof. Unlike an optimistic rollup, Era does not make users wait to see whether someone challenges an accepted invalid transition. The cost moves into circuit correctness, witness generation, prover availability, verifier correctness, and upgrade control.

5. Batch execution and withdrawal. After the proof is accepted, the L1 contracts execute the batch under the protocol sequence. Lina's later L2 withdrawal creates an L2-to-L1 message. Once the batch containing that message has reached the required L1 stage, she finalizes the withdrawal through the L1 bridge. ZKsync documents warn that withdrawals may experience an additional delay as a security measure; software should read the live status rather than promise that every validity-rollup withdrawal is immediate.⁷

Trust and upgrade assumptions

A validity proof prevents a transition outside the proved rules from being accepted, assuming the proof system, circuit, verifier contract, and cryptographic assumptions are sound. It does not prove that the sequencer included Lina promptly, that the user interface showed the right recipient, or that an upgrade is benign. It also does not create data availability by itself. Users need the data path specified by the deployment to reconstruct state and prepare transactions.

Prover failure is normally a liveness failure: an unproved batch cannot advance through the settlement pipeline, but a prover should not be able to make Ethereum accept an arbitrary state without a valid proof. This makes prover diversity and recovery important even when the prover is not trusted for correctness. Sequencer and operator failures can delay inclusion, publication, proofs, and withdrawals. The bridge and system contracts remain high-value code. Upgrade transactions are especially important because a new verifier, bootloader, system-contract set, or circuit version can change what later proofs mean. ZKsync's L1-to-L2 documentation explicitly gives upgrade transactions a defined initiation, bootloader, commit, possible revert, and execute lifecycle.⁶

A deployment assessment should therefore identify who can authorize an upgrade, which delay and emergency powers apply, how old and new proof versions overlap, and whether a user has a usable exit path before the new rules take effect. "Validity proved" describes a batch relative to a program. It does not answer who chose that program.

Observable consequences and failure path

The pipeline leaves concrete evidence: Lina's L2 receipt names a block; the block maps to a batch; Ethereum records the batch commitment, proof verification, and execution transactions; the bridge records whether her withdrawal message has been consumed. Operators can publish queue ages for uncommitted, unproved, and unexecuted batches. A long **unproved** queue points toward witness or prover trouble. A long **committed but unexecuted** queue points toward a later settlement stage. Those are different incidents even if both appear to a user as a delayed withdrawal.

Now let the primary prover fail after a batch is committed. The correct response is not to bypass proof verification. The batch remains unproved while another prover reconstructs the witness and submits a proof. Lina's prior accepted balance remains safe under the assumed contract rules, but her withdrawal cannot complete on schedule. If the circuit contains a bug that accepts an invalid transition, proof verification can succeed while the economic state is wrong; validity systems move much of the correctness burden into circuit and verifier engineering. If an authorized upgrade installs a faulty verifier, governance has changed the proof boundary itself. And if batch data required for reconstruction is unavailable, users may be unable to independently derive the state even though a proof attests to the transition.

The comparison with Nitro is now precise. Nitro needs an honest, timely challenge to reject a false assertion. Era needs a sound proof system and verifier to prevent one from being accepted. Both need ordered cross-layer messages, available data, safe contracts, transparent upgrade control, and a user-visible account of where a transaction sits.

Production Rollup Comparison on One Payment-and-Withdrawal Workload

The table fixes the workload: deposit ETH from Ethereum, make one L2 payment, publish the batch data required by the named deployment, establish state correctness, and withdraw canonically to Ethereum. "Production" means the named public deployment is live; it does not claim that every decentralization or governance goal is complete. OP Mainnet and Base share the OP Stack family but are separate deployments with separate operators and governance contexts.⁸

Deployment	Execution and ordering	Data and correctness path	Canonical withdrawal boundary	Main failure or control to inspect
Arbitrum One (Nitro/BoLD)	Nitro execution; fast sequencer feed, with an L1 delayed path for bypass and force inclusion	Ordered batches reach Ethereum by blobs or calldata; validators replay Nitro and BoLD adjudicates disputed assertions	Supporting assertion confirmed after the optimistic dispute process, then L1 bridge execution	Sequencer delay, effective validator participation and bonds, Nitro/dispute-contract versions, governance and upgrade powers
OP Mainnet (OP Stack)	OP Stack sequencer produces L2 blocks; derivation reconstructs L2 from Ethereum data	Output claims are challengeable through the OP fault-proof system; Ethereum data drives derivation	Withdrawal is initiated on L2, proved against an output, waits through the applicable finalization period, then is finalized on L1	Sequencer/proposer liveness, fault-proof configuration, guardian and upgrade powers, portal implementation ⁹
Base (OP Stack)	OP Stack execution and sequencing operated for Base	Same broad OP derivation and fault-proof architecture, with deployment-specific contracts and controls	Same initiate-prove-finalize shape through the deployment's canonical bridge	Do not infer Base's live security configuration solely from OP Mainnet; inspect Base's contracts, operators, and current stage
ZKsync Era	EraVM, bootloader; sequencer, L2 blocks grouped into L1 batches	Rollup data path plus commit-prove-execute pipeline; Ethereum verifies a validity proof	Batch containing the L2-to-L1 message reaches required proof/execution stage, then bridge finalization; a security delay may apply	Circuit/verifier correctness, prover liveness, operator queues, system-contract and upgrade controls
Starknet	Starknet sequencer orders and executes Cairo transactions; transaction receipts expose staged status	State updates are proved through Starknet's proving pipeline, including SHARP aggregation, and verified on Ethereum	StarkGate releases after the relevant state/message is accepted and the bridge conditions are met	Sequencer and prover liveness, Cairo/prover/verifier versions, data reconstruction, messaging and upgrade controls ^{10 11}

Deploy-ment	Execution and ordering	Data and correctness path	Canonical with-drawal bound-ary	Main failure or control to in-spect
Scroll	EVM-compat-ible L2 execu-tion; blocks are grouped into chunks and batches	Scroll documents exe-cution, batch/data commitment, proof generation, and final-ization; a commit transaction records batch data and a final-ize transaction veri-fies proof and ad-vances finalized state	Withdrawal message be-comes finaliz-able after the containing batch is final-ized and gate-way checks pass	Sequencer/ batcher/prover liveness, codec and circuit compatibility, gateway con-tracts, upgrade control ^{12 13}

The table should not be used to rank systems by one label. It is a checklist of where to look. For example, two validity rollups may use different virtual machines, proof aggregation, data encodings, upgrade authorities, and withdrawal delays. Two OP Stack chains may share software while exposing different operational and governance risk. The useful comparison holds the user action constant and follows the evidence all the way from a signature to asset release.

Optimistic Rollups

Optimistic rollups assume a proposed state is valid unless someone proves otherwise. After a batch is posted, a challenge period allows a verifier to submit a fraud proof.

A modern fault-proof system narrows disagreement to a specific execution step, then asks Layer 1 to evaluate that step. This avoids re-executing an entire batch on-chain.

Benefits

- close compatibility with the EVM;
- relatively simple proving assumptions;
- computation is performed off-chain unless disputed;
- mature developer tooling and ecosystems.

Trade-Offs

- canonical withdrawals can wait through the challenge period;
- at least one honest party must be able to reconstruct and challenge bad state;
- the fault-proof implementation and upgrade mechanism are security-critical;
- fast third-party bridges add liquidity and counterparty risk.

The honest verifier requirement does not mean every user must personally watch the chain. It means the system needs an open, economically sustainable set of watchers with access to the data and a working dispute path.¹⁴

Validity Rollups

Validity rollups, often called ZK rollups, attach a succinct proof that the new state was computed correctly. The Layer 1 verifier checks the proof rather than replaying the entire batch.

The proof may use a **SNARK** (succinct non-interactive argument of knowledge) or **STARK** (scalable transparent argument of knowledge). Both let a verifier check a computation with a proof much smaller than replaying all the work. "Succinct" emphasizes small, cheap-to-check proofs. "Non-interactive" means the final verifier needs one proof rather than a back-and-forth protocol. "Transparent" means the system avoids a secret trusted-setup ceremony. These families differ in proof size, prover cost, verification cost, transparency, and cryptographic assumptions. The phrase "zero knowledge" describes the ability to hide witness data, but a validity rollup can use validity proofs without offering transaction privacy.

Benefits

- invalid state cannot pass the verifier if the proof system and implementation are sound;
- withdrawals do not require a fraud-proof challenge window;
- verification cost can remain small even for a large computation;
- recursive proofs can aggregate many blocks.

Trade-Offs

- proving is computationally demanding;
- circuits or zkVMs add implementation complexity;
- bugs in circuits, verifiers, or trusted setup procedures can be severe;
- EVM equivalence and rapid protocol upgrades are difficult engineering problems.

The course's STARK case study emphasizes **scalable proofs**: the verifier performs much less work than the prover, making it possible to amortize verification over a batch.

Where the Data Lives

Validity proves that a transition is correct; it does not by itself make the underlying data available.

- A **rollup** publishes transaction data to the base layer.
- A **validium** stores data off-chain, often with a data availability committee.
- A **volition** lets users or applications choose between on-chain and off-chain data.

Validium can reduce cost, but an unavailable committee can prevent users from reconstructing balances or exiting, even if it cannot forge a validity proof. This is why state validity and data availability must be evaluated separately.

Compression and the Cost Model

For many rollups, data publication is the largest variable cost. A batch saves money by:

- removing repeated signature and transaction fields;
- encoding values compactly;
- sharing one Layer 1 transaction across many L2 transactions;
- using blobs rather than expensive EVM calldata where supported.

EIP-4844 introduced blob-carrying transactions with their own fee market. Blobs are committed to by Ethereum consensus and retained for a limited period, which is sufficient for rollup reconstruction and challenges while avoiding permanent EVM storage.¹⁵

A rollup's user fee can be viewed as:

**L2 execution cost + allocated data-publication cost + proving/
operation cost + margin**

High batch utilization lowers the data cost per user. Empty blocks or fragmented liquidity reduce those economies of scale.

Bridges and Forced Exits

The canonical bridge locks an asset on Layer 1 and represents it on Layer 2. Withdrawals burn or release the L2 representation and unlock the L1 asset after the required proof or challenge process.

The bridge contract is often the rollup's largest pool of value. Security depends on:

- correct message authentication;
- replay protection;
- the state-root acceptance rule;
- proof verification;
- upgrade and emergency powers;
- a usable escape path if the sequencer stops.

A rollup is not trustless merely because it has a proof system. Users should examine whether contracts are upgradeable, who controls the upgrade keys, whether there is a delay, and whether users can exit before a disputed upgrade takes effect.

Optimistic vs Validity Rollups

Feature	Optimistic Rollup	Validity Rollup
Correctness rule	Valid unless challenged	Proof required before acceptance
Main proof	Fraud/fault proof	SNARK or STARK
L1 computation	Mostly during disputes	Verify each aggregate proof
Canonical withdrawal	Delayed by challenge period	After proof and L1 finality
Prover burden	Lower in normal operation	Significant and continuous
Compatibility	Mature EVM compatibility	zkEVM/zkVM complexity
Core assumption	One honest challenger with data	Sound proof system and available data

Neither design dominates every workload. Optimistic systems benefit from simple execution compatibility. Validity systems benefit from concise finality and proof aggregation. Both still face sequencing, governance, bridging, and data-availability choices.

From Rollups to Appchains

The course asks why applications are becoming chains. A dedicated rollup can choose its fee token, block time, execution environment, governance, and sequencing policy. It can isolate congestion and capture more of the economics generated by its users.

The cost is fragmentation. Users must bridge assets, liquidity is split, and synchronous calls across rollups become asynchronous messages. Superchain

and hyperchain designs try to standardize bridges, messaging, and upgrades across related rollups. Shared sequencers and proof aggregation aim to restore some atomicity and economies of scale.

Worked Example: From Transaction to Withdrawal

Assume 1,000 users submit transfers to an optimistic rollup. The sequencer checks signatures and nonces, chooses an order, and returns quick receipts. It executes the batch, produces a new state root, and publishes compressed transaction data to Ethereum. The rollup contract records the proposed root.

A verifier re-executes the data. If its root differs, it opens a dispute. A bisection game narrows the disagreement until the contract checks one disputed machine step. A dishonest proposal is rejected and its bond can be penalized. If nobody challenges during the window, the root becomes final under the rollup contract.

A canonical withdrawal proves that finalized L2 state contains the withdrawal message. A liquidity bridge can pay earlier, but that is a separate service accepting delay and reorganization risk for a fee.

In a validity rollup, a prover generates a proof and the contract verifies it before accepting the root. The user avoids a fault-proof challenge window but may still wait for proving, batch publication, Ethereum inclusion, deployment-specific contract stages or delays, and L1 finality. "Instant finality" must be separated into sequencer confirmation, proof acceptance, and settlement finality.

Sequencer Failure and the Escape Hatch

A sequencer outage should reduce convenience, not destroy ownership. Force inclusion lets a user submit data to Layer 1; after a timeout, the rollup must process it or permit an exit. This path must remain usable when many users need it simultaneously.

Upgrade control is part of the same threat model. If an administrator can replace the verifier immediately, the proof system cannot protect users from that administrator. Delayed upgrades and an exit window make the cryptographic guarantee operationally credible.

Rollup State Commitments and Inboxes

A rollup contract usually maintains an ordered inbox and a sequence of accepted state commitments. The inbox binds the L2 to data posted on L1, including force-included transactions. A simplified commitment might be:


```
BatchCommitment {
    previous_state_root
    new_state_root
    inbox_start
    inbox_end
    transaction_data_hash
    l2_block_range
}
```

The transition rule states that executing inbox items and batch data from `previous_state_root` must produce `new_state_root`. An optimistic rollup lets a proposer assert this relation subject to challenge. A validity rollup requires a proof of it.

This structure prevents a proposer from proving an arbitrary computation unrelated to user messages. It also lets bridge contracts authenticate an L2 withdrawal against an accepted root.

Interactive Fault Proofs

Re-executing a large batch on Ethereum would remove the scaling benefit. An interactive game instead commits both parties to execution traces. If a trace contains 2^n steps, repeated bisection isolates one disputed step in about n rounds.

Suppose the proposer claims a final machine state and the challenger computes another. They compare a midpoint commitment. Whichever half disagrees becomes the next interval. Eventually the L1 contract executes one instruction against an agreed pre-state and decides which trace is correct.

Implementation details matter:

- the machine state must have a canonical hash;
- instruction semantics on L1 must match L2 exactly;
- deadlines prevent one party from stalling;
- bonds cover verification cost and discourage spam;
- the challenger needs all batch data;
- the system needs at least one working path to submit the challenge.

A permissionless game can still be practically centralized if proof software is difficult to run or the bond is prohibitively large.

Validity-Proof Pipeline

A validity rollup converts execution into an arithmetic statement. The witness contains transaction data, signatures or signature-verification inputs, prior state paths, and intermediate values. The circuit or zkVM constrains

each transition and exposes public inputs such as old root, new root, and batch commitment.

A production pipeline includes:

1. **trace generation**, turning VM execution into witness data;
2. **witness generation**, filling circuit columns or zkVM memory;
3. **proving**, committing to the trace and producing the argument;
4. **aggregation**, recursively combining several proofs;
5. **verification**, checking one compact proof on the settlement layer.

The prover is an availability component even when it cannot violate safety. If only one prover implementation exists and it crashes on a valid block, finality stalls. Multiple provers, deterministic trace formats, and the ability to reproduce witnesses improve resilience.

Circuit Correctness

Cryptographic soundness proves the encoded relation, not the intended protocol. If a circuit forgets to constrain a value, a proof can be valid for an invalid state transition. Teams use specification tests, differential execution against a reference VM, formal methods for critical gadgets, and independent audits.

Upgrading a zkVM changes the relation being proven. Settlement contracts must bind a proof to a verifier and program version. Upgrade delays give users time to examine new rules and exit.

Data Encoding and Fee Estimation

Batchers reduce bytes by omitting values that can be inferred from order or state, replacing addresses with indices, compressing signatures, and aggregating repeated fields. The decoder must be canonical; two decodings of the same bytes would threaten consensus.

A rough rollup fee estimator is:

```
user fee = L2 gas × L2 gas price
          + user data bytes × expected blob byte price
          + allocated proving and operation cost
          + risk margin
```

Blob prices vary with demand, so batchers estimate future publication cost and may delay low-priority batches. Delay improves compression and cost per transaction but increases latency and the amount of unposted state at risk during a sequencer failure.

Rollup Operations Checklist

A production rollup should document and monitor:

- sequencer uptime, reorganization policy, and forced-inclusion delay;
- batch submission lag and unposted transaction volume;
- DA publication success and retrieval;
- proof or challenge status for every commitment;
- canonical bridge balances and pending withdrawals;
- contract implementation, administrator, and upgrade delay;
- prover diversity and backlog;
- the tested cost and capacity of forced exits.

A block explorer that shows only L2 blocks covers the first step of a longer settlement pipeline.

Deriving a Canonical Withdrawal

Assume a rollup stores withdrawals in a Merkle tree. A user withdrawing 10 tokens receives a message leaf:

```
leaf = hash(
    source_rollup,
    destination_chain,
    withdrawal_nonce,
    sender,
    recipient,
    token,
    amount
)
```

After the state root is accepted, the user submits the message, Merkle path, and root identifier to the bridge. The bridge verifies inclusion, confirms finality under the rollup rule, marks the nonce spent, and transfers the asset.

Marking consumption before transfer avoids reentrancy and replay. Binding both domains prevents the same proof from being reused on another chain. Token mapping must distinguish native assets from representations and handle tokens with unusual transfer behavior.

Fee Estimation Under Volatility and Reorganizations

A user authorizes a maximum fee before the rollup knows the exact batch compression, DA inclusion price, proof allocation, and settlement outcome. The estimator must absorb uncertainty without silently converting every margin into operator revenue.

Quote object

```

FeeQuote {
  chain_id,
  transaction_hash_or_template,
  fee_asset,
  maximum_total,
  execution_component,
  estimated_da_component,
  proof_and_settlement_component,
  price_observation_height,
  quote_expiry,
  refund_rule,
  estimator_version
}

```

A quote is not consensus truth, but binding it to inputs and version makes errors measurable. Wallets should distinguish estimated, maximum authorized, charged, and refunded amounts.

Inclusion delay and price movement

The sequencer observes a base-layer data price at time t_0 , accepts a transaction, and may publish at t_1 . If price rises between them, it can absorb the loss, delay publication, or charge under a previously disclosed rule. It must not retroactively exceed the user's cap.

A quote expiry bounds this exposure. Accepted transactions need a maximum publication delay or cancellation path; otherwise the operator holds free optionality to wait for favorable prices.

Safety margins and reconciliation

Suppose estimated components are:

execution	1,800 gwei
DA	2,500 gwei
proof + settlement	300 gwei
subtotal	4,600 gwei
20% uncertainty margin	920 gwei
maximum	5,520 gwei

If actual allocated cost is 4,850 gwei, a transparent refund returns 670 gwei. If margin is a stated fixed service charge rather than refundable reserve, label it separately. Users should not need to infer operator spread from unexplained quote error.

Compression attribution

Batch compression creates shared savings. Transaction A may reduce B's marginal bytes by repeating the same address. Charging based on final marginal order lets the sequencer choose who receives the benefit.

Stable choices include charging uncompressed bytes, assigning dictionary costs by a canonical rule, or socializing actual batch cost by weighted uncompressed size. Each is imperfect but auditable. Publish both charged proxy and realized compressed cost.

Reverted and dropped transactions

A reverted transaction used execution and publication resources, so a zero fee is not generally sustainable. Charge actual metered work under the signed cap and refund unused gas or reserved components canonically.

A transaction dropped before publication should not pay DA cost. A sequencer acknowledgement that reserves resources may have an explicit cancellation fee, but it must be in the quote and triggered by an objective state.

Reorganizations

An L1 reorganization can remove a published batch. Reposting consumes additional DA and settlement cost. Decide who bears it:

- the operator as normal business risk;
- a transparent insurance reserve;
- users through a bounded protocol rule;
- a faulting party when accountable evidence exists.

Charging the same user twice without renewed authorization violates fee finality. If the original transaction remains valid and is replayed, preserve its total cap across attempts or expose a new quote before resubmission.

Failed batches

A malformed or invalid batch is an operator fault, not user resource consumption under normal assumptions. Users should not fund the replacement publication. A batch rejected because the base layer reorganized or a data market halted is a different class; accounting should name the cause.

Maintain per-batch ledgers linking user charges, posted data transaction, proof job, settlement transaction, refunds, and subsidies. Aggregate profitability cannot reveal whether one user was overcharged.

Fee-token exchange rates

When fees are paid in another token, the quote converts settlement costs using an oracle and spread. Bind oracle value, timestamp, staleness bound, decimal precision, and fallback.

Suppose DA cost is 0.002 ETH and ETH is quoted at 2,000 fee tokens per ETH. Base conversion is 4 tokens. A 3 percent spread gives 4.12 tokens. If the oracle becomes stale, rejecting new quotes is safer than silently using an operator-selected price.

Rate changes between quote and charge follow the signed quote policy. The user should not take unlimited exchange risk after acceptance.

Sponsored and subsidized transactions

A sponsor can pay fees without changing protocol resource use. Bind sponsorship to chain, transaction or policy, maximum amount, expiry, and replay identifier. Prevent an application from exhausting an unlimited sponsor allowance with adversarial calls.

Show the user whether a transaction is fully sponsored, partially sponsored, or may later require another fee for withdrawal. "Free" sequencing can still leave an unaffordable L1 escape.

Estimator evaluation

Replay the historical distribution and inject sudden DA price multiples, compressibility changes, proof backlog, reorganization, fee-token volatility, and batch rejection. Report:

- quote-to-charge error percentiles;
- overcharge refunded and operator deficit;
- acceptance-to-publication delay;
- transactions expired or canceled;
- realized versus allocated compression;
- cost by failure and repost class;
- subsidy and sponsor exhaustion.

Invariants

For each authorization:

```
charged + refund <= maximum authorized
charged = named resource charges + named service charge - subsidy
```

Across a batch:

$$\text{allocated shared costs} = \text{realized shared costs} + \text{disclosed reserve adjustment}$$

Allowing rounding dust requires a bounded, named destination.

Fee estimation is part of user safety. A scalable rollup should expose uncertainty, bound authorization, reconcile shared costs, and preserve the fee cap through failure rather than using operational complexity as permission for arbitrary charges.

Fault-Proof Game Economics

A challenger spends compute to replay batches and capital to post bonds. If rewards do not cover monitoring and transaction cost, "one honest challenger" may exist in theory but not operation.

The protocol can pay successful challengers from proposer bonds. Bonds must be large enough to deter false assertions and cover dispute cost, but not so large that only a few actors can participate. Challenge transactions also compete for L1 inclusion during congestion. Systems can sponsor challengers, run several independent watchers, and pre-fund accounts.

Permissionless participation should be tested: a new challenger using public software and ordinary infrastructure must be able to reproduce state, detect an invalid assertion, and complete the game.

Prover Performance Engineering

Proof generation is a pipeline of CPU, GPU, memory, storage, and network work. Trace generation may be sequential even when polynomial commitments parallelize. Large witnesses may exceed accelerator memory and require partitioning.

Measure proofs per unit time, time to first proof, peak memory, accelerator count, energy, and cost. Report the program and workload because cryptographic operations, memory accesses, and control flow affect circuits differently.

Recursive proof systems split a block into segments, prove segments in parallel, and aggregate them. Segmentation shortens the critical path but adds recursive overhead. A scheduler balances segment size, available hardware, and settlement deadline.

Prover Market Scheduling and Failure Recovery

A validity rollup may use several independent provers competing or taking assignments. Competition can reduce latency and operator dependence, but

the market must preserve deterministic inputs, proof compatibility, confidentiality, and a safe response when every prover misses the deadline.

Proving job

A scheduler should identify one immutable unit of work:

```
ProvingJob {
  chain_id,
  batch_range,
  pre_state_root,
  post_state_root,
  data_commitment,
  execution_program_hash,
  circuit_version,
  public_input_hash,
  proof_system,
  deadline,
  maximum_payment,
  job_nonce
}
```

Every bidder prices the same job. If the program hash, circuit, or public inputs can change after assignment, price comparison is meaningless and a proof may verify the wrong transition.

Large batches may be split into segment proofs and recursively aggregated. The dependency graph must bind segment order and boundaries so two individually valid segments cannot omit or overlap a transaction range.

Assignment models

A market can use:

- **open race:** first valid proof earns payment;
- **auction:** a prover commits to price and deadline;
- **round robin:** registered provers receive predictable assignments;
- **redundant assignment:** several provers work in parallel;
- **primary/standby:** one starts immediately and backups start after checkpoints.

An open race minimizes scheduling but wastes compute. A single winner auction is efficient under normal operation but creates deadline risk. Redundant assignment costs more but may be rational for high-value or time-critical batches.

Bid and bond

A bid should bind job hash, price, delivery deadline, prover identity, software or capability class, and bond. The bond penalizes objectively provable non-delivery or malformed submission only under rules that distinguish prover failure from unavailable inputs or settlement outage.

Do not slash a prover because the scheduler sent inconsistent data. Publish receipt hashes for assigned inputs and make cancellation states explicit. A dispute needs evidence that both parties can verify.

Suppose a proof pays \$180, expected compute and energy cost is \$120, and a missed deadline loses a \$300 bond. If the prover estimates a 5 percent miss probability:

$$\text{expected profit} = 180 - 120 - (0.05 \times 300) = \$45$$

At a 25 percent miss probability, expected profit becomes $180 - 120 - 75 = -\$15$; a rational prover should decline or bid higher. Markets that hide deadline risk attract unreliable bids or centralize around operators able to absorb losses.

Input availability

A prover needs batch data, prior state or witnesses, execution trace, program artifacts, and parameters. The scheduler should provide content-addressed inputs from redundant stores. A valid commitment without retrievable bytes cannot produce a proof.

Measure time to acquire inputs separately from proving time. If a 40 GB witness takes 12 minutes to download and 8 minutes to prove, optimizing the circuit by 20 percent saves less than improving distribution.

Confidential transactions or private application state may require trusted execution environments, encryption, or restricted assignment. State what the prover learns. A proof hides witness details from the verifier only if the proof system is zero knowledge; sending the witness to a prover is a separate disclosure.

Checkpoints and resumability

Long proving jobs should emit authenticated progress checkpoints when the proof system permits. A checkpoint is useful only if another compatible worker can resume it or if it proves that an assignment reached a payment milestone.

Bind checkpoints to the exact job, prover software format, and stage. Treat deserialization as hostile input. Version formats, cap sizes, and never let a resume artifact change public inputs.

If jobs cannot migrate, call checkpoints telemetry rather than failover. A backup must restart from inputs, and deadline planning must include that cost.

Proof verification and payment

Payment follows successful verification of the proof against the registered program and public inputs. A scheduler's "complete" status is insufficient. Submitters must not be able to replay one proof for several jobs or claim both segment and aggregate rewards without policy.

Record proof hash, verifier version, job nonce, submitter, verification result, settlement transaction, and payment. If verification is performed off-chain before on-chain submission, the on-chain verifier remains authoritative.

Capacity planning

Let batches arrive every 10 seconds, while one prover needs 40 seconds per batch. Ignoring variance, at least four equally capable provers are needed to match arrival rate:

$$40 \text{ s proving} / 10 \text{ s arrival} = 4 \text{ concurrent provers}$$

At 70 percent target utilization for bursts and failures:

$$4 / 0.70 \approx 5.72$$

Provision at least six equivalent prover slots. Heterogeneous hardware, aggregation bottlenecks, witness generation, and queue variance require a measured model rather than rounding this formula into a guarantee.

Track arrival rate, service rate, queue age, p50/p95/p99 proof latency, failure rate by stage, GPU memory, host bandwidth, and aggregation depth. A queue can be stable on average while deadline misses cluster during large or adversarial batches.

Market concentration

Count effective capacity, wins, payments, hardware supply chain, cloud regions, codebases, and ownership. Ten registered provers renting the same scarce accelerator and using the same prover implementation can fail together.

Avoid reputation rules that permanently favor incumbents. Publish qualification tests, permit new provers to shadow or prove historical jobs, and cap the damage a new prover can cause without preventing entry. Verification protects correctness; scheduling policy mainly protects latency and resources.

Missed deadlines

When the primary misses:

1. keep the prior accepted state safe;
2. expose whether failure is input, witness, compute, aggregation, verification, or settlement;
3. assign backups using the same immutable job;
4. extend publication only under a bounded protocol rule;
5. stop accepting an unbounded amount of new unproven state;
6. activate the documented escape path if the proof window cannot recover.

Continuing sequencing indefinitely while proofs lag creates a growing roll-back and withdrawal boundary. Define a maximum unproven batch count, elapsed time, or value at risk.

Adversarial tests

Submit malformed bids, duplicate job claims, stale circuit versions, correct proofs for wrong roots, oversized checkpoints, partial input stores, slow proofs that hold assignments, and valid proofs just after deadline. Kill the primary at every proving and upload boundary. Remove the dominant cloud region and common GPU model.

Assert one payment per accepted job, no state acceptance without a valid registered proof, deterministic reassignment, bounded queue growth, correct bond outcomes, and user exit when the market cannot recover.

A prover market decentralizes liveness only when jobs and artifacts are portable, verification remains permissionless, capacity survives correlated loss, and missed proofs stop the unsafe frontier before they endanger already accepted user state.

Sequencer Architecture

A sequencer commonly has an RPC/mempool layer, admission controls, ordering engine, execution engine, state database, block builder, and batch publisher. High availability can use an active-passive replica or a consensus group.

Replicating a sequencer introduces its own fork-choice problem. If two replicas issue conflicting soft confirmations, users need a rule for which survives. One approach gives only an elected leader signing authority; another uses a quorum certificate for each L2 block. The latter improves fault tolerance but adds latency.

Admission control protects the sequencer from transactions designed to consume simulation or storage resources without paying. Nonce gaps, replace-

ment transactions, invalid signatures, and underpriced data must be bounded before they fill queues.

Decentralized Sequencing Trade-Offs

Rotating sequencers reduce dependence on one operator but require consensus on L2 order and state. Permissionless participation needs stake or another Sybil-resistance mechanism, networking, penalties, and a way to distribute fees.

Decentralization can worsen latency and make reorganization behavior visible to users. A protocol may separate fast preconfirmations from final L2 consensus. Wallets and applications must know which promise they received.

Based sequencing inherits ordering from L1 proposers. Shared sequencing amortizes consensus across rollups. Each choice changes censorship, MEV, latency, and cross-rollup composition rather than producing one universal measure of sequencing decentralization.

Sequencer Decentralization and Failover Protocol

A decentralized sequencer is not merely several machines behind one endpoint. The system must define who may order transactions, how a leader is selected, what a signed soft confirmation means, how state passes to the next leader, and how users make progress when the service fails.

A sequencer set can use rotating leaders, proof-of-stake consensus, a shared external sequencer, or ordering inherited from the settlement layer. Each choice changes latency, censorship resistance, equivocation evidence, and the complexity of recovery.

Sequencer state

Every sequencer replica should persist at least:

```
SequencerState {
    chain_id,
    epoch,
    view,
    leader,
    unsafe_head,
    safe_l1_origin,
    inbox_cursor,
    next_l2_height,
    accepted_transaction_ids,
    confirmation_signing_state,
    protocol_version
}
```

`unsafe_head` may include unpublished blocks; `safe_l1_origin` identifies the settlement data from which derivation is stable under policy. Mixing them causes a failover replica to build on data that other nodes cannot reconstruct.

Persist signing state before releasing a confirmation. After a crash, a replica must not sign a conflicting block or preconfirmation for the same height and view merely because its memory was lost.

Normal leader rotation

1. replicas agree on the current epoch and leader schedule;
2. the leader selects transactions plus mandatory inbox messages;
3. replicas validate the proposed block and its L1 origin;
4. the required quorum certifies the order;
5. the service returns a confirmation naming its strength and expiry;
6. the batcher publishes enough data for independent derivation;
7. the next leader starts from the highest certified and available block.

If one operator controls every signing key or every replica database, quorum messages do not create an independent failure domain. Report operators, keys, hosting, client implementations, and settlement access separately.

Soft confirmation semantics

A confirmation should bind:

```
Preconfirmation {
  chain_id,
  l2_height,
  transaction_hash,
  ordered_position,
  l1_origin,
  expiry,
  sequencer_epoch,
  view,
  signer_or_quorum,
  protocol_version
}
```

The wallet must distinguish "received," "sequencer-ordered," "data published," "state accepted," and "settlement final." A preconfirmation can support low-risk UX, but it cannot be presented as settlement if another rule can still remove or reorder the transaction.

Define objective equivocation evidence. Two valid signatures for incompatible commitments in the same domain should be slashable or otherwise penalized. A promise whose violation cannot be proven is a service-level claim, not a cryptographic guarantee.

Crash before publication

Suppose leader S1 confirms 2,000 transactions and crashes before publishing the block. Successor S2 has three possible starting points:

- a certified block body replicated before confirmation;
- a certified commitment but unavailable body;
- no certified record beyond the last published batch.

Only the first allows seamless continuation. With a commitment but no body, replicas must recover data before building on it or abandon it under a rule users understand. With no durable certificate, transactions return to the pending pool and earlier soft confirmations expire or are marked broken.

Replicate the complete block body to a quorum or DA service before issuing a strong confirmation. Replicating only a hash proves what was promised but does not make the state recoverable.

View-change protocol

On leader timeout, replicas send signed new-view messages containing their highest certified block and publication status. The next leader selects the highest safe parent under deterministic tie-breaking and repropose pending work.

Timeouts that are too short cause churn during latency spikes; timeouts that are too long extend censorship and outage windows. Use adaptive operational alerts, but keep consensus transitions deterministic. Expose current view, leader, timeout cause, highest certificate, unavailable block bodies, and mandatory-inbox age.

A view change must not skip forced messages whose deadline is near. Reserve block resources before ordinary transaction selection and carry the authoritative inbox cursor into the new view.

Settlement-layer reorganization

A leader may derive from L1 origin 0 that later reorganizes away. The sequencer set should rewind to the common safe origin, deterministically discard or requeue dependent L2 transactions, and issue a visible status change.

Transactions sourced from reverted deposits cannot remain valid unless protocol rules explicitly fund them another way. Transactions independent of the reverted input may be replayable, but replay should re-evaluate nonce, fees, and state rather than copying an old result.

All replicas must use the same L1 fork-choice inputs. If some see one provider and others another, the set can stop or equivocate. Operate independ-

ent L1 nodes and cross-check finalized and safe heads; a load balancer over one provider account is not diversity.

Membership change

Activate a new sequencer set at a settlement-authenticated epoch boundary. The transition must bind old and new sets, threshold, activation height, protocol version, and pending confirmation policy.

Outstanding preconfirmations need a rule: the old set publishes them before handoff, the new set inherits the certified block bodies, or they expire before activation. Rotating keys without transferring availability can make valid promises impossible to fulfill.

Test removal of a malicious member, emergency threshold loss, and a delayed member still signing the old epoch. Domain separation must make old-epoch signatures invalid after activation without invalidating already settled history.

Censorship recovery

A decentralized set can still censor if a quorum agrees or all ingress passes through one gateway. Users need independent submission routes and a settlement-layer inbox whose consumption deadline is enforced.

Measure time from forced submission to inclusion under sequencer outage, not only ordinary API latency. Test transactions that competitors dislike, transactions from blocked network regions, low-fee valid transactions, and forced messages arriving during view changes.

Worked availability envelope

Assume seven sequencer operators and a five-of-seven ordering threshold. The protocol tolerates two unavailable operators. If three operators share one cloud region, a regional outage can remove the threshold even though no operator individually failed.

If block bodies are stored by only the leader and one backup, ordering may continue to certify unavailable data. Require the availability threshold to match the recovery claim. For example, a five-signature certificate could require each signer to attest that it has the body, but operations must test whether those copies are independently retrievable and durable.

Failover drill

Run a continuous workload while:

1. killing the leader before proposal, after proposal, after certificate, after user confirmation, and during publication;
2. partitioning a minority, then a threshold of replicas;

3. corrupting one replica's persisted view and signing state;
4. reorganizing the L1 origin;
5. filling the forced inbox near its deadline;
6. changing membership with outstanding confirmations;
7. removing the largest common cloud or network dependency;
8. restoring nodes from durable state.

Assert no conflicting settled order, no duplicate inbox consumption, deterministic parent selection, bounded broken confirmations, eventual forced inclusion after assumptions recover, and exact reconciliation between confirmed, published, dropped, and queued transactions.

A decentralized sequencer is production-ready when ordering keys, data, ingress, settlement views, and operations survive independent failures. Replica count alone is not the property; safe handoff and user recovery are.

Rollup Upgrade and Escape Testing

Before an upgrade, operators should replay historical blocks against the new implementation, compare state roots, test bridge messages, and execute the forced path. A canary deployment or shadow prover can find divergence before activation.

The upgrade announcement should publish code hashes, verifier addresses, activation time, audit results, and user exit deadline. Emergency changes need narrower scope and a postmortem. An escape hatch that an upgrade can silently disable is not independent protection.

Fault-Proof Pipeline: From Assertion to One-Step Dispute

An optimistic rollup accepts an assertion unless a challenger proves it inconsistent with the transition rules. The production system includes more than a challenge window: it needs an assertion graph, bonds, authenticated data, interactive narrowing, a one-step verifier, clocks, and permissionless participation.

Assertion

A proposer submits a claim derived from a parent:

```
Assertion {
  rollup_id,
  parent_assertion,
  l2_block_range,
  pre_state_root,
  post_state_root,
  data_commitment,
```



```

    inbox_position,
    vm_version,
    proposer_bond
}

```

The contract checks structural rules and starts a clock. It does not re-execute the batch. The data commitment and VM version must select the bytes and transition function a challenger will use.

An assertion may be valid or invalid independent of who proposed it. A trusted proposer is not a correctness proof; a permissioned proposer primarily controls liveness and censorship.

Challenger reproduction

A challenger retrieves the committed batch data, reconstructs the pre-state or required witnesses, and executes the pinned VM. If its computed post-state differs, it opens a dispute before deadline and posts the required bond.

This path must be economically and operationally open. If data access, state snapshots, or a proprietary VM build are available only to the operator, "permissionless challenge" is nominal.

Bisection

Re-executing an entire batch on L1 is too expensive. The parties commit to an execution trace and repeatedly narrow the disagreement.

If the trace has N steps, binary bisection takes approximately:

```

ceil(log2(N))

```

rounds to isolate one transition. For $N = 2^{30}$ machine steps, at most 30 bisection choices locate the disputed step, though each on-chain round also consumes confirmation and response time.

At each round, both sides bind their claimed intermediate state. The protocol chooses the half where commitments disagree. Domain separation includes game identity, round, interval, and trace commitment so a proof cannot be replayed into another dispute.

Clocks and timing

A chess-clock design gives each side a total response budget rather than re-starting a full window every round. The contract must define whose clock runs, when a move becomes effective, how L1 reorganizations affect inclusion, and what happens when both parties submit near a boundary.

Suppose each side has a 3.5-day clock and ordinary L1 inclusion p99 is two minutes. Thirty interactive rounds do not automatically require 60 minutes

because parties may consume variable time, but automation needs enough margin for monitoring, proof construction, fee spikes, and reorgs. Timeout tests should target exact block boundaries and delayed transactions.

One-step proof

After narrowing, the L1 verifier checks one transition from pre-step state to post-step state. Depending on design, it verifies a VM instruction, memory and register witnesses, Merkle proofs, inbox access, and output commitment.

The one-step verifier is a consensus-critical implementation of VM semantics. A mismatch between native rollup execution and this verifier can reject valid state or accept invalid state. Differential vectors must cover opcodes, exceptions, memory expansion, calls, precompiles, gas, and host inputs.

Resolution and bonds

If the one-step proof shows the assertion invalid, the contract rejects it and descendants depending on it. If the challenge is invalid or times out, the assertion advances toward acceptance. Bond allocation rewards useful participation and deters spam, but must not price honest challengers out.

Economics need to cover:

- L1 gas across worst-case rounds;
- capital locked for the full game;
- data retrieval and state reconstruction;
- computation and monitoring;
- concurrent disputes and denial-of-service;
- volatility of the bonded asset.

A proposer bond lower than extractable bridge value may still work if invalid assertions cannot finalize while one honest challenger acts. The bond funds deterrence and operations; cryptographic/game correctness protects the state. However, challenger rewards must support an actual monitoring market.

Multiple games and denial of service

An attacker may create many claims or challenges to exhaust honest capital and computation. Limit policies must avoid giving one administrator power to suppress a valid challenge. Defenses include per-claim bonds, bounded game trees, shared computation, proof caching, and priority for games closest to finalization.

Game implementations need garbage collection. Resolved descendants, bonds, and trace data should be finalized without deleting evidence before appeals or monitoring complete.

Fault-proof assertions

A release test should assert:

1. an invalid state root cannot pass when one challenger has data and responds on time;
2. a valid assertion survives malicious challenges;
3. every bisection round shrinks the disputed interval and binds one trace;
4. timeout results are deterministic at block boundaries and through shallow reorgs;
5. the one-step verifier agrees with native execution on generated and adversarial vectors;
6. duplicate and concurrent games cannot settle the same bond or assertion twice;
7. parent rejection invalidates dependent state safely;
8. any qualified challenger can participate without operator credentials;
9. worst-case gas, time, and capital fit the published challenge assumptions.

Run the complete game periodically in production-like staging. A deployed contract that has never resolved an intentionally invalid assertion remains an untested safety mechanism.

Circuit Versioning, Trusted Setup, and Verifier Migration

A validity proof is meaningful only relative to a precise program and verifier. Upgrading the application while leaving an old verifier reachable, or upgrading a verifier without binding the proof's circuit version, can create a path that accepts the wrong transition.

Program identity

Define one proof domain:

```
ProofDomain {
    chain_id,
    rollup_id,
    circuit_family,
    circuit_version,
    execution_program_hash,
    verifier_hash,
    public_parameter_hash,
    activation_height
}
```

Public inputs and proof submissions bind this domain. A proof generated for a test deployment, earlier circuit, different fork, or different setup ceremony must fail even if its byte format is otherwise valid.

The **execution program hash** identifies the state-transition logic being proven. The **verifier hash** identifies the on-chain code or verification key. Both matter: correct program logic with a misconfigured verifier is unsafe, and a correct verifier for an older program can accept obsolete semantics.

Trusted setup

Some proof systems use public parameters created through a **trusted setup ceremony**. Participants contribute randomness and should destroy secret intermediate material. Security holds if at least one honest participant destroys its secret under the ceremony's assumptions.

A **universal setup** can support many circuits up to stated limits; a circuit-specific setup applies to one circuit. "Universal" does not mean valid for every size or proof system. Record curve, maximum degree or constraint bound, transcript hash, contribution software, and verification procedure.

Users should be able to verify the final transcript independently. A ceremony with many names but no reproducible transcript is social evidence, not cryptographic verification.

If toxic-waste secret material survives, an attacker may forge proofs without being detected by ordinary verification. Operational controls cannot compensate after the fact; migration requires a new trusted parameter set and verifier, plus a response for state accepted under the compromised system.

Transparent systems

Transparent proof systems avoid a secret setup but still use public parameters such as hash functions, field choices, and security levels. Avoid saying "no assumptions." The trade may include larger proofs, more verification work, or different post-quantum properties.

Version transition

A safe migration chooses a finalized activation boundary H . Batches before H use old circuit C_0 ; batches at or after H use C_1 . The settlement contract rejects proofs whose version does not match the batch range.

Pending jobs need an explicit rule. A batch proved under C_0 but submitted after activation can remain valid if its height is before H ; submission time alone should not redefine semantics. Conversely, a C_1 proof must not validate a pre-activation batch unless the migration specification says so.

Dual verification window

During a shadow period, generate both old and new proofs for the same batches and compare public outputs. The production contract can continue

accepting only C_0 while operators validate C_1 off-chain. This detects mismatches without creating two canonical state paths.

A dual on-chain acceptance window is riskier. If both verifiers can accept the same batch and disagree, an attacker chooses the more permissive path. Prefer height-partitioned acceptance and one canonical state root per batch.

Key and contract deployment

Verify deployment bytecode, constructor inputs, verification key, proxy implementation, initialization state, owner, upgrade authority, and chain ID. Reproduce the address when deterministic deployment is part of the plan.

A verifier contract that is correct but uninitialized may let the first caller set an owner or key. An old implementation left reachable through another proxy or bridge route can remain an acceptance path. Map every caller, not only the main interface.

Proof compatibility matrix

Test:

Proof	Batch	Expected result
C_0 valid	before H	accept
C_0 valid	at/after H	reject
C_1 valid	before H	reject unless specified
C_1 valid	at/after H	accept
wrong chain/rollup	any	reject
altered public input	any	reject
correct proof, wrong verifier key	any	reject
malformed version encoding	any	reject before expensive work when possible

Add empty batches, maximum-size batches, recursive aggregates spanning H, proofs already in the mempool at activation, and settlement reorganization across H.

Recursive and aggregated proofs

An aggregation circuit that verifies child proofs embeds or commits to their verifier definitions. Upgrading a leaf circuit without updating the aggregation path can make new proofs unusable; accepting a generic child verifier identifier can make old or unintended proofs admissible.

Bind child circuit version, batch range, order, and public-input hash into the aggregate. Prevent overlap and gaps. A proof covering batches 10-20 plus another covering 20-30 must not double-apply batch 20.

Emergency response

If a soundness bug is suspected, stop accepting new proofs under the affected domain and preserve the last independently justified state. Do not switch to an operator-signed root as if it carried the same guarantee.

Record all accepted proofs and batches in the vulnerable window. Re-execute from a known safe state with corrected logic, compare user balances and messages, and publish the canonical recovery evidence. The response may require governance, but governance should choose among verifiable recovery artifacts rather than invent state.

Release assertions

Before verifier activation, require:

- reproducible program and verifier hashes;
- verified setup transcript or documented transparent parameters;
- complete cross-version negative tests;
- shadow proof agreement over production-shaped workloads;
- recursive aggregation compatibility;
- settlement reorganization testing around activation;
- old acceptance paths disabled as specified;
- user exit window when assumptions materially change;
- rollback or forward-fix runbook that preserves asset and message accounting.

Circuit upgrades are consensus upgrades for the rollup. Treat proof domains, setup artifacts, verifier code, activation boundaries, and recovery evidence with the same discipline as an L1 state-transition change.

Proving Pipeline: From Execution Trace to L1 Verification

A validity rollup does not prove "the block" as an informal object. It proves that a precisely encoded program accepted private witness data and public inputs. The engineering pipeline must keep execution, trace generation, arithmetization, proof creation, and contract verification on the same version.

Statement and witness

Public inputs commonly bind:

```

rollup identity
protocol and circuit version
parent and post-state roots
transaction-data commitment
inbox and withdrawal roots
batch number or range

```

The witness contains transaction fields, signatures or signature-verification auxiliaries, Merkle paths, pre-state values, execution intermediates, and any tables needed by the proof system. Data may be private to the proof while still needing separate publication for rollup availability.

A proof that omits the rollup identity may replay across deployments. A proof that omits the data commitment can establish a state transition without tying it to the bytes users downloaded. A proof that omits the program version may verify under rules different from the batch's declared semantics.

Constraint generation

An execution trace is converted into algebraic constraints. Each row or step encodes machine state, opcode semantics, memory, storage, gas, and transitions to the next step. Lookup arguments can prove that values belong to fixed tables such as byte ranges or opcode metadata without repeating every constraint.

The prover must constrain failure paths as carefully as success. A signature failure, out-of-gas exception, revert, and invalid opcode each have deterministic effects on state and receipts. An unconstrained branch can let a prover choose a convenient result not produced by the VM.

Circuit capacity is often measured in rows or constraints, not transactions. One cryptographic operation can consume more proving work than many transfers. A batcher therefore tracks execution gas, published bytes, and proving shape separately.

Witness generation and proving

Witness generation re-executes or instruments the block to fill every constrained cell. It is frequently memory- and storage-intensive. Proof generation then commits to the trace, derives challenges, constructs polynomial or hash-based arguments, and emits a succinct proof.

A production job record includes:

```

ProofJob {
  job_id,
  batch_range,
  program_hash,
  public_input_hash,

```

```
witness_commitment,
priority,
deadline,
attempts,
prover_build,
result_proof_hash
}
```

Jobs must be idempotent. A worker restart should either resume safely or reproduce the same public statement. Different valid proofs may have different bytes because of proof randomness, so compare their verified public inputs and result, not only proof hashes.

Recursion and aggregation

When one batch exceeds circuit capacity, a rollup can prove segments, then recursively verify segment proofs inside an aggregation circuit. Aggregation amortizes L1 verification but introduces another program version and dependency graph.

The aggregator must bind segment order, continuity of state roots, complete batch coverage, and absence of duplicate segments. If segment i ends at root R , segment $i+1$ must begin at R . Sorting proofs by an untrusted job identifier without constraining root continuity can combine individually valid pieces into the wrong history.

On-chain verification

The settlement contract reads the proof and public inputs, selects the correct verifier, checks that the parent root equals the last accepted root, and updates state only after successful verification. Verifier upgrades need a timelock and explicit circuit-version activation.

Verification gas should be measured with worst-case public inputs and contract storage behavior. A cheap cryptographic verifier can still become expensive if it writes many roots, messages, or accounting records.

Prover failure and fallback

Proof correctness protects safety; prover availability controls liveness. A queue can grow because of hardware loss, a pathological transaction, witness-service outage, circuit bug, or demand burst. Operators should expose oldest unproved batch, queue work in constraint-seconds, attempt count, GPU/CPU utilization, witness generation time, and estimated settlement delay.

Redundant workers help only if they do not share one code build, cloud region, witness database, or coordinator. A fallback prover should be exercised before an incident. If the protocol permits an escape mode after prolonged

proof failure, the activation condition and state reconstruction data must be public and testable.

Proving capacity calculation

Suppose batches arrive every 12 seconds. Average proof time is 42 GPU-seconds, but p95 is 72 seconds. Mean offered proving load is:

$$42 / 12 = 3.5 \text{ GPU equivalents}$$

Four workers provide only 12.5 percent mean headroom and cannot absorb long p95 jobs or one-worker failure. At six workers, mean utilization is about 58 percent. If one worker fails, utilization becomes 70 percent. Queue simulation should use the measured proof-time distribution and correlated batches, not only the average.

If recursion aggregates 32 batch proofs and takes an additional 180 GPU-seconds, its amortized load is:

$$180 / (32 \times 12) \approx 0.47 \text{ GPU equivalents}$$

That stage needs its own queue and redundancy. A stable leaf-proof queue can coexist with an unstable aggregation queue.

Differential and adversarial testing

For each VM test vector, compare native execution, witness generation, proof verification, and a second independent implementation where available. Mutate every public input and confirm verification fails. Generate invalid traces for signature, nonce, balance, gas, memory, storage, logs, and withdrawal roots.

Test circuit boundaries: zero transactions, maximum rows, one step over capacity, largest lookup table, deepest call stack, maximum public inputs, and version transition. Crash workers after witness generation, during proof creation, after upload, and before coordinator acknowledgement. The coordinator should avoid duplicate state acceptance while allowing redundant proof production.

A proof system is production-ready when the statement is complete, execution and circuit semantics agree, every version is pinned, queues remain stable under realistic distributions, and independent parties can reproduce verification. Succinct proof size alone establishes none of those properties.

Cross-Rollup Withdrawal and Liquidity Operations

A user moving assets between rollups may use canonical withdrawals, third-party liquidity, or a bridge protocol. Fast liquidity changes who fronts the waiting period; it does not make settlement finality instantaneous.

Canonical route

For rollups settled on the same L1:

1. burn or lock the asset on source rollup A;
2. include the withdrawal message in A's state;
3. publish data and accept A's proof or challenge result;
4. finalize the containing L1 state;
5. consume the message through a bridge or destination inbox;
6. mint or release on rollup B under its own inclusion and finality rules.

The source message binds both rollup domains, asset, amount, sender, recipient, nonce, expiry, and version. A proof valid for A's bridge must not be replayable into B's other deployments.

Liquidity provider route

A liquidity provider (LP) pays the user on B before the canonical route completes, then claims the delayed asset on A or L1. The LP prices finality delay, reorganization, proof, bridge, inventory, and fee risk.

The user exchanges waiting risk for LP and contract risk. Require a signed quote with exact output, destination, expiry, fee, and refund conditions. The LP's payment must reach the intended recipient with the required finality before its claim becomes releasable.

Inventory imbalance

Flow may be mostly A-to-B, depleting the LP's B inventory while accumulating claims on A. Rebalancing uses the canonical bridge, another LP, market trades, or net settlement.

Track available, reserved, paid-pending-claim, claimable, and disputed inventory per domain. A displayed balance that includes pending claims can overpromise liquidity.

Worked utilization

An LP has 1,000 units available on B and wants 20 percent reserve. It can quote at most 800 units. After accepting three pending transfers of 200 each:

$$\text{available for new quotes} = 800 - (3 \times 200) = 200 \text{ units}$$

If one user cancels before payment under the quote rule, release that reservation atomically. If payment was made, cancellation cannot restore inventory without a refund path.

Quote race and reservation

Two users may accept quotes concurrently. Reserve destination inventory before returning a firm acceptance, with a short expiry and unique quote ID. Database reservation and on-chain payment need reconciliation after crashes.

Idempotency keys prevent paying twice when the user or relayer retries. A payment transaction that times out at the RPC may still land; check canonical chain state before resubmission.

Price and fee risk

Bridge fees and gas can change while a quote is open. Bind a maximum input, minimum output, and quote expiry. The LP bears movement inside that promise unless the quote states an objective adjustment.

For volatile assets, destination output may be fixed in units or value. An oracle-based value promise adds oracle identity, freshness, spread, and manipulation risk.

Reorganizations

If the LP pays after weak source confirmation and source state reorganizes, its claim may disappear. Define source evidence threshold by asset value and cap. Fast routes can require more confirmations for large transfers.

A destination reorganization can remove the user's payment after the LP claims input if claim verification accepts weak evidence. Bind claim release to destination finality, or use collateral that covers reorg risk.

Partial and failed execution

Destination tokens may charge transfer fees or callbacks may fail. Verify effective delivery under supported token semantics. If the LP sent less than the quote after fees, the claim should not receive full input.

A destination call can be optional after asset delivery. Separate "asset received" from "application call succeeded" so one revert does not strand or double-send funds.

Netting

Multiple opposite flows can be netted to reduce canonical bridge transactions. Netting saves fees but creates a batch obligation. Bind included transfer IDs, gross amounts, fees, net positions, and settlement boundary.

A failed net settlement must not erase individual user claims. Preserve a ledger that reconstructs each obligation and prevents one transfer entering two nets.

Insolvency and run risk

An LP can appear liquid while pending claims are invalid, slow, or pledged elsewhere. Publish verifiable on-chain inventory and liabilities where possible. Proof of assets without obligations is insufficient.

Rate limits cap new exposure when claim age, dispute rate, or utilization rises. A circuit breaker can stop quotes while letting users with valid completed payments submit claims or refunds.

User statuses

Expose:

- quote reserved;
- source transfer observed but not final;
- destination payment submitted;
- destination payment final;
- LP claim pending;
- transfer complete;
- refund available;
- disputed with a named boundary.

"Bridged" is too vague for a multi-step fast route.

Reconciliation

Per transfer:

```

user input
= LP claim + protocol fee + refund

quoted destination output
= delivered output + explicit shortfall/refund

```

Across the LP:

```

opening inventory + inflows - finalized payments
= available + reserved + explained adjustments

```

Production tests

Test concurrent quote acceptance, reservation expiry, RPC ambiguity, duplicate relays, source and destination reorgs, fee tokens, transfer-fee tokens,

failed calls, inventory exhaustion, invalid claims, netting failure, operator crash, and LP insolvency.

Fast cross-rollup liquidity is trustworthy when speed comes from transparent inventory and bounded risk, every claim is linked to final delivery, and users retain a canonical or refundable path when the LP disappears.

Rollup Fee Market and Batch-Packing Trace

A rollup fee pays for more than execution. The operator must recover local execution and storage cost, the cost of publishing compressed data, settlement transactions, proof or dispute infrastructure, and a risk margin for volatile base-layer prices.

A useful fee decomposition is:

```

user fee = L2 execution fee
          + allocated DA fee
          + proof or dispute fee
          + settlement overhead
          + operator margin
          - explicit subsidy

```

Each component should be observable or governed by a documented estimator. A single opaque gas price hides which resource is scarce.

Two-dimensional demand

A transaction may be cheap to execute but expensive to publish, or expensive to execute with little data. Model at least:

- L2 execution gas or compute units;
- compressed bytes added to the batch;
- state writes or persistent storage burden;
- proof cost, when transaction shape changes proving work.

One scalar fee can still be presented to users, but its calculation should price each constrained resource and avoid cross-subsidies that attackers can exploit.

Worked fee estimate

Suppose a transaction uses 90,000 L2 gas at 0.02 gwei per gas:

```

90,000 × 0.02 gwei = 1,800 gwei

```

Its batch contribution is estimated at 140 compressed bytes. If L1 or DA publication costs 18 gwei per byte after the protocol's conversion and safety margin:

$$140 \times 18 \text{ gwei} = 2,520 \text{ gwei}$$

Allocate another 300 gwei for proof and settlement overhead:

$$\begin{aligned} \text{estimated fee} &= 1,800 + 2,520 + 300 = 4,620 \text{ gwei} \\ &= 0.00000462 \text{ ETH} \end{aligned}$$

This is an illustrative estimator, not a live network quote. The wallet should show the fee asset, maximum charged amount, refund rule, and which base-layer price observation the estimate used.

Compression is contextual

The marginal compressed size of a transaction depends on its neighbors. Repeated addresses and zero bytes may compress well; random signatures and high-entropy calldata may not. Measuring one transaction alone can overstate or understate its batch contribution.

A deterministic protocol may calculate charges from uncompressed bytes or a stable proxy while the operator bears compression variance. If fees use actual compressed output, transaction ordering could alter what each user pays. Define attribution so a builder cannot move compression benefits to favored transactions.

Batch-packing policy

A batcher chooses transactions subject to several limits:

```
execution_gas <= G_max
compressed_bytes <= B_max
proof_complexity <= P_max
state_writes <= S_max
```

A transaction fits only if it leaves all limits valid. Greedy sorting by fee per gas can fill execution capacity while wasting scarce DA bytes. Sorting only by fee per byte can starve compute-heavy, data-light work.

One approach calculates expected revenue against the transaction's vector of resource use, then packs while maintaining reserves for system messages, forced transactions, and proof constraints. The exact optimization may be heuristic, but consensus must define the resulting batch validity independently of the heuristic.

Forced-inclusion reserve

If users can place transactions in an L1 inbox, ordinary batches need capacity to consume them before the force deadline. Reserving zero capacity until

the deadline lets a sequencer fill every batch with profitable private traffic and then face an impossible backlog.

Let maximum forced-inbox growth be 200 kB per L1 interval and safe rollup consumption be 250 kB. Only 50 kB of recovery margin remains. A short DA-price spike or missed batch can make the queue grow. Monitor arrival and service rates, oldest-message age, and the number of batches needed to clear the queue.

A force path should specify whether inbox work has prepaid L1 cost, pays L2 execution later, or can fail for insufficient funds. Invalid forced messages must not block later messages forever; define skippable failure semantics while preserving order commitments.

Price volatility

The sequencer estimates a future publication cost but may post the batch after the base-layer fee changes. Use a bounded moving estimate, explicit safety margin, and a reconciliation rule. Overcharging without refunds can become a hidden margin; undercharging can make the operator delay publication and weaken user guarantees.

Separate a user's fee cap from the operator's publication decision. A transaction accepted under one quote should have a deadline or cancellation policy if base-layer prices make timely publication uneconomic.

If the system smooths prices across batches, maintain a reserve and publish its accounting. A reserve can absorb short spikes but should not silently socialize persistent losses or let governance redirect user prepayments.

Fee tokens and conversion risk

Charging in a token other than the settlement asset introduces an exchange rate. State the oracle, update frequency, stale-price rule, spread, and who bears conversion risk. During a rapid token decline, an old conversion rate can make fees too low and expose liveness to spam.

Fallback should be deterministic: reject new transactions, require the settlement asset, or apply a bounded conservative rate. An emergency operator quote with no on-chain rule adds discretionary access control.

Congestion and priority

A fee market needs a clear inclusion objective. First-price priority is easy to explain but exposes users to estimation error. Posted prices smooth user experience but need an adjustment rule. Auctions or MEV-aware ordering add complexity and information leakage.

Whatever the policy, system deposits, withdrawals, forced messages, proof updates, and escape transactions may require protected capacity. Document

these lanes and prevent operators from labeling arbitrary private traffic as system-critical.

Refunds and failed execution

A reverted transaction still consumes execution and publication resources. Charge measured work up to the failure boundary, but refund unused fee cap under a canonical rule. Ensure the sequencer cannot manufacture a different refund by changing a non-consensus execution limit.

Deposits that fail on L2 need a recoverable credit or retry path. Users should not lose funds merely because the destination call ran out of gas. Separate asset custody from destination-call success.

Fee-market tests

Replay realistic mixed workloads while varying L1/DA price, compressibility, proof complexity, forced-inbox arrivals, and fee-token exchange rate. Inject missed publications and sequencer restarts. For each transaction, reconcile:

```
maximum authorized
- actual charged
- canonical refund
= zero unexplained remainder
```

Assert that resource limits never overflow, forced work meets its deadline under the stated arrival envelope, failed transactions cannot block the queue, and restarting does not forget prepaid fees or issue duplicate refunds.

Publish estimator error distributions, not only averages: quoted versus charged fee at p50, p95, and p99; operator surplus or deficit; batch utilization by each resource; forced-queue age; and time from sequencer acceptance to DA publication.

A rollup fee market is credible when it prices the resources users consume, preserves mandatory safety traffic under congestion, reconciles every payment, and does not turn base-layer volatility into an undisclosed right for the operator to delay finality.

End-to-End Implementation Example: A Rollup Payment

Consider a minimal account-based rollup supporting deposits, transfers, and withdrawals. The example is intentionally small enough to audit, but its boundaries match production systems.

State and transaction format

The L2 state maps an account identifier to a balance, nonce, and public key. A transfer contains:


```

Transfer {
  chain_id,
  rollup_contract,
  sender,
  receiver,
  amount,
  fee,
  nonce,
  expiry,
  signature
}

```

`chain_id` and `rollup_contract` prevent the signature from being replayed on another deployment. The `nonce` prevents replay within this rollup. `Expiry` bounds how long a censored or delayed transaction remains valid. The signed payload includes `fee` and `receiver` so an intermediary cannot change either.

A deposit is not an ordinary signed L2 transfer. It begins as an L1 event emitted after the bridge receives funds. The rollup derives a unique deposit identifier from the source block, transaction, and event position. The state transition marks that identifier consumed before crediting the account. A reorganization policy defines how much L1 finality is required before the deposit can enter a batch.

Batch construction

The sequencer validates syntax and signatures, rejects stale nonces, selects an order, and executes against a parent state root. It creates a batch header:

```

BatchHeader {
  rollup_id,
  batch_number,
  parent_state_root,
  post_state_root,
  transaction_data_commitment,
  inbox_cursor,
  timestamp,
  protocol_version
}

```

The batch number and parent root make the state chain explicit. The inbox cursor proves which forced L1 messages and deposits have been consumed. The protocol version chooses one deterministic transition function. The data commitment binds the encoded transactions used to reproduce the post-state root.

Before signing the header, an implementation checks three invariants: the parent is the last accepted state, every mandatory inbox item through the cursor was processed exactly once, and re-execution from published data produces the proposed post-state root.

Publication and proof

The operator publishes compressed transaction data to the chosen DA path and submits the header to the settlement contract. These actions must be linked. A contract that accepts a state root without binding it to available transaction data can leave users unable to reconstruct state.

An optimistic design starts a challenge window. Challengers download the data, reproduce execution, and dispute an invalid transition. A validity design proves a statement equivalent to:

```
given parent_state_root and committed batch data,
valid decoding + signatures + nonce rules + balance rules
produce post_state_root and inbox_cursor
```

The public inputs must bind the rollup identity, protocol version, roots, data commitment, and inbox position. Omitting one can make a proof valid for the wrong deployment, program, or batch.

Withdrawal lifecycle

A withdrawal transition debits L2 funds and inserts a message leaf containing source rollup, destination chain, recipient, asset, amount, and unique nonce. After the batch is accepted under the rollup's proof rule, the user supplies a Merkle proof to the L1 bridge.

The bridge verifies the accepted state or message root, checks domain separation and finality, marks the message consumed, then transfers the asset. Marking before transfer follows checks-effects-interactions and blocks reentrancy-based replay. The consumed key should bind every field that distinguishes one withdrawal from another.

A wallet should not show one undifferentiated "complete" state. Useful statuses are:

1. **received** - sequencer accepted the signed transfer;
2. **included** - transaction appears in an L2 block;
3. **data published** - independent nodes can reconstruct it;
4. **state accepted** - proof or challenge rule accepted the batch;
5. **settlement final** - the relevant L1 block is final under policy;
6. **withdrawal executed** - the destination bridge consumed the message.

Failure and recovery table

Failure	Safe behavior	Recovery path
Sequencer stops before inclusion	User funds and nonce remain unchanged	Submit through forced inbox or another sequencer
Sequencer equivocates on soft confirmations	Conflicting promises are visible but not final	Follow canonical published batch; apply preconfirmation penalty if defined
Batch data is missing	Do not accept a state that depends on unavailable data	Reconstruct from DA network or reject/halt under protocol rule
Optimistic batch is invalid	Challenger prevents final acceptance	Execute fault-proof game before deadline
Validity proof is unavailable	State cannot advance, but prior accepted state remains safe	Fail over to another prover or use delayed escape mode
Settlement chain reorganizes	Do not release against the reverted commitment	Re-evaluate batch and message after required finality
Bridge transaction is replayed	Consumed-message check rejects it	No recovery needed; retain evidence and alert
Upgrade changes transition rules	Old and new versions must not silently diverge	Timelocked activation, shadow execution, and user exit window

Test harness

An end-to-end test starts from a known L1 and L2 genesis, deposits funds, transfers them, publishes a batch, proves or challenges it, withdraws, and verifies final balances on both layers. Repeat the flow after process restarts and at every persistence boundary.

Then inject faults: reorder inbox messages, duplicate a deposit, change one encoded amount after commitment, prove against the wrong program version, withhold data, stop the primary prover, reorganize an unfinalized deposit, replay a withdrawal, and activate an incompatible upgrade. Assert both a safety result and an observable status. "The transaction failed" is insufficient; the user and operator need to know which boundary failed and which recovery action is valid.

This small example demonstrates why rollup correctness is not one proof or contract. It is an agreement among transaction encoding, deterministic execution, data publication, proof rules, settlement finality, bridge replay protection, operator persistence, and user-facing status.

Conclusion

Rollups scale execution by batching work and turning Layer 1 into a verifier and data-publication layer. Optimistic rollups use disputes; validity rollups use cryptographic proofs. Their real security also depends on sequencers, bridges, data availability, upgrades, and exit mechanisms.

Rollups do not remove the need to scale Layer 1. They make Layer 1 data capacity more valuable. The next chapters study the modular architecture behind this relationship and the data-availability problem that makes it possible.

References

1. Arbitrum Docs. "Inside Arbitrum Nitro." <https://docs.arbitrum.io/how-arbitrum-works/inside-arbitrum-nitro>. ↩ ↩2
2. Arbitrum Docs. "Overview of BoLD." <https://docs.arbitrum.io/how-arbitrum-works/bold/gentle-introduction>. ↩ ↩2
3. Arbitrum Docs. "Transaction lifecycle on Arbitrum." <https://docs.arbitrum.io/how-arbitrum-works/deep-dives/transaction-lifecycle>. ↩ ↩2
4. Arbitrum Docs. "The Sequencer and Censorship Resistance." <https://docs.arbitrum.io/how-arbitrum-works/deep-dives/sequencer>. ↩
5. ZKsync Docs. "ZKsync protocol overview." <https://docs.zksync.io/zksync-protocol/rollup>. ↩
6. ZKsync Docs. "L1 <-> L2 communication." https://docs.zksync.io/zksync-protocol/era-vm/transactions/l1_l2_communication. ↩ ↩2 ↩3
7. ZKsync Docs. "Withdrawal delay." <https://docs.zksync.io/zksync-protocol/security/withdrawal-delay>. ↩
8. Optimism Docs. "OP Mainnet." <https://docs.optimism.io/op-mainnet>. ↩
9. Optimism Docs. "Withdrawal flow." <https://docs.optimism.io/op-stack/bridging/withdrawal-flow>. ↩
10. Starknet Docs. "Transactions." <https://docs.starknet.io/learn/protocol/transactions>. ↩
11. Starknet Docs. "SHARP." <https://docs.starknet.io/learn/protocol/sharp>. ↩
12. Scroll Docs. "Rollup Process." <https://docs.scroll.io/en/technology/chain/rollup/>. ↩
13. Scroll Docs. "Transactions." <https://docs.scroll.io/en/technology/chain/transactions/>. ↩
14. Ethereum.org. "Optimistic Rollups." <https://ethereum.org/developers/docs/scaling/optimistic-rollups/>. ↩
15. Buterin, Vitalik, et al. "EIP-4844: Shard Blob Transactions." <https://eips.ethereum.org/EIPS/eip-4844>. ↩

Chapter 7: Modular vs Monolithic Blockchains

Introduction

A monolithic blockchain asks one protocol and validator network to provide execution, settlement, consensus, and data availability. A modular blockchain separates some of these functions so that specialized systems can perform them independently.

Modularity is not automatically more decentralized or more secure. It creates explicit interfaces and lets each layer scale differently, but the end-to-end system inherits the assumptions and failure modes of every layer it uses.

From One Machine to a Stack of Services

A **monolithic blockchain** asks one validator network to order transactions, execute them, agree on the result, and make block data available. "Monolithic" does not mean badly designed; it means the core jobs share one protocol and security boundary.

A **modular blockchain stack** separates some jobs into different protocols. A rollup may execute transactions, Ethereum may settle its state claims, and a data-availability network may publish the batch bytes. Separation lets each layer specialize, but the user now depends on the interfaces between them.

Think of sending a parcel. One company can collect, transport, clear, and deliver it end to end. A modular route may use a storefront, international carrier, customs broker, and local courier. Specialization can lower cost, but tracking "shipped" is ambiguous unless it names which service completed which step.

Four jobs, four questions

- **Execution:** Given an ordered transaction and prior state, what new state results?
- **Settlement:** Which claimed result is accepted, and where can a dispute or proof be enforced?
- **Consensus:** Which ordered data history is canonical?

- **Data availability:** Can participants obtain the data needed to verify or reconstruct that history?

Consensus can agree on a data commitment without understanding an application's transactions. Settlement can accept a proof without storing every historical query. Execution can calculate a result before another layer considers it final.

Sovereign and settled rollups

A **settled rollup** uses a settlement contract to decide which state root is valid. Its bridge can release assets according to that contract.

A **sovereign rollup** publishes ordered data but its own nodes interpret the rules and choose upgrades or forks. The DA layer establishes what data was ordered, not which software version the rollup community should follow. A bridge to a sovereign rollup must therefore decide how it identifies the canonical fork.

The word **sovereign** describes rule choice, not automatic safety. Users still need data, honest verification, and an explicit bridge or light-client policy.

Messages between layers

Layers communicate through commitments and proofs. A message envelope usually binds source and destination, sender and recipient, payload, nonce, timeout, and protocol version. **Domain separation** prevents valid evidence from one chain or channel from being replayed in another.

A **relayer** transports evidence. When verification is complete, the relayer need not be trusted for correctness: it can delay delivery, but it cannot change a proven amount or recipient. Permissionless relaying means another party can submit the same evidence.

End-to-end security

Security does not add like independent features. A valid execution proof cannot rescue unavailable data; final data does not rescue a bridge with an upgrade key that can mint assets; correct layers do not rescue a decoder that interprets their interface differently.

Use a chain of custody: name the evidence emitted by one layer, the verifier at the next layer, the finality required, and the recovery if evidence stops. The weakest accepted transition bounds the user outcome.

The Monolithic Model

In a monolithic chain, validators:

1. receive and order transactions;

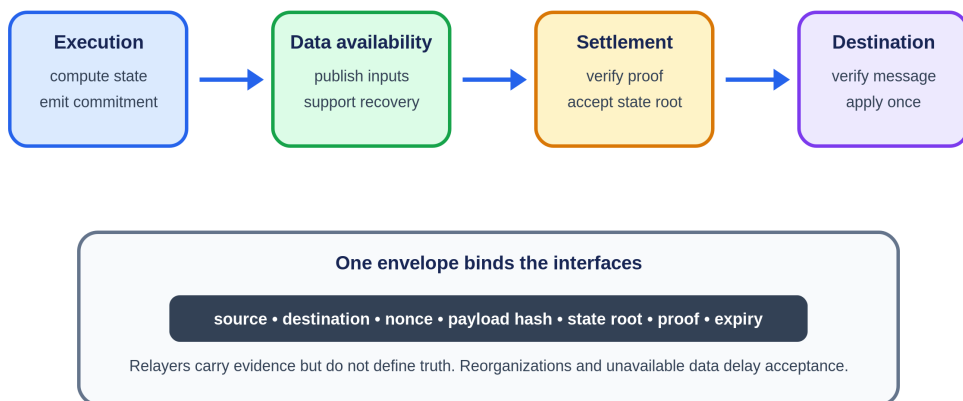
2. execute them;
3. agree on a block;
4. store and distribute the block data;
5. maintain the resulting state.

The strength of this model is integrated security and synchronous composability. A transaction can call several contracts against one state and either complete atomically or revert. Developers and users reason about one fee market, one finality rule, and one validator set.

The weakness is replicated work. Increasing capacity raises the requirements of the same nodes that secure the network. Congestion in one popular application can affect every other application.

The Modular Stack

A modular transaction crosses explicit trust boundaries



End-to-end safety is the composition of every verifier; end-to-end liveness is limited by the slowest required service.

Figure 7.1: A modular transaction crosses execution, data availability, settlement, and destination interfaces. A domain-separated message envelope binds the evidence carried between them. Original figure for this book.

A modular system separates the four jobs:

- **Execution layers** run transactions and compute state transitions.
- **Settlement layers** verify proofs, resolve disputes, and define canonical state.
- **Consensus layers** order data and finalize blocks.

- **Data availability layers** publish enough data for verification and reconstruction.

An Ethereum rollup is a practical example. The rollup provides execution, while Ethereum supplies settlement, consensus, and data availability. A sovereign rollup may instead use a data availability layer for ordering and publication while its own nodes define the canonical state and fork-choice rules.

Why Specialization Helps

Modularity allows different scaling techniques for different bottlenecks:

- execution can use parallel VMs or application-specific logic;
- validity proofs can compress verification;
- data layers can use erasure coding and sampling;
- settlement can remain deliberately conservative;
- many execution layers can share one security base.

It also gives applications sovereignty. A game may prioritize low latency, an exchange may optimize parallel order processing, and a privacy application may use a custom proof system without asking an entire Layer 1 community to change its execution environment.

The Hidden Costs of Modularity

Fragmented Liquidity and State

Assets and applications spread across execution layers. Moving between them requires bridges and asynchronous messages. The integrated composability of one chain becomes a distributed-systems problem.

More Trust Boundaries

A modular transaction can depend on a sequencer, a proof system, a settlement contract, a data layer, and a bridge. A failure in any one can delay finality or block exits.

Different Meanings of Finality

A user may see sequencer confirmation in seconds, data publication later, and settlement finality later still. Applications must decide which stage is sufficient for deposits, withdrawals, and cross-chain messages.

Operational Complexity

Developers need indexers, relayers, proof infrastructure, bridge monitoring, and policies for upgrades across several layers. The stack is easier to customize but harder to observe as a whole.

Celestia and Data Availability Specialization

Celestia focuses on ordering transactions and making their data available rather than executing application state. Light nodes sample small portions of erasure-coded blocks. With enough random samples, they gain high confidence that the complete block can be reconstructed by the network.¹

This allows many rollups to publish data without requiring Celestia validators to execute each rollup. The rollup chooses whether its state is validated by a settlement layer, validity proofs, fraud proofs, or its own full nodes.

The security statement must be precise: the data layer can show that data was published, but it does not necessarily prove that a particular rollup's state transition was valid.

Named Architecture Trace: An OP Stack Chain With Celestia DA

Maturity label checked September 2026: beta integration specification; deployment status is chain specific and requires separate verification.

The OP Stack separates sequencing, derivation, execution, data availability, settlement and governance into named components. The OP alternative data availability specification explicitly labels the integration beta and warns that backward incompatible changes may occur.² Celestia documents an OP Stack integration that moves transaction batch data from Ethereum to Celestia while retaining an Ethereum-facing commitment and retrieval path through configured alternative-DA components.^{3 4 5} The following trace explains the specified architecture. It does not assert that every deployment has production-hardened fallback, fault proofs, bridge configuration, or governance.

Trace Maya's payment on an OP Stack chain configured for Celestia DA. Maya signs an EVM transaction and sends it to the chain's sequencer. The sequencer's execution client checks and executes it, while op-node coordinates L2 block production. Maya receives a fast L2 result under the sequencer's ordering promise. At this point the transaction has executed in the operator's view, but independent derivation still depends on publication.

The batcher collects L2 blocks and compresses their transaction data. In the default Ethereum-DA configuration, that data is posted in Ethereum calldata or blobs. In the Celestia alternative-DA configuration, the batcher sends the

payload to Celestia and receives a commitment or reference under the integration protocol. It then posts the compact DA commitment through the OP Stack's Ethereum-facing data-availability contract path. An independent node reads Ethereum's canonical inputs, sees the alternative-DA reference, retrieves the payload from Celestia, verifies that the bytes match the commitment, and feeds them into the OP derivation pipeline. The execution engine reproduces the same L2 blocks and state.^{3 5}

The settlement path remains another module. An output proposal or dispute-game claim commits to L2 state on Ethereum under that chain's fault-proof configuration. A challenger reconstructing a disputed state needs the Celestia payload. If it cannot retrieve the data required by the configured DA rule, it cannot safely treat the execution claim as reproducible merely because an Ethereum contract stored a short commitment. The integration must define how unavailable alternative data blocks state acceptance or activates a challenge.

A canonical withdrawal now crosses all of these boundaries. Maya initiates an L2-to-L1 message. The message is in an L2 block derived from Celestia-backed batch data. A state claim covering that block is proposed on Ethereum and passes the applicable dispute period. Maya proves and finalizes the withdrawal through the OP bridge contracts. Ethereum holds the escrow and judges the fault-proof game, but the transaction data needed to reconstruct the claim came from Celestia. End-to-end security is the composition, not the strongest module named in the diagram.

Exact trust and liveness assumptions

The sequencer can delay or reorder the fast path. Users need the chain's configured L1 submission or recovery mechanism to bypass it. Celestia validators and data-availability sampling secure publication under Celestia's model, while retrievers and archives determine whether challengers can still obtain the batch throughout the required window. Ethereum orders the DA commitments, state claims and bridge actions and executes the settlement contracts. OP derivation and execution clients must agree on the alternative-DA encoding. The fault-proof program and challenger set must be effective. Governance can replace contracts, chain configuration, DA adapters or proof components.

This deployment does **not** inherit Ethereum blob availability for its batch bytes. It does **not** ask Celestia to execute the EVM transaction or decide the OP Stack state root. Blobstream can relay Celestia data-root commitments to an EVM chain, but it proves facts about Celestia consensus and commitments, not correctness of Maya's application execution.⁶ The OP fault-proof path establishes execution correctness relative to available input and the deployed program.

Observable consequences and failure paths

Operators should connect one batch across systems with durable identifiers: L2 block range and batch hash; Celestia namespace, height and blob commitment; Ethereum alternative-DA commitment transaction; derived L2 safe head; output or dispute-game claim; and withdrawal message. A dashboard that monitors only Ethereum batch transactions can report healthy settlement while Celestia retrieval is failing. A Celestia explorer can show available bytes while the Ethereum commitment adapter or OP derivation client is broken.

Suppose the sequencer produces 100 L2 blocks, publishes their payload to Celestia, but crashes before posting the alternative-DA commitment to Ethereum. Celestia contains bytes, yet canonical OP derivation has no authenticated pointer in its Ethereum input stream. A replacement batcher needs an idempotent recovery rule that posts the same commitment and block range, not a new encoding that forks derivation.

Suppose the Ethereum commitment lands, but Celestia data cannot be reconstructed. Nodes stop advancing the safe derived chain at that input. They must not replace unavailable bytes with a batch fetched privately from the sequencer unless the protocol authenticates the same commitment and the DA acceptance rule is satisfied. The rollup's proof and withdrawal timers must not outrun the challenge data.

Suppose Celestia reorganizes a not-yet-final block containing the blob while Ethereum retains the reference. The integration needs a finality policy and a response: wait before posting, update or invalidate the reference under allowed rules, or halt derivation. A reference to a noncanonical DA block must not silently become valid because Ethereum finalized the reference transaction.

Suppose Blobstream or another bridge carrying Celestia commitments pauses. Celestia can remain available while Ethereum-side verification and settlement stop. This is a liveness failure at the bridge module. If the bridge accepts a false root because its validator or contract assumptions fail, that becomes a safety failure. A circuit breaker can limit value or stop new withdrawals, but it cannot by itself reconstruct missing data or prove the correct state.

Deployment comparison

Layer	Standard OP Stack with Ethereum DA	OP Stack with Celestia DA
Execution	OP execution client/EVM	OP execution client/EVM
Sequencing	Configured OP sequencer	Configured OP sequencer

Layer	Standard OP Stack with Ethereum DA	OP Stack with Celestia DA
Batch bytes	Ethereum calldata or blobs	Celestia blob under integration namespace/rules
Canonical reference	Ethereum batch input	Ethereum alternative-DA commitment/reference
DA security	Ethereum protocol and blob/calldata availability	Celestia consensus, coding/sampling and retrieval assumptions
Settlement	Ethereum OP contracts and fault proofs	Ethereum OP contracts and fault proofs, now dependent on Celestia input retrieval
Added module	None beyond normal OP path	DA adapter, Celestia client/retriever, commitment verification or Blobstream path
Distinct failure	Ethereum data price/capacity	Cross-layer finality mismatch, retriever failure, adapter/bridge failure

The modular design can reduce publication cost or add capacity, and it lets execution and DA evolve separately. Its cost is a longer proof chain. A deployment claim should therefore state the complete route: "OP Stack execution, Celestia data availability, Ethereum settlement through these contracts and this bridge version," followed by the exact maturity, fallback, and upgrade controls. "Secured by Ethereum" alone leaves out the layer whose data makes an Ethereum challenge possible.

Sovereign and Settled Rollups

A **settled rollup** submits state roots to a settlement contract that enforces its validity or dispute rules. Its canonical bridge depends on that contract.

A **sovereign rollup** uses a data layer for ordering and availability, while rollup nodes interpret the data and choose the canonical fork. Upgrades may occur through social consensus among those nodes rather than through a settlement contract.

This is a governance difference as much as a technical one. Settled rollups gain shared enforcement and canonical bridges. Sovereign rollups gain freedom over their state machine and upgrade process.

Monolithic vs Modular

Dimension	Monolithic	Modular
Execution and security	Integrated	Split across layers
Composability	Synchronous within one state	Often asynchronous across layers

Dimension	Monolithic	Modular
Scaling	Bounded by shared validator resources	Specialized and horizontally expandable
Application control	Constrained by base protocol	Custom VMs, fees, and governance
Failure analysis	Fewer interfaces	More dependencies and bridge risk
User experience	One account and fee market	Bridging and varied finality

The choice is not binary. A monolithic chain can host modular rollups, and a modular data layer still has a monolithic consensus protocol. Real systems sit on a spectrum.

A Framework for Evaluating an Architecture

Ask the following questions:

1. Where is execution performed?
2. Who orders transactions, and can users bypass that party?
3. Where is data published, and for how long?
4. Who verifies state transitions?
5. Which layer defines finality?
6. How can users exit during a failure?
7. Who can upgrade each layer?
8. Which messages or bridges connect fragmented state?

These questions reveal more than labels such as L2, appchain, validium, or modular chain.

End-to-End Transaction Walkthrough

Imagine a game rollup that executes on a custom VM, posts data to Celestia, and settles validity proofs on Ethereum. A player signs a move. The sequencer orders it and gives a soft confirmation. The VM updates the player's objects. Encoded data is included by Celestia consensus. A prover generates a transition proof, and Ethereum records the new state root after verification.

The move crosses several finality boundaries. The sequencer can promise an order but fail to publish. Celestia can finalize the data while knowing nothing about the game rules. Ethereum verifies the state only after the commitment and proof arrive. A bridge or marketplace must decide which boundary is sufficient before releasing an asset.

This architecture scales because no validator set performs every job. It is harder to debug: a delayed withdrawal may be stuck at sequencing, DA submission, proving, Ethereum inclusion, or relaying. User interfaces need to name the stalled layer rather than report a generic "pending" status.

Security Composition

End-to-end safety is bounded by the weakest assumption on the path. Correct execution with unavailable data prevents users from constructing new state. Available data with a broken bridge allows theft. Sound proofs with an immediate upgrade key leave governance in control.

A useful engineering artifact is a dependency graph. For each edge, document what is trusted, how failure is detected, what finality is assumed, and whether users can recover without cooperation from the failed component.

Designing the Interfaces Between Modules

A modular architecture succeeds or fails at its interfaces. Each interface should carry enough authenticated information for the receiving layer to verify its responsibility without silently assuming another layer did the work.

Execution to Settlement

An execution layer sends a state assertion containing the prior root, new root, input range, data commitment, and proof metadata. The settlement contract verifies a fault or validity proof and records the accepted root. It should not accept a root without binding it to an exact data range, because otherwise a proof about one batch may be replayed or interpreted against another.

Execution to Data Availability

The rollup publishes a namespaced blob or batch and receives a commitment plus inclusion proof. Rollup nodes need a deterministic rule that maps a settlement assertion to the DA commitment. If the DA chain reorganizes after settlement accepts the assertion, recovery depends on the finality assumptions embedded in the bridge or oracle connecting them.

Settlement to Bridge

A bridge verifies that a withdrawal message belongs to an accepted state root and has not already been consumed. The message needs source chain, destination chain, sender, recipient, asset, amount, nonce, and version. Domain separation is what keeps a proof from one rollup or deployment from being valid in another.

Sequencer to User

A sequencer receipt is a promise, not necessarily final state. It should identify the ordered transaction, L2 block, sequencer signature, and expiry or reorganization policy. Wallets can then label it accurately as pending publication or pending settlement.

Sovereign Rollup Fork Choice

A sovereign rollup does not ask a settlement contract to choose its canonical state. Full nodes read ordered data from the DA layer and apply the rollup's fork-choice and execution rules locally. An upgrade may be a social fork: users choose software interpreting the same data differently.

This gives the community sovereignty but complicates light clients and bridges. A light client needs a way to identify the accepted rollup rules and validator or proof system. A bridge needs its own decision about which fork is canonical. A settled rollup outsources that decision to the settlement contract; a sovereign rollup cannot avoid defining it somewhere.

Modular Liveness Matrix

Failed component	Immediate effect	Safety impact	Recovery path
Sequencer	No fast ordering	Usually none if force inclusion works	Submit through fallback or L1 inbox
Batch submitter	New state lacks published data	Unposted confirmations can disappear	Another submitter posts the batch
DA network	New batches cannot be proven available	Accepting unavailable data can break recovery	Halt settlement, switch only through governed upgrade
Prover	Validity finality stalls	Normally none	Another prover reconstructs witness
Settlement chain	Withdrawals and finality stall	Depends on reorganization/finality failure	Wait or invoke documented social recovery
Bridge relayer	Messages delayed	None if anyone can relay	Permissionless relay
Upgrade multisig	Routine upgrades delayed	None	Governance replacement under existing rules

The matrix distinguishes safety from liveness. A component can be operationally centralized without being able to steal assets, yet its outage can still make the application unusable.

Implementation Checklist for a Modular Stack

1. Pin protocol and verifier versions in every cross-layer message.
2. Use chain and rollup domain identifiers in signatures and proofs.
3. Define finality delays for each source chain before consuming messages.
4. Make relaying permissionless where possible.
5. Expose each pipeline stage in user receipts and monitoring.
6. Reconstruct state from DA using an independent node before launch.
7. Test sequencer, prover, submitter, DA, and settlement outages separately.
8. Document coordinated upgrades when one interface changes.

Cross-Layer Message Envelopes

A modular stack should standardize the envelope carried across layers. One conceptual format is:

```
MessageEnvelope {  
    protocol_version  
    source_domain  
    destination_domain  
    source_height  
    nonce  
    sender  
    target  
    payload_hash  
    expiry  
}
```

The source-domain state commits to the envelope. A relay supplies the envelope plus proof to the destination. The destination verifies source finality, domain and version, expiry, and replay status before dispatching the payload.

Versioning belongs in the signed or committed message. Otherwise an upgrade can cause two layers to decode the same bytes differently. `payload_hash` avoids ambiguous variable-length encodings; the full payload follows a canonical serialization. Nonces can be global, per sender, or per channel, but the destination must enforce exactly one model.

Ordered and Unordered Channels

An ordered channel processes nonce n only after $n-1$. This is easy for applications needing sequence, but one missing message blocks all later work. An unordered channel accepts any unused nonce and requires the application to manage dependencies.

IBC makes this distinction explicit. Modular rollup systems need the same clarity. A token bridge may use unordered transfers, while a replicated state machine may require order.

Relayers Are Replaceable, Not Trusted

A relayer observes a source event and submits its proof to a destination. If the proof is complete and verification is on-chain, a dishonest relayer cannot forge a message; it can only delay or censor its own delivery.

This liveness property depends on permissionless replacement. Message data and proofs must be publicly retrievable, and another relayer must be allowed to submit them. Exclusive relayers turn a replaceable service into a trust boundary.

Fee design pays relayers without letting them alter recipients or amounts. The source message can name a maximum fee, or the destination can reimburse the submitter under protocol rules. Applications should handle duplicate relay attempts safely.

Cross-Layer Reorganizations

A destination consuming a message before source finality risks accepting an event that later disappears. Waiting longer reduces this risk but increases latency. Different source chains provide deterministic finality, probabilistic confirmations, or optimistic assertions.

A bridge policy maps source evidence to destination acceptance. For probabilistic chains, it may require a confirmation depth. For BFT chains, it verifies a finality certificate. For rollups, it waits for challenge or validity-proof completion. This policy should be explicit and upgradeable only with a delay because changing it alters the security budget of every pending message.

Light-Client Bootstrap and Checkpoint Trust

A light client can verify new headers cheaply once it has a trusted starting point. The first accepted header, validator set, or commitment is therefore part of the security boundary. "Verify, do not trust" begins after bootstrap; it does not explain bootstrap itself.

Bootstrap package

A client installation should identify:

```
Bootstrap {
  chain_id,
  genesis_hash,
  checkpoint_height,
```

```

    checkpoint_hash,
    validator_or_committee_root,
    consensus_version,
    checkpoint_expiry,
    distribution_signatures[],
    software_build_hash
  }

```

The package binds the intended network and verification rules. Chain names and token symbols are insufficient because testnets, forks, and malicious deployments can reuse them.

Sources of initial trust

A client may begin from:

- genesis plus verification of every transition;
- a checkpoint embedded in reviewed software;
- a recent checkpoint authenticated by a social or governance process;
- a proof verified by another already trusted chain;
- several independent checkpoint providers under a stated threshold.

Each route has a different cost and assumption. Genesis verification can be impractical for a mobile wallet and may still require weak-subjectivity information in proof-of-stake systems. Multiple providers are useful only when their identities, operations, and failure domains are independent.

Weak subjectivity

In some proof-of-stake protocols, validators can withdraw and later sign an alternative history without risking current stake. A client offline for too long may see two internally valid histories and lack enough current accountability evidence to choose.

A **weak-subjectivity checkpoint** is a sufficiently recent trusted state from which ordinary consensus verification resumes. The client must know the maximum safe checkpoint age under protocol assumptions. "Latest" from one RPC endpoint is not authentication.

If the checkpoint expires, fail closed and ask for a fresh authenticated package. Silently extending its life turns a bounded trust assumption into permanent trust.

Checkpoint distribution

Distribute checkpoints through several authenticated channels: signed release metadata, official domains, package repositories, hardware-wallet updates, or another chain. The channels should publish the exact hash and height, not a link that redirects to mutable content.

Threshold signatures can reduce dependence on one publisher, but signer independence and key recovery matter. A threshold controlled by one build administrator is one trust domain.

Clients should show the checkpoint age and source class in diagnostics. Bridge operators need alerts before any source client approaches expiry.

Eclipse resistance

An **eclipse attack** surrounds a client with attacker-controlled peers and hides honest network data. Even a correct consensus verifier can follow stale history or fail to learn a newer finalized header when all inputs come from the attacker.

Use peers from diverse networks and discovery paths, pin known-good boot-nodes without relying only on them, compare headers through independent transports, and rate-limit peer churn. A wallet using one hosted RPC is not operating a peer-diverse light client.

Conflicting valid evidence should freeze progress and preserve both branches for investigation. Conflicting unauthenticated RPC responses are a provider problem, not proof of consensus failure; diagnostics should distinguish them.

Validator-set and committee sync

A BFT light client must verify how the trusted set authorizes the next set. A sync-committee client must verify committee periods and participation thresholds. Skipping periods may require a proof chain or protocol-specific update that remains within a trust window.

Bound update size and verification work. An attacker should not be able to force the client to process years of useless transitions before rejecting a bad final header.

Clock and freshness assumptions

Some light-client rules compare header timestamps, trusting periods, and local time. A badly wrong device clock can accept stale information or reject valid updates. State the allowed clock drift and obtain time from more than the untrusted peer being checked.

Freshness is application-specific. A read-only balance display can tolerate more delay than a bridge releasing assets. Bind bridge acceptance to an explicit maximum client age and finality policy.

Software and parameter upgrades

A consensus upgrade may change header fields, signature schemes, validator transitions, or domain separation. The light client must select verification

rules by authenticated height or version, not by whatever decoder accepts the bytes.

Ship test vectors across the transition. If old software can no longer verify new headers, it should stop with an actionable version error rather than treating the chain as permanently halted or accepting a compatibility short-cut.

Worked offline-return trace

A wallet last synchronized at height 1,000 with a 14-day trust period and returns after 30 days:

1. it recognizes that its checkpoint is expired;
2. it refuses to accept a new head solely from its normal RPC;
3. it downloads a newly signed checkpoint package through two independent channels;
4. it verifies chain ID, height, hash, signer threshold, software compatibility, and package freshness;
5. it resumes header verification from the new checkpoint;
6. it cross-checks the resulting finalized head across diverse peers.

If the channels disagree, the wallet remains read-only and reports the conflict. User convenience cannot resolve which chain controls real assets.

Production tests

Test first install, offline return just before and after expiry, wrong chain ID, stale but correctly signed checkpoint, compromised minority signer, threshold loss, bad local clock, eclipse by all initial peers, validator-set transition, consensus upgrade, and conflicting checkpoint channels.

A light client is independently useful when it makes bootstrap trust explicit, bounds it in time, verifies every later transition, obtains data through diverse paths, and stops safely when its trusted basis expires or conflicts.

Light-Client Bridge Verification Trace

A light-client bridge verifies the source chain's consensus evidence on the destination chain instead of trusting a fixed signer set. This can reduce discretionary custody, but only if the destination verifier correctly follows source consensus, validator-set changes, and finality.

Consider chain A sending a 10-token transfer to chain B. The bridge on B stores:

```
ClientState {
  source_chain_id,
```

```

latest_height,
latest_commitment_root,
current_validator_set,
trusting_period,
frozen_height,
verification_version
}

```

The transfer message on A is committed under root R_h at height h :

```

Packet {
  source_port,
  source_channel,
  destination_port,
  destination_channel,
  sequence,
  sender,
  recipient,
  asset_id,
  amount,
  timeout_height,
  timeout_timestamp,
  version
}

```

Normal verification path

1. The sender escrows 10 tokens on A, and execution commits packet hash P under R_h .
2. A relayer submits header H_h , its finality certificate, any validator-set transition proof, and a Merkle proof from P to R_h on B.
3. The client contract checks chain ID, header linkage, validator signatures and weights, trusting period, monotonic height, and the consensus rule that makes H_h final.
4. The packet verifier checks the commitment proof, destination, channel version, sequence, timeout, and denomination mapping.
5. The bridge records the packet as consumed before minting or releasing 10 representation tokens.
6. An acknowledgement can be proven back to A; timeout handling is mutually exclusive with successful receipt.

Relayers provide data but no authority. If one relayer withholds a packet, another can submit the same evidence. If a relayer changes the recipient or amount, the commitment proof fails.

Validator-set update

Source validators change from set V_n to $V_{\{n+1\}}$. The destination cannot accept a header solely because it is signed by unknown $V_{\{n+1\}}$. It verifies the transition according to source consensus, commonly by checking that a trusted set finalized a commitment to the next set, then using that set for later headers.

Skipping many heights may require adjacent transition proofs or a protocol-specific overlap rule. The implementation must bound proof length and signature work. An attacker should not be able to submit a million empty transitions to exhaust gas.

Trusting-period expiry

Some clients are secure only if updated within a trusting period shorter than the source chain's unbonding or accountability window. After expiry, the old validator set may no longer be slashable and could sign a conflicting history.

The safe response is to stop, not to accept a convenient new header. Recovery needs an explicitly governed checkpoint or a fresh trusted state. Operators should alert well before expiry and permit anyone to keep the client updated.

Conflicting finality evidence

If two valid-looking finalized headers exist at the same height, the client freezes at that height and rejects new packets. Evidence may indicate source consensus failure, a verifier bug, or forged signatures. Continuing automatically can double mint assets on B.

A frozen client needs a recovery policy: verify accountable misbehavior and a canonical checkpoint, upgrade the verifier if necessary, and reconcile packets whose status is uncertain. "Governance will decide" is not enough; specify who decides, the delay, loss limits, and how users exit.

Replay and channel confusion

Packet sequence 42 on channel x must not authorize packet 42 on channel y , another deployment, or a fork with the same asset symbol. Commitments and consumed-packet keys need domain separation across chain IDs, ports, channels, versions, and packet identifiers.

Mark consumption before the external token call, or use a reentrancy-safe checks-effects-interactions pattern. Test tokens that call back, return no value, charge a transfer fee, rebase, or pause.

Timeout race

Suppose the packet expires at destination height 500. The sender tries to reclaim escrow on A while a relayer tries to prove receipt on B. A safe pro-

toloc defines exactly which chain's height or timestamp controls expiry and requires proof of non-receipt for refunds. Receipt and refund must not both succeed.

Clock-based timeouts inherit assumptions about timestamp drift and light-client freshness. Height-based timeouts inherit assumptions about destination progress. Applications should add slack rather than treating nominal wall-clock equivalence as exact.

Security budget

A light-client bridge is no safer than:

- source consensus and its accountable stake;
- the on-destination consensus-client implementation;
- commitment-proof correctness;
- upgrade and freeze governance;
- destination chain safety;
- wrapped-token and escrow contracts;
- client-update availability before timeout or trusting-period expiry.

Value at risk should be capped below the loss a plausible failure can create. Rate limits, delayed large withdrawals, per-asset caps, and circuit breakers reduce blast radius but do not repair invalid verification.

Production tests

Run adversarial vectors for invalid signature weights, duplicate validators, malformed keys, skipped validator transitions, expired clients, conflicting headers, proof-depth limits, wrong chain IDs, replayed packets, timeout/receipt races, and reentrant tokens. Differentially test the on-chain verifier against the source chain's reference implementation.

Finally, rehearse a relayer outage, near-expired trusting period, frozen client, emergency verifier upgrade, and orderly channel shutdown. A bridge is production-ready when operators can explain every accepted proof, bound the value exposed while evidence is ambiguous, and recover without silently choosing a chain history.

Bridge Accounting, Limits, and Circuit Breakers

A bridge is both a verification system and an accounting system. Correct proofs do not help if token mapping, fee behavior, rounding, or pending liabilities let destination supply exceed source backing.

Conservation equation

For a lock-and-mint bridge at one observation boundary:

```

source escrow
= destination circulating representation
+ finalized burns awaiting source release
+ finalized deposits awaiting destination mint
+ explicitly identified fees and reserves

```

The exact terms depend on message timing, but every unit must occupy one named state. Reconcile by asset identifier and message ID, not only aggregate dollar value.

For burn-and-mint across native domains, track authorized supply changes rather than escrow. For liquidity-network bridges, track provider liabilities and claims separately from canonical issuance.

Asset identity

Bind source chain, source contract, destination chain, destination contract, decimals, version, and bridge route. Symbols such as USDC are display labels and can collide.

Decimal conversion needs a canonical rounding rule. Bridging 1 unit from an 18-decimal token to a 6-decimal representation can create dust. Decide whether dust remains escrowed, accumulates for later claims, or is rejected. It must not vanish into operator discretion.

Fee-on-transfer and rebasing assets break naive "requested amount equals received amount" assumptions. Measure the escrow's effective balance change and define which asset behaviors are supported. An upgradeable source token can change behavior after onboarding, so monitor its implementation authority.

Message lifecycle

Use one state machine:

```

observed -> finalized -> proven -> releasable -> consumed
          \-> expired / canceled / disputed

```

Transitions are monotonic and keyed by a domain-separated message identifier. "Consumed" is recorded before or atomically with external asset release. A failed token call must leave a retryable state without allowing a second successful release.

A relay retry should submit the same proof and converge on one state. Operator databases are caches; the contract or protocol state is authoritative.

Rate limits

Rate limits bound loss from a verifier, key, or accounting failure. They do not make an invalid release correct.

Possible limits include:

- amount per asset per hour or day;
- amount per destination and route;
- one-message maximum;
- net outflow relative to inflow;
- total value at risk;
- new-asset probation caps;
- slower lanes for unusually large claims.

Use deterministic windows or token buckets. Define timestamp source, boundary behavior, retries, and whether unused capacity accumulates. A miner-controlled timestamp should not permit a large window jump.

Worked token bucket

Suppose a route allows a sustained 100 tokens per minute and a burst capacity of 500. The bucket refills at $100 / 60 \approx 1.67$ tokens per second up to 500.

After a 400-token withdrawal, 100 tokens remain. A new 250-token withdrawal must wait for 150 tokens of refill:

$$150 / 1.67 \approx 90 \text{ seconds}$$

The interface should show the limiting route and earliest estimated eligibility. Splitting the withdrawal into smaller messages must not bypass the shared bucket.

Value-aware limits

Token amounts are not comparable across assets. A price-based global limit introduces oracle risk: stale or manipulated prices can admit excessive value or freeze safe transfers.

Use conservative price sources, staleness checks, per-asset hard caps, and a deterministic fallback. Newly listed or thinly traded assets should not inherit a large cap from an unreliable spot price.

Caps should consider source finality and bridge verification delay. The maximum loss before detection can include all releases during the alert, governance, pause, and settlement windows.

Circuit breakers

A circuit breaker pauses a narrow transition when objective conditions hold:

- conflicting finality evidence;
- accounting imbalance;
- proof verification anomaly;
- withdrawal rate above the configured envelope;
- stale light client or oracle;
- unexpected contract implementation change;
- consumed-message collision;
- monitored supply change without a matching bridge event.

Automatic halts should be conservative and observable. An attacker must not be able to keep a bridge halted cheaply with unauthenticated reports. Manual pause keys add power and need thresholds, scope, expiry, and public evidence.

Separate deposit, mint, burn, and release controls. It may be safe to stop new deposits while allowing previously proven withdrawals, or necessary to stop release while preserving proof submission and accounting visibility.

Fast and slow lanes

Small transfers can use an ordinary lane. Large transfers may require stronger finality, more confirmations, delayed release, additional proof review, or liquidity-provider collateral.

Publish the threshold and timing. Hidden discretionary review makes fungible users receive different security without knowing it. Attackers can split claims unless limits aggregate by sender, asset, route, intent, and system-wide outflow where appropriate.

Monitoring and reconciliation

Continuously compare:

- source escrow balance;
- destination total supply;
- pending deposits and withdrawals by status;
- consumed identifiers;
- rate-limit state;
- fees, dust, and reserves;
- contract implementation and authority;
- light-client and oracle freshness.

Run reconciliation from independent indexers and direct chain state. Matching dashboards that share one database are not independent evidence.

Failure and recovery

If an imbalance appears, stop the release or mint transition that can enlarge it, preserve every message and proof, and identify the last balanced boundary. Do not "fix" totals by burning a user asset or changing escrow without message-level reconciliation.

A recovery manifest lists affected IDs, expected and observed states, correction transactions, signers, before/after supply equations, and user remedies. Reopen under low caps and an observation window.

Production tests

Test decimal mismatches, dust, rebasing, transfer fees, paused and callback tokens, duplicate proofs, failed external calls, window boundaries, split claims, oracle staleness, source reorganization, implementation upgrades, and circuit-breaker recovery.

A bridge is operationally safe when every unit has a named state, every release has unique final evidence, limits bound loss during response, and operators can reconcile and resume without inventing balances.

Operating a Modular Stack

Observability should correlate one transaction across every layer. Assign a stable journey identifier and record:

- sequencer receipt and L2 block;
- DA transaction and commitment;
- proof job, version, and completion;
- settlement transaction and accepted root;
- outbound message nonce;
- relay submission and destination execution.

Alerts should name which service owns the delay. Service-level objectives can then distinguish fast confirmation, publication deadline, proof deadline, settlement deadline, and message-delivery deadline.

Runbooks need safe halt conditions. If DA finality is uncertain, settlement should stop accepting new assertions rather than guess. If the prover is down, sequencing may continue only within a bounded unpublished or unproven window. If a bridge verifier has a bug, a pause key may limit loss, but its scope and recovery governance must be documented.

Modular Cost Accounting

The application pays several providers: sequencer, execution nodes, DA layer, prover, settlement chain, and relayers. Some costs are variable per transaction; others are fixed infrastructure or security subsidies.

Unit economics should allocate batch and proof cost by the resource each transaction consumes, not divide equally. A large data-heavy transaction should pay more DA cost; a computation-heavy transaction should pay more proving cost. Mispricing invites denial of service against the subsidized resource.

Worked Failure Trace: A Cross-Rollup Swap

Consider a user swapping an asset on rollup R1 for an asset on rollup R2. Both rollups publish data to DA network D and settle to chain S. A solver provides liquidity and a relayer carries proofs. The desired outcome is atomic from the user's perspective: either the user receives the minimum output on R2, or the input on R1 remains recoverable.

A safe intent can bind:

```
SwapIntent {
  source_domain,
  destination_domain,
  input_asset,
  max_input,
  output_asset,
  min_output,
  beneficiary,
  nonce,
  expiry,
  refund_address
}
```

The signature covers domains, assets, bounds, beneficiary, expiry, and refund. It should not authorize arbitrary calls chosen later by a solver. A settlement design may escrow input on R1, accept evidence that output was delivered on R2, then release input to the solver.

Normal path

1. the user signs the bounded intent and escrows input on R1;
2. R1 publishes the escrow transition to D and later anchors its state to S;
3. a solver pays the beneficiary on R2 before expiry;
4. R2 publishes and settles the output transition;

5. the solver submits authenticated R2 evidence to the escrow contract or verifier;
6. after the required finality policy, the input releases to the solver;
7. the intent nonce is marked consumed on every accepting domain.

Fast execution on both rollups does not make this path synchronously atomic. It is a state machine with pending, paid, claimable, released, expired, and refunded states.

Partial failures

Solver disappears before paying. The escrow remains locked until expiry, after which the user calls refund. The expiry must account for clock semantics and source-chain finality. A solver acknowledgement is not evidence of payment.

Solver pays, but the relayer fails. Any party should be able to carry the same proof. The relayer is replaceable because destination verification checks authenticated state, not relayer identity. If only one allowlisted relayer can complete the claim, it is part of the liveness boundary.

Output appears in an unfinalized R2 block that reorganizes. Releasing input immediately can leave the solver paid on neither canonical history while the user receives a refund or double benefit. The claim rule must name R2 finality, settlement finality on S, and how conflicting preconfirmations are treated.

R2 data is unavailable. A state root or validity proof may show a transition while independent parties cannot reconstruct user state. The escrow contract may still verify a succinct proof, but the system's user-recovery claim has weakened. The product should not label this state equivalent to one with published, retrievable data.

Settlement chain S halts. Both rollups may continue offering provisional execution while cross-domain claims cannot achieve the required finality. Operators need a policy for pausing new intents, extending expiries, or letting existing escrows refund without accepting contradictory histories.

One rollup upgrades its message format. Version and verifier identity belong in the envelope. A decoder should reject an unknown version rather than interpret fields under the old layout. Upgrade timing must leave already-open intents claimable or refundable.

Safety and liveness matrix

Dependency	Safety role	Liveness role	Recovery
R1 execution	correct escrow and refund state	accepts user and claim transactions	forced inclusion or exit

Dependency	Safety role	Liveness role	Recovery
R2 execution	correct beneficiary payment	accepts solver payment	alternate solver or refund
DA network D	data bound to settled transitions	reconstruction and proof generation	independent retrieval/repair; halt if unavailable
Settlement S	authentic final state for both rollups	advances finality and bridge claims	wait, bounded emergency rule, or social recovery
Solver	cannot exceed signed bounds	supplies destination liquidity	competition and expiry refund
Relayer	no correctness power if proof is complete	transports claim evidence	permissionless replacement
Upgrade governance	can alter verifiers and formats	can repair broken deployment	timelock, monitoring, old-version exit

Observability

The user interface should report the current state and its controlling deadline:

- **escrow pending publication;**
- **escrow final on source;**
- **solver payment observed, not final;**
- **destination payment final;**
- **claim submitted to source;**
- **input released;**
- **refund available at a named time;**
- **blocked by DA, settlement, or verifier condition.**

Retries are safe only when operations are idempotent. The intent nonce, output payment identifier, and claim identifier must make duplicate submissions converge on one state rather than pay twice.

Integration tests

Test every cut between steps: crash after escrow but before publication, pay output twice, relay the same proof twice, cross the expiry while a claim waits in the mempool, reorganize each unfinalized chain, withhold DA data, stop settlement finality, and upgrade one decoder with in-flight intents.

For each cut, assert conservation of assets, one terminal outcome, permissionless proof delivery, bounded lock time, and an observable recovery action. A cross-domain protocol is ready only when partial completion is a designed state, not an exception handled by an administrator.

Conclusion

Monolithic blockchains offer integrated security and composability but require validators to repeat all major work. Modular architectures separate execution, settlement, consensus, and data availability so each can specialize and many execution layers can share a base.

The gain is flexibility and scale. The cost is fragmentation and a larger set of dependencies. The next chapter examines the least intuitive of these services - proving that block data is actually available without requiring every node to download all of it.

References

1. Celestia Docs. "Data Availability." <https://docs.celestia.org/learn/celestia-101/data-availability/>. ↩
2. Optimism. "Alt-DA Mode." *OP Stack Specification*. <https://specs.optimism.io/experimental/alt-da.html>. ↩
3. Optimism Documentation. "OP Stack components." <https://docs.optimism.io/op-stack/protocol/components>. ↩ ↩2
4. Optimism Documentation. "Transaction flow." <https://docs.optimism.io/op-stack/transactions/transaction-flow>. ↩
5. Celestia Documentation. "OP Stack integration." <https://docs.celestia.org/build/stacks/op-alt-da/introduction/>. ↩ ↩2
6. Celestia Documentation. "Blobstream." <https://docs.celestia.org/learn/blobstream/>. ↩

Chapter 8: Data Availability Scaling

Introduction

A block header can commit to a large body of transactions with a small hash. The hash proves what the data should be, but it does not prove that anyone received the data. A malicious producer could publish the header, reveal only selected pieces, and prevent validators from checking the state transition.

This is the **data availability problem**. It appears whenever nodes want assurance about a large block without downloading all of it. The problem is central to sharding, light clients, and rollups.

Intuition: A Correct Answer Needs Recoverable Inputs

Suppose a teacher writes only "the class total is 742" on the board. The number might be correct, but students cannot check it or continue calculating averages without the individual scores. A commitment or proof of correct processing does not necessarily reveal the underlying data.

Data availability (DA) means the data needed to verify or reconstruct a block was published and can be obtained when the protocol requires it. It is not the same as:

- **validity**: whether transactions followed the rules;
- **retrievability**: whether a convenient service answers a request now;
- **permanent storage**: whether the data will remain archived years later.

A protocol can guarantee availability near publication while applications pay separate archives for historical queries.

Withholding attacks

A producer can publish a block header containing a commitment while sending the actual block to too few peers. The header looks compact and valid, but users cannot reconstruct state or prove that hidden transactions were invalid.

The difficult case is a **partial withholding attack**. The producer serves requested pieces selectively so some nodes believe data exists while no honest group possesses enough to reconstruct the whole block. Merely asking one server for one copy does not solve this.

Erasure coding

Erasure coding expands k original pieces into n encoded pieces so any sufficiently large subset can reconstruct the original. It resembles cutting a document into pieces and adding carefully designed redundancy: losing some pieces is harmless, but preventing recovery requires hiding many.

This differs from ordinary replication. Three complete copies tolerate loss of two whole hosts. Erasure coding spreads smaller redundant pieces and can use bandwidth more efficiently, but clients must verify that the producer encoded them consistently.

Reed-Solomon coding is one mathematical family used for this purpose. Readers do not need its polynomial algebra to follow the security argument: correct encoding creates redundant shares, and a reconstruction threshold defines how many are needed.

Sampling

A light client cannot download the entire large block without losing the scalability benefit. Instead, it asks for random encoded shares and verifies proofs that each share belongs to the committed block. This is **data availability sampling (DAS)**.

If an attacker must hide a large fraction to prevent reconstruction, random requests have a chance of hitting hidden pieces. Repeating independent samples makes missing every hidden piece exponentially less likely.

If hidden fraction is h and the client takes s independent samples, the chance that all samples avoid the hidden fraction is:

$$(1 - h)^s$$

For $h = 0.5$ and $s = 10$:

$$(1 - 0.5)^{10} = 0.5^{10} = 1 / 1,024 \approx 0.098\%$$

This number is a model, not a complete guarantee. Requests must be unpredictable, peers must be diverse, proofs must be valid, and attackers must not show each client a different view.

Commitments and inclusion proofs

A block header commits to encoded shares. An **inclusion proof** shows that one returned share occupies a particular position under that commitment. It prevents a peer from answering with invented data.

The proof does not by itself show that every other share exists. Sampling combines many authenticated spot checks with the coding threshold. Peer-to-peer exchange then helps sampled shares spread among honest nodes.

Two dimensions and namespaces

Some designs arrange data in rows and columns and apply coding in both directions. Two-dimensional coding gives clients structured samples and supports reconstruction from rows or columns.

A **namespace** labels data belonging to one application or rollup. A namespaced Merkle tree lets that application retrieve and prove its portion without downloading unrelated data. Namespace separation improves selective access; it does not let an application ignore the chain's overall availability assumptions.

How to read DA claims

Ask:

1. What exact bytes are committed?
2. Who performs and checks erasure coding?
3. How many shares reconstruct the data?
4. How are random sample positions chosen?
5. Can one peer or gateway answer every request?
6. When does consensus treat availability as sufficient?
7. How long is data retained, and who archives it afterward?
8. What does the dependent rollup do when availability is uncertain?

The rest of the chapter turns this intuition into probability, client, network, and integration details.

Validity Is Not Availability

State validity asks whether a transition followed the protocol rules. Data availability asks whether the inputs behind that transition can be obtained.

Fraud proofs require the data needed to identify an invalid step. Even validity proofs do not necessarily reveal the new state to users. If transaction data is withheld, users may be unable to reconstruct balances, create future transactions, or exit.

A commitment therefore provides **integrity**, not **availability**. Seeing a Merkle root is not the same as seeing the block.

The Withholding Attack

Suppose a light client receives a header for a block divided into many shares. It asks peers for several shares and receives all of them. This is weak evidence: a malicious producer may selectively answer the light client while withholding different shares from the wider network.

The objective is to encode and distribute data so that withholding enough shares to prevent reconstruction is likely to be detected by random sampling.

Erasure Coding

Erasure coding expands k original data shares into n shares, where the original block can be reconstructed from a threshold subset. Reed-Solomon codes are a common construction.

The producer commits to the extended data. If it withholds enough shares to make reconstruction impossible, it must hide a substantial fraction of the encoded block. A light node that requests random shares then has a meaningful probability of encountering a missing share.

After s independent samples, the probability of missing a withholding attack falls exponentially. This turns a binary download requirement into a tunable confidence level.

Erasure coding alone is not sufficient. Nodes also need confidence that the encoding was performed correctly. Two-dimensional coding, fraud proofs for incorrect encoding, and polynomial commitments are used to make malformed shares detectable.¹

Data Availability Sampling

A light node performing data availability sampling (DAS):

1. obtains the block header and commitment;
2. requests randomly selected shares from different peers;
3. verifies each share against the commitment;
4. accepts availability after enough successful samples.

No individual light node reconstructs the block. Across many independently sampling nodes, the network requests enough shares for honest full nodes to recover it.

This produces an unusual scaling property: as more light nodes join and sample, the network can support larger blocks while maintaining strong confidence. The details still depend on peer-to-peer distribution, sampling independence, encoding correctness, and the assumed adversary.

Celestia's Approach

Data availability sampling makes withholding detectable

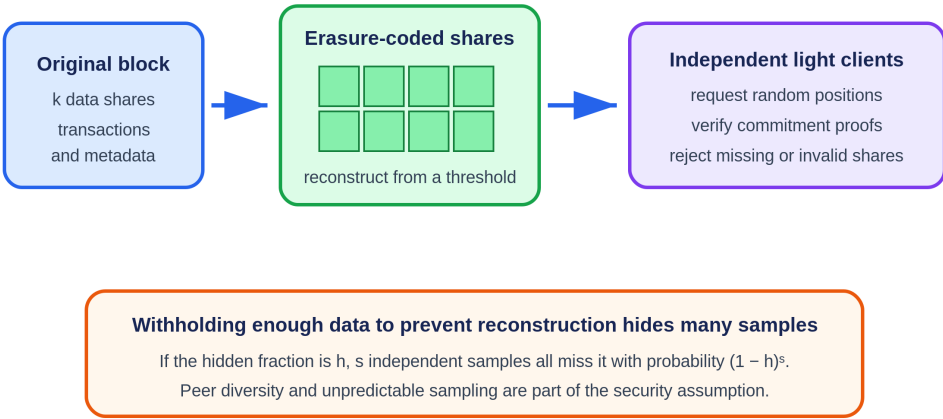


Figure 8.1: Erasure coding forces an availability attacker to hide many shares, while independent light clients test random positions. Original figure for this book, based on Celestia's data availability design.

Celestia arranges block data in a square, applies two-dimensional Reed-Solomon encoding, and commits to rows and columns. Light nodes sample shares and verify inclusion proofs. Namespace Merkle trees allow an application to retrieve the portions of data relevant to its namespace without processing every rollup's data.²

Celestia provides consensus and data availability. It does not execute each rollup. Rollup nodes interpret the published data and apply their own validity rules.

Ethereum Blobs and PeerDAS

EIP-4844 added blobs as a separate data type for rollups. The EVM cannot directly read blob contents, but it can access cryptographic commitments. Blob data is priced separately from ordinary execution and retained for a limited period.³

This is enough because rollups need the data during the period when nodes reconstruct state and, for optimistic designs, issue challenges. Permanent historical storage can be supplied by archival services rather than every consensus node.

PeerDAS activated on Ethereum mainnet with Fusaka on December 3, 2025.

⁴ EIP-7594 extends blobs with one-dimensional erasure coding, divides the extended data into cells and columns, assigns each node deterministic custody responsibilities, and has nodes sample columns from peers each slot. A node can reconstruct the full data matrix after acquiring at least half of the columns under the specified coding scheme.⁵ PeerDAS reduces the fraction each node must download; it does not turn short-term protocol availability into permanent archival storage.

External Data Availability Layers

Rollups can publish data in several places:

- **Layer 1 calldata or blobs** - stronger integration with settlement, generally higher cost;
- **dedicated DA network** - greater capacity and lower cost, with a separate consensus assumption;
- **data availability committee** - cheap and simple, but a small group can withhold data;
- **local storage** - suitable only when users accept operator trust or have another recovery path.

EigenDA and similar systems use restaked or dedicated operators to disperse encoded data and attest to availability. The main evaluation questions are who signs, what threshold is required, how data is retrieved, and what happens after withholding.

Named Case Study: One Rollup Batch on Celestia, EigenDA, and Avail

Deployment labels: Celestia, EigenDA, and Avail DA are production networks. They can all carry data for a rollup, but they do not give the settlement contract identical evidence or rely on the same validator set. The comparison below fixes one input: a 600 kB compressed rollup batch containing ordered transactions and a header that binds the rollup identifier, batch number, parent state root, encoding version, and byte length.

The rollup first serializes the batch deterministically. This is the common boundary. If different publishers can encode the same transactions into different bytes, no DA system can repair the ambiguity. Let B be the exact 600 kB byte string and $H(B)$ its application commitment. The rollup stores $H(B)$ alongside whichever network-specific receipt or inclusion proof it uses. A verifier accepts the resulting state only if it can bind the DA evidence back to B , the batch header, the correct DA network, and a sufficiently final DA block.

Celestia path: namespaced shares and sampling

A publisher submits B as one or more Celestia blobs under a namespace allocated to the rollup. Celestia's block data is split into shares, arranged and erasure-coded into an extended data square, and committed through namespaced Merkle structures. The namespace lets a rollup retrieve and prove its own shares without treating every other application's bytes as its payload. Celestia consensus orders the transaction and commits the data root; light nodes sample shares to gain probabilistic confidence that the extended block data is available.^{2 6}

Trace the batch. The publisher broadcasts a blob transaction. After Celestia includes it, the rollup records the Celestia height, namespace, commitment, share range, and transaction or blob identifier. A verifier retrieves the namespaced shares from independent nodes, checks inclusion against the data commitment, reconstructs B , checks length and $H(B)$, decodes transactions under the bound version, and re-executes the rollup. A settlement integration such as Blobstream can convey Celestia data-root commitments to another chain, but the application still needs a rule connecting the particular namespace shares and finality level to its accepted rollup batch.⁷

The availability assumption is tied to Celestia's consensus and data-availability sampling design. A light client does not prove that a permanent archive will serve B months later. Celestia documentation distinguishes availability around block production from later retrievability and pruning.⁸ The rollup therefore runs or contracts archival retrieval if its fraud-proof or recovery window exceeds ordinary node retention.

Failure path: the publisher receives an RPC acknowledgement but the blob never reaches a final Celestia block. No settlement state should advance. If the block is final but the rollup's retrieval peers are eclipsed, those clients may report failure while the wider network still has the shares; peer diversity and independent sampling matter. If enough data is genuinely withheld for reconstruction, sampling should make acceptance unlikely under the stated model, but a verifier must fail closed rather than substitute the publisher's local copy. If the namespace or share range is wrong, a valid proof for someone else's bytes must not satisfy this batch.

EigenDA path: dispersal to operator quorums

EigenDA uses a different shape. A disperser accepts B , erasure-codes it into chunks, distributes assigned chunks to EigenDA operators, and collects authenticated acknowledgements from the configured quorum or quorums. The client polls the blob status and receives a certificate or confirmation data that downstream components can verify under the integration's rules. EigenDA's official V2 guide makes the API boundary visible: prepare bytes, send data, receive a blob key, and check status.⁹

Trace the same batch. The rollup submits B with payment and dispersal parameters. The disperser returns an identifier before availability is necessarily certified, so the rollup marks the batch **dispersing**, not available. Operators validate and store their assigned chunks and sign acknowledgements. Once the threshold for every required quorum is met, the batch becomes confirmed. A verifier or retriever uses the blob key and certificate data to request chunks, reconstructs B , verifies its commitment and encoding, and re-executes the rollup transition.

The trust boundary is not Celestia consensus. EigenDA's security model depends on operator quorums, chunk assignment, the reconstruction threshold, the certificate confirmation threshold, cryptographic commitments, and the stake controlled by an adversary. Its specification requires the configured thresholds to leave enough honestly held chunks for reconstruction; in its notation, $\text{ConfirmationThreshold} - \text{SafetyThreshold} \geq \text{ReconstructionThreshold}$.¹⁰ Each quorum linked to an attestation supplies a redundant availability guarantee under its own stake distribution and thresholds.

The current security documentation also states a separate liveness assumption: until decentralized dispersal replaces the present model, a trusted disperser must not censor a client's request, and adversarial stake must remain below the configured liveness threshold. If operators certify data that later proves unavailable, the documented recovery is not an automatic cryptographic repair. A community member raises a data-unavailability alarm and, after social confirmation, the EIGEN-token mechanism can use a fork to penalize the dishonest side.¹¹ A rollup must decide whether that delayed, intersubjective recovery meets its loss and availability objectives. The settlement contract must also know exactly which certificate format, quorum set, thresholds, reference block, and commitment it accepts.

Failure path: the disperser crashes after accepting B but before enough operators acknowledge it. The blob remains unconfirmed and the rollup must retry through a healthy path without changing B or creating an ambiguous second batch. If a threshold certificate exists but retrievers cannot reconstruct, the incident targets operator storage, assignment, or serving assumptions and may trigger the protocol's accountability path. If the rollup configures one highly correlated operator set, nominal operator count can overstate resilience. If a settlement contract accepts an outdated quorum configuration or fails to bind the certificate to $H(B)$, a formally valid certificate can authorize the wrong bytes.

Avail path: application identifiers and validity-backed sampling

Avail DA orders data submissions in its own proof-of-stake chain. A publisher submits B in an extrinsic associated with an **application identifier**. Avail ex-

tends block data with erasure coding and commits to it with polynomial commitments; light clients sample cells and verify proofs against the block commitment. The application identifier gives the rollup a selective data stream, while the block commitment binds all included data.^{12 13}

Trace the batch. The rollup signs and submits the data extrinsic under its application identifier. It records the Avail block, extrinsic position, application identifier, commitment, and finality evidence. A verifier checks finality, filters or proves the rollup's data, obtains enough shares or the complete application payload, reconstructs B , checks $H(B)$, and executes it. As with Celestia, a bridge or proof system that transports Avail commitments to another settlement chain is an additional component. DA-chain finality does not automatically update an Ethereum rollup contract.

The trust assumption centers on Avail's validator consensus, erasure-coding and commitment implementation, light-client sampling, and any bridge used by the settlement layer. Sampling says the committed block data was likely available under the model. It does not promise indefinite retention. The rollup still needs archival policy, correct application-ID filtering, and a recovery route when its preferred RPC or light-client network is unavailable.

Failure path: a publisher labels B with the wrong application identifier. Avail may correctly include and make the bytes available, while the rollup's normal selective retriever never sees them. That is an integration failure, not DA-chain withholding. A client that trusts one full-node RPC without checking commitments has reduced the system to provider trust. A settlement bridge that lags leaves the batch available on Avail but not yet usable by the rollup's settlement contract. A deep reorganization or validator-set safety failure on the DA chain requires the rollup to roll back or halt according to its explicit finality policy.

Same batch, different evidence

Question for batch B	Celestia	EigenDA	Avail DA
Publication unit	Blob transaction split into namespaced shares	Blob dispersed as coded chunks to operator quorums	Data extrinsic associated with an application identifier
Primary availability evidence	DA-chain block commitment, namespaced inclusion, and sampling/retrieval evidence	Threshold operator acknowledgements and the accepted EigenDA certificate/configuration	DA-chain finality, block polynomial commitment, inclusion/filtering, and sampling/retrieval evidence
Selective retrieval key	Namespace, height, share range or blob commitment	Blob key/certificate plus retriever interfaces	Application identifier, block and extrinsic/data position

Question for batch B	Celestia	EigenDA	Avail DA
Who orders publication	Celestia validator consensus	Dispersal/certificate flow; the rollup separately orders its own batches	Avail validator consensus
Settlement integration	Verify or relay Celestia commitments, often through a bridge such as Blobstream	Verify the configured EigenDA certificate and reference state	Verify or relay Avail commitments through the chosen bridge/proof path
Main correlation risk	DA validators, sampling peers, and archival providers may share infrastructure	Many operators may share cloud, code, stake dependencies, or one disperser path	Validators, sampling peers, bridge and archive services may share infrastructure
Retention statement	Availability at publication is separate from later retrievability	Certificate/storage duration must match the rollup's challenge and recovery needs	Sampling availability is separate from application archival retention

For all three, the safe acceptance rule has the same outer form:

```
accept rollup state only if
  network_id is expected
  and batch header binds H(B), length, namespace/app/quorum context,
  and encoding version
  and network-specific availability evidence satisfies current
  policy
  and settlement has authenticated that evidence
  and an independent verifier can reconstruct and decode B
```

The systems differ inside network-specific availability evidence. Hiding that field behind a generic boolean such as `data_available = true` removes the information an auditor needs. A production dashboard should expose the source network, finality height or reference state, certificate or commitment, retrieval success from independent paths, retention deadline, settlement-bridge status, and the exact batch commitment that execution consumed.

Availability, Retrievability, and Permanence

These terms should not be confused:

- **Availability** means data was disseminated so the network could obtain it during the required window.
- **Retrievability** means a user can fetch it from some service now.
- **Permanence** means historical data remains stored indefinitely.

A consensus protocol can guarantee availability at publication without requiring every validator to retain the data forever. Applications that need historical queries should define separate archival assumptions.

Trade-Offs

Design	Validator Load	Security Integration	Main Risk
Full replication	High	Native	Capacity limited by weakest validators
L1 blobs + sampling	Distributed	Native settlement consensus	Complex networking and coding
Dedicated DA layer	Specialized capacity	Separate validator set	Cross-layer assumption
DA committee	Low cost	Small signer group	Coordinated withholding

A cheaper data layer can materially reduce rollup fees. It also changes the system's failure model. Cost comparisons without the availability assumption are incomplete.

Sampling Probability by Example

Suppose erasure coding expands a block so that an adversary must hide at least half the shares to prevent reconstruction. One uniformly random request misses the attack with probability at most one-half. Twenty independent requests miss every hidden share with probability at most $(1/2)^{20}$, roughly one in a million.

The arithmetic is simple; its assumptions are not. The formula assumes each requested position is sampled uniformly against one fixed pattern of hidden shares. A malicious peer that selectively serves every position requested by one client while withholding the rest violates that model; it does not merely make the numerical samples slightly less independent. Samples must be unpredictable, commitments must authenticate cells under a correctly encoded block, and the network must prevent one producer or eclipse set from presenting incompatible availability views. Practical DAS therefore combines coding and commitments with custody rules, peer diversity, and distribution protocols.

A Data-Withholding Failure

Consider an optimistic rollup operator that publishes a state root but withholds the batch. Users can see the commitment, and the operator's interface might still display balances. Independent verifiers cannot replay the batch

and therefore cannot construct a fraud proof. The root does not visibly look invalid; nobody can test it.

An integrated DA layer changes this. A block is accepted only after encoded shares have been disseminated. Sampling nodes check availability, and reconstruction nodes recover the full data. Once the batch is available, one honest verifier can challenge an invalid transition. The DA layer does not perform the challenge, but it preserves the possibility of doing so.

Validity and availability are therefore complementary. The system needs both a rule for correct transitions and access to the information those rules operate on.

Two-Dimensional Reed-Solomon Encoding

A common DAS construction arranges $k \times k$ original shares in a square. Each row is extended to $2k$ shares with Reed-Solomon coding. Each resulting column is then extended to $2k$, producing a $2k \times 2k$ extended square. The block header commits to row and column roots.

If a producer withholds enough shares to prevent reconstruction, it must hide a noticeable fraction of the square. Random sampling detects that fraction with increasing probability. If the producer encodes a row incorrectly, an encoding-fraud proof can identify inconsistent shares. Polynomial-commitment designs can instead prove that samples belong to correctly encoded polynomials.

The network protocol matters as much as the code. A sampler requests coordinates from several peers. Full or bridge nodes reconstruct rows and columns when enough shares arrive, then redistribute recovered shares. Sampling success by one isolated client does not establish global dissemination; peer exchange and custody rules are designed to make selective disclosure difficult.

Namespaced Data and Selective Retrieval

A general DA layer may carry batches for thousands of applications. A rollup should not download every other rollup's data. Namespace Merkle trees organize shares by namespace while allowing proofs that all shares for a namespace were returned.

A namespaced commitment supports two statements: a share belongs to the block, and a range contains all data under a given namespace. This lets a rollup node retrieve its own batches while light clients sample across the full block.

Namespaces are not access control. Data remains public; the namespace is an indexing and proof mechanism.

Blob Commitments and Retention

EIP-4844 transactions carry blob commitments. Consensus clients disseminate blob sidecars alongside blocks and verify that commitments match the block. Execution sees a versioned hash of the commitment rather than the blob bytes.

A rollup contract can therefore bind a batch assertion to blob data without making that data permanent EVM storage. Consensus nodes retain blobs for the protocol window; rollup nodes, indexers, and archives keep longer history according to application needs.

The retention boundary creates an operational deadline. A new rollup node joining after blob expiry needs a snapshot or historical provider. Trust can be minimized by checking the reconstructed state against finalized roots, but availability of old history remains a service assumption.

Implementing a DA Client

A light DA client should:

1. follow finalized or appropriately confirmed headers;
2. derive unpredictable sample coordinates;
3. request samples from diverse peers;
4. verify each share against the header commitment;
5. reject the header if required samples are missing by deadline;
6. store evidence of invalid encoding or inconsistent responses;
7. expose confidence and peer health to dependent rollups.

A rollup full node additionally fetches every share in its namespace, reconstructs missing data, decodes batches canonically, and confirms that the settlement assertion refers to that exact data.

DA Threat Model Checklist

- What fraction of shares must be hidden to prevent reconstruction?
- How many independent samples reach the desired failure probability?
- Can a producer answer samplers selectively?
- Who verifies correct erasure encoding?
- How are samples distributed across peers?
- When is a header considered final?
- How long is consensus data retained?
- Who serves historical data afterward?
- What does the settlement layer do if the DA layer halts or reorganizes?

- Can a governance key switch DA commitments retroactively?

Availability Certificates and Their Limits

Some DA systems disperse encoded chunks to operators and collect signatures. An availability certificate proves that a threshold attested to receiving assigned data. If enough signers are honest and retain chunks for the required period, the block can be reconstructed.

The certificate is only as strong as membership, threshold, custody challenge, and slashing. A signer may acknowledge data and delete it later. Proofs of custody or periodic challenges test continued possession. Slashing needs objective evidence that a signer failed, which is harder for a network timeout than for an invalid signature.

A committee certificate differs from DAS by light clients. The former relies on a signer threshold; the latter derives confidence from encoded data and random samples. Hybrid systems use both.

Selective Disclosure and Eclipse Attacks

A producer can try to answer requested shares only to selected samplers while hiding them from reconstruction nodes. If the attacker controls a client's peers, it can create a false view of availability.

Peer sampling should diversify network paths and avoid revealing all future coordinates to one peer. Nodes can gossip received shares so answering one client helps distribute data. Sampling requests may be parallelized across peers, and clients monitor peer overlap or autonomous-system concentration.

An eclipse-resistant DAS design therefore includes peer discovery and networking assumptions. Cryptographic verification rejects wrong shares but cannot force an isolated client to meet an honest peer.

Reconstruction and Repair

When enough shares are available, a full node reconstructs missing rows or columns. Recovered shares are verified against commitments and redistributed. Repair keeps data retrievable when some custodians leave.

Reconstruction consumes CPU and bandwidth. An adversary may repeatedly provide just enough shares to trigger expensive repair while withholding others. Implementations bound concurrent jobs, prioritize finalized blocks, and charge or rate-limit requests.

For long retention, storage networks can pin complete blobs after the consensus availability window. Their correctness is checked against old commit-

ments, but their economic model determines whether data remains retrievable years later.

Sampling Networks, Peer Diversity, and Adversarial Serving

Data availability sampling assumes more than correct mathematics. Clients must obtain unpredictable samples through a network that prevents a producer from showing favorable pieces to each client while withholding enough globally to block reconstruction.

Sampling request

```
SampleRequest {
  chain_id,
  block_height,
  data_commitment,
  row_or_column,
  share_index,
  request_nonce,
  client_version
}
```

A response includes the share bytes and an inclusion proof. The client checks the requested position, commitment, encoding domain, and proof before counting success.

Do not let the server choose sample positions. Derive them from client randomness committed after the producer fixes the data commitment, or from a protocol randomness source whose timing prevents grinding.

Selective serving

A malicious producer can serve shares to well-known monitors while withholding from ordinary nodes. Random clients should exchange observations and shares through diverse peers. A success result from one gateway says only that gateway answered one request.

Privacy matters because a server that links all requests from one client learns its complete sample set and can answer exactly those positions. Clients can distribute requests among peers or use privacy-preserving transports, but correlated infrastructure may still join them.

Peer diversity

Count independent autonomous systems, regions, operators, and implementations, not IP addresses. One provider can expose thousands of endpoints.

Discovery uses several sources: bootstrap peers, peer exchange, DNS records, on-chain identities, and cached known-good peers. No source should control

every initial connection. Rate-limit new peers and retain diversity during churn.

A client selects peers across failure domains and caps the fraction of samples answered by one domain. If diversity falls below policy, it reports reduced confidence rather than silently treating repeated answers as independent.

Eclipse and partition attacks

An eclipse attacker surrounds a client and controls every response. It can serve an old chain, delay samples, or reveal only pieces matching its attack.

Cross-check finalized headers over a separate transport, maintain long-lived authenticated peers, limit address-table poisoning, and compare network observations. A light client should distinguish "share missing" from "all current peers are one untrusted domain."

During a network partition, two groups may each obtain different subsets. Reconstruction and consensus rules define whether the block progresses. Sampling confidence cannot choose between conflicting commitments; consensus finality is still required.

Grinding sample positions

If sample positions are predictable before block construction, a producer may search over block encodings or commitments to make monitored samples land on available shares. This is **grinding**.

Bind positions to randomness unavailable when the producer commits, and include block/commitment domain separation. Analyze how many alternative encodings, nonces, or block proposals a producer can try. One bit of producer freedom doubles its search space.

Correlated samples

The formula $(1-h)^s$ assumes independent samples, where h is hidden fraction and s is sample count. Repeating the same share does not improve confidence. Sampling without replacement changes exact probability but usually helps slightly.

If 100 clients each take 20 samples but all use the same deterministic seed, the network has only 20 distinct positions, not 2,000. Mix client-specific or unpredictable randomness while preserving auditability.

Serving load

Sampling creates many small random requests, which can stress disk I/O and connection overhead more than bulk block download. Cache recent shares, batch proofs, use range requests carefully, and cap unauthenticated work.

Suppose 50,000 light clients take 20 samples from each 12-second block:

$$50,000 \times 20 / 12 \approx 83,333 \text{ sample responses/second}$$

At 2 kB per response including proof, egress is roughly:

$$83,333 \times 2 \text{ kB} \approx 167 \text{ MB/s}$$

This is ecosystem demand, not one node's obligation. Distribute it across serving nodes and measure p99 latency under churn and repair traffic.

Negative caching

A missing response may mean withholding, slow peer, wrong request, or local outage. Negative caching prevents repeated expensive requests but can prolong a transient failure.

Record reason and expiry. Retry through another domain before classifying a share missing. Do not let one unauthenticated "not found" response poison the global cache.

Share exchange and reconstruction

Sampled shares can be gossiped so honest clients collectively approach reconstruction. Verify before forwarding. Deduplicate by commitment and position, and bound storage by finalized height and retention policy.

When enough shares exist, reconstruct and verify the original commitment. A successful decode with a mismatched commitment is invalid, not "mostly available." Store evidence of inconsistent encoding for fraud or operator diagnosis.

Confidence reporting

Expose:

- distinct valid positions sampled;
- number requested and failed by reason;
- peer and failure-domain diversity;
- header finality status;
- coding and commitment version;
- confidence under the stated hidden-fraction model;
- whether full reconstruction was attempted or achieved.

Avoid a single green badge when independence or network diversity is unknown.

Adversarial tests

Test predictable seeds, repeated positions, one gateway answering all requests, sybil peer churn, selective serving to monitors, slow final shares, malformed proofs, inconsistent rows, provider-region outage, network partition, and load spikes.

A sampling network supports scalable verification only when sample choice is unpredictable, responses are authenticated, observations span independent paths, and serving capacity remains available during the same attacks that make availability important.

DA Capacity Planning

Let each block contain B original bytes, expand by coding factor r , and arrive every t seconds. The network disperses approximately rB/t bytes per second before protocol overhead. Each node's custody and sampling share may be smaller, but reconstruction nodes and producers handle more.

Capacity tests should measure producer upload, peer fanout, sample latency, reconstruction time, and behavior under missing shares. Increasing block size until average bandwidth is saturated leaves no room for repair or adversarial peers.

Rollups also need publication deadlines. If a sequencer executes faster than DA accepts blobs, unpublished batches accumulate. Set a maximum pending-data window and stop accepting new soft confirmations before recovery becomes unbounded.

DA Integration Test

A useful integration test creates a batch, encodes and publishes it, samples it from independent light nodes, deletes selected shares, reconstructs them, then verifies that a rollup node decodes the same transactions and state commitment. Negative cases include malformed encoding, wrong namespace, incomplete range, old commitment, DA reorganization, and expired history.

Testing only successful upload verifies storage API behavior, not data availability security.

Worked Integration Trace: Publishing a Rollup Batch

A rollup integrating an external DA layer needs more than an upload API. The state commitment accepted by settlement must be cryptographically tied to the bytes that DA nodes encoded and made available.

Assume the rollup constructs canonical batch bytes B . Canonical means every verifier agrees on field order, integer encoding, compression, and version.

The publisher computes or receives a commitment $C(B)$ and submits B to the DA network under a namespace.

```
BatchReference {
    rollup_id,
    batch_number,
    parent_state_root,
    post_state_root,
    da_network_id,
    da_height,
    namespace,
    data_commitment,
    encoding_version
}
```

The settlement transition should bind the state roots to `data_commitment`, DA domain, and encoding version. Binding only `da_height` is ambiguous because a DA block contains many items. Binding a commitment without a network identity enables cross-domain substitution when two systems use compatible proof formats.

Publisher path

1. deterministically encode the ordered L2 transactions into B ;
2. compute a local digest and retain it with the batch job;
3. submit B to several independent DA ingress peers;
4. wait for the protocol-defined inclusion and availability evidence;
5. verify that returned evidence commits to the local digest;
6. submit the batch reference and state proof to settlement;
7. continue serving B during the required reconstruction window.

The publisher must treat an RPC acknowledgement as receipt, not availability. It should not discard local bytes after one gateway says "accepted." Evidence may require a finalized DA header, inclusion proof, namespace proof, or availability certificate, depending on the network.

Verifier path

A verifier obtains the authenticated DA header through a light client or settlement integration, verifies inclusion of $C(B)$, retrieves enough shares or the full namespace data, reconstructs B , and recomputes the rollup state transition.

```
verify(reference):
    header = authenticate_da_header(reference.da_height)
    verify_inclusion(header, reference.namespace,
reference.data_commitment)
```

```
bytes = retrieve_and_reconstruct(reference)
assert commit(bytes) == reference.data_commitment
assert execute(reference.parent_state_root, decode(bytes))
    == reference.post_state_root
```

A validity-rollup verifier may use a succinct proof instead of locally executing. It still needs the data for users and future state reconstruction unless the design explicitly accepts a validium-style withholding assumption.

Failure matrix

Failure	Detection	Safe response
Gateway accepts but never broadcasts	Independent peers cannot retrieve or prove inclusion	Retry through another ingress; do not post state reference
Blob is included but encoding is invalid	Share or encoding proof fails	Reject availability; retain evidence
Publisher cites wrong namespace	Commitment cannot be found under bound namespace	Reject batch reference
DA chain reorganizes before finality	Authenticated header is replaced	Re-evaluate inclusion; do not finalize dependent message
Some peers selectively withhold shares	Random sampling or reconstruction reports missing positions	Diversify peers, gossip requests, and follow rejection threshold
Certificate signers attest unavailable bytes	Retrieval fails despite threshold signature	Halt dependent state; invoke slashing/governance only under specified evidence
Data expires from consensus nodes	Archival retrieval no longer succeeds	Use separately funded archives; this is permanence, not publication availability
Rollup decoder version differs	Reconstructed bytes yield divergent transactions	Bind encoding version and publish cross-client vectors

Capacity calculation

Suppose batches arrive every 8 seconds and average 600 kB after compression. The mean publication rate is:

$$600 \text{ kB} / 8 \text{ s} = 75 \text{ kB/s}$$

Mean rate is insufficient for provisioning. If the p99 batch is 1.8 MB and the system must catch up three missed batches within 16 seconds, recovery traffic alone is:

$$3 \times 1.8 \text{ MB} / 16 \text{ s} = 337.5 \text{ kB/s}$$

Add erasure-coding expansion, inclusion proofs, peer overhead, sampling responses, retransmission, and unrelated DA tenants. Capacity tests should drive this burst while one ingress peer and one retrieval peer are unavailable.

Integration assertions

A release test should prove that:

- identical batch inputs produce identical bytes and commitments across clients;
- changing any transaction or ordering changes the bound commitment;
- an inclusion proof for another namespace, height, or network is rejected;
- settlement does not accept a post-state root before required DA evidence;
- verifiers can reconstruct from independent peers after publisher shutdown;
- a DA reorganization rolls back or delays dependent state under policy;
- archived bytes remain retrievable for the application's stated history period;
- alerts distinguish publication delay, inclusion failure, sampling failure, and archival loss.

This trace separates four events often collapsed into "posted": an ingress accepted bytes, consensus included a commitment, the data was available for reconstruction, and an archive can still retrieve it later. Each event supports a different claim.

Conclusion

Data availability scaling allows nodes to gain strong confidence that block data exists without downloading all of it. Erasure coding makes severe withholding easier to detect, while random sampling gives light clients a tunable confidence level.

This technology connects Layer 1 sharding and rollup-centric scaling. Execution can move elsewhere only if users and verifiers can obtain the data required to reconstruct and challenge state. The next chapter turns from scaling data to scaling execution itself.

References

-
1. Al-Bassam, Mustafa, Alberto Sonnino, and Vitalik Buterin. "Fraud and Data Availability Proofs." <https://arxiv.org/abs/1809.09044>. ↵
 2. Celestia Docs. "Data Availability." <https://docs.celestia.org/learn/celestia-101/data-availability/>. ↵ ↵2

3. Buterin, Vitalik, et al. "EIP-4844: Shard Blob Transactions." <https://eips.ethereum.org/EIPS/eip-4844>. ↩
4. Ethereum Foundation. "Fusaka Mainnet Announcement" (November 6, 2025). <https://blog.ethereum.org/2025/11/06/fusaka-mainnet-announcement>. ↩
5. Feist, Dankrad, et al. "EIP-7594: PeerDAS." <https://eips.ethereum.org/EIPS/eip-7594>. ↩
6. Celestia Documentation. "The lifecycle of a celestia-app transaction." <https://docs.celestia.org/learn/celestia-101/transaction-lifecycle/>. ↩
7. Celestia Documentation. "Blobstream." <https://docs.celestia.org/learn/blobstream/>. ↩
8. Celestia Documentation. "Data retrievability and pruning." <https://docs.celestia.org/learn/celestia-101/retrievability/>. ↩
9. EigenDA Documentation. "EigenDA Payment and Data Dispersal Guide." <https://docs.eigencloud.xyz/eigenda/integrations-guides/quick-start/v2/>. ↩
10. EigenDA Specification. "Security Parameters." <https://layr-labs.github.io/eigenda/protocol/architecture/security-parameters.html>. ↩
11. EigenDA Documentation. "Security Model." <https://docs.eigencloud.xyz/eigenda/core-concepts/security/security-model>. ↩
12. Avail Project. "Avail Light Client README." <https://github.com/availproject/avail-light/blob/main/client/README.md>. ↩
13. Avail Project. "Application client implementation." https://github.com/availproject/avail-light/blob/main/core/src/app_client.rs. ↩

Chapter 9: Parallel Execution

Introduction

Most blockchain virtual machines execute transactions in a block one after another. This makes verification deterministic and simple, but it leaves modern multi-core processors underused. Parallel execution asks which transactions can run at the same time without changing the final state.

The problem is not merely starting several threads. Transactions can read and write shared accounts and contracts. A parallel engine must detect conflicts and produce exactly the result required by the chain's canonical transaction order.

Intuition: Which Transactions Can Run Together?

A block defines an ordered list of transactions. The simplest executor processes transaction 1, updates state, then processes transaction 2 against that new state. This **sequential execution** is easy to reason about but uses only one execution path at a time.

Parallel execution asks whether independent transactions can run simultaneously on several CPU cores while producing exactly the result the canonical order requires.

Imagine two cashiers. Alice transfers funds between accounts A and B while Chen transfers funds between C and D. The account sets do not overlap, so both updates can be calculated together. If Chen instead spends from B, his result depends on Alice's earlier update and the two operations conflict.

Reads, writes, and conflicts

A transaction's **read set** is the state it inspects. Its **write set** is the state it changes. Two transactions conflict when canonical order can affect the result:

- both write the same value;
- one writes a value the other reads;
- a dynamic call discovers another overlapping key.

Two reads of the same value do not conflict because neither changes it.

A **conflict graph** represents transactions as points and conflicts as connecting lines. Transactions with no line between them can be candidates for the

same parallel wave. This is a planning model; actual virtual-machine behavior may discover more accesses.

Declared and optimistic execution

A **declared-access** system requires a transaction to identify accounts or objects it may touch before execution. The scheduler can separate non-overlapping declarations. Over-declaring reduces parallelism; under-declaring must fail or follow a protocol rule.

An **optimistic executor** starts transactions in parallel without knowing every access. It records what each attempt read and wrote. At commit time, it checks whether an earlier canonical transaction changed something the attempt read. If so, the attempt is **aborted** and retried against newer state.

"Optimistic" here means doing speculative work first and validating conflicts later. It does not change the canonical transaction order or permit different nodes to keep different outcomes.

Multi-version state

A **multi-version database** keeps several versions of a value tagged by transaction position. A speculative transaction reads the newest version that should precede it. If an earlier transaction later creates a newer relevant version, validation detects that the speculative read was stale.

Think of draft pages numbered by order. A worker assigned page 8 may read the latest accepted edit before 8. If page 5 later changes that paragraph, page 8's work must be checked again.

Determinism and commitment

CPU thread timing differs between machines. One worker may finish first on one validator and last on another. Consensus remains safe only if every valid schedule commits the same ordered result.

The executor may schedule freely internally, but state roots, receipts, logs, gas, return values, and failure outcomes must match the reference sequential semantics. Parallelism is an implementation method, not a new meaning for the block.

The hot-state limit

A popular counter, liquidity pool, or market price can become **hot state** touched by many transactions. More CPU cores do not help when every update must follow one another. This is like adding checkout lanes when every cashier still needs the same single stamp.

Applications improve concurrency by splitting independent balances or markets, using append-only claims, aggregating later, and avoiding unnecessary

global counters. The protocol can schedule better only when the workload contains real independence.

Reading speedup

If a fraction of work must remain serial, it limits total speedup. Eight cores cannot make a program eight times faster when half its work uses one ordered path. Retries, validation, database locking, and final root construction add more overhead.

Worked examples later show attempts and milliseconds explicitly. Count all speculative attempts, including discarded ones; reporting only successfully committed transactions hides wasted resources.

Why the EVM Is Commonly Sequential

Parallel execution preserves one canonical result

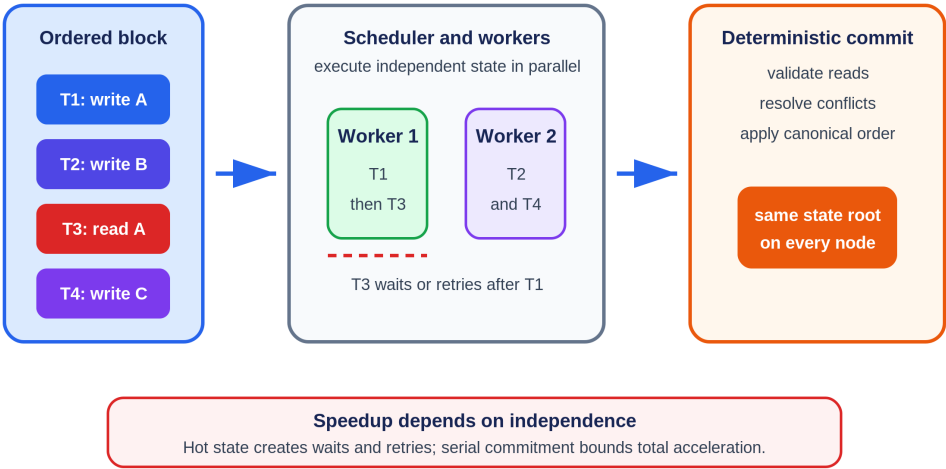


Figure 9.1: Independent transactions can run on different workers, while conflicting reads wait or retry and commitment preserves the canonical result. Original figure for this book, based on Block-STM and SC6019 Lecture 05.

An Ethereum transaction may call arbitrary contracts, which may call other contracts and touch storage addresses not obvious from the transaction envelope. The effect of transaction B can depend on the state produced by transaction A. Executing B early may therefore return a different value.

Sequential execution solves this by applying transactions in order. It is deterministic but limits execution throughput to a single ordered pipeline. Faster CPUs help, but adding cores does not automatically increase gas per second.

Conflict Graphs

Two transactions can run concurrently when their state accesses do not conflict. A conflict exists when:

- both write the same state location; or
- one writes a location the other reads.

Two read-only accesses are compatible. If the access sets are known, an engine can build a dependency graph and schedule independent transactions together.

The main challenge is discovering those access sets. Smart contracts generate addresses dynamically, and calls may depend on prior reads.

Declared Access Lists

Some systems ask transactions to declare the accounts or objects they will access. The scheduler can group non-overlapping transactions before execution.

Solana transactions specify accounts and whether they are read-only or writable. The Sealevel runtime can execute transactions that do not contend for writable accounts in parallel.¹

Sui uses an object-centric model with two ordering paths. Address-owned mutable objects can use the Mysticeti fast path, while shared and party objects require consensus sequencing.² The current implementation integrates fast-path votes and certificates into Mysticeti directed acyclic graph (DAG) blocks, so the fast path reuses the consensus communication structure without waiting for a total-order consensus commit.³ Explicit object inputs also expose conflicts, allowing transactions over disjoint objects to execute in parallel.

The benefit is predictable scheduling. The cost is a programming model that exposes concurrency to developers and users. Popular shared contracts can still become hot spots.

Optimistic Parallel Execution

Optimistic execution starts transactions speculatively, detects conflicts afterward, and re-executes work when necessary. It is similar to software transactional memory in databases.

Block-STM, used by Aptos, executes an ordered block with multiple worker threads. Transactions record reads and writes in multi-version data structures. If a read is invalidated by an earlier transaction, the affected transac-

tion is aborted and retried. Despite speculative execution, the committed result respects the original block order.⁴

This approach preserves a familiar transaction model and discovers dependencies at runtime. Its performance depends on workload contention. Independent transfers scale well; thousands of transactions updating one shared counter do not.

Determinism and Consensus

Validators must agree on the same state root. Parallel scheduling itself may be nondeterministic, but the committed semantics cannot be.

A safe engine defines:

1. a canonical transaction order;
2. rules for which version of state each read observes;
3. conflict detection and validation;
4. deterministic abort and retry behavior;
5. a final write order equivalent to sequential execution.

A performance bug can become a consensus bug if different validators commit different results. Parallel runtimes therefore need testing that combines database concurrency control with adversarial smart-contract behavior.

The Hot-State Problem

Parallel execution does not help when a workload concentrates on one piece of state. An automated market maker pool, popular NFT mint, or global counter can serialize the block.

Applications can reduce contention through:

- partitioned order books or liquidity pools;
- per-user nonces and balances;
- commutative updates;
- batching and aggregation;
- actor or object models;
- delayed reconciliation.

This is co-design: the VM exposes concurrency, and applications organize state so the concurrency can be used.

Parallel Execution vs Sharding

Parallel execution uses multiple cores within a validator. Sharding divides work across validator groups.

Dimension	Parallel Execution	Sharding
Main scale unit	CPU cores in one validator	Validators and committees
State view	Often shared	Partitioned
Composability	Can remain synchronous	Cross-shard calls often asynchronous
Main limit	Contention and hardware	Committee security and communication

The techniques complement each other. Each shard can execute transactions in parallel, and a rollup can use a parallel VM while relying on another layer for data and settlement.

Named Transaction Traces: Solana, Sui, and Aptos

Deployment label: all three are production networks. They expose concurrency differently. Solana transactions declare account access before execution. Sui makes object identity and ownership central to its transaction model. Aptos orders transactions and uses Block-STM to discover conflicts during speculative execution. The useful comparison is not a peak transactions-per-second number. It is what information the scheduler has, what happens when two transactions collide, and which result becomes canonical.

Solana: declared accounts become runtime locks

Trace two users buying different items from one marketplace program. Each Solana transaction contains signatures, a message, recent blockhash or durable-nonce context, instructions, and an account-key list. Each instruction identifies the program and the accounts it will read or write. The message marks accounts as writable or read-only. This declaration is part of the signed transaction, not a scheduler guess.^{1 5}

The processing pipeline receives and deserializes the transaction, verifies signatures, sanitizes structure, checks compute budget and age, validates nonce and fee payer, loads accounts, executes instructions, then commits or rolls back.⁵ Before execution, the runtime can see that transaction A writes inventory account `item_A` while transaction B writes `item_B`. If their other writable sets do not overlap, workers can execute them in parallel even though both call the same marketplace program. Program identity alone does not force serialization; writable state overlap does.

Now add one global writable counter recording all sales. Both transactions name it. The runtime cannot safely execute both writes concurrently, so the

counter becomes a lock conflict and a throughput bottleneck. If a client forgets to provide an account an instruction needs, the program cannot reach out to arbitrary undeclared state; execution fails. If a transaction declares an account writable when it only reads it, correctness survives but concurrency falls. If it understates access, the transaction fails rather than quietly bypassing locks.

Observable evidence includes the signed account list, writable flags, instruction list, compute-unit use, execution logs, status, and commit slot. A benchmark should report writable-account overlap and account-lock failures. Ten programs touching one hot token or market account may serialize more than ten calls to one program over disjoint accounts.

Sui: Mysticeti integrates fast path and consensus

Sui represents state as objects with identifiers, versions, ownership, and digests. A transaction names the objects it consumes or mutates. This makes dependencies explicit at an application level: two transfers over unrelated address-owned objects can follow independent fast-path certification and execution, while two calls mutating one shared or party object require consensus-assigned versions and contend on that object's state.^{2 3}

Trace a coffee-shop payment using address-owned coin objects. The wallet constructs a transaction naming the gas object, payment coin, recipient, commands, and current object versions. The user signs it and submits it through a full node to a validator. The validator checks authorization and the object references and includes the transaction in its signed Mysticeti block. Peer blocks carry accept votes. Once the transaction has votes from $2f + 1$ distinct validators it is certified; Mysticeti-FPC finalizes it either when $2f + 1$ validators support that certificate or when a Mysticeti-C commit contains the certificate in its causal history.³ Validators can then execute the Move commands and produce effects naming created, mutated, wrapped, or deleted objects.

This is a fast path in the ordering sense: the payment need not wait for total-order consensus over unrelated transactions. It is not a single-validator trust path. Its certificate still requires Byzantine-quorum evidence, and its votes are embedded in the same DAG communication used by Mysticeti consensus. The current general lifecycle page describes a consensus-commit path, effects acknowledgements, and a certified-checkpoint fallback; implementations and clients must state which path and certificate they observed rather than flattening both into one RPC status.⁶

Owned-object authorization still matters. Separate customers spending separate coin objects do not contend on one account balance row. Two transactions consuming the same owned-object version cannot both succeed. A shared object such as one public auction can become hot because independ-

ent users race to mutate the same object. Shared and party inputs carry consensus-managed versioning, so a client does not guess their latest access version. The object model therefore chooses both a versioning path and an execution conflict domain.

Failure follows object versions. If two signed transactions try to consume the same address-owned coin version, both cannot succeed. Once one effect is final, replay or the conflicting spend is rejected by the version and ownership rules. A wallet built against a stale fast-path object reference must refresh and reconstruct rather than retry opaque bytes indefinitely. For a shared or party object, consensus determines the access version; the transaction identifies the object under the protocol's consensus-object rules rather than guessing its latest mutable version. A globally shared object can become the same hot-state bottleneck seen in other systems, even when unrelated objects execute in parallel.

Observable evidence includes transaction digest, ownership type, input object IDs and versions, user signature, fast-path certificate or consensus evidence, certified effects, and checkpoint inclusion. Certified effects and checkpoint inclusion can both prove finality through documented paths, but the checkpoint is also the durable stream consumed by state sync and indexers. An application should name which evidence it verified instead of treating an RPC acknowledgement as equivalent to either.

Aptos: Block-STM discovers conflicts speculatively

Aptos transactions do not need to declare a complete read/write set in advance. Consensus establishes a block order. The execution engine then applies Block-STM, a software transactional memory design that speculates on multiple ordered transactions, records versioned reads and writes, validates those reads, and re-executes work whose assumptions were invalidated by an earlier transaction.^{7 8}

Trace an ordered block containing T_0 , T_1 , and T_2 . T_0 updates Alice's balance. T_1 touches an unrelated resource. T_2 reads Alice's balance and sends funds. Workers may start all three. T_1 can finish independently. T_2 may first read an older multi-version value while T_0 is still running. Validation later sees that canonical earlier transaction T_0 changed the resource. Block-STM aborts and re-executes T_2 against the right version. The final state must match sequential execution in consensus order even though the work overlapped.

This optimistic design extracts parallelism without requiring programmers to predict all accesses. Its failure cost is wasted work. An adversarial or badly designed workload can let many transactions run almost to completion and then conflict on one late-read resource. Re-execution increases CPU, cache and memory pressure while committed throughput falls. Correctness requires aborted attempts to leave no durable events, writes, gas result, or ex-

ternal effect. The final receipts and state root must be independent of worker count and thread timing.

Aptos's end-to-end transaction path is broader than Block-STM. A client submits to a fullnode; transactions propagate toward validators, enter the consensus and Quorum Store pipeline, are ordered, executed, and committed to storage.⁸ A fast speculative attempt is not client finality. The user observes submission, pending state, block order, execution result, and commit.

Fixed workload comparison

Workload	Solana	Sui	Aptos
Two payments over disjoint state	Parallel when declared writable account sets do not conflict	Address-owned objects can certify on independent Mysteri fast paths and execute in parallel	Block-STM speculates in parallel and validates successfully
Two updates to one hot market	Shared writable account creates lock contention	Consensus sequences the shared or party object; mutable access remains an execution conflict	Speculation conflicts; later ordered transaction may re-execute
Dependency information	Signed account list and writable flags before execution	Object IDs, versions, ownership and shared-object status	Read/write dependencies discovered during speculative execution
Safe collision behavior	Conflicting locks prevent unsafe concurrent writes	Fast-path certificates or consensus ordering plus object-version rules prevent incompatible effects	Validation aborts stale speculation and retries in canonical order
Performance trap	Overbroad writable lists and global accounts	One popular shared object	Late conflicts and repeated speculative work
Audit evidence	Accounts, instructions, logs, compute, slot/status	Inputs and versions, effects, checkpoint	Ordered block index, attempts/conflicts, receipt, committed state

All three preserve a deterministic canonical result. They differ in when dependency information appears and who bears mistakes. Solana makes the client and program expose accounts. Sui makes object topology part of application design. Aptos lets the runtime discover dependencies but may spend extra work on wrong speculation. A credible comparison replays the same contention distribution and reports committed results, retries or lock conflicts, latency percentiles, and state-commit cost.

Benchmarking Parallel Engines

Peak TPS is especially misleading here. A benchmark should report:

- transaction mix and conflict rate;
- number and type of CPU cores;
- state size and storage medium;
- block size and latency;
- abort/re-execution rate;
- throughput under hot-state contention;
- time to compute and verify the state root.

The correct question is not how many ideal transfers fit in a second, but how performance degrades as real contracts share state.

Worked Example: Three Transactions, Two Cores

Start with Alice holding ten tokens and Bob holding none. T1 transfers four from Alice to Bob. T2 reads Bob's balance and sends half to Carol. T3 updates an unrelated game object.

T1 and T3 can run concurrently because their read/write sets do not overlap. T2 cannot safely commit before T1 because it must observe Bob's balance of four. If T2 speculatively reads zero, validation detects that T1 wrote the same location earlier in canonical order. T2 aborts, reads version four, and retries. The result matches sequential execution while useful work from T3 ran in parallel.

Replace T3 with 10,000 swaps against one liquidity pool and the workload serializes because every swap touches the same reserves. More cores do not remove a contract-level dependency. Transfer-only benchmarks hide this behavior.

Designing Contracts for Concurrency

An exchange can partition markets so ETH/USDC trades do not conflict with BTC/USDC. A game can store inventories as owned objects and batch global rankings. A rewards contract can accumulate per-user claims rather than increment one global counter.

These designs trade immediate global consistency for parallel work and need explicit reconciliation rules. The VM cannot invent independence that the application's state model does not contain.

Multi-Version State and Validation

Optimistic parallel execution needs a versioned view of state. When transaction τ_i writes key k , the executor records version (i, value) . A later transaction reads the highest version with index less than its own canonical index.

Workers may execute out of order, so a read can initially observe a speculative value. Validation checks whether the read version is still the correct predecessor after earlier transactions finish. If not, the transaction and dependent work are scheduled again.

Conceptually:

```
execute(i):
    for each read(k):
        v = latest_version_before(k, i)
        record_read_dependency(k, v.version)
    buffer writes as version i

validate(i):
    for each recorded_dependency(k, j):
        require j == latest_committed_version_before(k, i)
    if any requirement fails:
        abort i and dependent transactions
```

A real implementation must coordinate incarnation numbers, speculative writes, memory reclamation, and dependency wake-ups without turning the scheduler itself into a bottleneck.

Static Scheduling With Access Lists

Declared-access systems know conflicts before execution. Build a graph whose vertices are transactions and whose edges connect conflicting read/write sets. Transactions without edges between them can share a parallel wave.

The declaration needs enforcement. If a transaction touches an undeclared writable account, execution must fail rather than silently access it. Over-declaration is safe but reduces parallelism. Users or builders therefore have an incentive to provide narrow access lists.

Block construction can become concurrency-aware. A builder may choose a set of lower-fee independent transactions that uses all cores instead of a slightly higher-fee set contending on one account. This creates a new fee-market question: how should blockspace price scarce sequential state access versus abundant parallel capacity?

Deterministic State Commitment

Even after execution, validators must compute one state root. If parallel workers update a shared Merkle tree with locks, root construction can serialize. Alternatives batch writes, sort them by key, and update independent subtrees concurrently before combining roots.

The state-commitment scheme affects witness generation and stateless validation. Vekle or sparse Merkle structures have different update, proof, and storage costs. VM throughput measured without root computation can overstate full-block performance.

Parallel Execution Failure Modes

Abort storms. Many speculative transactions read values repeatedly invalidated by earlier writes. Adaptive scheduling can serialize hot keys after conflicts are detected.

False conflicts. Coarse account-level locks serialize transactions touching independent storage slots in one contract. Finer keys improve concurrency at the cost of tracking overhead.

Nondeterministic host behavior. Time, random numbers, floating-point differences, or iteration over unordered structures can produce different results. The VM must define deterministic inputs and arithmetic.

Denial of service. An attacker constructs transactions that trigger expensive speculative work and repeated aborts. Fees must cover wasted execution or the scheduler must bound speculation.

State-root bottleneck. Execution scales across cores but commitment remains sequential. End-to-end benchmarks reveal this gap.

Benchmark Matrix

A useful benchmark varies both transaction complexity and conflict rate:

Workload	Conflict pattern	What it measures
Independent transfers	Distinct accounts	Best-case scheduler scaling
Hot counter	One shared write	Worst-case serialization
DEX pairs	Partitioned markets	Application-level parallelism
NFT mint	Shared supply plus user writes	Burst contention
Mixed contracts	Read/write distribution from production	Realistic abort behavior
Adversarial conflicts	Deliberate invalidations	DoS resilience

Report execution, validation, aborts, root computation, memory, and speedup against the same engine running one worker.

Scheduling Algorithms

A scheduler can use several strategies.

Greedy waves place a transaction in the earliest wave whose writes do not conflict with earlier reads or writes. This works well when access lists are known and graph construction is cheap.

Work stealing gives each worker a queue and lets idle workers take ready transactions from others. It balances irregular execution times, but dependency bookkeeping must prevent a transaction from running before required predecessors.

Speculative windowing executes only a bounded range ahead of the validated frontier. A large window exposes more parallelism but wastes more work when early transactions invalidate later reads.

Hot-key serialization detects repeatedly conflicting keys and routes their transactions to an ordered lane while leaving unrelated work parallel.

The best scheduler depends on conflict distribution, transaction duration, and cost of abort. Benchmarks should include short and long transactions; counting transactions alone hides load imbalance.

Parallel Execution Memory, I/O, and NUMA Effects

Adding execution workers can move the bottleneck from CPU instructions to memory bandwidth, cache coherence, or state-database I/O. A scheduler benchmark that reports core count without memory and storage behavior can mistake hardware saturation for protocol contention.

Shared memory pressure

Workers read account metadata, contract code, storage values, version tables, and execution results. If their working sets exceed cache, they fetch from main memory. More workers then compete for finite memory bandwidth.

Suppose one worker executes 8,000 transactions per second and reads 40 kB of memory per transaction. Its read demand is:

$$8,000 \times 40 \text{ kB} = 320 \text{ MB/s}$$

Sixteen ideal workers would request 5.12 GB/s before writes, proofs, database overhead, and cache misses. On a machine sustaining 4 GB/s for this access pattern, speedup saturates before sixteen workers even without logical conflicts.

Measure cache misses, memory bandwidth, allocation, garbage collection, and lock wait. CPU utilization below 100 percent can coexist with a memory bottleneck.

Non-uniform memory access

Multi-socket servers often have **non-uniform memory access (NUMA)**: a core reaches memory attached to its own socket faster than memory attached to another. A shared version table allocated on one socket can make remote workers slower.

Pin worker threads, place memory by access locality, partition state caches, and report socket topology. A result from one large NUMA server does not automatically predict performance on many smaller machines.

Scheduler correctness must not depend on thread pinning. NUMA tuning changes performance only; state roots and receipts remain identical.

Database I/O

Cold state misses memory and reaches storage. Random reads can dominate latency even when nominal SSD bandwidth is high. Report IOPS, queue depth, read/write amplification, compaction, cache size, and state locality.

An optimistic executor may read state during speculative attempts that later abort. If 30 percent of attempts abort after cold reads, storage traffic can rise without committed throughput. Cache pollution can also slow the successful path.

Use an execution cache keyed by state version and invalidate it deterministically. A stale cache result must be detected before commit; cache correctness is consensus-critical even when cache policy is not.

False sharing and locks

Two workers can update independent logical values stored on the same cache line. Hardware repeatedly transfers ownership of that line, causing **false sharing** even though the transactions do not conflict at protocol level.

Separate frequently written worker counters, use per-worker buffers, and merge them deterministically. Instrument lock hold time and contention by data structure. A global metrics lock can destroy scalability while the VM itself is parallel.

State commitment bottleneck

Execution may run in parallel while Merkle-tree or other state commitment updates serialize near shared ancestors. Batch and deduplicate leaf updates, calculate independent subtrees concurrently, then combine roots in canonical order.

Do not report execution completion before commitment when validators need the new root to accept the block. Measure execution, validation, writeback, root calculation, and database flush separately.

Oversubscription

More software workers than hardware threads can hide I/O latency, but excessive workers increase context switching and memory use. GPU or prover tasks on the same host may also compete for CPU, memory, and I/O.

Sweep worker counts rather than choosing the maximum. Record where throughput plateaus and tail latency or abort rate worsens. The best count can change with workload locality and state size.

Persistence and crash recovery

Parallel workers produce intermediate results that must become durable atomically with the committed block boundary. A crash after some database writes but before root persistence must roll back or replay idempotently.

Use write batches, journals, generations, or copy-on-write structures. Recovery rechecks the last durable state root and must not expose partial receipts or logs to indexers.

Hardware-aware benchmark matrix

Vary:

- cores, sockets, and simultaneous multithreading;
- memory channels, capacity, and NUMA placement;
- hot versus cold working set;
- storage medium, IOPS, queue depth, and compaction;
- worker count and scheduler policy;
- conflict distribution and abort depth;
- commitment scheme and flush policy;
- background snapshot, proof, and indexing load.

Report throughput and p99 block completion with hardware counters. A 10x VM microbenchmark speedup may become 2x end-to-end when state commitment and storage dominate.

Operational assertions

Assert identical outputs across worker counts and placements, bounded memory under adversarial conflicts, recovery from crashes at every persistence boundary, no starvation under cold-state load, and stable catch-up while snapshots or compaction run.

Parallel blockchain execution scales when independent work survives both logical conflicts and physical resource limits. The scheduler, memory hierarchy, database, commitment tree, and persistence path form one executor.

Fee Markets for Contention

A transaction can consume little CPU yet block many others by writing a popular key. Traditional gas meters its direct execution but not the opportunity cost of serializing a block.

A concurrency-aware market could charge for declared writable accounts, price hot keys dynamically, reserve parallel lanes, or let builders optimize total fee under a conflict graph. Each proposal affects predictability and manipulation. A user might over-declare reads to exclude competitors, while a builder might prefer independent low-fee transactions that fill idle cores.

The protocol must keep consensus deterministic. If scheduling affects inclusion, validators still verify one canonical ordered block; local thread decisions cannot change fees or results.

Parallel Smart-Contract Patterns

Sharded Counters

Replace one global counter with k buckets selected by account hash. Updates spread across buckets; reads sum them. This improves writes but makes reads more expensive and may provide only eventual snapshots.

Escrowed Objects

Move an asset into an order-specific object before matching. Independent orders then touch independent state. Settlement combines only matched objects.

Commutative Accumulators

Some updates can be combined regardless of order, such as adding independent reward deltas. The VM or contract batches deltas and applies a deterministic reduction.

Epoch Batching

Collect actions during an epoch and compute one aggregate update. This reduces shared writes but adds latency and requires rules for ordering within the batch.

Patterns should preserve invariants under retries and partial execution. A parallel-friendly design that weakens accounting correctness is not an optimization.

Testing Determinism

Run the same block many times with different worker counts, queue seeds, thread timing, and storage latency. Every run must produce identical receipts, logs, gas, and state root.

Differential tests compare the parallel engine with a simple sequential reference. Fuzzers generate transactions with nested calls, reverts, dynamic storage, and conflict patterns. Crash tests stop a worker after speculative writes and verify that recovery discards uncommitted versions.

Race detectors help implementation safety, but protocol determinism is a higher-level property. Two race-free executions can still disagree if iteration order or error precedence is unspecified.

End-to-End Speedup Limits

Amdahl's law bounds speedup when part of execution remains serial. If fraction p is parallel and fraction $1-p$ is serial, speedup on N workers is at most:

$$1 / ((1 - p) + p/N)$$

If 20% of block processing is serial, infinite workers cannot exceed $5\times$ speedup. Signature checks may parallelize while ordering, hot state, receipt assembly, and root commitment remain serial.

Measure the entire validation pipeline before projecting core scaling. Optimizing the parallel fraction can make the unchanged serial section the dominant cost.

Worked Scheduler Trace: Speculate, Validate, Commit

Consider four transactions in canonical block order:

```
T1: read A; write B = A + 1
T2: read C; write D = C + 1
T3: read B; write C = B + 1
T4: read E; write F = E + 1
```

T1, T2, and T4 can begin from the same snapshot because their initial access sets do not conflict. T3 reads B, which T1 writes, and writes C, which T2 read. If T3 speculates before those dependencies are resolved, validation must abort and retry it against the state that includes earlier canonical transactions.

One possible trace is:

```
worker 1: execute T1 at version 0 -> tentative write B
worker 2: execute T2 at version 0 -> tentative write D
```

```

worker 3: execute T3 at version 0 -> reads old B, tentative write C
worker 4: execute T4 at version 0 -> tentative write F

validate T1 -> success; commit version 1
validate T2 -> success; commit version 2
validate T3 -> stale read of B; abort tentative C
validate T4 -> success; commit when canonical prefix permits
retry T3 after T1/T2 -> read committed B; write C; commit version 3
commit T4 as version 4

```

Workers can finish out of order, but the visible result must match sequential execution of T1, T2, T3, T4. An engine may buffer T4 even after successful validation until every earlier transaction has a committed result or deterministic abort.

Multi-version state

Optimistic engines commonly tag values with transaction versions. A speculative read records both key and version observed. Validation asks whether an earlier canonical transaction wrote that key after the read's snapshot. If so, the read is stale.

```

ReadRecord = (transaction, key, observed_version)
WriteRecord = (transaction, key, new_value)

```

A retry must clear or supersede every tentative write and derived event from the aborted execution. Leaving one log, gas refund, message, or cache entry visible can create a state root that no sequential execution produces.

Dynamic access

The scheduler may not know T3 touches C until execution. Contract calls can compute keys from prior reads and invoke other contracts. Declared access lists improve planning, but the runtime must define behavior when a transaction touches an undeclared key: reject it, expand its lock set and retry, or execute it on a conservative path. Silently continuing breaks the scheduler's conflict assumptions.

Deterministic failure

Transactions that revert still consume protocol-defined resources and may emit no durable writes. Parallel execution must reproduce the same success/revert decision and gas accounting as sequential execution. Sources of non-determinism include host clocks, thread order, unordered map iteration, floating-point behavior, random number generators, external I/O, and races in native precompiles.

Consensus inputs such as block time or randomness must enter through canonical block fields. The VM should expose no process-local value that differs among validators.

Scheduler invariants

For each committed block, test:

1. parallel and reference sequential execution produce identical state roots;
2. receipts, events, gas totals, and return data also match;
3. every committed read observes the newest earlier canonical write;
4. aborted attempts leave no durable state or externally visible event;
5. retries terminate or hit a deterministic block limit;
6. worker crashes and restarts do not change the committed prefix;
7. transaction outcome does not depend on worker count or thread interleaving.

Differential testing should run the same generated block with one worker, several worker counts, randomized scheduling seeds, and the sequential reference. Persist the seed and access trace on failure so the race can be reproduced.

Contention and Admission Control

Retries consume real CPU, memory bandwidth, and cache capacity without increasing committed throughput. An attacker can construct transactions that appear independent early, then converge on one key late in execution. Even reverted attempts may exhaust the executor.

A production scheduler needs limits and pricing for speculative work. Options include capping attempts per transaction, charging for repeated execution under protocol rules, routing known-hot contracts to a serial lane, and using recent access history to avoid speculation likely to conflict. Any heuristic may affect performance but must not affect the canonical outcome.

Suppose 1,000 transactions each take 1 millisecond on the first attempt. With 16 workers and no conflicts, ideal execution time is about 62.5 milliseconds before serial overhead. If 40 percent abort once after completing their work, the executor performs 1,400 transaction-attempts, increasing ideal worker time to 87.5 milliseconds. If retries serialize on one hot key, the critical path can approach 400 milliseconds plus parallel work. Reporting only successful transactions hides this amplification.

Expose attempt count, aborted work, conflict keys, serial-lane depth, validation time, commit time, and state-database stalls. Capacity is the rate of com-

mitted canonical work under the target contention distribution, not the rate of speculative starts.

Conclusion

Parallel execution turns transaction independence into throughput. Declared-access systems expose dependencies before execution; optimistic systems discover them during execution and retry conflicts. Both must preserve deterministic, sequentially valid results.

The limit is contention. A parallel VM is most effective when its application model avoids global hot state. The next chapter examines a different bottleneck: agreement among many validators.

References

1. Solana. "Transactions and Instructions." <https://solana.com/docs/core/transactions>. ↩ ↩2
2. Sui Documentation. "Types of Object Ownership." <https://docs.sui.io/develop/objects/object-ownership/>. ↩ ↩2
3. Danezis, George, et al. "Mysticeti: Reaching the Latency Limits with Uncertified DAGs." <https://docs.sui.io/paper/mysticeti.pdf>. ↩ ↩2 ↩3
4. Gelashvili, Rati, et al. "Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing." <https://arxiv.org/abs/2203.06871>. ↩
5. Solana Documentation. "Transaction processing pipeline." <https://solana.com/docs/core/transactions/transaction-pipeline>. ↩ ↩2
6. Sui Documentation. "Life of a Transaction." <https://docs.sui.io/develop/transactions/transaction-lifecycle>. ↩
7. Aptos Documentation. "Execution." <https://aptos.dev/network/blockchain/execution>. ↩
8. Aptos Documentation. "Life of a Transaction." <https://aptos.dev/network/blockchain/blockchain-deep-dive>. ↩ ↩2

Chapter 10: Consensus Scaling

Introduction

Consensus allows independent nodes to agree on one ordered history despite failures and malicious behavior. It is also a scalability bottleneck. More validators improve fault tolerance and decentralization, but create more messages, signatures, and network delay.

Consensus scaling therefore seeks lower communication cost and faster finality without hiding the network and adversary assumptions that make those results possible.

Consensus From First Principles

Nodes receive messages at different times. Two valid transactions may compete for the same funds, and two producers may propose different next blocks. **Consensus** is the protocol that lets independent nodes converge on one ordered history despite delay and faulty participants.

Consensus does not decide whether a contract rule is wise. It decides which valid proposal becomes canonical under agreed rules.

Fault models

A **crash fault** stops or restarts. A **Byzantine fault** can lie, sign conflicting messages, or coordinate with others. Blockchain consensus also needs **Sybil resistance**, a rule that prevents one attacker from creating unlimited voting identities. Proof of work weights computational work; proof of stake weights bonded stake; permissioned BFT systems use an admitted validator set.

The fault threshold is part of the claim. "Tolerates one-third Byzantine weight" means safety or liveness is proven only while adversarial voting weight stays below a specified boundary and the network model holds.

Network models

- **Synchronous:** messages arrive within a known maximum delay.
- **Partially synchronous:** such a bound eventually holds, but nodes may not know when stable conditions begin.
- **Asynchronous:** no fixed delivery bound is assumed.

A timeout cannot prove that a leader is malicious; the leader or network may be slow. Timeouts are tools for progress under a model, not evidence of intent.

Nakamoto-style consensus

Bitcoin-like consensus allows producers to extend a chain of valid blocks. Nodes follow the valid chain with the most accumulated work or protocol-defined weight. Temporary forks resolve as one branch becomes heavier.

Finality is **probabilistic**: deeper blocks become increasingly costly or unlikely to replace, but no single vote makes them mathematically irreversible. Applications choose a confirmation depth based on value and risk.

BFT-style consensus

A Byzantine fault tolerant (BFT) protocol uses explicit proposals and votes among a known validator set. With four equal replicas and at most one Byzantine, a quorum of three is common. Any two groups of three overlap in at least two replicas; because only one can be Byzantine, the overlap includes an honest replica.

Honest replicas follow locking or voting rules that prevent them from supporting incompatible commits. The overlap carries safety from one quorum certificate to another.

A **quorum certificate (QC)** is compact evidence that enough distinct validator or weight voted for a proposal. It may contain individual signatures or one aggregated signature plus a signer bitmap. The verifier must still check membership and weight.

Views, leaders, and pacemakers

A **view** is one numbered leader attempt. The leader proposes a block; replicas validate and vote. If progress stalls, a **pacemaker** uses timeouts and signed messages to move replicas to a higher view.

The new leader must carry forward the highest safe certificate or locked block. Replacing a stalled leader without this evidence could let different groups commit conflicting branches.

Safety versus liveness

Safety asks whether two honest nodes can commit conflicting histories. **Liveness** asks whether valid work eventually commits. During a severe partition, a protocol may halt to preserve safety. Once communication assumptions recover, view changes should restore liveness.

Think of safety as "do not approve two incompatible ledgers" and liveness as "do not remain stuck forever." A fast protocol that sometimes approves both

is unsafe; a perfectly consistent protocol that never produces another block is not live.

Finality and fork choice

A **fork-choice rule** selects the branch nodes should build on now. A **finality rule** identifies a prefix that should not revert under the fault assumptions. Some protocols combine them; others use a longest-chain head plus a voting-based finality gadget.

A transaction can therefore be included at the head but not finalized. Bridges and high-value applications often wait for the stronger boundary.

Why consensus throughput is not execution throughput

Voting on a block hash can be fast while distributing and executing the block body is slow. Empty-block benchmarks measure agreement on almost no application work. Complete tests include payload propagation, signature verification, execution, state commitment, leader failure, and catch-up.

Later sections introduce HotStuff, threshold signatures, DAG mempools, weighted quorums, and protocol traces. Each builds on the same questions: what evidence is signed, which quorums overlap, what state survives a crash, and how progress resumes after delay.

Safety, Liveness, and the Finality Boundary

A consensus analysis must separate two protocol properties from the decision boundary applications observe:

- **Safety** means honest nodes do not finalize conflicting blocks while the stated fault and protocol assumptions hold.
- **Liveness** means valid work eventually reaches the protocol's commit or finality condition once its timing and quorum-reachability assumptions hold.
- A **finality rule** identifies the evidence that marks a block or prefix committed. Reversing a deterministically finalized BFT block requires some assumption to have failed, such as excessive Byzantine weight, broken authentication, or an implementation violating its voting rules. Nakamoto-style protocols instead provide a probability of reorganization that generally decreases with additional confirmations.

Network timing matters. A synchronous model assumes a known message-delay bound. A partially synchronous model assumes the bound eventually holds but its starting time is unknown. An asynchronous model makes no timing bound.

These assumptions determine both fault tolerance and latency. Performance claims that omit them are incomplete.

Nakamoto and BFT-Style Consensus

Nakamoto consensus selects a chain through accumulated proof of work. It scales to an open validator set and tolerates temporary forks, but finality is probabilistic. Larger or faster blocks increase stale-block pressure and may favor well-connected miners.

Classical Byzantine fault tolerant protocols use voting rounds. A common $3f + 1$ partially synchronous design preserves safety with up to f Byzantine replicas and regains liveness after the network becomes timely and enough honest replicas share a view. A quorum certificate is evidence for one protocol phase; it is not automatically a commit. Once the protocol-specific sequence of certificates, locks, or voting phases satisfies its commit rule, finality is deterministic under the stated assumptions.

The challenge is communication. A naive all-to-all vote exchange requires quadratic messages as the validator set grows.

HotStuff

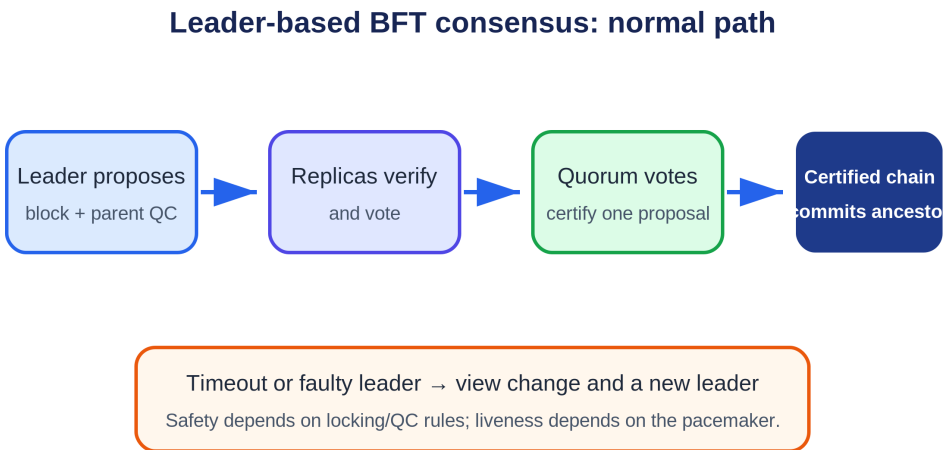


Figure 10.1: A leader proposes, replicas vote, votes form a quorum certificate, and chained certificates drive commitment. A timeout moves the protocol to a new view. Original figure for this book, based on the HotStuff paper.

HotStuff is a leader-based BFT protocol designed for linear communication in the normal case. Replicas vote on a leader's proposal, and the leader aggreg-

ates votes into a quorum certificate. A sequence of certified proposals creates the locking conditions needed for safety.¹

One QC does not finalize its block in chained HotStuff. A later certified chain advances the logical prepare, pre-commit, and commit phases for ancestors, and only the specified commit rule makes an ancestor final. This distinction is essential when reading explorers or implementations that expose "certified" and "committed" as separate states.

Its key contributions include:

- linear communication with threshold signatures or aggregated votes;
- optimistic responsiveness after the network becomes timely;
- a pacemaker abstraction for leader changes;
- pipelining in chained HotStuff.

A bad leader can still delay a view, but the protocol rotates leaders. The pacemaker and timeout rules are therefore as important to liveness as the happy path.

Sync HotStuff

The course examines Sync HotStuff, a synchronous state-machine replication protocol. In the steady state, a leader broadcasts a proposal, replicas echo it, and a replica can commit after waiting for the maximum round-trip delay unless it observes equivocation.²

The stronger synchrony assumption permits tolerance of up to one-half Byzantine replicas, compared with the familiar one-third bound under partial synchrony. The paper reports steady-state latency near 2Δ , where Δ is the known delay bound, and optimistic responsiveness when fewer than one-quarter of replicas fail to respond.

The trade-off is explicit: stronger fault tolerance and a simple fast path depend on a meaningful bound on network delay. A wide-area public blockchain may find that assumption harder to defend than a consortium network or data-center service.

Reducing Communication

Consensus protocols use several techniques to scale:

Leader Aggregation

Validators send votes to a leader instead of broadcasting every vote to everyone. The leader distributes one certificate.

Threshold and Aggregate Signatures

Many signatures are compressed into a smaller proof of quorum. This saves bandwidth and verification work, although distributed key generation and signer accountability introduce complexity.

Committees

A smaller rotating committee reaches agreement on behalf of a larger set. Random selection reduces targeted capture, but committee size determines the probability of a Byzantine majority.

Pipelining

Different consensus stages for consecutive blocks overlap. Throughput rises even if the latency of one block is unchanged.

DAG-Based Mempools

A **directed acyclic graph (DAG)** is a set of items connected by one-way references with no path that loops back to its start. Unlike one chain where each block has one parent, a DAG can record several independently disseminated batches at the same time. A DAG mempool spreads and certifies transaction data before a later consensus step chooses its order.

Protocols can separate transaction dissemination from ordering. Validators first certify batches in a directed acyclic graph, then consensus orders references to those batches. This reduces duplicated data transmission and keeps the ordering layer small.³

Block Propagation Is Part of Consensus

A theoretically efficient vote protocol can still be bottlenecked by distributing block data. Compact blocks, erasure-coded broadcast, relay networks, and separating headers from transaction bodies reduce this pressure.

There is also a censorship and centralization risk. Specialized relays and powerful block builders improve propagation but can become privileged infrastructure. Consensus performance must be measured with the network topology and block payload, not votes alone.

Consensus and Execution Separation

Consensus orders transactions; execution computes their effects. Separating them allows consensus to agree on compact batch references while execution and data dissemination proceed in parallel. Modular systems take this fur-

ther by assigning consensus and data availability to one layer and execution to another.

The layers still interact. Consensus cannot finalize unavailable data safely, and execution cannot finalize state without a canonical order.

Named Case Studies: How Consensus Families Reached Production

A protocol family is not a straight genealogy in which one paper becomes one product unchanged. Production systems borrow a safety rule, split one subsystem into two, replace the broadcast layer, or redesign the commit path. The three traces below separate what carried over from an earlier design from what did not.

From HotStuff to DiemBFT and AptosBFT

Deployment labels: Diem was a production-oriented implementation that did not become a public production network; AptosBFT is production. HotStuff supplies a chained Byzantine fault-tolerant core: rotating leaders propose blocks, weighted quorum certificates summarize votes, and a pacemaker moves replicas between views. DiemBFT turned that paper structure into an implementation with transaction execution, persistent safety state, epoch changes, networking, and operational recovery. AptosBFT descends from that code and design line but runs in a live network with a distinct data-dissemination path.^{4 5}

Trace an Aptos transfer from Alice to Bob. Alice submits a signed transaction to a fullnode, which forwards it toward validators. Validators validate and disseminate the transaction. Aptos documentation separates mempool, **Quorum Store**, consensus, execution, and storage. Quorum Store packages transaction batches and makes their availability independently certifiable. The round leader can propose references to available batches rather than bearing the full cost of first distributing every transaction inside the consensus proposal. Validators order the proposal through AptosBFT, execute the ordered transactions, vote, and commit after the required certificate path. Storage persists the committed result and the client can observe the transaction as committed.⁴

The HotStuff inheritance is visible in rounds, leaders, quorum certificates, chained certified blocks, and timeout-driven progress. The important production idea is still that an honest validator does not vote in a way that violates its locked or preferred branch rule, and a new leader carries sufficient certificate evidence to propose safely. What did **not** carry over unchanged is a simple mental model in which the leader broadcasts one complete block

and consensus alone handles all payload movement. Quorum Store separates payload availability from ordering. Aptos also couples the protocol to weighted stake, epochs, state synchronization, execution, and a real transaction pipeline. Calling all of that "HotStuff" conceals the components that dominate throughput and recovery.

Suppose the round leader fails. Validators time out, exchange timeout information, advance the round, and a later leader proposes from safe certificate state. That is the pacemaker liveness path. Suppose Quorum Store cannot form availability proofs for new batches. Consensus may still exchange empty or previously available proposals, but application throughput falls because ordering cannot safely refer to unavailable payloads. Suppose a validator loses its persisted safety state and signs conflicting rounds after restart. The mathematical quorum intersection argument cannot save a node that violates its own voting rule; slashing evidence and key safety become part of the production protocol.

For the user, the observable boundaries are submission to a fullnode, admission, batch availability, proposal, certification, execution, and commit. An RPC response that merely accepts Alice's transaction is not consensus finality. A useful incident report says whether the bottleneck was transaction dissemination, Quorum Store availability, leader progress, execution, state synchronization, or storage commit.

From Narwhal and Bullshark to Sui Mysticeti

Deployment labels: Narwhal and Bullshark are implemented research systems and prior production components; Mysticeti is Sui's production consensus. Narwhal separates reliable transaction dissemination from consensus ordering by building a directed acyclic graph (DAG) of certified batches. Bullshark orders that DAG. The conceptual gain is that validators continuously disseminate data instead of waiting for one leader to carry a large block to everyone.³

Sui's current protocol uses **Mysticeti-C** for total-order consensus and integrates **Mysticeti-FPC** for transactions that can use the owned-object fast path.⁶

⁷ Validators create signed blocks that reference earlier blocks, forming a DAG. The blocks carry transaction information, causal references, and fast-path votes. Mysticeti-C applies a decision rule to DAG structure and stake to commit leaders and derive an order for consensus transactions. Mysticeti-FPC can certify and finalize a nonconflicting fast-path transaction before a Mysticeti-C commit, while reusing the DAG blocks rather than running a separate signature-gathering protocol.

Trace a Sui transaction that touches a shared or party object and therefore needs consensus. The user signs it and sends it to a full node. The full node's transaction driver forwards it to a selected validator, which checks authoriza-

tion, object access, and gas and includes it in a proposed Mysticeti block. Peer validators validate the transaction and place accept votes in later blocks. Mysticeti-C commits the DAG position and yields the deterministic order and consensus-managed object versions. Validators then execute against Sui's object model, return effects, and the full node obtains quorum acknowledgements or waits for a certified checkpoint.⁶

For comparison, an address-owned fast-path transaction is also carried and voted on in Mysticeti DAG blocks, but it does not need a total order against unrelated transactions. The Mysticeti paper defines certification after votes from $2f + 1$ distinct validators and permits finalization either from quorum support for that certificate or when a Mysticeti-C commit contains it in its causal history.⁷ This reconciles two descriptions that can otherwise look inconsistent: fast-path traffic is integrated into Mysticeti's communication DAG, yet it can bypass total-order consensus.

What carried over from Narwhal/Bullshark is the DAG-first intuition: dissemination and causal references continue across rounds, payload availability is not reduced to one leader's broadcast, and ordering can use accumulated DAG evidence. What did **not** carry over unchanged is the two-box textbook picture "Narwhal mempool plus Bullshark consensus." Mysticeti changes the DAG and commit protocol to reduce latency and integrates with Sui's transaction model. A reader should not assume that every Narwhal certificate type, Bullshark wave, or earlier round structure remains a live Mysticeti mechanism merely because the systems share authors and lineage.

Now let the designated leader's block arrive late. In a linear leader-broadcast design, replicas may wait and then start a view change. In a DAG design, validators can continue producing and referencing other blocks, while the decision rule handles a missing or unsupported leader. Throughput and liveness still depend on enough weighted validators exchanging DAG blocks. A partition that prevents quorum-weight communication stops finality. Equivocating validator blocks must be detected and treated under the protocol's rules. If data named by a DAG block is unavailable, a reference is not a substitute for the payload needed to validate and execute transactions.

The visible evidence is therefore richer than "block height." Operators inspect DAG round or slot progress, blocks received by stake weight, leader decisions, missing ancestors, transaction certification, execution effects, and epoch state. The production question is whether the exact Mysticeti decision and persistence rules are safe through equivocation, restart, epoch transition, and asymmetric delay, not whether the earlier Narwhal paper was sound.

From the PBFT family to Tendermint and CometBFT

Deployment label: CometBFT is production infrastructure used by application-specific chains. Practical Byzantine Fault Tolerance (PBFT) established the familiar setting of a fixed validator group, fewer than one-third Byzantine faults, authenticated messages, and quorum intersection. Tendermint adapted the family for blockchains with repeated heights, stake-weighted validators, rotating proposers, locking, and two named voting steps. CometBFT is the maintained state-machine replication engine in that lineage and exposes the application through the Application Blockchain Interface (ABCI).^{8 9}

Trace one height. A proposer chooses a block for height h and round r . Validators receive and validate the proposal against consensus and application rules. They broadcast **prevotes** for the proposal or nil. If a validator observes more than two-thirds of voting power prevote a block, it can lock according to the protocol rule and broadcast a **precommit** for that block. More than two-thirds precommits for the same block at the height form the commit evidence; the engine advances to the next height and the application commits the resulting state. If no proposal or quorum arrives before a timeout, validators move to a higher round with a new proposer.⁸

The PBFT inheritance is quorum intersection, Byzantine authentication, a proposer-led normal path, and safety from vote restrictions. What changed is as important. CometBFT does not use PBFT's client-request, pre-prepare, prepare, commit vocabulary as a drop-in transcript. It repeats propose-prevote-precommit rounds at each blockchain height, uses locks to preserve safety across rounds, weights votes by validator power, changes validator sets at defined boundaries, gossips blocks and votes over a peer network, and calls an external deterministic application through ABCI. It also commits a block before moving to the next height rather than using HotStuff's pipelined chain of several certified descendants.

Suppose the proposer is offline. Validators time out, prevote nil when required, fail to form a block commit in that round, and proceed to the next proposer. Suppose a validator saw a quorum prevote and locked block A, then receives block B in a later round. It may not simply follow the newest proposal; the lock and proof-of-lock rules govern when it can vote differently. This is the safety bridge across rounds. A validator that precommits conflicting blocks for the same height and round creates evidence of equivocation. A network partition with neither side holding more than two-thirds voting power preserves safety but halts commits.

Application behavior is part of the trace. CometBFT orders and replicates calls, but the ABCI application must be deterministic. If two correct validators return different results for the same ordered block because of local time,

floating-point behavior, or an external API, consensus cannot turn those divergent applications into one state. Likewise, a valid consensus commit says that the validator quorum agreed to the block; it does not prove a bridge oracle, governance decision, or application price feed was economically correct.

Lineage comparison

Earlier family	Production descendant	Ideas that carried over	Ideas that changed or did not carry over
HotStuff	DiemBFT, then Aptos-BFT	Rotating leaders, chained quorum certificates, safe voting state, pacemaker/view change	Payload dissemination is split through Quorum Store; stake epochs, execution, state sync, and operations are part of the real system
Narwhal/Bullshark	Sui Mysticeti-C and Mysticeti-FPC	DAG dissemination, causal references, continued data flow despite an imperfect leader	Mysticeti changes the DAG decision protocol and embeds fast-path certification; do not project every Bullshark wave or Narwhal certificate onto it
PBFT family/Tendermint	CometBFT	Authenticated Byzantine quorum, proposer, quorum intersection, lock-like safety across retries	Propose-prevote-precommit rounds, weighted validators, repeated block-chain heights, gossip, ABCI, and non-pipelined commit distinguish the production engine

These are not three brand names for one algorithm. AptosBFT emphasizes a chained-certificate core with separate payload availability. Mysticeti lets the DAG carry total-order evidence and integrated fast-path votes. CometBFT uses explicit prevote and precommit rounds with locking at each height. All still need quorum reachability, persisted signing safety, authenticated validator sets, deterministic execution, and tested recovery when timing assumptions fail.

Comparison

Family	Finality rule	Typical fault condition	Communication idea	Main trade-off
Nakamoto	Reorganization probability generally falls with confirmations	Honest majority of effective block-production resource in the standard model	Gossip and longest/heaviest chain	Slow certainty and fork risk
Practical Byzantine Fault Tolerance (PBFT)-style	Deterministic commit under assumptions	Usually Byzantine weight < 1/3	Multi-phase voting	Quadratic communication in basic form

Family	Finality rule	Typical fault condition	Communication idea	Main trade-off
HotStuff	Deterministic commit after the certified-chain rule	Byzantine weight $< 1/3$ under partial synchrony for liveness	Leader aggregation and certificates	Leader and pacemaker complexity
Sync HotStuff	Deterministic commit after synchronous waiting and echo rules	Byzantine weight $< 1/2$ under the known delay bound	Synchronous echo and wait	Relies on a defensible delay bound
DAG + BFT	Protocol-specific deterministic commit rule	Often Byzantine weight $< 1/3$; protocol dependent	DAG dissemination and ordering evidence	More protocol components

Worked Example: Why a Quorum Certificate Matters

Take four replicas, A through D, with at most one Byzantine replica. A quorum contains three votes. A leader proposes block X and gathers votes from A, B, and C. The certificate proves that at least two honest replicas voted for X.

Could conflicting block Y also obtain three votes at the same height? Any two sets of three among four overlap in at least two replicas. At least one overlapping replica is honest, so locking and voting rules prevent both certificates under the safety assumptions.

A certificate compresses quorum evidence, but it does not alone define commitment. HotStuff uses a chain of certified proposals so replicas can distinguish a safe extension from a conflict. That structure permits safe leader replacement.

The Liveness Path

If leader A is offline, replicas time out, move to a new view, and send their highest certificates to leader B. B extends the safest certificate. Once messages arrive within the eventual network bound, it gathers a quorum and progress resumes.

Timeouts that are too short cause needless view changes during latency spikes. Timeouts that are too long make a faulty leader expensive. Pacemakers often back off after failures and reset after stable progress. Two implementations of the same paper can therefore show very different latency.

Throughput Is Not Finality Latency

Pipelining can commit one block per round after the pipeline fills even if each block needs several rounds to become final. Throughput measures spa-

cing between committed blocks; latency measures one transaction's journey. Benchmarks should report both, plus validator count, geography, payload size, signature scheme, and failure conditions.

HotStuff State and Voting Rules

A HotStuff replica tracks more than the latest block. It stores the current view, the highest quorum certificate it knows, and a lock that prevents votes creating conflicting commits.

A proposal contains a block extending a parent and a justify QC. A replica votes only when the proposal is safe according to the locking rule. A simplified check is:

```
safe_to_vote(proposal):
    return proposal.extends(locked_block)
       or proposal.justify_qc.view > locked_qc.view
```

The first condition follows the current lock. The second permits progress when the proposal carries newer quorum evidence. Exact protocol variants differ, but the invariant is the same: honest replicas do not vote in a way that lets two conflicting chains both satisfy the commit rule.

In chained HotStuff, each proposal can serve as a phase for an earlier block. Consecutive QCs pipeline prepare, pre-commit, and commit evidence. Once the required certified chain exists, an ancestor commits. The pipeline raises throughput because a new proposal advances several blocks' phases at once.

Pacemaker and View Synchronization

Safety can hold in a fully asynchronous period, but liveness requires honest replicas to spend enough time in the same view with an honest leader. The pacemaker creates this overlap.

When a timer expires, a replica broadcasts a timeout containing its highest QC. A timeout certificate or threshold of timeout messages justifies moving to the next view. The new leader collects the highest safety evidence and proposes an extension.

Timeout selection is adaptive. A fixed timeout small enough for normal operation may cause endless view changes during an outage. An exponential backoff eventually exceeds actual delay after the network stabilizes. Implementations reset or reduce timeouts carefully after progress so latency returns to normal without synchronizing replicas into another failure cycle.

Threshold Signatures and Accountability

A threshold signature compresses $2f + 1$ votes into one constant-size certificate. Validators first participate in distributed key generation or receive shares under another trusted process. No coalition below the threshold can sign.

Aggregation saves bandwidth, but a single threshold signature can hide individual signers. Systems needing slashing evidence may carry signer bitmaps or use aggregate signatures that preserve attribution. Key rotation, lost shares, and membership changes also add operational cost.

A consensus design should say whether a QC proves only quorum weight or also identifies every signer. This affects forensic analysis and punishment after equivocation.

DAG Mempools

Consensus proposals often carry large transaction batches. If every leader retransmits those bytes, bandwidth and leader performance dominate. A DAG mempool separates dissemination:

1. each validator broadcasts a transaction batch;
2. peers sign availability acknowledgements;
3. a certificate proves enough peers received the batch;
4. later consensus proposals order compact certificates rather than raw transactions.

The DAG records causal references among certificates. Ordering can continue with small messages after data dissemination has completed. Safety still depends on the ordering consensus, while availability depends on the certificate threshold and retrieval protocol.

This architecture adds queues and garbage collection. Nodes must retain certified batches until committed, recover missing parents, and prevent an attacker from flooding unreferenced data.

Consensus Implementation Checklist

- Define the exact block, vote, QC, timeout, and view-change messages.
- Domain-separate every signature by chain, epoch, message type, and view.
- Persist locks and safety state before sending votes.
- Test crash recovery between persistence and network send.
- Bound proposal and certificate sizes.
- Measure leader change under packet loss and skew.

- Include real block dissemination in benchmarks.
- Specify validator-set transitions and old-key retirement.
- Expose highest QC, lock, current view, and timeout metrics.
- Run conflicting-message and malformed-certificate tests across all clients.

The most dangerous consensus bugs live in rare transitions: restart, epoch change, delayed old messages, and view changes during partial network recovery.

Epoch and Validator-Set Changes

Consensus safety proofs often assume a fixed validator set, while proof-of-stake systems change membership. An epoch transition must bind the new set to a finalized decision by the old set.

A transition block can commit to validator public keys, weights, activation height, and protocol version. New validators begin voting only after the transition is final. Old signatures remain valid for old views but cannot authorize new-epoch blocks.

Light clients need a chain of authenticated set changes. Skipping directly from an old checkpoint to a current header without verifying intermediate transitions lets an attacker invent a validator set. Sync committees or succinct proofs compress this chain under additional assumptions.

Long-range attacks occur when old validators, whose stake is no longer slashable, sign an alternative history. Weak subjectivity addresses this by requiring clients to obtain a recent trusted checkpoint within a defined period. This is a social/bootstrap assumption distinct from short-range BFT safety.

Slashing Safety and Validator Key Operations

Slashing penalizes objectively provable validator behavior such as signing incompatible blocks. It strengthens incentives only when keys, domains, evidence, and appeal/recovery rules are correct. Operational mistakes can trigger the same signature evidence as malicious behavior.

Signing domain

Every consensus signature binds:

```
SigningDomain {
  chain_id,
  genesis_or_fork_id,
  protocol_version,
  epoch,
  view_or_slot,
  message_type,
```

```

    payload_hash
  }

```

A vote, proposal, timeout, and aggregate have distinct message types. Reusing a signature across testnet/mainnet, old/new forks, or proposal/vote contexts must fail.

Slashing protection database

Before signing, a validator records enough history to reject a double vote, surround vote, conflicting proposal, or other prohibited pair under protocol rules. Persist the record before releasing the signature.

A remote signer must enforce protection itself; relying only on the validator client allows two clients to request conflicts. The database needs atomic writes, authenticated backups, versioning, and a merge procedure when migrating hosts.

Active-active danger

Running two validators with one key for availability can cause equivocation during network partitions. A load balancer that sends requests to both is not safe redundancy.

Use active-passive fencing, one remote signer, consensus-aware replicas, or a protocol that explicitly supports redundant shares. Promotion verifies that the old signer cannot continue and imports the latest protection state.

Key hierarchy

Separate withdrawal, governance, consensus-signing, fee-recipient, and operator API keys. Compromise has different effects. A consensus key may equivocate without being able to withdraw stake; a withdrawal key may steal funds without voting.

Keep online signing keys in hardened signers or hardware appropriate to latency needs. Backup and rotate under a documented ceremony. Never test recovery for the first time during an outage.

Threshold signing

A threshold validator splits signing authority among members. A signature requires t shares, reducing one-device compromise risk. It adds network latency, share availability, distributed key generation, and membership operations.

The threshold group represents one protocol validator, not t independent consensus votes. Slashing protection must be consistent across share signers so two subsets cannot sign conflicts.

If $t=3$ of 5, any three shares sign. Two disjoint groups of three cannot exist, but groups $\{A, B, C\}$ and $\{C, D, E\}$ overlap only at C. If C signs both because its local state is inconsistent, conflicting signatures can form. Shared or cryptographically enforced signing state remains necessary.

Evidence lifecycle

Slashing evidence includes both signed messages, signer identity, domain, and proof that they violate a rule. Nodes verify evidence before gossip and bound storage to the accountable window.

Duplicate submission must not slash twice. Evidence from expired accountability windows, old forks, or wrong validator sets is rejected. Retain data long enough for network delays and censorship recovery.

False positives and software faults

A slashing condition should be objective from signed bytes. Monitoring can be wrong, but the on-chain verifier must not slash from an unauthenticated alert.

A common client bug can make many validators sign invalid or conflicting messages. The protocol may still slash according to rules, but governance intervention changes economic expectations. Document extraordinary authority rather than assuming it will or will not act.

Validator migration

To move a validator:

1. stop and fence the old signer;
2. export and verify slashing-protection state;
3. confirm the last signed slot/view through independent logs;
4. import atomically on the new signer;
5. test a no-sign dry run and chain/domain configuration;
6. enable one active signing path;
7. monitor duplicate-key and stale-view requests.

If the old host cannot be proven off, wait through a safe inactivity window or rotate according to protocol. Missed rewards are cheaper than equivocation.

Clock and rollback

A restored virtual-machine snapshot can roll back the protection database while the chain advances. Exclude active signers from generic snapshot rollback, or make the signer consult a monotonic external store.

Wrong clocks can request signatures for stale or future slots. The signing domain prevents cross-slot replay, but repeated bad requests can exhaust resources or miss duties. Monitor time drift through independent sources.

Worked failure domain

An operator runs 100 validator keys on two hosts, each with a copy of all keys. It appears redundant, but one software fault affects 100 keys and active-active failover can double-sign all of them.

Splitting keys 50/50 lowers the immediate correlated set; using distinct clients, signers, regions, and staged upgrades lowers it further. Redundancy should reduce common-mode loss, not multiply signing authority.

Disaster drill

Test signer outage, database corruption, restore from backup, failed fencing, network partition during promotion, wrong chain ID, epoch upgrade, threshold-member loss, and evidence submission.

Assert no conflicting signatures, bounded missed duties, complete evidence verification, one penalty per violation, recoverable protection state, and separation of withdrawal authority.

Slashing supports consensus security when signing is treated as an irreversible state transition. Key custody, persistence, fencing, migration, and evidence handling are protocol operations, not generic server administration.

Slashing and Equivocation Evidence

A validator equivocates by signing conflicting messages that protocol rules prohibit, such as two blocks at one height or incompatible votes in one view. Slashing evidence contains both signatures and enough context to verify conflict.

Evidence must be objective and compact. A timeout is not proof of malice because networks fail. Conflicting signed votes are. Penalties can remove stake, remove future rewards, or eject validators. Excessive correlated slashing can threaten network recovery, so protocols distinguish accidental downtime from safety violations.

Accountability requires retaining signer identity. A threshold signature without attribution proves quorum but may not reveal which members equivocated. Systems may keep individual votes off-chain, publish signer bitmaps, or use aggregatable signatures preserving evidence.

Fork Choice and Finality Gadgets

Some systems separate a fork-choice rule from finality. Fork choice selects the head validators should build on now; a finality gadget periodically certifies checkpoints that should never revert under the fault bound.

During temporary disagreement, honest validators can see different heads while agreeing on the latest finalized checkpoint. Applications choose confirmation policy based on risk. A low-value payment may accept head inclusion; a bridge waits for checkpoint finality.

The interaction matters. Votes used for finality may also influence fork choice, and network delay can cause validators to build on stale heads. Specifications must define tie-breaking, latest-message handling, justified checkpoints, and behavior when finality stalls.

Consensus Safety Testing

A model checker explores small validator sets and message schedules, looking for two conflicting commits. Property-based tests generate delays, duplicates, reorderings, restarts, and Byzantine messages. Network simulations scale to realistic committees and geography.

Critical scenarios include:

- leader equivocates across network partitions;
- replicas restart after voting but before persisting state;
- old votes arrive after a view change;
- epoch transition overlaps a timeout;
- validator weight changes near a quorum boundary;
- clock skew triggers premature timeout;
- malformed aggregate signatures stress verification;
- a minority floods valid but useless certificates.

Testing cannot replace proof, and proof cannot replace implementation testing. The model, specification, and code must encode the same protocol.

Consensus Network Partitions and Recovery

A network partition prevents groups of honest validators from communicating reliably. Consensus must decide whether any group can continue and how branches reconcile when connectivity returns.

Quorum reachability

In a BFT protocol requiring more than two-thirds voting weight, a partition with 70 percent on one side and 30 percent on the other can let the larger

side progress while the smaller cannot form a certificate. A 50/50 partition halts both sides, preserving safety if honest replicas do not violate locking rules.

Validator count is insufficient when weights differ. Compute reachable weight from the authenticated epoch set and avoid counting duplicate signers or replicas of one key.

Partition or slow network?

A replica observes missing or delayed messages, not a labeled partition. Pace-maker timeouts advance views, but rapid view changes can add load to an already congested network.

Expose peer reachability, vote weight observed, message latency, timeout certificate formation, highest QC, and block-body availability. Operators should not manually force a leader or lower quorum because progress is slow.

Asymmetric partitions

Communication may work from A to B but not B to A, or small messages may pass while large block bodies fail. Votes can form for data some replicas cannot retrieve.

Test direction, payload size, and protocol phase separately. Availability certificates should mean signers actually possess retrievable data under the stated rule, not merely that they saw a header.

Healing and view convergence

When connectivity returns, replicas exchange highest certificates and locked state. The pacemaker brings them to a common higher view, and the next valid leader extends the safe parent selected by protocol rules.

Do not choose the branch with the most transactions or the operator's preferred payments. Certificate and lock rules determine safety. Transactions from abandoned uncommitted proposals may return to the mempool after nonce and state revalidation.

Persistence

A replica persists its current epoch, last vote by view/height, highest QC, lock, and committed boundary before sending messages whose replay could violate safety. After restart, it reloads this state before voting.

Disk failure can make a node "forget" a vote and sign a conflict. Remote signers need monotonic slashing protection or consensus-state fencing. Two active replicas sharing one validator key can equivocate during a partition unless only one has signing authority.

Worked weighted partition

Suppose validator weights are:

A 28, B 24, C 20, D 16, E 12; total $W = 100$

A certificate needs at least 67 under a $\text{floor}(2W/3)+1$ rule. If A, B, and E can communicate, they have 64 and cannot progress. A, C, D, and E have 76 and can.

If B's operator also controls E, they are separate protocol weights but one operational failure domain. Consensus arithmetic and decentralization analysis answer different questions.

Reconfiguration during a partition

Do not activate a validator-set change independently on two sides. The new set is authenticated by a finalized or committed boundary under the old set, and every certificate is checked against the set active for its domain.

A partition across the boundary may leave some nodes unaware of the new epoch. Their old-epoch votes must not count in the new epoch. Handoff rules may require overlap or a joint certificate.

Client and implementation divergence

What looks like a network partition can be a deterministic client split: implementations reject each other's blocks. Network dashboards show healthy connections while votes divide by client.

Compare validation errors, state roots, protocol versions, and payload hashes. Preserve the offending input. Restarting peers or adding bandwidth will not repair divergent execution.

Recovery procedure

1. identify last committed/finalized block agreed across independent nodes;
2. preserve votes, timeout messages, proposals, and network observations;
3. restore connectivity without changing quorum or lock rules;
4. let protocol view synchronization choose the safe parent;
5. verify abandoned transactions before requeue;
6. reconcile proposer rewards, slashing evidence, and user status;
7. test catch-up from the minority side and a fresh node.

If conflicting commits exist, ordinary recovery assumptions have failed. Halt value release and treat the event as a safety incident; do not call one branch canonical without accountable evidence and the protocol's extraordinary governance process.

Partition test matrix

Run 70/30, 50/50, rotating minorities, one-way links, high loss, delayed votes, header-only connectivity, body withholding, signer restart, duplicated signer, epoch transition, and mixed-client divergence.

Measure committed throughput, finality latency, timeout/view rate, bandwidth amplification, fork depth, catch-up time, and transaction replay. Assert no conflicting commits under the modeled fault bound and eventual progress once the network and honest-leader assumptions recover.

Consensus scales responsibly when recovery uses the same certificates and locks as normal operation. A partition is not permission to replace protocol safety with administrator judgment.

Consensus Operations

Operators monitor view duration, proposal delay, vote arrival distribution, missed leaders, QC formation time, finality lag, peer diversity, and clock offset. A rise in view changes may indicate a faulty leader, regional network problem, overloaded verification, or timeout too close to normal p99 delay.

Incident response should preserve signed messages and timing evidence. Restarting every validator simultaneously can destroy liveness or forensic context. Staged recovery keeps enough replicas online and avoids violating lock persistence.

Capacity planning includes signature verification, block execution, data propagation, and state commitment. Increasing committee size without network and verification headroom can lower security in practice by causing honest validators to miss deadlines.

Worked Protocol Trace: Four Replicas and One Fault

Consider four replicas A, B, C, and D, with at most one Byzantine fault. A quorum contains three votes. Any two quorums of three intersect in at least two replicas; because at most one is Byzantine, the intersection contains at least one honest replica. Locking rules use that honest overlap to prevent incompatible commits.

Normal path

Assume A leads view 12 and proposes block x extending the highest known quorum certificate. Each replica checks the proposal's parent, view, payload, state-transition inputs, and signature domain before voting.

```
view 12
A -> {B,C,D}: PROPOSE( $x$ , parent_qc)
```



```
{A,B,C} -> A: VOTE(X, 12)
A: aggregate three votes into QC(X, 12)
```

A quorum certificate proves that three replicas voted for x ; it does not by itself mean a client should treat x as committed. In chained HotStuff, later certified descendants satisfy the protocol's commit rule. Pipelining allows a new proposal to carry $QC(x, 12)$ while votes for the next block are collected.

Before replica B sends its vote, it persists the safety state required by the protocol. If it crashes after sending but before recording its lock, it might restart and vote for a conflicting branch. Write-ahead persistence therefore belongs on the safety path, not as optional operational logging.

Faulty leader and timeout

Now assume A is faulty in view 13 and sends no valid proposal. Replicas' pacemakers expire. They sign timeout messages containing their highest known QC and send them to the next leader, B.

```
view 13
A: silent or invalid proposal
{B,C,D}: TIMEOUT(13, highest_qc)
B: form timeout certificate

view 14
B -> {A,C,D}: PROPOSE(Y, highest_safe_qc, timeout_certificate)
{B,C,D} -> B: VOTE(Y, 14)
B: form QC(Y, 14)
```

The timeout certificate proves enough replicas abandoned view 13. The highest-QC rule ensures the new leader extends a branch compatible with honest replicas' locks. A leader cannot choose an older convenient parent merely because it has three fresh participants.

Timeout values affect liveness, not the underlying safety proof, provided replicas enforce voting and locking rules. Too short a timeout creates needless view changes under ordinary jitter. Too long a timeout makes every failed leader expensive. Implementations often increase timeout after repeated failures and reduce it after stable progress.

Equivocation

Suppose a Byzantine leader sends Y to B and Z to C in the same view. The signed proposals are equivocation evidence. Honest replicas still apply their voting rules. With four replicas, the faulty leader cannot form quorums of three for both conflicting blocks unless at least one honest replica votes incompatibly.

This is why a signature check alone is insufficient. The implementation must index votes by chain, epoch, view, message type, and block identity, reject a second prohibited vote, and preserve evidence. An aggregate signature should retain a signer bitmap or underlying votes needed for accountability.

Client finality

A client observes at least three distinct notions:

1. **proposal reception** - one leader advertised a block;
2. **certification** - a quorum voted for it;
3. **commit/finality** - the protocol's chained-certificate rule makes it irreversible within the fault model.

An application must name which event it uses. Showing a proposal as a provisional status is reasonable. Releasing bridged assets against it is not. Even a committed block is conditional on validator-set authentication and the client's weak-subjectivity checkpoint when membership changes.

Trace assertions

A deterministic test harness for this trace should assert:

- no honest replica votes twice in one prohibited context;
- every accepted QC contains the required distinct weight;
- a new-view proposal extends the safe parent selected from timeout evidence;
- persisted lock state survives a crash at every send boundary;
- two honest replicas never commit conflicting blocks;
- after the network stabilizes and an honest leader is selected, progress resumes;
- metrics identify the timed-out view, missing leader, highest QC, and time to recovery.

Run the same trace with duplicate, reordered, and delayed messages. Then vary the fault: an invalid payload, a malformed aggregate, conflicting epoch numbers, stale QCs, and a validator-weight change at the boundary. The state machine should reject each invalid transition for a specific reason without discarding evidence needed for diagnosis.

Quorum Arithmetic With Weighted Validators

Replica counts are only a special case. If validators have weights summing to w , a certificate commonly requires weight greater than $2w/3$, under an assumption that Byzantine weight is less than $w/3$.

Let two certificates each have weight greater than $2W/3$. Their intersection has weight greater than:

$$2W/3 + 2W/3 - W = W/3$$

If Byzantine weight is less than $w/3$, the overlap includes honest weight. Boundary comparisons matter. An implementation using integer weights must define whether a threshold is $\text{floor}(2W/3)+1$ or an equivalent exact rule. Rounding one path differently in vote collection and certificate verification can split clients at the quorum boundary.

Weight changes should activate at an authenticated epoch boundary. Calculating one certificate against old weights and another against new weights can invalidate the intersection argument. Test totals not divisible by three, maximum integer weights, duplicate signers, zero-weight entries, and set transitions immediately before and after a timeout.

Conclusion

Consensus scaling is the engineering of messages, signatures, leaders, committees, and timing assumptions. HotStuff makes the normal path linear and pipeline-friendly. Sync HotStuff shows what becomes possible under synchrony. DAG-based systems separate data dissemination from ordering.

No protocol is unconditionally "faster consensus." Each result depends on its network model, fault threshold, committee construction, block payload, and recovery behavior. The final chapter looks at how these techniques may converge in future blockchain architecture.

References

1. Yin, Maofan, et al. "HotStuff: BFT Consensus in the Lens of Blockchain." <https://arxiv.org/abs/1803.05069>. ↵
2. Abraham, Ittai, et al. "Sync HotStuff: Simple and Practical Synchronous State Machine Replication." <https://arxiv.org/abs/2005.13432>. ↵
3. Danezis, George, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. "Narwhal and Tusk." <https://arxiv.org/abs/2105.11827>. ↵ ↵2
4. Aptos Documentation. "Life of a Transaction." <https://aptos.dev/network/blockchain/blockchain-deep-dive>. ↵ ↵2
5. Diem Documentation. "Consensus crate." <https://diem.github.io/diem/consensus/index.html>. ↵
6. Sui Documentation. "Life of a Transaction." <https://docs.sui.io/concepts/sui-architecture/transaction-lifecycle>. ↵ ↵2
7. Danezis, George, et al. "Mysticeti: Reaching the Latency Limits with Uncertified DAGs." <https://docs.sui.io/paper/mysticeti.pdf>. ↵ ↵2

8. CometBFT Documentation. "Byzantine Consensus Algorithm." <https://docs.cometbft.com/v0.37/spec/consensus/consensus>. ↩ ↩2
9. CometBFT Documentation. "ABCI 2.0." <https://docs.cometbft.com/main/spec/abci/>. ↩

Chapter 11: Future Directions

Introduction

Blockchain scalability is moving from one-chain throughput contests toward specialized stacks. Execution, proving, sequencing, data availability, settlement, and interoperability are becoming separate services that can improve independently.

The next bottleneck is therefore not one number such as TPS. It is coordination: making many fast execution environments feel like one coherent, usable system with explicit end-to-end security assumptions.

How to Read This Future-Facing Chapter

This chapter combines deployed ideas, active prototypes, and research proposals. A basic reader should not treat every named mechanism as a product that is already available or as one inevitable roadmap.

For each idea, separate five questions:

1. **What problem is it trying to solve?** Examples include proof delay, fragmented liquidity, censorship, or validator storage.
2. **What is the mechanism?** Name the messages, proofs, committees, or markets that change the outcome.
3. **Which assumption moved?** Faster service may require a new operator, timing bound, hardware class, or governance key.
4. **What evidence exists?** A paper, prototype, testnet, audited deployment, and long-running permissionless production system provide different confidence.
5. **How does failure appear to a user?** A wallet needs states such as pending, preconfirmed, proven, settled, refundable, or blocked.

A glossary for the emerging stack

A **real-time proof** is a validity proof produced quickly enough to fit a protocol's next acceptance deadline. "Real time" is relative to a slot or batch interval, not literally instantaneous.

Proof aggregation combines evidence for many computations so a verifier checks one smaller proof. **Recursion** means one proof verifies other proofs

inside its proven computation. This is like checking a signed summary whose calculation includes checking the signed summaries beneath it.

A **shared sequencer** orders transactions for several rollups. It may improve cross-rollup coordination, but the rollups still need settlement, DA, and rules for sequencer failure. A **based rollup** derives ordering from base-layer proposers rather than operating an entirely separate sequencer.

A **preconfirmation** is a signed promise about future inclusion or order before full consensus finality. It is useful only when the signer, promise domain, expiry, violation evidence, and penalty are explicit.

An **intent** states an authorized outcome, such as receiving at least 100 units of one asset for no more than 50 units of another. A **solver** chooses a route that satisfies it. The signature should bind recipient, assets, limits, expiry, fees, and settlement rules; the solver receives path freedom, not permission to change the outcome.

Proposer-builder separation (PBS) separates the consensus participant proposing a block from specialized builders assembling profitable payloads. A **relay** may check and forward blinded bids between them. The design must handle withholding, censorship, relay failure, and builder concentration.

An **encrypted mempool** hides transaction contents until an ordering point. **Threshold encryption** splits decryption power among members so a threshold must cooperate. It reduces content-based front-running only if the committee cannot decrypt early and still releases shares reliably.

Stateless validation lets a validator verify using transaction data plus witnesses rather than a complete state database. **State expiry** removes inactive state from the active set while preserving a commitment and revival path. Someone still needs to retain or serve the expired values.

Chain abstraction hides some chain and bridge choices from the user. It is an interface goal, not a security model: software still selects routes, assets, finality, and recovery on the user's behalf.

Read maturity labels literally

- **Paper:** mechanism and argument, possibly without production code.
- **Prototype:** code demonstrates feasibility under limited conditions.
- **Testnet:** multiple parties can test behavior without normal production value.
- **Limited production:** real users or value, often with restricted operators or caps.
- **Permissionless production:** open participation and real value, still subject to bugs and governance.

A mechanism can be mature in cryptography but immature in operations. The detailed sections keep formalism because implementation details decide safety; this guide supplies the intuition and vocabulary needed to follow them.

Real-Time Validity Proving

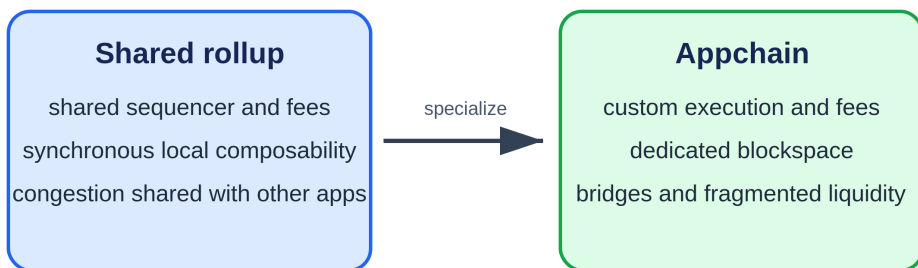
Validity proofs once required long proving times and specialized circuits. Faster provers, hardware acceleration, recursive aggregation, and general-purpose zkVMs are reducing that delay.

Real-time proving would let a validity proof arrive within one block interval. This could shorten finality, simplify bridges, and allow even base-layer execution to be re-verified succinctly. The remaining constraints are prover cost, memory bandwidth, circuit correctness, and the centralization risk of specialized hardware.

A future proof market may separate block production from proving. Multiple provers compete to generate a proof, while the chain verifies only the winning result. This needs mechanisms for redundancy and censorship resistance so one prover outage does not stop the chain.

Shared Sequencing and Cross-Rollup Composability

From shared blockspace to an application chain



The gain is control. The cost is another security and interoperability boundary.

Figure 11.1: The future appchain trade-off: dedicated execution and customization versus bridges and fragmented liquidity. Original figure for this book, based on SC6019 Lecture 06.

Independent sequencers fragment ordering. A transaction cannot easily execute atomically across rollups, and cross-rollup messages wait for settlement.

Shared sequencers aim to order transactions for several rollups. If the same sequencer sees both sides, it can offer stronger inclusion guarantees and forms of atomic execution. Based rollups go further by deriving ordering from the base-layer proposer.

The benefit is interoperability and reduced sequencer trust. The risk is moving concentration into a shared service. A shared sequencer must define leader selection, failure recovery, fees, and what a rollup can do when the service censors it.

Proof Aggregation

Instead of verifying every rollup proof separately, an aggregator can recursively prove that many proofs were valid. Layer 1 then verifies one proof.

Aggregation reduces settlement cost and lets small rollups share proving economies. It may also support canonical bridges between rollups that settle through the same proof system. The design must preserve data availability and identify which state roots were actually covered.

More Data Through Sampling

EIP-4844 created blob space for rollups, and PeerDAS activated on Ethereum mainnet with Fusaka on December 3, 2025.^{1 2} PeerDAS uses erasure-coded cells, custody assignments, peer requests, and sampling so an individual node need not download every blob. Further blob-capacity increases and full danksharding remain evolving protocol work; deployment reviews should separate the live PeerDAS mechanism from later roadmap stages.

Dedicated data layers will also compete on throughput, sampling security, retention, and integration. The market may support different tiers: tightly integrated base-layer blobs for high-value settlement and cheaper external availability for applications willing to accept another consensus assumption.

Statelessness and State Expiry

Data availability concerns recent block data, while state growth concerns the long-lived database of accounts and contracts. If state grows forever, running a validator becomes increasingly expensive.

Stateless validation moves toward blocks that include witnesses proving the state values they touch. Verkle trees were proposed to make these witnesses smaller. State expiry or rent would remove or charge for inactive state.

These ideas reduce validator storage but move responsibility elsewhere. Users or archival services may need to retain old state and provide proofs when it becomes active again.³

Parallel and Specialized Virtual Machines

Block-STM, object-centric execution, and declared access lists show how multi-core hardware can be used. Future VMs may make state dependencies explicit and provide programming tools that warn about hot-state bottlenecks.

Application-specific rollups will specialize further. An exchange may build native order-book primitives, a game may use an object model, and an AI-agent network may optimize frequent micropayments and verifiable computation. The trade-off is portability: specialized execution makes applications faster but less interchangeable.

Account Abstraction, Bundlers, and Paymasters

Account abstraction lets contract logic define how a user's operation is authorized and paid instead of requiring every account to follow one fixed signature and fee model. It can support recovery keys, multisignature policy, session permissions, sponsored fees, and batching. It also introduces new mempool, simulation, and sponsor failure paths.

User operation

A user signs a structured request rather than a native transaction:

```
UserOperation {
    chain_id,
    account,
    nonce,
    call_data,
    gas_limits,
    fee_limits,
    authorization_data,
    paymaster_data,
    validity_window,
    entry_point,
    version
}
```

The signature or authorization binds every field that can change destination, value, fee, replay scope, or code path. A wallet displays the effective calls, not only an opaque hash.

A contract account validates the operation under its own policy. The policy may require one key for small payments, several keys for large payments, a guardian delay for recovery, or a session key restricted to one application and amount.

Entry point and bundler

A **bundler** collects user operations, simulates them, and submits a native transaction to an **entry-point contract**. The entry point validates accounts, charges fees, executes calls, and reconciles payments.

The bundler is replaceable for correctness when another bundler can include the same valid operation. It still controls short-term inclusion and pays native transaction cost upfront. A wallet needs several submission paths or a direct fallback.

Bundle execution must isolate failures. One invalid user operation should not revert every unrelated valid operation unless the protocol explicitly prices and handles that risk.

Nonces and parallel lanes

A single increasing nonce serializes all operations from one account. Structured nonces can provide independent lanes, such as one for routine payments and another for application sessions. The account contract must prevent reuse within each lane and define cancellation.

Parallel nonces improve throughput but complicate user expectations. Canceling nonce 10 in one lane should not cancel nonce 10 elsewhere. Wallets must bind lane, sequence, and validity window in the signature.

Simulation and state changes

Bundlers simulate validation before inclusion, but chain state can change before execution. Another operation may consume a nonce, revoke a key, spend a deposit, or change a paymaster policy.

Consensus execution remains authoritative. Simulation failure should produce a specific reason and safe retry; success is not a guarantee of later inclusion. Bundlers protect themselves with conservative admission, reputation, deposits, and resource limits.

Validation code must be bounded and deterministic. If it reads volatile or unrestricted state, attackers can create operations that simulate successfully and fail on-chain, forcing bundlers to pay gas.

Paymasters

A **paymaster** sponsors fees or accepts payment in another asset. It validates a policy and deposits native fee assets with the entry point.

Sponsorship data binds operation, chain, maximum charge, expiry, and replay identifier. An application should not sign an unlimited promise that an attacker can attach to arbitrary calls.

A paymaster can fail because its deposit is empty, oracle is stale, policy changed, or post-execution accounting reverts. The user's operation should report that sponsorship failed rather than appearing as an unexplained wallet failure.

Worked sponsorship budget

Suppose a paymaster deposits 10 ETH and caps one operation at 0.002 ETH. Ignoring refill, the theoretical maximum is:

$$10 / 0.002 = 5,000 \text{ operations}$$

At a 70 percent planning limit, admit at most 3,500 maximum-cost outstanding operations before requiring refill or lower reservations. Otherwise many individually valid promises can compete for one exhausted deposit.

Reserve against maximum authorized cost, then reconcile actual cost and release the remainder. Concurrent bundlers must see one authoritative reservation state or can overbook the deposit.

Session keys

A session key gives limited authority for a period. Its policy includes allowed contracts and methods, per-call and cumulative value, fee ceiling, chain, expiry, and revocation.

Checking only the first call is unsafe when that call delegates, batches, or triggers token approvals. Validate the effective call graph or restrict operations to contracts with understood behavior. A token approval can grant value beyond the immediate zero-value call.

Revocation must be available through a stronger key and take effect under a clear finality rule. An operation already in a public mempool may race revocation; policy defines which canonical order wins.

Recovery and upgrades

Social recovery replaces lost authorization after a delay and guardian threshold. Guardians should not gain immediate spending power. A pending recovery must be visible, cancelable by the current owner when safe, and domain-separated from other accounts and chains.

Contract-account upgrades can replace every validation rule. Bind upgrade authority, delay, implementation hash, and escape behavior. A wallet calling an account "multisignature" is misleading if one upgrade key can remove the threshold instantly.

Mempool fragmentation

Different account and paymaster rules make validation heterogeneous. Bundlers may support only common implementations, creating a de facto permission boundary. Publish compatibility, rejection reasons, and inclusion latency by account type.

Private bundlers improve UX but can censor or observe operations. A shared mempool needs anti-spam rules that do not require executing unbounded custom validation for free.

Production tests

Test duplicate and parallel nonces, key revocation races, malformed authorization, bundle partial failure, paymaster depletion, stale exchange rates, simulation/execution divergence, session-key limits, recovery cancellation, entry-point upgrade, bundler outage, and settlement reorganization.

Reconcile user maximum fee, sponsor reservation, actual native gas, token charge, refund, and bundler compensation for every path.

Account abstraction improves usability when flexible authorization remains explicit, bounded, recoverable, and portable across bundlers. It becomes a scalability tool when batching and sponsorship reduce friction without turning one entry point, paymaster, or wallet service into hidden custody.

Chain Abstraction

Users should not need to know which chain holds gas, which bridge route is safe, or when a message changes finality domains. Chain abstraction combines smart accounts, intent systems, solvers, and cross-chain messaging to present one interface.

The security challenge is making hidden routing legible. A solver can improve price and speed but introduces execution and censorship questions. Interfaces should show which chain settles a transaction, when it is final, and which recovery path exists.

Decentralizing the Full Stack

Many scaling systems launch with centralized sequencers, provers, upgrade keys, or data committees. This can be a practical deployment stage, but decentralization should be measured rather than promised.

Useful milestones include:

- permissionless proof or fault-proof participation;
- force inclusion and forced withdrawal;
- multiple independent sequencers;
- delayed and transparent upgrades;
- distributed data custody;
- open-source clients from more than one team.

Decentralizing one component can expose another. A decentralized sequencer does not fix a multisig-controlled bridge, and a permissionless prover does not fix unavailable data.

MEV and Proposer-Builder Separation

Scaling creates more blockspace but does not remove maximum extractable value. Faster sequencing and cross-domain transactions create new opportunities for reordering and latency advantages.

Proposer-builder separation divides block construction from consensus proposal. Specialized builders compete to assemble valuable blocks while validators choose among bids. Protocol designs must prevent builder concentration, censorship, and timing games. Encrypted mempools and inclusion lists are possible counterweights.

Proposer-Builder Separation in Practice

Proposer-builder separation (PBS) divides block construction from consensus proposal. Specialized builders search transaction orderings and assemble payloads; a proposer chooses among bids and publishes the winning block. This can improve block construction and isolate some MEV work, but it creates a new market and relay path whose failure behavior must be designed.

A minimal blinded flow is:

1. users submit transactions to public or private orderflow endpoints;
2. builders simulate transactions and construct candidate payloads;
3. a builder commits to a payload and bids for the right to have it proposed;
4. relays validate the payload and show the proposer a header and bid without revealing the full body;
5. the proposer signs the selected header;
6. the winning payload is revealed and propagated;
7. consensus validates that the revealed payload matches the signed commitment.

The proposer should not sign two headers for one slot. The builder should not receive proposer commitment and then withhold the body. The relay should not be able to substitute a different payload after the signature.

Bid object

```
BuilderBid {  
  chain_id,  
  slot,  
  parent_hash,  
  payload_commitment,  
  fee_recipient,  
  gas_or_resource_limits,  
  builder_pubkey,  
  bid_value,  
  protocol_version,  
  signature  
}
```

Bind every bid to chain, slot, parent, payload, fee recipient, limits, and version. A high bid for a stale parent is not executable value. Compare bids only after validating their common domain and the proposer's policy constraints.

Relay validation

A relay commonly simulates payload validity before forwarding a bid. Validation should include parent availability, state transition, transaction signatures, resource limits, fee accounting, withdrawals or system operations, and commitment consistency.

Simulation introduces latency and denial-of-service exposure. Builders can send expensive invalid blocks to consume relay capacity. Relays need admission control, reputation or bonds, parallel validation, strict deadlines, and specific error metrics. They must not weaken validation near the slot deadline merely to show more bids.

Relays also see commercially sensitive bids and payloads. Log access, operator conflicts, data retention, and information leakage matter. Multiple relay URLs do not create independence if they share one operator, codebase, cloud, or upstream builder set.

Builder payment and value

The bid value must be enforceable in the payload. If the builder promises 2 ETH but constructs a payment that can fail, the proposer receives less than the advertised bid. Validation should compute the effective balance change under protocol rules rather than trust an off-chain number.

A rational proposer compares expected value after failure risk, not only face value:

expected value = bid $\times$ probability of timely valid reveal - failure penalty

Suppose builder A bids 2.0 ETH with a 99.95 percent timely-reveal rate and builder B bids 1.99 ETH with 99.999 percent reliability. Ignoring other costs:

A: $2.0 \times 0.9995 = 1.9990$ ETH
 B: $1.99 \times 0.99999 \approx 1.98998$ ETH

A still has greater expected value, but the calculation omits the consensus and reputational cost of a missed slot. A proposer may rationally apply a larger failure penalty than lost payment alone.

Withholding and fallback

After a proposer signs a blinded header, a builder or relay may fail to reveal the body. The proposer cannot safely construct another body for the same signed commitment. The result may be a missed slot.

A fallback policy must operate before signing: query independent relays, maintain a locally built payload, reject bids after a deadline that leaves too little reveal time, and choose local production when no bid clears a risk-adjusted threshold. Once a header is signed, fallback cannot change its committed body.

Test relay disconnection before bid selection, during signature submission, and after header commitment. Measure missed-slot rate, reveal latency percentiles, time remaining for propagation, and whether local fallback remains valid on the selected parent.

Censorship

PBS can concentrate transaction inclusion decisions among builders even while proposers remain numerous. Measure the share of slots, bids, and winning value by builder and relay; exclusive orderflow; transaction inclusion delay; and whether compliant transactions excluded by leading builders enter through local construction or an inclusion mechanism.

An inclusion list can require the final payload to contain eligible transactions named by the proposer, subject to validity and resource rules. The specification must prevent a proposer from listing invalid or deliberately conflicting transactions and prevent a builder from claiming the block was full when it strategically displaced listed items.

Censorship resistance depends on transaction visibility. Private orderflow can improve user execution but starve public builders and monitoring. Applications should disclose who receives orders, what those parties can do with them, how long exclusivity lasts, and what fallback returns an order to a public path.

Builder concentration

Builders benefit from low-latency orderflow, proprietary search, capital, simulation infrastructure, and cross-domain inventory. Winning-builder count can therefore fall even when anyone is technically allowed to connect.

Report concentration by blocks and value, not registration count. Examine common ownership, financing, colocated infrastructure, exclusive orderflow, and relay preference. Simulate the sudden loss of the largest builder and the largest relay set; the chain should continue producing valid blocks, even if revenue falls.

Timing budget

For a 12-second slot, allocate an explicit budget:

bid collection	7.5 s
selection and signing	0.3 s
payload reveal	0.5 s
validation and propagation	2.2 s
safety margin	1.5 s

	12.0 s

These numbers are illustrative. Measure geographic p99s and correlated delays. Extending bid collection may raise revenue while reducing propagation margin and increasing fork risk. Optimize the full consensus outcome rather than auction revenue in isolation.

Production assertions

Before enabling a PBS path, assert that:

- signed headers bind every consensus-critical payload field;
- effective payment equals the validated bid under success and failure cases;
- stale-parent and wrong-slot bids are rejected;
- no relay can make invalid payloads valid;
- the proposer never signs conflicting headers;
- late reveal produces a measured missed-slot outcome, not an unsafe fallback;

- local construction remains tested and ready;
- builder, relay, and orderflow concentration are observable;
- censorship alarms use inclusion delays and eligible transaction tests;
- upgrades preserve domain separation across bid and payload versions.

PBS should be judged as a consensus-adjacent market with safety, liveness, timing, privacy, and competition requirements. Higher bids are useful only when the block is valid, arrives in time, propagates safely, and does not make transaction inclusion depend on a small hidden supply chain.

Named Implementation Atlas: What Runs, What Is Piloted, What Is Proposed

The mechanisms above are easier to judge when attached to a named system. The labels below describe the implementation stage documented in September 2026, not an assurance that a deployment is decentralized, bug-free, or permanent. A production protocol can still depend on a concentrated service; a pilot can handle real transactions while retaining explicit limits; a research design can have working code without a settled operating model.

ERC-4337: account abstraction through an alternate mempool

Maturity label: production standard and production deployments. ERC-4337 implements account abstraction without changing Ethereum's consensus transaction format. Its official documentation defines a `UserOperation`, an alternate mempool, bundlers, the singleton-style `EntryPoint` contract, smart accounts, and optional paymasters.^{4 5} This is a named instance of the account, bundler, and sponsor mechanics developed earlier in the chapter.

Trace one sponsored operation. Maya's smart account signs a `UserOperation` that calls a game contract. The object binds the account address, nonce, call data, gas limits, fee caps, paymaster data, and signature. She sends it to a bundler rather than directly to Ethereum's native transaction mempool. The bundler simulates validation against the specified `EntryPoint`. The entry point calls the account's `validateUserOp`; when sponsorship is requested it also calls the paymaster's validation logic. The bundler groups Maya's operation with others and sends one native transaction invoking `handleOps`. The entry point validates each operation, executes its call, accounts for gas, and pays the bundler's beneficiary from an account or paymaster deposit.

The observable consequence is that Maya can transact without first holding the chain's native fee asset, while Ethereum still sees a normal transaction from the bundler. Her wallet should expose separate states: accepted by one bundler, included in a bundle, executed by `EntryPoint`, and successful at the application call. A simulation result is not inclusion. A bundler can refuse an otherwise valid operation, so portability across bundlers is part of liveness.

The failure path is concrete. If the paymaster's deposit is exhausted or its policy rejects the operation, sponsorship fails even if Maya's account signature is valid. If state changes after simulation, on-chain validation remains authoritative. A malformed operation can cost the bundler resources, which is why the standard limits validation behavior and uses staking or reputation rules for some entities. The account must explicitly trust the intended entry-point contract; a look-alike dispatcher must not gain authorization. Finally, a wallet upgrade key can replace Maya's validation policy. ERC-4337 makes programmable authorization possible, but the smart-account implementation decides whether recovery, session keys, and upgrades are safe.

MEV-Boost: proposer-builder separation through relays

Maturity label: production middleware on Ethereum. Flashbots describes MEV-Boost as open-source middleware run by validators to access a competitive block-building market. It is an implementation of proposer-builder separation outside Ethereum's consensus protocol, sometimes called proposer-builder separation through an external market.⁶

Trace one proposal slot. Searchers and users send transactions through public or private paths to builders. Builders assemble full execution payloads and include a payment to the validator's registered fee recipient. They submit payloads to relays. A relay validates a payload and returns a blinded header and bid rather than revealing the transactions immediately. The validator's MEV-Boost sidecar asks its configured relays for bids and forwards the selected header to the consensus client. The proposer signs the blinded block. After checking that signature, the winning relay returns the full payload, which the proposer publishes for Ethereum attestation.⁷

This division protects a builder's payload before the proposer commits, but it introduces timing and service dependencies. The proposer observes bid value, relay identity, response latency, payload delivery, and whether the full block arrived before the slot deadline. A local block-building path is the liveness fallback when MEV-Boost or its relays fail. The Flashbots risk documentation explicitly calls out liveness and local fallback, builder centralization, builder-relay collusion, malicious relays, and hidden MEV.⁸

Suppose the winning relay withholds the full payload after the proposer signs. The validator misses the slot: signing a different payload for the same slot would risk an equivocation violation. A safe local fallback must be selected before the proposer signs the blinded header, using deadlines and a circuit breaker that leave time to build and publish locally. Suppose instead one builder controls most profitable order flow. The blocks remain valid, but censorship and market power can concentrate. A relay can also advertise a fraudulent bid that MEV-Boost itself cannot fully verify from a blinded header. Monitoring and reputation can detect abuse, but they are not the same as

eliminating the relay assumption. MEV-Boost therefore shows both the value and the limit of a production PBS market: specialization can increase proposer revenue and builder competition while leaving relay trust and concentration for later protocol work.

Espresso: shared settlement and sequencing integration

Maturity label: production network with integration-dependent guarantees. Espresso's current documentation describes a proof-of-stake network that provides decentralized settlement and finality for blocks proposed by integrated chains and applications. Each application retains its own execution environment and ordering rules; Espresso does not execute those application transactions or prove that their state transition was correct.⁹ This is an important correction to the loose phrase "shared sequencer": shared ordering or settlement does not automatically validate application execution.

Trace an integrated rollup payment. The rollup receives Maya's transaction and executes it under its own virtual machine. It submits the resulting block or commitment through the Espresso integration. Espresso consensus orders and finalizes the submitted object. Another application can verify Espresso state when coordinating with the rollup, subject to the integration and verification path. The rollup still publishes or serves the data its users need and still applies its own proof or dispute rule for execution correctness.

The visible boundary is an Espresso finality certificate or commitment associated with the rollup block. An application can distinguish "accepted by the rollup operator" from "finalized by Espresso" and from "execution accepted at the rollup's settlement contract." Those are separate claims. If Espresso stalls, an integration needs an explicit fallback or the rollup's finality path stalls with it. If the rollup sequencer constructs an invalid state transition, Espresso can faithfully finalize the submitted block without detecting the application error because execution correctness remains outside Espresso's stated responsibility. If at least the threshold required to corrupt Espresso consensus is dishonest, ordering finality itself can fail. A deployment review must therefore inspect the exact integration, validator and staking state, fallback rules, DA choice, and how a target chain verifies Espresso commitments.

Across: intents with relay capital and optimistic settlement

Maturity label: production cross-chain intent protocol. Across supplies a named implementation of the chapter's intent and solver path. Its documented V3 lifecycle has three phases: initiation, fill, and settlement. A user deposits on an origin-chain SpokePool; a relayer advances its own funds on the destination; later, protocol settlement verifies the fill and repays the relayer.¹⁰

Trace Maya moving an asset from an origin chain to a destination. She calls `depositV3`, binding destination chain, output token, output amount, recipient, fill deadline, optional message, and any exclusivity fields. The origin `SpokePool` escrows her input and emits `V3FundsDeposited`. Relayers watch that event. A relayer that accepts the quote calls `fillV3Relay` on the destination `SpokePool` using its own inventory. The destination event records fulfillment and prevents another ordinary fill of the same intent. Maya can treat the destination fill as delivered without waiting for the relayer's reimbursement cycle.

Settlement happens later. A dataworker aggregates covered deposits and fills into a root bundle containing relayer refunds, token-rebalancing instructions, slow-fill data, and block ranges. It proposes that bundle to the Ethereum `HubPool` with a bond. During the challenge period, one honest monitor can dispute an invalid bundle. A dispute is then resolved by UMA's Data Verification Mechanism, whose token-holder vote determines the winning bond under the documented model.¹¹ The often-stated "one of N honest" assumption therefore covers detection and initiation of a dispute; final disputed settlement also depends on the DVM and its governance and economic assumptions. After an undisputed or successfully resolved bundle is accepted, relayers receive repayment and liquidity is rebalanced through the protocol's routes.

The architecture moves latency from the user to the relayer's balance sheet. Maya sees a deposit transaction, quoted deadline, destination fill event, and eventual settlement status; the relayer additionally tracks inventory, repayment chain, bundle coverage, and dispute state. If no relayer fills before the deadline, the system needs the documented slow or refund path rather than inventing a destination transfer. If a relayer fills the wrong recipient or too little output, the event does not satisfy Maya's signed intent. If a proposed root bundle repays a nonexistent fill, an honest verifier must dispute it during the optimistic window. If destination-chain finality reverts after a fill is observed, settlement rules must not blindly repay it. The protocol is fast because a relayer fronts value, not because cross-chain finality becomes atomic.

CoW Protocol gives a complementary, same-chain example. Users sign trade intents; solvers compete over an auction; the selected solution settles through the protocol's settlement contract.¹² Across emphasizes cross-chain delivery and later reimbursement, while CoW emphasizes batch auctions, coincidence of wants, and on-chain trade settlement. In both, the signature constrains the acceptable outcome and the solver chooses a path. Neither design gives a solver permission to alter recipient, limit price, fee cap, expiry, or replay scope.

Succinct Prover Network: a market for SP1 proofs

Maturity label: production prover service moving through a protocolized mainnet market. Succinct documents a prover network and migration path for SP1 applications, as well as a protocol architecture that connects proof requesters and provers through auctions and Ethereum settlement.^{13 14} The distinction matters: a hosted proving service can be production-ready before every auction, staking, and decentralized settlement component reaches its intended end state.

Trace one proof request. A rollup or application submits the program, inputs or input commitments, a maximum prover-gas bound, maximum fee, minimum prover stake, deadline, and verification key. Eligible provers bid in a proof contest. The assignment service chooses a bid under the market rule. The winning prover runs SP1, returns a proof before the deadline, and receives payment when fulfillment is accepted. The requester verifies the proof against the expected program and public inputs before using it to advance an application state.

The market separates **correctness** from **delivery**. A sound verifier rejects a fabricated proof regardless of which prover won. Staking and deadlines address liveness and economic performance: a prover that accepts work and misses the deadline can lose stake under the documented rules. The user-visible evidence includes request identifier, program and verification-key identity, assigned prover, deadline, proof hash, verification result, fee, and settlement state.

Suppose the cheapest winning prover crashes halfway through a deadline-sensitive rollup batch. The coordinator must reassign or the application misses its proof deadline. A second prover helps only if it can obtain the program and inputs and does not share the same software, cloud region, or hardware bottleneck. Suppose the off-chain auctioneer censors Maya's request. The on-chain proof may remain cryptographically sound, but the service has a liveness and access failure. Suppose the requester names the wrong verification key. The network can correctly prove the wrong program. These failures show why a prover market needs requester-side program binding, redundant capacity, transparent assignment, deadline enforcement, and a route to recover funds or reissue work.

Mechanism-to-Implementation Map

Mechanism in this chapter	Named implementation	September 2026 label	What the implementation proves - and what it does not
Account abstraction, bundlers, paymasters	ERC-4337	Production standard/deployments	EntryPoint enforces account and sponsor validation; a bundler receipt does not guarantee inclusion, and account upgrades still define authorization
Proposer-builder separation	Flashbots MEV-Boost	Production middleware	Relay-validated blinded bids connect builders and proposers; external relays and concentrated builders remain operational and censorship risks
Shared sequencing/settlement	Espresso	Production network, guarantee depends on integration	Consensus finalizes submitted application blocks; it does not execute or validate each application's state transition
Intents and solver delivery	Across; CoW Protocol	Production protocols	Signed limits constrain outcomes and contracts record fulfillment; relayers/solvers still control route choice and can fail to deliver
Decentralized prover market	Succinct Prover Network	Production service and protocolized mainnet market	Verifiers decide proof correctness; auctions, stake, redundancy, and deadlines decide whether a usable proof arrives

The maturity column is deliberately more specific than "live." Production says real systems and value use the mechanism. It does not erase the failure path. A reader evaluating a later release should revisit the linked official documentation, identify the deployed version and contracts, and update the label when participation, controls, or fallbacks change.

How to Judge Future Claims

New systems should be evaluated across the entire stack:

- 1. What workload produced the throughput result?
- 2. What hardware and validator count were used?
- 3. Where are execution and data stored?
- 4. How is invalid state rejected?
- 5. Can users force inclusion and exit?
- 6. Which keys can upgrade the system?
- 7. What finality does the quoted latency represent?
- 8. What happens during sequencer, prover, bridge, or DA failure?

A scalability result is meaningful only when its security and operating assumptions are stated beside it.

A Plausible End-State Architecture

A future wallet may accept an intent - "swap this asset and pay that merchant" - without asking which chain should execute it. Solvers compete to route the action. A shared sequencer reserves positions across two rollups. Each rollup executes in a parallel VM. Its prover produces a recursive proof. Blob data is dispersed through PeerDAS, and an aggregate proof settles on Ethereum. The wallet shows one confirmation while tracking several finality stages.

Every component exists in an early form. Composition is the hard part. If one rollup reverts, atomic settlement must unwind the other. If a solver disappears, the user needs a refund. If the shared sequencer censors the intent, each rollup needs independent inclusion. If data is available but proving stalls, another prover must take over.

Scalability comes from dividing work, but division creates interfaces. Future systems will be judged by whether those interfaces remain verifiable during failure.

Research Problems Still Open

Cross-rollup atomicity is difficult without a shared trust or timing domain. Decentralized sequencing must resist censorship without recreating global consensus latency. Proof systems need lower hardware barriers and stronger circuit assurance. Data sampling needs resilient peer-to-peer networking at larger scale. State expiry needs a usable way to revive old state. MEV mitigation must operate across several domains, not one mempool.

The field also has a measurement problem. Theoretical capacity, testnet bursts, and sustained mainnet throughput are routinely presented as equivalent. Reproducible workloads, hardware disclosure, adversarial tests, and measurements of recovery and decentralization are needed alongside speed.

Based and Shared Sequencing Protocols

A based rollup lets the L1 proposer order rollup transactions. Users submit through L1-aware builders or inclusion mechanisms, and rollup execution follows the L1 order. This inherits L1 liveness and censorship properties more directly but ties latency and throughput to L1 proposal timing.

A shared sequencer runs a separate ordering protocol for several rollups. It can promise atomic bundles such as "execute on rollup A only if the corresponding action is ordered on rollup B." The promise is meaningful only if

both rollups verify the same ordering certificate and define compatible failure behavior.

A shared-sequencing message may bind:

```
AtomicBundle {
    bundle_id
    participating_rollups[]
    transactions[]
    common_ordering_height
    expiry
}
```

Each rollup must decide what happens if another rollup rejects its transaction during execution. True atomicity may require preconditions, escrow, or a later settlement protocol; common ordering alone does not make arbitrary state transitions atomic.

Proof Markets and Aggregation Trees

A proof market separates execution from the right to prove. Executors publish traces or commitments, and provers bid to generate proofs before a deadline. Redundant provers reduce liveness dependence on one operator.

Aggregation combines proofs in a tree. Leaf proofs cover rollup blocks; intermediate proofs verify groups; a root proof covers many rollups. The settlement layer verifies one root and records the list or commitment of included state roots.

The market needs data access, deterministic witness formats, payment rules, and a fallback when no bid arrives. Aggregation also creates latency: waiting for more leaves lowers verification cost per block but delays settlement. Operators choose an aggregation window much like batchers choose a transaction window.

Intents and Solver Safety

An intent expresses an outcome rather than exact transactions. A user might authorize receiving at least 100 units of asset B before a deadline in exchange for at most 1 unit of asset A. Solvers choose routes across exchanges and chains.

The signed intent must bound solver authority:

- input asset and maximum amount;
- minimum output and recipient;
- permitted chains or settlement contracts;
- expiry and nonce;

- fee limit;
- whether partial fill is allowed;
- cancellation rule.

Settlement should be atomic from the user's perspective: either the output condition is proven and payment releases, or the input remains recoverable. Off-chain solver reputation is not a substitute for enforceable bounds.

Chain abstraction can make routing invisible while keeping the signed conditions visible. Wallets should still report which domains hold funds and when the result reaches finality.

Intent Protocol and Solver Settlement Trace

An intent states an outcome the user authorizes rather than one exact execution path. A solver can choose routes, venues, bridges, and timing that satisfy the constraints. This can hide cross-domain complexity, but it also moves safety into signatures, quote competition, settlement proofs, and solver incentives.

A user exchanging asset X on chain A for asset Y on chain B might sign:

```
Intent {
  owner,
  input_chain,
  input_asset,
  max_input,
  output_chain,
  output_asset,
  min_output,
  recipient,
  expiry,
  nonce,
  partial_fill_policy,
  fee_limit,
  settlement_contract,
  intent_version
}
```

The signature must bind every constraint and domain. A wallet should display the maximum input, minimum received amount, recipient, destination, expiry, fees, partial-fill rule, and approval scope. "Get the best result" is not a safe authorization boundary.

Quote and fill path

1. The user publishes an intent or requests private quotes.
2. Solvers compute routes and submit signed quotes tied to the intent hash.

3. The user or an auction rule selects a quote without broadening the signed intent.
4. A solver delivers the required output on B or locks a verifiable guarantee.
5. Settlement verifies destination delivery and releases no more than `max_input` on A.
6. The intent nonce and fill amount are updated atomically so replay or overfill fails.

A solver is replaceable only before it receives exclusive rights or user funds. If the protocol grants an exclusive fill window, specify its duration and the fallback when the solver disappears.

Delivery proof

A destination transaction hash alone is not proof of final delivery. Settlement must verify that the right asset and amount reached the bound recipient on the right chain under the required finality policy.

The proof may use a light client, canonical bridge message, validity proof, optimistic claim, or bonded oracle. Each gives a different safety and latency boundary. Name it in the interface. A solver payment observed by an indexer is not equivalent to a finalized, on-chain delivery proof.

Fee-on-transfer, rebasing, callback-capable, frozen, or upgradeable tokens complicate delivery. Verify the recipient's effective balance change or a protocol-defined transfer event whose semantics are stable. Token symbols are not identifiers; bind chain and contract address or another canonical asset ID.

Atomicity without synchronous chains

Chains A and B do not share one atomic transaction. The protocol therefore chooses who fronts risk. A solver can pay output first and later claim input with proof. The user is protected if input remains recoverable on expiry; the solver is protected if valid delivery guarantees the input claim.

If input is released first, the user takes solver performance risk unless collateral or another mechanism guarantees output. Do not call this atomic merely because both operations normally complete.

Partial fills

Suppose the user permits up to 100 X for at least 200 Y, with proportional partial fills. After a solver fills 30 X for 61 Y, remaining authorization is at most 70 X. Rounding must preserve the user's minimum rate.

For a fill of x , require:

```
output >= ceil(x × min_output / max_input)
```

With 30 of 100 input and a minimum 200 output:

```
ceil(30 × 200 / 100) = 60 Y
```

The 61 Y fill is valid. Track cumulative input and output because individually rounded fills can otherwise overconsume input or underdeliver aggregate output. Define whether fees are inside or outside the minimum.

Nonces, cancellation, and races

A nonce prevents replay, but cancellation is itself a state transition. If a solver delivers on B while the user cancels on A, one side can lose unless the protocol defines a cutoff and proof order.

Cancellation may be valid only before an exclusive fill acceptance, or it may require a delay long enough for destination delivery evidence to arrive. Wallets should show "cancellation requested" separately from "canceled and no longer fillable."

A replacement intent must not accidentally reactivate an old allowance. Bind permits to the settlement contract, intent hash, asset, amount, nonce, and expiry. Revoke unused token approval after settlement when possible.

Solver economics

Solvers price gas, bridge latency, inventory imbalance, reorganization risk, and adverse selection. A quote that looks one basis point better but relies on an unsafe bridge is not automatically better execution.

Quote comparison should expose expected output, worst-case output, all fees, estimated completion, finality assumption, solver collateral, and recovery path. Auctions need rules for late bids, bid withdrawal, ties, private orderflow, and information leakage.

Collateral should cover plausible non-performance or invalid claims, but proof conditions must be objective. A bond that governance can slash only after a discretionary vote provides a different guarantee from automatic on-chain settlement.

Denial of service and griefing

Users can request quotes they never accept, and solvers can win auctions they never fill. Rate-limit unauthenticated requests, require bounded quote validity, and consider small objective bonds where griefing costs are material. Do not impose deposits that make ordinary comparison inaccessible.

An attacker may create many intents sharing one allowance or nonce, submit destination dust transfers to confuse indexers, or race proofs across replicas. Settlement state must serialize fills by intent hash and reject evidence already consumed elsewhere.

Privacy and information leakage

Public intents reveal desired assets, size, deadline, and willingness to trade. Solvers can infer urgency and move markets. Private requests reduce public leakage but give the request-for-quote operator power over access and quote visibility.

Support size buckets, short quote lifetimes, multiple independent endpoints, and delayed public audit records where appropriate. State which party sees plaintext and when. "Private mempool" does not mean private from its operator.

Failure matrix

Failure	Safe state	Recovery
No solver quotes	user retains input	retry or use another route
Selected solver disappears before delivery	input remains locked or unspent	exclusivity expires; another solver fills
Delivery occurs but proof is delayed	recipient has output; solver claim pending	permissionless relayer submits proof
Destination reorganizes	input is not released against reverted delivery	wait for policy finality and reprove
Claim is replayed	consumed fill identifier rejects it	alert; no second release
Intent expires unfilled	unused input returns to user	permissionless expiry transaction
Settlement pauses	no ambiguous release	documented unpause or user escape delay

Production assertions

Test exact, better-than-minimum, partial, duplicate, expired, canceled, and overfill cases. Reorganize each chain around delivery and claim; delay proof relayers; change token behavior; crash during nonce persistence; submit conflicting solver quotes; and exercise escape while governance is unavailable.

Assert conservation of every asset, no input release without qualifying delivery, no fill beyond authorization, one consumption of each proof, eventual user refund after expiry, and an observable state for every delay.

Intent systems improve usability only when outcome freedom remains inside a narrow signed envelope. The solver may choose the path; it may not choose

a different recipient, asset, cost ceiling, deadline, finality rule, or recovery outcome than the user authorized.

Stateless Validation Pipeline

A stateless block includes or makes available witnesses for every state access. Validators begin with the prior state root, verify each witness, execute transactions, update touched commitments, and compute the new root without storing the entire state.

Block builders now carry more responsibility: they need state to generate witnesses. If only a few builders maintain complete state, validation becomes cheap while block production centralizes. Distributed state providers, witness markets, and state expiry rules aim to balance this.

A state-expiry design must answer how dormant state returns. A user may present the old value plus a proof against an archived root, pay to reactivate it, and place it in current state. The archive can be untrusted for correctness if proofs verify, but it must remain available.

Research Evaluation Milestones

A future technology should move through increasingly strong evidence:

1. **Correct model.** Security and liveness are proved under explicit assumptions.
2. **Reference implementation.** The protocol runs and interoperates against test vectors.
3. **Adversarial testnet.** Faults, withholding, reorganization, and overload are injected.
4. **Independent implementations.** More than one team reproduces the protocol.
5. **Measured production.** Sustained workloads and recovery paths are published.
6. **Reduced control.** Sequencing, proving, DA, and upgrades gain independent operators.

Roadmap language often mixes these stages. Readers should distinguish a research proposal from deployed, permissionless, failure-tested infrastructure.

Decentralized Prover Networks

A prover network accepts jobs containing a program version, public inputs, witness commitment, deadline, and reward. Provers may submit proofs directly or commit before revealing to prevent copying.

The network must avoid several failure modes. One prover can underbid and fail near the deadline. A coordinator can censor jobs. Witness data can leak private application inputs. Different hardware may produce proofs at unequal cost, concentrating supply.

Redundancy policies assign important jobs to several provers or maintain a fallback prover. Proof verification makes incorrect output harmless to safety, but repeated missed deadlines harm liveness. Reputation, bonds, and slashing can price that behavior if failure is objectively attributable.

Preconfirmations

A preconfirmation is a signed promise by a proposer or sequencer about future inclusion or order. It can give users sub-block latency before consensus finality.

A useful promise states transaction, target slot or height, maximum position or ordering relation, fee, expiry, and penalty. The signer needs collateral or future revenue at risk. Otherwise a conflicting promise is merely evidence of bad behavior without compensation.

Preconfirmations create a market for near-term blockspace and can improve user experience. They can also favor sophisticated builders and private order flow. Protocols must define whether promises are transferable, how conflicts are resolved, and what happens during reorganization.

Encrypted Mempools and Threshold Decryption

Encrypting transaction content until order is fixed can reduce front-running. Users encrypt under a committee key; consensus orders ciphertexts; a threshold of members releases decryption shares afterward.

The committee can withhold shares and halt execution. If members decrypt early, they regain ordering advantage. Distributed key generation, verifiable shares, penalties, and fallback timeouts address these risks.

Encryption hides content but not all metadata. Sender, ciphertext size, timing, and fee may still reveal strategy. It also complicates simulation and fee estimation because builders cannot inspect execution before ordering.

Threshold-Encrypted Mempool Protocol Trace

A threshold-encrypted mempool hides transaction contents until an ordering point, reducing opportunities to copy, front-run, or selectively exclude transactions based on their payload. It does not hide arrival metadata, eliminate all ordering power, or guarantee that decryption will be live.

Let an epoch committee hold shares of a decryption key. Users encrypt transactions under public key PK_e and submit ciphertext envelopes:

```
EncryptedTransaction {
  chain_id,
  epoch,
  ciphertext,
  commitment,
  sender_fee_cap,
  expiry_height,
  encryption_version
}
```

The outer fields let nodes reject wrong-network, stale, oversized, or underfunded envelopes before decryption. They must not reveal enough application detail to recreate the MEV problem.

Commit, order, decrypt, execute

1. The user builds and signs an ordinary transaction, pads it under a canonical rule, and encrypts it to PK_e .
2. Gossip nodes validate envelope bounds and propagate the ciphertext.
3. Consensus orders ciphertext commitments without seeing plaintext.
4. After the ordering boundary is final enough under policy, committee members publish decryption shares.
5. Nodes verify shares, combine the threshold, decrypt the payload, check the transaction signature, and execute in committed order.
6. Invalid plaintext consumes the sender's charged envelope allowance and cannot reorder later valid transactions.

Decryption shares must bind chain, epoch, ciphertext commitment, and ordering boundary. Otherwise a share from one epoch or fork may help decrypt an unintended transaction.

Key generation and rotation

Distributed key generation should produce PK_e without one party learning the full secret. The protocol must authenticate participants, complaints, exclusions, and the final participant set. A transcript that completes with different public keys at honest nodes is a consensus failure.

Rotate keys at a defined epoch. Accepting old-key ciphertext too close to rotation can strand it; accepting new-key ciphertext early may leak or misroute it. Publish an overlap policy with explicit last-submission, ordering, share-release, and key-destruction heights.

Back up availability, not the assembled private key. Committee members need recoverable shares or secure replacement rules, but reconstructing a central master secret removes the threshold assumption.

Early decryption and collusion

If the threshold is t of n , any coalition with t valid shares can decrypt before ordering. Choose committee composition and threshold against the actual collusion and compromise model, including common cloud, operator, and jurisdiction failures.

Slashing can deter publicly provable early share release, but private collusion may leave no evidence. Threshold encryption shifts trust from the sequencer to a decryption committee unless the committee is broad, rotating, and difficult to corrupt before the transaction expires.

Withholding and liveness

Fewer than t timely shares prevent execution. The protocol needs a deadline and a fallback that does not let strategic members choose which transactions become visible.

Possible policies include extending the share window, rotating in standby members, falling back to public plaintext after user-authorized expiry, or skipping the entire encrypted batch. Selective fallback is dangerous: revealing only profitable ciphertexts gives the fallback controller ordering information.

Suppose a 16-member committee requires 11 shares. The system tolerates five unavailable members, but six withholding members halt decryption. Correlated failure analysis should ask whether six members share one provider, client implementation, or operator.

Invalid-ciphertext denial of service

An attacker can submit ciphertext that passes cheap envelope checks but decrypts to malformed or computationally expensive input. Charge for bytes and reserved execution before inclusion, cap plaintext expansion, and make decryption and decoding costs predictable.

Padding hides transaction length classes but raises bandwidth. If payloads are padded into 1, 4, 16, and 64 kB buckets, a 4.1 kB transaction consumes 16 kB, nearly four times its plaintext size. Report privacy gain and capacity loss together.

Censorship before decryption

Encryption hides content, not sender IP, timing, fee cap, ciphertext size, or submission endpoint. A sequencer can censor all traffic from a user or

private relay and can delay ciphertexts until expiry. Redundant public ingress, inclusion receipts, proposer inclusion rules, and measured inclusion latency remain necessary.

After decryption, a block producer must not drop a valid but unfavorable transaction while retaining later ones. The committed-order rule should make such omission objectively invalid or require an explicit batch-abort rule whose cost cannot be targeted cheaply.

Reorganizations

Releasing shares after a weak ordering signal can expose plaintext before the order is stable. A reorganization then lets a builder place transactions around already revealed content. Waiting for stronger finality reduces this risk but adds latency.

Specify the release boundary and test reorganization depths around it. Shares for an orphaned commitment should not authorize another fork. If plaintext is revealed, the system cannot cryptographically make it secret again; recovery is about fair re-inclusion, not confidentiality restoration.

Production tests

Test malformed proofs of share, duplicate shares, equivocation, wrong epoch and fork domain, committee changes, delayed finality, fewer than threshold shares, early threshold collusion, invalid plaintext, maximum padding, ciphertext flooding, and a reorganization immediately after release.

Expose ciphertext queue age, time to ordering, shares observed by member, threshold completion time, invalid-decryption rate, abort reason, fallback use, and plaintext-to-ciphertext expansion. Rehearse losing the largest correlated member group.

Threshold encryption is ready only when transaction order is fixed before contents become available under the stated adversary, decryption completes under realistic correlated failures, and abort or fallback cannot be selectively used to restore the very MEV advantage encryption was meant to reduce.

Cross-Domain MEV

A transaction on one rollup can change an asset price used on another. Whoever observes or controls message timing may arbitrage the difference. Shared sequencing can coordinate order, but settlement delays and independent reorganization rules remain.

Cross-domain MEV analysis follows information: when does each actor learn an order, price, proof, or bridge message? It also follows control: who can

delay publication, proof, or relay? A modular stack may move MEV from one block producer to sequencers, solvers, relayers, or proof markets.

Hardware Acceleration and Decentralization

GPUs, FPGAs, ASICs, fast networking, and high-bandwidth memory increase execution and proving. Specialized hardware can lower unit cost while raising the capital required to compete.

Open hardware designs, commodity-compatible algorithms, proof markets, and cheap verification can preserve access. Measure market concentration and switching cost, not only benchmark speed. A protocol tied to one vendor's hardware inherits supply-chain and censorship risk.

Post-Quantum Considerations

Large-scale quantum computers would threaten common signature and commitment schemes. Migration is difficult because old accounts and bridge keys may remain vulnerable even after the protocol supports new signatures.

A roadmap needs quantum-resistant account authorization, validator signatures, proof commitments, and a process for inactive users to migrate. Post-quantum signatures are larger, affecting data availability and bandwidth. Hash-based proof systems have different assumptions but still use signatures and commitments elsewhere in the stack.

This is long-term research, but scalability and cryptographic agility interact: larger keys and proofs consume the capacity future protocols are trying to create.

What "Finished" Means for a Scaling System

A system is not finished when its fast path reaches mainnet. It approaches maturity when independent clients interoperate, users can force inclusion and exit, proof and DA systems survive operator failure, upgrades are delayed and visible, bridges have bounded risk, benchmarks are reproducible, and incident history demonstrates recovery.

Roadmaps should track removal of trust assumptions alongside throughput. The final product is not maximum speed; it is sustained, verifiable service under realistic faults.

Research-to-Production Gate

Future techniques are easiest to misread when a paper result, benchmark, testnet, and production service are described with the same tense. A deployment gate should require evidence at each level.

1. Security statement

Write the exact property and assumptions. For a preconfirmation, define what is promised, who signs, how conflicts are proven, and which collateral can be penalized. For an encrypted mempool, define confidentiality before ordering, the decryption threshold, withholding behavior, and metadata leakage. For a prover market, define correctness, deadline, witness privacy, and fallback.

A proof in one model does not cover implementation bugs, economic grieving, key compromise, or dependencies omitted from the model. List those separately.

2. Interoperable specification

The specification needs canonical encodings, domains, state machines, error behavior, version negotiation, test vectors, and upgrade rules. Two teams should implement it without sharing code and agree on every accepted and rejected vector.

Specifications should make unsafe ambiguity impossible. "Recent block," "sufficient collateral," or "available data" needs a parameter or verification rule. Unknown versions should fail closed while preserving a recovery path for in-flight work.

3. Reference and independent implementations

A reference client demonstrates one interpretation; an independent client tests whether the specification is complete. Differential tests compare outputs across generated and adversarial inputs. Reproducible builds, pinned dependencies, and public issue histories help reviewers distinguish protocol properties from one implementation.

4. Adversarial network

A testnet should inject withholding, equivocation, censorship, partitions, clock skew, key rotation, malformed proofs, worker loss, and overload. Rewards for breaking assumptions are useful only when the target and disclosure process are clear.

Measure recovery, not only whether the network restarted. Did users retain assets? Could independent operators reconstruct state? Did queued work converge without duplicates? Which privileged action was required?

5. Bounded production launch

Limit value, throughput, upgrade delay, and dependency scope while incident response is still being learned. Publish the exact controls. A rate limit can

bound loss but may also block honest exit. An emergency pause can protect funds but creates a governance key that belongs in the security model.

Increase limits only after observing realistic workload, failures, and independent operation. Time in production is not evidence when one team still runs every critical role.

6. Control reduction

Maturity should remove powers and single dependencies: permissionless challengers and provers, multiple sequencers or a tested forced path, independent DA retrieval, client diversity, delayed upgrades, key separation, and user exit under old rules.

A roadmap item is complete when its trust assumption is removed or bounded and the replacement path has survived failure tests, not when a governance vote changes a label.

Worked Decision: Should an Exchange Use Preconfirmations?

Assume an exchange rollup wants 200-millisecond order acknowledgements while settlement finality takes minutes. A sequencer can sign a promise that a valid order will appear before slot s with a maximum position and fee.

The promise improves user feedback but introduces questions:

- Can the sequencer issue conflicting positions to two traders?
- What evidence proves a missed inclusion deadline?
- Is the penalty larger than the profit from breaking the promise?
- Does a base-layer reorganization excuse performance?
- Can users submit without accepting a preconfirmation?
- Is the promise valid after an upgrade or sequencer-set change?

A useful envelope is:

```
Preconfirmation {
  chain_id,
  rollup_id,
  transaction_hash,
  promised_slot,
  ordering_constraint,
  max_fee,
  sequencer_set_version,
  expiry,
  signer
}
```

The exchange can display "preconfirmed" as a separate state, never as settled. Risk limits may allow small reversible actions after preconfirmation while withdrawals and cross-domain releases wait for stronger finality.

Suppose a conflicting promise can earn the sequencer at most \$50,000 during a stressed market. A \$10,000 slash does not create credible deterrence. Collateral must cover plausible extractable value, and enforcement must be timely and objective. If a coalition controls both ordering and the evidence path, nominal collateral may not be reachable.

The launch test should have the sequencer intentionally miss and conflict promises, rotate keys with outstanding promises, and operate through an L1 reorganization. Users should see the state change, claim process, and final outcome without relying on a private support decision.

Technology Watch Template

Track developing mechanisms with a dated table:

Field	Question
Claim	What measurable improvement is promised?
Stage	Paper, prototype, testnet, limited production, or permissionless production?
Assumptions	Network, cryptography, honest parties, hardware, governance?
Implementation	Which code and commit implement the claim?
Evidence	Proof, benchmark, test vectors, audits, incidents?
Dependencies	Sequencer, prover, DA, settlement, relayer, keys?
Recovery	What happens when each dependency fails?
Control	Who can upgrade, pause, censor, or change membership?
User status	What can a wallet truthfully display at each boundary?
Next gate	Which falsifiable test must pass before wider use?

Update the table when evidence changes. Do not silently convert a future roadmap into a present property. That discipline keeps a future-directions chapter useful after individual project timelines change.

Conclusion

The future is likely to combine rollups, real-time proofs, sampled data, parallel VMs, shared sequencing, and abstracted cross-chain interfaces. This stack can support far more activity than a single replicated machine.

Its central challenge is preserving verifiability while complexity moves between layers. The successful systems will not be those with the largest TPS claim. They will make their assumptions visible, provide credible recovery paths, and let ordinary users benefit from scale without becoming experts in every layer beneath them.

References

1. Feist, Dankrad, et al. "EIP-7594: PeerDAS." <https://eips.ethereum.org/EIPS/eip-7594>. ↵
2. Ethereum Foundation. "Fusaka Mainnet Announcement" (November 6, 2025). <https://blog.ethereum.org/2025/11/06/fusaka-mainnet-announcement>. ↵
3. Ethereum.org. "The Verge." <https://ethereum.org/roadmap/verkle-trees/>. ↵
4. ERC-4337 Documentation. "ERC-4337." <https://docs.erc4337.io/core-standards/erc-4337.html>. ↵
5. ERC-4337 Documentation. "The EntryPoint Contract." <https://docs.erc4337.io/smart-accounts/entrypoint-explainer.html>. ↵
6. Flashbots Docs. "MEV-Boost Overview." <https://docs.flashbots.net/flashbots-mev-boost/introduction>. ↵
7. Flashbots Docs. "MEV-Boost Block Proposal." <https://docs.flashbots.net/flashbots-mev-boost/architecture-overview/block-proposal>. ↵
8. Flashbots Docs. "MEV-Boost Risks and Considerations." <https://docs.flashbots.net/flashbots-mev-boost/architecture-overview/risks>. ↵
9. Espresso Documentation. "Rollup Architecture." <https://docs.espressosys.com/network/learn/rollup-architecture>. ↵
10. Across Docs. "Intent Lifecycle in Across." <https://docs.across.to/guides/concepts/intent-lifecycle>. ↵
11. Across Docs. "Security Model and Verification." <https://docs.across.to/introduction/security>. ↵
12. CoW Protocol Documentation. "Flow of an order." <https://docs.cow.fi/cow-protocol/concepts/how-it-works/flow-of-an-order>. ↵
13. Succinct Docs. "Protocol Architecture." <https://docs.succinct.xyz/docs/protocol/spn/architecture>. ↵
14. Succinct Docs. "Migrate to Mainnet." <https://docs.succinct.xyz/docs/sp1/prover-network/migration-guide>. ↵

Glossary

This glossary defines terms as they are used in this book. Some projects use the same word differently; the surrounding security model always takes precedence over the label.

Start Here: Core Blockchain Terms

Address

A public identifier used as a transaction sender, recipient, account, or contract location. An address is not a person's verified real-world identity.

Block

An ordered package of transactions and protocol data linked to prior history. A block can be proposed or included before it reaches the protocol's strongest finality.

Block header

The compact part of a block containing metadata and commitments such as the parent hash and state or transaction roots. The header binds a larger body without containing all of it.

Canonical chain

The history currently selected by the protocol's fork-choice and finality rules. Temporary competing branches may exist before convergence.

Cryptographic hash

A fixed-length fingerprint of data. A small input change produces a different fingerprint, and finding useful collisions should be infeasible under the assumed hash function.

Digital signature

Cryptographic evidence that the holder of a private key authorized specific bytes. It does not by itself prove sufficient funds, correct execution, inclusion, or finality.

Finality

The condition under which a block or transaction should not be reverted under stated protocol and fault assumptions. Finality may be probabilistic or based on an explicit protocol commit rule. A sequencer confirmation,

proposal, or quorum certificate is not settlement or BFT finality unless that commit rule explicitly makes it so.

Full node

Software that independently checks blocks under protocol rules. Storage and historical-retention choices vary; "full" does not always mean retaining every old byte forever.

Merkle proof

A small set of neighboring hashes that proves one item is included under a Merkle root. It authenticates an item but does not guarantee that every committed item is available.

Node

A computer running blockchain protocol software and communicating with peers. Nodes may validate, produce blocks, serve data, or perform only a subset of roles.

Private key and public key

A private key is secret signing material. A corresponding public key lets others verify signatures. Control of a key establishes protocol authorization, not necessarily legal identity or rightful ownership after theft.

Smart contract

Program code executed under blockchain rules. A contract deterministically changes state from inputs; it does not infer intent beyond its code and authenticated messages.

State

The latest live values used by execution, such as balances, ownership, nonces, and contract storage. State differs from history, the record of how those values changed.

Transaction

A signed instruction submitted to the network. Its final outcome depends on admission, ordering, execution, fees, current state, consensus, and finality.

Validator

A participant that checks proposals and contributes votes, attestations, or other consensus evidence. The exact authority and penalty depend on the consensus protocol.

Wallet

Software that manages keys, constructs transactions, and displays observed status. A wallet usually relies on nodes or service providers for chain data unless it verifies through its own node or light client.

Architecture and State

Application chain (appchain)

A blockchain or rollup dedicated to one application or a related group of applications. It can customize execution, fees, sequencing, and governance, but must supply or rent consensus, data availability, settlement, and bridging.

Bridge

A protocol that authenticates messages or assets between security domains. A canonical rollup bridge is enforced by its settlement contracts. A sidechain bridge normally depends on the sidechain's consensus, a light client, a committee, or a multisignature account.

Commitment

A short cryptographic value binding a party to larger data. A Merkle root and a polynomial commitment are examples. A commitment proves integrity when data is revealed; it does not by itself prove that data is available.

Composability

The ability of applications or contracts to interact. Synchronous composability allows several calls to complete atomically in one transaction. Asynchronous composability uses messages and requires explicit handling of delay and partial completion.

Execution

Applying ordered transactions to prior state to compute new state.

Layer 1 (L1)

The base blockchain whose consensus defines its canonical history and native state.

Layer 2 (L2)

A protocol that updates state outside an L1 while using that L1 for settlement, correctness enforcement, or exits. Merely connecting a separate chain with a bridge does not make it inherit L1 security.

Monolithic blockchain

A system whose validator network performs execution, settlement, consensus, and data availability.

Modular blockchain

An architecture that separates one or more of execution, settlement, consensus, and data availability into specialized systems.

Settlement

The function that accepts canonical state commitments and resolves disputes. Rollups commonly settle to an L1 smart contract.

State root

A cryptographic commitment to the full application or blockchain state at a point in history.

Layer 2 Systems**Canonical bridge**

The bridge enforced by a rollup's settlement contracts rather than an external liquidity provider.

Challenge period

The interval during which an optimistic state assertion can be disputed. Expiry without a successful challenge makes the assertion acceptable under the protocol's challenger, data, timing, and contract assumptions; it is not a validity proof of the transition.

Channel

A protocol in which a fixed or constrained set of participants exchange signed state updates and use the blockchain to open, close, or resolve disputes.

Commit-chain

An operator-based off-chain system that posts state commitments to a base chain and provides an exit or challenge mechanism. The exact term is used inconsistently, so data and validation assumptions must be stated.

Fault proof or fraud proof

Evidence that an asserted state transition is invalid. Interactive systems narrow a disagreement to a small computation that the settlement contract checks.

Force inclusion

A mechanism allowing a user to bypass a sequencer, usually by submitting a transaction through the settlement layer.

Optimistic rollup

A rollup that accepts state assertions unless they are successfully challenged with a fault proof.

Plasma

A family of child-chain designs that commit roots to a parent chain while users retain proofs and use exit games. Data withholding and mass exits are central design problems.

Rollup

A system that executes transactions outside its settlement layer, publishes the data needed to reconstruct its state, and uses fraud or validity proofs to enforce correctness.

Sequencer

The party or protocol that orders L2 transactions and produces L2 blocks. It can provide low-latency confirmation but may introduce censorship, liveness, and ordering risks.

Sidechain

A separate blockchain connected by a bridge. It normally has independent consensus and does not automatically inherit the base chain's security.

State channel

A channel for arbitrary application state rather than payments alone.

Validity rollup

A rollup whose state commitments must be accompanied by a cryptographic proof of correct execution. Often called a ZK rollup, even when the proof is not used for privacy.

Validium

A validity-proof system that stores transaction data outside the settlement layer. Correctness can remain proven while data withholding prevents users from reconstructing state.

Volition

A system that lets users or applications choose between on-chain and off-chain data availability.

Watchtower

A service that monitors the chain and responds to dishonest or stale channel closures for an offline user.

Data Availability and Proofs**Blob**

In Ethereum, a temporary data object introduced by EIP-4844 for rollup publication. Consensus commits to blobs, while EVM execution cannot directly read their contents.

Data availability (DA)

The property that data required to verify or reconstruct a state was disseminated and obtainable during the protocol's required window.

Data availability committee (DAC)

A set of parties attesting that off-chain data is available. It reduces publication cost but introduces a threshold trust assumption.

Data availability sampling (DAS)

A method by which light nodes request random pieces of erasure-coded data to gain confidence that the full block can be reconstructed.

PeerDAS

Ethereum's peer data availability sampling protocol, activated in the Fusaka upgrade on December 3, 2025. Nodes custody and sample subsets of erasure-coded blob columns rather than every node downloading every blob.

Erasure coding

Encoding that expands data into redundant shares so the original can be reconstructed from a threshold subset.

KZG commitment

A polynomial commitment scheme used by Ethereum blob transactions. It binds a proposer to blob data and supports compact evaluation proofs.

SNARK

A succinct non-interactive argument of knowledge. Different SNARKs make different setup, cryptographic, proof-size, and prover-cost trade-offs.

STARK

A scalable transparent argument of knowledge. STARKs avoid a trusted setup and use hash-based assumptions, generally with larger proofs than many SNARK systems.

Validity proof

A cryptographic proof that a computation or state transition followed specified rules. Validity does not by itself prove data availability.

Witness

Auxiliary data proving that a computation used particular state values. Stateless validation gives validators witnesses instead of requiring all state locally.

Zero knowledge

A property allowing a proof to reveal that a statement is true without revealing its private witness. Succinct validity proofs do not have to be zero knowledge.

Messaging, Operations, and Governance**Censorship resistance**

The ability of a valid transaction to reach execution despite one or more actors trying to exclude it. A force-inclusion path is useful only if its delay and cost are bounded in practice.

Domain separation

Binding a signature, hash, or message to a chain, protocol version, epoch, and message type so valid data from one context cannot be replayed in another.

Escape hatch

A mechanism allowing users to withdraw or advance state without the normal operator. Escape hatches depend on available data, executable contracts, and affordable base-layer capacity.

Forced inclusion

A path by which a user submits a transaction through a more trusted layer when the normal sequencer censors it. Forced inclusion guarantees should state delay, fee, ordering, and failure behavior.

Light client

A client that verifies headers, committees, or succinct proofs rather than downloading and executing every transaction. Its security depends on how it authenticates consensus and validator-set changes.

Preconfirmation

A signed promise about future transaction inclusion or ordering made before consensus finality. A credible promise specifies expiry and an enforceable consequence for violation.

Reorganization (reorg)

Replacement of a previously preferred but not final chain segment. Cross-chain systems need explicit rules for messages observed before their source chain is final.

Relayer

An actor that transports a message or proof between systems. A well-designed bridge does not trust a relay for correctness and allows any party to replace a failed relay.

Remote procedure call (RPC)

An application interface through which a client asks a node or gateway to submit transactions or read chain data. An RPC response reports what that endpoint observed or accepted; it is not consensus evidence unless the client independently verifies the corresponding proof, certificate, or finalized chain state.

Social recovery

Recovery that requires human coordination, governance, or a trusted checkpoint rather than following the ordinary protocol automatically. It may restore service but changes the trust model.

Timelock

A mandatory delay between scheduling and executing an upgrade or privileged action. A timelock is protective only if users and monitors can detect the action and respond before execution.

Weak subjectivity

The requirement for a proof-of-stake client to begin from a sufficiently recent trusted checkpoint, limiting long-range histories signed by validators whose stake is no longer slashable.

Performance and Reliability

Admission control

A mechanism that limits or prices incoming work so queues and resources remain within an operating objective. Fee markets are one form of admission control.

Backpressure

A signal from a saturated downstream component that slows upstream production. Without backpressure, a sequencer can create batches faster than a prover or DA layer can process them.

Capacity

The maximum sustainable offered load that meets a stated latency and error objective. Capacity is measured for a workload, hardware configuration, network, duration, and completion boundary.

Capacity knee

The region where increasing offered load causes tail latency and queue depth to rise rapidly while completed throughput grows slowly or stops growing.

Critical path

The longest dependency chain determining completion time. Parallel work outside the critical path may improve resource use without reducing user latency.

Headroom

Unused capacity reserved for demand variance, component loss, retries, compaction, view change, or recovery traffic. Headroom should be chosen separately for each resource.

Idempotence

The property that repeating an operation has the same durable effect as applying it once. Message consumption, deposits, withdrawals, and retried jobs need idempotent identifiers.

Offered load

The rate at which clients submit work, whether or not the system accepts or completes it. Reporting completed throughput without offered load hides overload behavior.

Operating envelope

The workloads and fault conditions under which a system meets its throughput, latency, safety, liveness, cost, and recovery objectives.

p50, p95, p99 latency

Percentile latencies. p99 is the value at or below which 99 percent of measured operations complete. Percentiles require a defined population, window, and completion boundary.

Queue stability

A condition in which backlog remains bounded over time. A temporary high throughput result is not sustainable when queued data, proofs, or transactions continually grow.

Recovery point objective (RPO)

The maximum accepted loss of durable work or state after failure. A block-chain safety design often targets no loss of finalized state, while auxiliary indexes may permit replay from an earlier point.

Recovery time objective (RTO)

The target time to restore a specified service after failure. State which service and finality boundary the timer covers.

Saturation

A state where one resource is fully utilized and limits completed work. CPU, storage I/O, network bandwidth, DA publication, proving, or consensus can each saturate independently.

Service-level objective (SLO)

A measurable target such as p99 settlement within ten minutes for 99.9 percent of withdrawals. An alert should map to user impact or exhaustion of the SLO's error budget.

Tail latency

Latency of slower requests, commonly represented by high percentiles. Tail behavior reveals queues, pauses, retries, and skew hidden by averages.

Throughput

Completed work per unit time under a defined completion rule. Submitted, sequenced, executed, proved, and finalized throughput are different metrics.

Utilization

The fraction of a resource's capacity in use. Running near 100 percent average utilization usually produces unstable queues when service time or arrivals vary.

Workload mix

The distribution of transaction or request types, state-access patterns, sizes, and bursts used in a test. A throughput result applies to that mix, not to an abstract average transaction.

Execution and Consensus**Block-STM**

An optimistic parallel execution engine that speculates on transactions, detects invalidated reads, and retries conflicts while preserving a canonical order.

Byzantine fault

Arbitrary faulty behavior, including conflicting messages, collusion, and malicious deviation from protocol.

Committee

A subset of validators assigned to vote on or process part of the system.

Consensus

Agreement on an ordered canonical history among independent nodes despite faults.

Fast path

A protocol route that reaches an outcome with fewer phases or less ordering work under favorable conditions. The term is system specific. In current Sui terminology, address-owned objects can use Mysticeti fast-path certification without waiting for total-order consensus, while shared and party objects use consensus sequencing.

Hot state

State accessed by many concurrent transactions. Hot state creates conflicts and limits parallel execution.

Liveness

The guarantee that the protocol eventually continues to process and finalize valid work under stated conditions.

MEV

Maximum extractable value: value captured by controlling transaction inclusion, exclusion, or ordering.

Nakamoto consensus

Longest- or heaviest-chain consensus introduced by Bitcoin, with probabilistic finality and Sybil resistance based on proof of work.

Pacemaker

The component of a leader-based BFT protocol that manages timeouts and view changes.

Parallel execution

Executing independent transactions concurrently while producing a deterministic result equivalent to the protocol's canonical semantics.

Quorum certificate (QC)

Aggregated evidence that a quorum of replicas voted for a proposal in one protocol phase. A QC does not by itself imply finality unless the protocol's commit rule explicitly says it does; chained protocols commonly require later certificates.

Quorum intersection

The property that two threshold quorums overlap in enough participants or weight to include an honest voter under the stated fault bound.

Prover market

A system that assigns proof-generation jobs among independent providers. Correctness follows from verification, while deadlines, witness privacy, job availability, and concentration remain operational concerns.

Safety

The guarantee that honest participants do not finalize conflicting states.

Sharding

Partitioning validators, transactions, execution, or state so different groups process different work concurrently.

Synchronous, partially synchronous, asynchronous

Network models. A synchronous protocol assumes a known message-delay bound. Partial synchrony assumes a bound eventually holds. An asynchronous model assumes no timing bound.

View change

The process that replaces a stalled or faulty consensus leader and carries forward the certificates or locks needed for safety.

Review Questions and Design Exercises

These questions are designed to test reasoning, not recall. A strong answer states its assumptions and distinguishes normal operation from failure recovery.

Basic-Knowledge Warm-Up

Use these questions before the chapter exercises. A reader should be able to answer them in plain language; formulas and protocol names are optional.

1. What is the difference between blockchain history and current state?
2. Why do many nodes repeat transaction checks instead of trusting one database?
3. What is the difference between a block being included and a transaction being final?
4. What evidence does a digital signature provide, and what does it not prove about a transaction's validity?
5. Why can raising transactions per second make latency or decentralization worse?
6. Explain safety and liveness using a system that must choose between halting and accepting conflicting payments.
7. Why is a sidechain bridge a separate security boundary?
8. In a payment channel, why must each participant keep the newest signed state?
9. What does a rollup post to its settlement or DA layers, and why is a state root alone insufficient for recovery?
10. Explain the difference between an optimistic fault proof and a validity proof.
11. Why can a validity proof establish correctness without establishing data availability?
12. Give an example of two transactions that can execute in parallel and two that conflict.
13. Why do two BFT quorums need to overlap in an honest participant?
14. For any future scaling claim, name one assumption and one failure state a wallet should expose.

Warm-up answer guide

1. History is the ordered record of past blocks and transactions; state is the latest live balances, ownership, and contract data.
2. Replication lets independent parties detect an invalid update or dishonest operator, at the cost of repeated work.
3. Inclusion places a transaction in a current canonical candidate; finality adds evidence or depth that makes reversal forbidden or sufficiently unlikely under a stated model.
4. A signature proves authorization by the corresponding key for the signed bytes. It does not prove sufficient balance, correct contract execution, inclusion, or finality.
5. Larger or faster blocks can take longer to propagate and demand stronger hardware; batching can increase throughput while individual users wait longer.
6. Safety prevents two conflicting payments from both becoming accepted. Liveness ensures valid payments eventually progress. A partition may force a protocol to halt to preserve safety.
7. The bridge decides which sidechain evidence releases assets elsewhere. The base chain does not automatically verify every sidechain transition.
8. An old state may assign balances differently. The newest signature, revocation rule, or monotonic sequence is evidence against stale settlement.
9. A rollup posts ordered data or its commitment, state claims, and proof/dispute evidence according to its design. A root binds values but does not reveal them.
10. An optimistic system accepts after a challenge window unless a challenger proves a fault. A validity system requires a compact proof of correct execution before acceptance.
11. A verifier can check a proof over hidden or missing inputs without those inputs being available for users to reconstruct future state.
12. Transfers over disjoint accounts can run together; two transactions writing one balance or reading a value another writes conflict.
13. Overlap plus honest voting rules prevents certificates for incompatible commits under the fault bound.
14. Acceptable answers include a sequencer outage and "forced-inclusion pending," prover delay and "unproven," or bridge finality delay and "not withdrawable."

Chapters 1-2: Measuring Scalability and the Trilemma

1. A chain reports 50,000 TPS. What information is needed before comparing it with Ethereum or a rollup?

2. Explain how throughput can increase while user latency becomes worse.
3. Why does adding validators to a fully replicated state machine improve redundancy without necessarily improving execution capacity?
4. Design a benchmark for an on-chain order book. Which workloads would expose hot-state contention?
5. Give one example in which a protocol improves the trilemma frontier and one in which it merely moves cost to a less visible layer.
6. Which forms of operational centralization are missed by validator count?

Chapters 3-5: Layers, Sharding, and Channels

1. Follow one token transfer through an L1, a rollup, and a sidechain. At which point is each transfer final?
2. Why is a cross-shard transfer naturally asynchronous? What prevents its receipt from being replayed?
3. A protocol samples 100 validators without replacement from 1,000, of whom 250 are adversarial. Safety fails at 34 adversarial committee members. Write the exact capture probability and calculate it. How does reducing committee size affect parallelism and tail risk?
4. A channel counterparty publishes an old signed state. What evidence lets the contract choose the newer state?
5. Why can a payment route have sufficient graph connectivity but insufficient liquidity?
6. Compare the failure of a channel operator, a sidechain bridge, and a rollup sequencer. Which failures threaten safety and which threaten liveness?

Chapters 6-8: Rollups, Modularity, and Data Availability

1. Separate sequencer confirmation, proof or challenge acceptance, and L1 finality for an optimistic and validity rollup.
2. Why does a validity proof not prove that users can reconstruct the state?
3. Describe the minimum escape hatch needed when a sequencer censors a withdrawal.
4. A modular rollup uses Celestia for data and Ethereum for settlement. What can each layer prove, and what can it not prove?
5. Derive the probability that 15 independent samples miss an attack hiding half the shares.
6. Compare L1 blobs, an external DA network, and a DAC across cost, validator integration, and withholding risk.

7. What observability should a wallet expose when a modular transaction is delayed?

Chapters 9-10: Parallel Execution and Consensus

1. Construct three transactions where two can run in parallel and the third must retry. Identify all read/write conflicts.
2. Why is deterministic commitment required even if thread scheduling is nondeterministic?
3. Suggest two state-model changes that reduce contention in an on-chain game.
4. In a four-replica BFT system tolerating one Byzantine replica, explain why two quorums of three intersect in an honest replica.
5. How does pipelining improve throughput without reducing a transaction's finality latency?
6. Compare the network assumptions and fault thresholds of HotStuff and Sync HotStuff.
7. Why should consensus benchmarks include block payload and leader failures?
8. Why does one HotStuff quorum certificate not by itself prove that the certified block is committed?
9. Compare a Sui address-owned fast-path transaction with a shared-object transaction. Which evidence is common, and which ordering requirement differs?

Chapter 11: Future Architecture

1. Draw the dependency graph of a transaction using a solver, shared sequencer, validity rollup, external DA layer, and Ethereum settlement.
2. For each edge, identify a safety failure, a liveness failure, and a recovery mechanism.
3. What new concentration risks can shared sequencing or proof markets create?
4. How can chain abstraction hide complexity without hiding security assumptions?
5. Propose a reproducible benchmark for a multi-rollup system. Which finality boundary ends the timer?
6. An MEV-Boost relay withholds the payload after a proposer signs the blinded header. Why can the proposer not safely publish a locally built payload for the same slot, and when must fallback be chosen?

Capstone Design Exercise

Design a blockchain architecture for one of the following:

- a global micropayment network;
- an on-chain game with one million daily players;
- a high-value exchange with a central limit order book;
- a public registry whose data must remain retrievable for decades.

Your design must state:

1. transaction workload and latency target;
2. execution environment and concurrency model;
3. ordering and consensus mechanism;
4. settlement and finality rule;
5. data availability and archival strategy;
6. bridge or messaging assumptions;
7. upgrade and emergency controls;
8. user exit or recovery path;
9. benchmark method and hardware disclosure;
10. the trilemma cost the design accepts.

Quantitative Laboratory: Capacity and Finality

Assume a rollup posts one batch every 10 seconds. Each batch contains 2,000 transactions, consumes 120 kB of compressed data, and takes a prover 24 seconds on one machine. The settlement chain finalizes a posted batch after 12 minutes. The sequencer has a 20-second forced-inclusion deadline.

1. Compute offered throughput when every batch is full.
2. Compute the sustained data rate in bytes per second, excluding commitment overhead.
3. How many independent prover workers are required to prevent an unbounded proof queue under steady load? Include utilization headroom rather than giving only the mathematical minimum.
4. Draw the user-visible milestones for sequencer receipt, batch publication, proof acceptance, and settlement finality.
5. Which milestone should a bridge use before releasing a high-value withdrawal, and why?
6. If the sequencer stops immediately after receipt, what evidence and deadline does a user need to invoke forced inclusion?
7. Repeat the capacity calculation when average compression worsens by 40 percent and the settlement data limit is fixed.

A strong submission shows units, separates offered load from completed throughput, and names assumptions about proof aggregation and parallelism.

Fault-Injection Laboratory

Run an implementation or simulator through this sequence:

1. operate at 60 percent of measured saturation for ten minutes;
2. disconnect the current consensus leader for two views;
3. delay 10 percent of data chunks beyond their normal retrieval deadline;
4. stop the primary prover while proofs are queued;
5. submit one malformed cross-domain message and one replay;
6. restore all components without deleting persistent state.

Record p50, p95, and p99 latency; queue depth; time to view change; data-repair traffic; proof backlog; finalized height; and every user-visible status transition. The report must identify whether each fault affected safety, liveness, latency, or only cost. It must also explain which component detected the fault and which component initiated recovery.

Design Review Rubric

Evaluate the capstone on five dimensions, each from 0 to 4:

- **Explicit assumptions:** network, trust, workload, finality, and failure assumptions are testable.
- **End-to-end correctness:** the transaction, message, withdrawal, and recovery paths are internally consistent.
- **Quantitative evidence:** calculations retain units, benchmarks reach saturation, and latency distributions are reported.
- **Adversarial depth:** censorship, withholding, equivocation, reorganization, upgrade compromise, and correlated outages are addressed.
- **User recovery:** escape paths are available, affordable under congestion, observable, and exercised in tests.

A score of 0 means the issue is absent. A score of 2 means it is described but not measured or tested. A score of 4 means another team could reproduce the evidence and challenge the assumptions.

Instructor Notes and Solution Sketches

These sketches identify the reasoning a strong answer should contain. They are not unique solutions. Quantitative answers should retain units and state assumptions.

Chapters 1-2

A credible throughput comparison fixes transaction semantics, state size, offered-load curve, duration, hardware, network topology, validator independence, completion boundary, and fault behavior. Throughput can rise while latency worsens when batching waits longer, queues grow near saturation, or finality requires more stages. Adding replicas to a fully replicated state machine adds fault tolerance and read capacity, but each replica still executes the same ordered writes.

For the order-book benchmark, vary the number of markets, price-level concentration, cancellation ratio, market-order depth, account skew, and burst arrival. Hot price levels reveal serial state. A protocol improves the trilemma frontier when it lowers production or verification cost without changing the claim being measured; it only moves cost when it replaces broad verification with a committee, operator, expensive recovery path, or hidden subsidy.

Chapters 3-5

An L1 transfer is complete under the L1's finality rule. A rollup transfer passes sequencer, publication, proof or challenge, and settlement milestones. A sidechain transfer follows its own consensus, and its bridge adds another finality rule.

Cross-shard execution is asynchronous because independent committees cannot atomically lock all state without coordination that erodes parallelism. The destination authenticates the source receipt and stores a consumed nonce or message identifier. For the stated committee, if x is the adversarial count:

```
P[X >= 34]
= sum from x=34 to 100 of
  C(250, x) C(750, 100-x) / C(1000, 100)
approx 0.02144, or 2.14%
```

The calculation assumes uniform sampling without replacement and counts validator identities rather than correlated control or stake weight. Smaller committees can increase parallel capacity while increasing capture tail risk. In a state channel, signatures authenticate updates and a monotonic nonce selects the newest state. Payment routes need directional balance on every hop, not only graph connectivity.

Operator failure has different effects: a channel counterparty can delay cooperative close but the adjudicator preserves funds; a sidechain bridge compromise may violate safety; a sequencer outage should affect liveness if forced inclusion and exit remain intact.

Chapters 6-8

An optimistic rollup has fast soft confirmation but final state waits through publication, challenge, and settlement. A validity rollup replaces the challenge period with proof generation and verification, while settlement finality still comes later. A validity proof establishes a transition statement; unavailable inputs can still prevent users from reconstructing state or exiting.

A minimum escape path authenticates forced transactions through L1, defines a bounded inclusion deadline, and permits state advancement or withdrawal without the sequencer. For an external DA design, the DA layer can prove that bytes were ordered and available under its rules; settlement verifies only the commitment or proof its contract understands.

If each independent sample has probability $1/2$ of missing a half-hidden block, 15 samples all miss with probability:

$$(1/2)^{15} = 1/32768 \approx 0.00305\%$$

The calculation assumes uniform unpredictable samples, independent observations, valid encoding, honest header authentication, and peers that cannot selectively identify and deceive the sampler.

A wallet should distinguish waiting for sequencing, data publication, proof, settlement finality, and destination execution. "Pending" alone does not tell a user whether retrying is safe.

Chapters 9-10

One conflict example is T1: read A, write B; T2: read C, write D; T3: read B, write C. T1 and T2 can speculate together from the same snapshot, while T3 depends on both and must wait or retry. A parallel schedule must commit a state equivalent to canonical order; otherwise validators can calculate different roots from identical blocks.

Reduce contention by partitioning per-player or per-market state, replacing one global counter with mergeable local counters, and avoiding synchronous writes to shared metadata. In a four-replica BFT system, two quorums of three intersect in two replicas; with at most one Byzantine, at least one overlap is honest. Pipelining overlaps different consensus stages across blocks, improving steady-state block rate without shortening one block's commit chain.

HotStuff assumes eventual network bounds for liveness and tolerates fewer than one-third Byzantine replicas in the common model. A synchronous variant uses a known bound and derives different protocol guarantees from that stronger assumption. Benchmarks must include payload dissemination and

leader failures because empty-block voting hides bandwidth and view-change cost.

A single HotStuff QC proves a quorum vote in one phase. Commitment depends on the protocol-specific certified chain and locking rule, so an explorer must not relabel every certified proposal as final. In Sui, both fast-path and consensus traffic use signed Mysticieti DAG blocks and Byzantine-quorum evidence. Address-owned fast-path transactions can certify and finalize without waiting for total-order consensus, while shared and party object transactions require consensus to assign an order and versions.

Chapter 11

A dependency graph should show user, solver, sequencer, execution rollup, DA network, prover, settlement chain, and destination bridge. Each edge needs authenticated data, finality, timeout, and recovery. Shared sequencing can concentrate order flow; proof markets can concentrate specialized hardware and witness access; solver markets can concentrate routing and censorship power.

Chain abstraction is safe when the interface hides mechanics but still exposes assets, maximum spend, destination, finality status, fees, and recovery. A multi-rollup benchmark fixes workload and route, records every domain boundary, and ends at a named milestone such as settlement finality or destination execution rather than the first sequencer response.

After a proposer signs a blinded payload header, publishing a different local payload for the same slot can create conflicting signed proposals and a slashable equivocation. Relay deadlines, local construction, and circuit-breaker fallback must select the local path before signing the blinded header. Withholding after the signature therefore causes a missed slot rather than a safe late substitution.

Quantitative Laboratory

With 2,000 transactions every 10 seconds, offered throughput is:

$$2,000 / 10 \text{ s} = 200 \text{ transactions/s}$$

The mean compressed data rate is:

$$120 \text{ kB} / 10 \text{ s} = 12 \text{ kB/s}$$

One proof job consumes 24 prover-seconds and arrives every 10 seconds, so minimum steady capacity is $24/10 = 2.4$ workers. Three workers give only 80 percent utilization under uniform jobs and no failures. Four workers give 60

percent and more useful headroom. A production answer should examine proof-time variance, aggregation, restart cost, and correlated hardware loss.

The bridge should normally wait for proof acceptance plus the chosen settlement finality, not sequencer receipt. Forced inclusion needs the signed transaction, evidence or timing rule showing the sequencer deadline expired, and enough L1 capacity to submit it.

If compression worsens by 40 percent, a batch uses 168 kB and mean data rate becomes 16.8 kB/s. With a fixed 12 kB/s limit, only about $12/16.8 = 71.4\%$ of the former transaction rate fits, or roughly 143 transactions/s under the same mix. The exact result depends on whether the stated limit was already saturated and whether batch overhead is fixed.

Capstone Review

A complete capstone has two diagrams: the normal transaction/finality path and the degraded recovery path. It includes capacity arithmetic for execution, data, proving, consensus, and state; a role and key inventory; at least one safety and one liveness failure per external dependency; a measured mass-recovery test; and a clear statement of the accepted trade-off.

Practitioner's Evaluation Handbook

A scaling architecture should be reviewed as a system, not as a collection of attractive components. This handbook turns the concepts in the book into a repeatable evaluation process for engineers, investors, researchers, and application teams.

How a Basic Reader Can Evaluate a System

Evaluation does not require trusting a large benchmark number or understanding every cryptographic detail. Start with one user action and follow its evidence.

Suppose Maya deposits into a rollup, makes a payment, and withdraws. Draw boxes for the wallet, sequencer, rollup executor, data publisher, proof or challenge system, L1, and bridge. Draw an arrow whenever one box sends data or evidence to another.

For every arrow, ask:

- What message is sent?
- How does the receiver know its source and version?
- Which signature, hash, proof, or validator vote is checked?
- Can the sender delay, replace, or censor it?
- What status does Maya see while it is missing?
- Is there another path when the sender fails?

This simple diagram is a **trust-boundary map**. A trust boundary is any point where one component accepts another component's statement. Verification narrows what must be trusted; it does not remove dependencies such as timing and data delivery.

Requirements before metrics

A **service-level objective (SLO)** states a measurable target, such as "99 percent of withdrawals settle within ten minutes." A **recovery time objective (RTO)** states how quickly service should recover. A **recovery point objective (RPO)** states how much recently accepted work may be lost or replayed after recovery.

These terms force a team to say what "working" means. A system can meet a throughput target while missing withdrawal SLOs or losing recent soft confirmations after restart.

Normal path and failure path

Write the normal path first, but evaluate the failure path separately. For each dependency, remove it and observe:

- 1. whether safety still holds;
- 2. whether new work stops;
- 3. whether existing users can exit;
- 4. which manual authority is required;
- 5. how long recovery takes;
- 6. which evidence proves recovery was correct.

A failure that halts safely differs from one that releases the wrong asset. A recovery that needs a governance vote differs from a permissionless retry.

Measured, simulated, and projected

- **Measured:** observed from a real run with recorded inputs and instrumentation.
- **Simulated:** produced by a model or controlled environment whose assumptions are stated.
- **Projected:** estimated by scaling measurements or theory beyond the tested range.

All three can be useful, but they are not interchangeable. If four provers handle 100 jobs per second, eight provers may not handle 200 when they share one database or aggregator.

Reading operational terms

A **runbook** is a tested sequence for responding to a known condition. **Telemetry** is measured system data such as queue age, errors, and resource use. An **alert** is a rule that calls attention to telemetry crossing a meaningful boundary. A **postmortem** reconstructs an incident and assigns testable follow-up work.

"We monitor it" is incomplete. Name the metric, threshold, user impact, responder, safe action, and test that proves the alert works.

A one-page beginner scorecard

Question	Evidence to request
What useful work completes?	transaction templates and success rule
When is it final?	named consensus, proof, and settlement boundary
Who can block progress?	role and failure-domain map
Who can create loss?	keys, thresholds, contracts, and value caps

Question	Evidence to request
Can data be recovered?	retrieval test from independent providers
Can a user exit?	exercised transaction and worst-case cost/time
Does capacity survive failure?	fault-injected benchmark and queue behavior
Can the rules change?	upgrade manifest, delay, authority, and exit window
Can another team reproduce it?	code, configuration, snapshot, raw data, and scripts

The rest of the handbook expands this scorecard. Readers can use it even when specialists perform the cryptographic review.

1. Define the Workload Before the Architecture

Start with actions users perform, not a target TPS. Record the expected and peak rates for transfers, contract calls, state reads, proof requests, deposits, withdrawals, and cross-domain messages. Measure transaction size and the number of state locations touched.

Separate independent traffic from contended traffic. Ten thousand transfers between distinct accounts may parallelize well. Ten thousand swaps against one pool may serialize. A game with one million daily users can still have a severe hot-state event during a popular mint.

Write latency objectives at the boundary users experience. A trading application may need a sequencer acknowledgement within 200 milliseconds but accept settlement after several minutes. A bridge releasing high-value collateral may require proof acceptance and L1 finality. These are different service-level objectives.

A workload statement should include:

- normal, peak, and burst arrival rates;
- transaction and batch sizes;
- read/write sets or expected contention;
- acceptable p50 and p99 latency;
- state growth and historical-query needs;
- deposit, withdrawal, and cross-chain volume;
- geographic distribution of users and validators;
- expected adversarial behavior.

2. Draw the Transaction and Trust Paths

Draw two diagrams. The **transaction path** follows a normal transaction from signing to finality. The **recovery path** follows the same asset when each privileged service is offline or malicious.

For a rollup, the normal path may be wallet -> RPC -> sequencer -> batcher -> DA publication -> prover or challenger -> settlement contract. The recovery path may be wallet -> L1 force-inclusion contract -> timeout -> forced withdrawal. Both diagrams need identities for upgrade administrators and bridge relayers.

At every edge, ask:

1. What message or commitment crosses the boundary?
2. How is it authenticated?
3. Which state or finality does the receiver assume?
4. Can the sender equivocate or withhold?
5. How is failure detected?
6. Can a user recover without the sender?

An architecture that has only a normal-path diagram is incomplete.

3. Identify the Capacity Equation

Model each major resource independently. For a transaction class i , let:

- e_i be execution work;
- d_i be published bytes;
- s_i be state I/O;
- p_i be proving work;
- c_i be consensus or messaging work.

If the system has sustainable capacities E , D , S , P , and C , then a workload with transaction rates x_i must satisfy:

$$\begin{aligned}\sum x_i e_i &\leq E \\ \sum x_i d_i &\leq D \\ \sum x_i s_i &\leq S \\ \sum x_i p_i &\leq P \\ \sum x_i c_i &\leq C\end{aligned}$$

The first tight inequality is the current bottleneck. Optimization moves that bottleneck; it rarely removes all limits. A rollup compression improvement may free DA capacity and expose proving. A faster VM may expose state commitment or networking. This model prevents teams from optimizing the most visible component instead of the limiting one.

4. Benchmark in Layers

Microbenchmarks

Measure signature verification, VM instructions, state reads and writes, hashing, proof generation, proof verification, encoding, and network serialization independently. Microbenchmarks explain why the system behaves as it does, but cannot establish end-to-end capacity.

Component Benchmarks

Run the execution engine against workloads with controlled conflict rates. Run consensus with realistic block payloads across realistic network delay. Run the prover with representative circuits and witness sizes. Run DA retrieval under missing peers and partial withholding.

End-to-End Benchmarks

Submit signed transactions through the same interfaces users will use. Include batching, publication, proof generation, settlement, indexing, and receipt delivery. Report each finality milestone rather than stopping the timer at the first sequencer response.

Failure Benchmarks

Remove the sequencer, prover, batch submitter, consensus leader, bridge relayer, or part of the DA network. Measure time to detect, time to recover, user action required, and cost of the recovery transaction. An escape hatch that has never been load-tested is a hypothesis.

5. Report Distributions, Not One Number

Averages hide queues and failure. Report p50, p90, p95, and p99 latency. Plot offered load against completed throughput and tail latency. The useful capacity is near the point where latency or errors begin to rise sharply, not the largest transient sample observed before collapse.

Run tests long enough to expose database compaction, cache exhaustion, state growth, garbage collection, proof queues, and fee-market changes. Publish warm-up and measurement periods. Repeat experiments and include variance.

Hardware disclosure should include processor, core count, memory, storage device, network link, operating system, client commit, compiler, and configuration. For distributed tests, disclose locations and injected delay or packet loss.

6. Evaluate State Growth

A chain can sustain high execution today while accumulating a state database that makes tomorrow's validator expensive. Measure bytes of new state per transaction, temporary data retention, history retention, snapshot size, synchronization time, and witness size.

Ask who pays for inactive state. Gas charged once may not cover decades of replication. Rent, expiry, stateless witnesses, and archival markets solve different parts of the problem. A plan to prune history is not a plan to serve historical queries.

7. Evaluate Data Availability Separately

Record exactly where transaction data is published and for how long. If blobs or an external DA network are used, describe the commitment, erasure coding, sampling, retrieval, and retention assumptions. If a committee signs attestations, publish its membership, threshold, rotation, and slashing rules.

Test a withholding event. Can an independent node reconstruct the batch? Can an optimistic verifier build a fault proof? Can a validity-rollup user recover the current state? Which service holds historical copies after the consensus retention window?

Avoid the phrase "data is on-chain" unless the chain, data type, and retention rule are named.

8. Evaluate Sequencing and MEV

Document how transactions enter the system, how they are ordered, and whether users can bypass the normal sequencer. Measure censorship time, reordering power, downtime, and recovery after conflicting sequencer views.

If there is a decentralized sequencer set, analyze its own consensus and membership. If sequencing is based on the L1 proposer, explain latency and inclusion. If a shared sequencer orders several rollups, define atomicity and what happens when only one rollup accepts the ordered batch.

MEV analysis should cover front-running, back-running, sandwiching, liquidations, cross-domain timing, and privileged order flow. More throughput does not remove ordering value.

9. Evaluate Proof Systems

For validity systems, record the statement being proven, circuit or VM version, proof system, cryptographic assumptions, setup procedure, verifier contract, prover hardware, proof time, memory, proof size, and verification cost.

Measure worst-case and average proving. A block filled with cryptographically expensive operations may be harder to prove than a transfer block with the same execution gas. Define what happens if proof generation misses its deadline or the only prover fails.

Circuit audits and differential testing matter as much as proof-system soundness. A proof can perfectly establish the wrong program if the circuit does not match intended execution.

For optimistic systems, document the fault-proof game, bond, challenge window, data dependency, game duration, and permission to challenge. Exercise the complete game against an intentionally invalid transition.

10. Evaluate Consensus Under Its Real Model

State whether the protocol assumes synchrony, partial synchrony, or another network model. Record its Byzantine threshold, quorum rule, leader-selection method, timeout behavior, view-change path, signature aggregation, and validator admission.

Benchmark the full block, not an empty proposal. Introduce slow and equivocating leaders. Partition a minority of validators. Test clock skew and delayed certificates. Report safety outcomes separately from liveness degradation.

Validator count should be accompanied by stake distribution, operator independence, client diversity, hosting, geography, and delegation. A thousand keys controlled by ten operators do not create a thousand independent failure domains.

11. Evaluate Bridges as Security-Critical Systems

Inventory every bridge contract, message verifier, relayer, committee, light client, upgrade key, rate limit, and pause mechanism. Follow deposits and withdrawals in both directions. Confirm replay protection, source and destination domain separation, nonce handling, and finality assumptions.

Model the largest credible loss, not only daily volume. A bridge may accumulate years of locked collateral. Rate limits and delayed large withdrawals can bound damage, but emergency controls create governance power that must be secured and disclosed.

Test recovery when one chain reorganizes, stops finalizing, or upgrades its message format. Cross-chain systems inherit the failure modes of both chains plus the bridge.

12. Evaluate Upgrades and Governance

List every address or governance process that can change execution code, proof verification, bridge rules, sequencer membership, fees, or emergency state. Record signature thresholds, signer independence, hardware security, timelocks, monitoring, and user exit periods.

An upgradeable rollout has two protocols: the code users see now and the process that can replace it. The second can dominate the first. A useful maturity report distinguishes immediate emergency powers from delayed routine upgrades and states whether a user can exit under old rules.

13. Make Recovery Affordable

Failure recovery often moves activity back to a scarce base layer. Estimate the cost of force inclusion, mass withdrawal, state reconstruction, and proof submission under congestion. Simulate many users invoking the path together.

A mechanism that is correct but unaffordable does not give ordinary users meaningful sovereignty. Batching recovery claims, rate-limited exits, proof aggregation, and pre-funded watchdogs can improve practicality, but each introduces new coordination rules.

14. Produce a Comparable Result

A professional evaluation report should publish:

1. architecture and recovery diagrams;
2. source code and client commits;
3. workload generator and transaction mix;
4. hardware and network configuration;
5. offered-load versus throughput curves;
6. latency distributions at each finality boundary;
7. state, data, and proof growth;
8. failure-injection results;
9. trust, upgrade, and bridge assumptions;
10. raw results and scripts needed to reproduce the test.

The conclusion should name the operating envelope: the workload and conditions under which the architecture meets its objective. It should also name the first bottleneck and the failure that creates the largest user burden.

Worked Example: Evaluating a Rollup Exchange

Consider a hypothetical validity rollup running a central-limit-order-book exchange. The operator claims 20,000 transactions per second and one-second confirmation. The number alone is not decision-ready.

Workload

Define four transaction classes: limit-order placement, cancellation, market execution, and collateral update. Use at least two account distributions: uniform accounts and a concentrated market where many traders touch the same order book and price levels. Include signature verification, failed orders, and state growth.

Ramp offered load rather than selecting one favorable rate. At each step, run long enough for queues and compaction to stabilize. The sustainable point is the highest rate where input queue, proof queue, and DA publication backlog remain bounded and p99 latency stays within the objective.

Resource budget

For rate λ , estimate:

- execution demand: $\lambda \times \text{CPU time per transaction}$;
- data demand: $\lambda \times \text{compressed bytes per transaction}$;
- proving demand: $\lambda \times \text{proving seconds per transaction-equivalent}$;
- state growth: $\lambda \times \text{net new state bytes per transaction}$;
- settlement demand: $\text{batches per second} \times \text{verification gas per batch}$.

Do not add unlike units into one score. Compare each demand with the capacity of its resource and identify the first utilization ratio approaching one. Leave headroom for variance and recovery.

Suppose the measured mix averages 45 compressed bytes and 0.8 milliseconds of execution per transaction. At 20,000 transactions per second, the rollup generates 900 kB/s of data and 16 CPU-seconds of serial execution each second. The execution target therefore needs at least 16 fully utilized cores before database, scheduling, and operating-system overhead. A production target at 60 percent utilization needs roughly 27 equivalent cores. This is a lower bound, not a hardware promise.

Finality timeline

Label four milestones:

1. the sequencer accepts and orders the transaction;
2. transaction data is published and recoverable;
3. the validity proof is accepted by settlement;

4. the settlement block reaches the application's chosen finality rule.

The advertised one second may describe only milestone 1. A wallet can display it as a provisional confirmation, but a high-value bridge should not represent it as settlement finality. Measure the distribution between every pair of milestones.

Failure tests

Stop the sequencer and submit through forced inclusion. Stop the prover and observe whether unproven batches accumulate safely. Withhold one DA chunk and verify reconstruction or rejection. Reorganize an unfinalized settlement block. Corrupt an upgrade proposal and confirm the timelock and monitor alerts. Exhaust bridge and exit paths under base-layer congestion.

For every test, record detection time, safety outcome, recovery time, operator action, and user action. A test that recovers only after an undocumented administrator intervention reveals an additional trust assumption.

Decision

The evaluation should finish with an operating envelope, not a winner label. For example: the exchange sustains a stated transaction mix at a stated rate on disclosed hardware; provisional confirmation stays below a latency target; proof and publication queues remain bounded; withdrawals reach settlement finality within a measured distribution; and the tested failure paths preserve funds while degrading service for a bounded period.

That conclusion is narrower than "20,000 TPS," but it is useful to an engineer, risk team, and user.

Evaluation Template

Use this short form before adopting or comparing a scaling system:

Question	Evidence
What workload was tested?	Transaction mix, contention, duration
Where does execution happen?	VM, hardware, determinism rule
Where is data published?	Commitment, retention, retrieval
Who orders transactions?	Sequencer/consensus and bypass path
How is state proven?	Fault/validity proof and implementation
What is finality?	Milestones and fault assumptions
How do users exit?	Normal and forced paths, measured cost
What can be upgraded?	Keys, threshold, delay, exit window
What fails under load?	Capacity curves and queues

Question	Evidence
What happens during failure?	Injection results and recovery time

Incident Readiness and Postmortems

A scaling system should be evaluated for diagnosability before an incident. When several layers are involved, the first visible symptom may be far from the failed component. A wallet reports a pending withdrawal, while the underlying cause is a DA retrieval gap, proof queue, settlement reorganization, or bridge verifier pause.

Evidence to retain

Every component should emit durable identifiers that join one user action across layers:

```
user transaction hash
sequencer request and L2 block
batch and data commitment
proof or dispute job
settlement transaction and state root
message and withdrawal nonce
destination execution
protocol, client, and verifier versions
```

Logs need synchronized clocks but must not treat timestamps as consensus truth. Retain signed protocol messages, state roots, commitments, queue transitions, and configuration changes. Protect private transaction content and prover witnesses according to the application's privacy model.

Alert design

Alert on violated objectives, not every transient event. Examples include an oldest-unpublished batch beyond its deadline, proof queue work increasing for several intervals, finality lag beyond the recovery budget, sampling failure above threshold, force-inclusion backlog, or bridge messages past expiry.

Each alert should name the affected boundary, assets or batches at risk, safe automated actions, and operator actions requiring approval. A generic "rollup unhealthy" page forces responders to rediscover the architecture under pressure.

Runbooks

A runbook for each critical dependency states:

- 1. how to confirm the symptom from an independent source;
- 2. which safety invariant must not be violated while restoring liveness;

3. which actions are reversible and which require governance or user notice;
4. how to preserve forensic evidence;
5. how to fail over without processing an item twice;
6. when to stop new activity and when existing users can exit;
7. how to verify recovery end to end.

Do not make "restart everything" the default. Simultaneous restarts can remove consensus quorum, discard useful caches, create proof duplication, or erase the timing evidence needed to understand a split.

Postmortem structure

A useful postmortem includes:

- user-visible impact and exact time interval;
- detection source and delay;
- normal path and recovery path affected;
- timeline using protocol identifiers and versions;
- triggering event, contributing conditions, and latent design weaknesses;
- why safeguards did or did not contain impact;
- manual and automated actions taken;
- verification that funds, state, messages, and queues reconciled;
- corrective actions with owners, tests, and completion evidence.

Avoid attributing a distributed failure to one operator mistake when missing validation, unsafe defaults, poor observability, or shared infrastructure allowed that mistake to become an incident.

Worked Adoption Decision

Assume a team is choosing between a monolithic L1, an L1-settled validity rollup, and an appchain with external DA for a high-value exchange. The exchange expects 1,500 mixed transactions per second, sub-second trading feedback, and withdrawals whose final settlement can take several minutes.

Start with hard requirements:

Requirement	Threshold
Trading acknowledgement	p99 below 800 ms
Canonical withdrawal	p99 below 10 min
Data recovery	independent reconstruction of every accepted batch
Safety	no single operator can create an unbacked withdrawal
Recovery	users can exit without the normal sequencer

Requirement	Threshold
Change control	routine upgrades delayed at least 7 days

Then score evidence, not architecture labels:

Evidence	Monolithic L1	Validity rollup	Appchain + external DA
Mixed workload reaches 1,500 tx/s	measured result required	measured result required	measured result re-quired
Sub-second feed-back	block/preconfirm-ation policy	sequencer pre-confirmation	appchain consensus/pre-confirmation
Withdrawal safety	native state	verifier, bridge, and L1 finality	appchain consensus, bridge, DA, settlement
Forced path	native transaction submission	L1 inbox and es-cape path	chain/bridge-specific re-covery
Independent recon-struction	L1 data/history	rollup data pub-lication	DA proof plus retrieval and archive
Upgrade boundary	protocol gov-ernance	rollup contracts and verifier	appchain, bridge, DA in-tegration

A decision cannot be completed from this table alone. Run the same exchange generator against each candidate, disclose hardware and topology, and inject sequencer or leader loss, proof delay, DA withholding, settlement reorganization, and withdrawal congestion.

Suppose the monolithic L1 reaches only 900 transactions per second at the latency objective. The rollup reaches 2,000 but its proof queue becomes unstable after one prover loss. The appchain reaches 4,000 but its bridge uses an immediate small multisignature upgrade. None yet meets every requirement.

The next action is not to pick the largest number. For the rollup, add prover redundancy and repeat the failure test. For the appchain, require delayed bridge upgrades and an old-version exit. For the L1, decide whether workload partitioning or lower demand is acceptable. Adoption follows the first candidate that supplies evidence for the full operating envelope.

Decision record

The final record names the selected commit and deployment, rejected alternatives, measured workload, finality policy, trust and key inventory, unresolved risks, launch limits, rollback trigger, and date for reassessment. A decision is temporary when protocols, control structures, or workloads change.

Incident Command for Scaling Systems

A scaling incident often crosses sequencer, prover, data availability, settlement, bridge, wallet, and governance teams. The response must preserve evidence and user safety while ownership is still uncertain. Fast recovery does not mean changing state before the failure boundary is understood.

Severity and declaration

Declare an incident when an observed condition threatens a published property: conflicting state, unavailable data, missed proof deadline, stalled forced inbox, incorrect fee or balance, bridge anomaly, prolonged finality loss, key compromise, or unsafe upgrade.

Use severity based on impact, not publicity:

- **SEV-0:** accepted conflicting state, unauthorized asset release, or active key compromise;
- **SEV-1:** safety at material risk, exits unavailable, or a bounded loss event in progress;
- **SEV-2:** major liveness or correctness degradation with safe state preserved;
- **SEV-3:** limited degradation, elevated error rate, or a near miss requiring follow-up.

The declaration records UTC time, detector, affected systems, first bad and last known good boundaries, current user impact, and incident commander.

Roles

Assign one incident commander who coordinates but does not personally execute every change. Separate:

- operations lead for sequencer, prover, DA, and node actions;
- protocol lead for safety and canonical-state analysis;
- bridge and asset lead for custody and supply reconciliation;
- communications lead for user and partner updates;
- evidence scribe for timeline, commands, hashes, and decisions;
- independent safety reviewer who can veto an unsafe recovery.

A role can have backups, but authority must be explicit. An unstructured group chat is not incident command.

First ten minutes

1. confirm the alert through an independent source;
2. freeze automated deploys, parameter changes, and key rotations;

3. identify the last known safe state and current unsafe frontier;
4. preserve logs, databases, proof artifacts, signatures, and provider responses;
5. stop irreversible value release if its verification boundary is uncertain;
6. keep read-only status and user exits available when they do not worsen the incident;
7. open the timeline and assign roles;
8. publish an initial factual status from an authenticated channel.

Do not restart every component at once. Restarts destroy volatile evidence and can make a deterministic fault look intermittent.

Safety before liveness

Choose an action by asking which property it can violate. If proof production stops but prior state remains valid, halt state acceptance before bypassing the verifier. If DA status is uncertain, do not finalize roots that users may be unable to reconstruct. If one bridge claim is suspicious, stop the relevant release path rather than rewriting unrelated chain state.

Pause scope should be narrow enough to preserve safe functions and broad enough to bound loss. Document contracts, assets, message domains, and versions affected. Test that the pause actually blocks the intended transition without blocking the recovery transaction.

Canonical-state worksheet

Record:

```
last known safe L1 block and hash
last safe L2 batch and state root
latest data-available batch
latest accepted proof or dispute result
sequencer unsafe head
bridge messages pending / consumed / disputed
forced-inbox cursor and oldest deadline
active protocol and verifier versions
```

Two teams independently derive the boundary from source data. If they disagree, preserve both hypotheses and halt at the earlier common safe point. Never choose a state root because it is operationally convenient.

Evidence preservation

Use immutable or append-only storage where possible. Hash exported logs and artifacts. Preserve original time zones and monotonic timestamps, soft-

ware versions, environment configuration, API responses, signer transactions, and network captures.

External dashboards can change after the event. Export the underlying data and query, not only a screenshot. Redact secrets and personal data in the review copy while retaining protected originals under access control.

Communication cadence

An initial notice states what is observed, user impact, safe actions, unsafe actions, and the next update time. Separate confirmed facts from hypotheses. Do not estimate restoration until the recovery path has been tested.

Update even when the state has not changed. Name the exact boundary: "withdrawal finalization paused" is more useful than "network issues." When users must act, give verifiable contract, chain, version, and deadline details through authenticated channels.

Keep internal forensic details that would enable active exploitation restricted during containment, but do not use security as a reason to hide user impact or custody status.

Recovery plan

A recovery proposal includes:

- the fault and affected boundary;
- chosen canonical state and evidence;
- exact commands, transactions, artifact hashes, and signers;
- asset and message reconciliation before and after;
- test or simulation result;
- rollback or forward-fix path;
- expected user-visible transitions;
- abort thresholds;
- independent reviewer approval.

Rehearse on a production-sized snapshot or fork. A patch that starts successfully is not enough; prove deposits, forced transactions, proofs, withdrawals, and monitoring across the repaired boundary.

Worked reconciliation

Suppose source escrow contains 1,000 tokens. Destination wrapped supply is 940, with 40 tokens in proven pending withdrawals and 20 in deposits finalized on source but not minted:

```
source escrow 1,000
= wrapped supply 940
```

- + pending withdrawal liability 40
- + pending deposit liability 20

The accounting balances. If wrapped supply is 950 instead, there is an unexplained 10-token excess. Do not resume the bridge until every identifier and owner is reconciled; aggregate totals alone can hide a duplicate claim and an offsetting omission.

Resumption gates

Resume only when:

- the fault is contained or the vulnerable transition is disabled;
- canonical state and data availability agree across independent implementations;
- assets and message identifiers reconcile;
- the repair was reproduced from preserved inputs;
- monitoring detects recurrence at the earliest boundary;
- keys and credentials exposed during response are rotated safely;
- the user exit and recovery paths have been exercised;
- the incident commander and independent reviewer sign the gate record.

Stage resumption. Start read paths and low-risk production, then proving, batching, and value release under caps. Remove temporary caps only after an observation window.

Postmortem

Publish a blameless but accountable timeline: detection, escalation, decisions, mitigations, restoration, and user impact. Identify technical causes, control failures, and why defenses did not detect or contain the issue sooner.

Every corrective action needs an owner, testable completion condition, and deadline. "Improve monitoring" is not an action; "alert when the forced-in-box cursor is within two batches of its deadline and test the alert in staging" is.

Include counterfactual loss: what would have happened without the alert, pause, or cap. Near misses reveal safety margin before users pay for the lesson.

Incident readiness is a scalability property. As throughput and cross-domain value grow, manual reconciliation and improvised recovery become slower than the system's failure rate. A credible design can stop at a known boundary, explain user status, reconcile state, and resume without trusting an administrator's undocumented choice.

Upgrade Operations and User Exit Windows

An upgrade changes the code or parameters that protect user state. Treat it as a protocol migration, not a software deployment. The operational goal is to prove that the proposed transition is authorized, understood, reproducible, reversible where possible, and gives users a meaningful chance to exit when trust assumptions change.

Upgrade manifest

Publish one machine-readable manifest:

```
UpgradeManifest {
  system_id,
  proposal_id,
  old_version,
  new_version,
  source_commit,
  build_digest,
  artifact_hashes[],
  verifier_or_contract_addresses[],
  state_migration_hash,
  activation_condition,
  activation_time_or_height,
  rollback_condition,
  governance_authority,
  user_exit_deadline
}
```

Signatures should authorize this exact manifest. A vote for prose that later resolves to different bytecode, initialization data, or activation height is not reproducible authorization.

Classify the change

Label whether the upgrade changes:

- execution semantics or gas schedule;
- state encoding or commitment scheme;
- proof circuit, verifier, or trusted setup;
- sequencer, validator, or prover membership;
- bridge custody or message verification;
- DA provider or retention assumption;
- fee market, asset, or oracle;
- pause, upgrade, or emergency authority.

A minor API release can still be a major security change if it redirects a proof endpoint or bridge verifier. Classification follows effect, not version number.

Pre-activation gates

Require reproducible builds, independent artifact hashes, differential execution, state migration rehearsal, security review, and rollback testing. Run old and new versions in shadow against the same finalized inputs and compare state roots, receipts, logs, fees, and messages.

Expected differences need machine-readable exceptions tied to test cases. "Roots differ because of the upgrade" is not an explanation. Enumerate which transactions or state fields change and prove that all other behavior remains equal.

For a verifier upgrade, generate valid and invalid proof vectors across both versions. Test old proofs pending at activation, new proofs submitted early, recursive proofs embedding the old verifier, and malformed version tags.

Timelock and exit window

A timelock is useful only if users can understand the change and complete an exit before activation. The window must include monitoring delay, challenge or proof time, settlement finality, bridge withdrawal, and congestion under simultaneous exits.

Suppose monitoring may take 24 hours, an optimistic withdrawal needs 7 days, settlement policy adds 20 minutes, and a mass-exit capacity model requires 2 days. A 48-hour upgrade delay is not a meaningful exit window. A conservative bound is at least:

$$24 \text{ h} + 7 \text{ d} + 20 \text{ min} + 2 \text{ d} \approx 10 \text{ days and } 20 \text{ minutes}$$

Add operational margin. If emergency upgrades can bypass the delay, disclose exactly who can invoke that path and which loss it is intended to contain.

State migration

Bind the pre-state root, migration program hash, parameters, and expected post-state root. Run the migration on a production-sized snapshot with constrained hardware. Measure wall time, peak memory, disk amplification, and downtime.

Make migration idempotent or persist phases so a restart cannot apply transformations twice. Validate supply, ownership, nonces, permissions, pending messages, and consumed replay identifiers before activation.

If rollback is possible after new transactions execute, define how those transactions are replayed or compensated. Database rollback alone can duplicate external messages or withdrawals. Often the safe recovery is a forward fix from a frozen state, not a silent binary downgrade.

Activation

Use an objective height, finalized timestamp, or governance state visible to every component. Confirm clock and chain assumptions. Freeze configuration changes that could alter the manifest near activation.

At activation, monitor version adoption, state-root agreement, block production, proof acceptance, bridge queues, forced messages, fee behavior, and user errors. Define abort thresholds before the event; do not invent them while under pressure.

An activation runbook assigns named roles for command authority, observation, communications, bridge/prover/sequencer operations, and independent safety review. One person should not both execute and attest that all checks passed.

Rollback and halt matrix

Condition	Safe response	Forbidden shortcut
artifact hash mismatch	stop activation	rebuild unreviewed binary live
state roots diverge in shadow	investigate and delay	label difference expected without vector
migration stops before commit	resume from verified phase or restore snapshot	rerun blindly over partial state
new verifier rejects valid proofs	halt unsafe frontier; preserve old accepted state	accept proofs off-chain by operator judgment
bridge messages diverge	pause release and reconcile identifiers	replay all pending messages
liveness degrades but safety holds	use bounded rollback/forward-fix rule	weaken validation to regain throughput
conflicting accepted state appears	halt finalization and preserve evidence	choose a root based on convenience

Governance key operations

Inventory every key and threshold that can schedule, cancel, accelerate, pause, or execute an upgrade. Verify device access, signer identity, transaction simulation, nonce, chain ID, and destination contract. Conduct a dry run with the same signer path.

A multisignature threshold is not independent if signers share custody software or one administrator can reset all accounts. Exercise loss of one signer and loss of the largest correlated signer group.

After activation, revoke obsolete roles, rotate temporary keys, and verify old implementation contracts cannot be reinitialized or called through an alternate proxy path.

User communication

Publish plain-language and technical notices from authenticated channels. State the exact effect, trust changes, hashes, activation boundary, exit deadline, known limitations, and where status updates will appear.

Wallets and interfaces should surface pending upgrades that affect custody or verification. Avoid claiming "no action required" when a user must accept a new governance, DA, bridge, or proof assumption by remaining in the system.

Post-activation proof

Record actual activation height, transaction, code hashes, post-migration root, signer set, tests, incidents, and any deviation from the manifest. Reconcile assets and messages. Keep the old and new artifacts plus migration inputs long enough for independent review.

An upgrade is complete only after the observation window passes, queues normalize, proofs and exits work, obsolete authority is removed, and the evidence package lets an outsider reproduce what changed.

Conclusion

Scalability engineering begins after the headline number. A defensible evaluation starts with workload, traces the transaction and recovery paths, models each resource, tests the real stack under failure, and publishes enough detail for another team to reproduce the result.

This method does not produce one universal winner. It produces an operating envelope and a visible set of trade-offs. That is the information builders need to choose an architecture responsibly.

Figure Credits

Figures are part of the technical argument, not decoration. Captions identify the source and the concept the figure supports.

Original Figures for This Book

- **Figure 1.1, Four functions behind blockchain scalability.** Original diagram created for this edition.
- **Figure 2.1, Measurable trilemma trade-offs.** Original diagram created for this edition.
- **Figure 3.1, Layer 1 and Layer 2 architecture.** Original diagram created for this edition.
- **Figure 4.1, Cross-shard receipt lifecycle.** Original diagram created for this edition based on SC6019 Lecture 02.
- **Figure 5.1, State-channel lifecycle and dispute path.** Original diagram created for this edition based on SC6019 Lecture 03.
- **Figure 6.1, Rollup lifecycle and completion boundaries.** Original diagram created for this edition.
- **Figure 6.2, Named optimistic and validity rollup traces.** Original diagram created for this edition.
- **Figure 7.1, Modular transaction and message boundaries.** Original diagram created for this edition.
- **Figure 8.1, Data availability sampling workflow.** Original diagram created for this edition based on Celestia's design.
- **Figure 9.1, Parallel scheduling and deterministic commitment.** Original diagram created for this edition based on Block-STM and SC6019 Lecture 05.
- **Figure 10.1, Leader-based BFT normal path.** Original diagram created for this edition based on the HotStuff protocol paper.
- **Figure 11.1, Shared rollup and appchain trade-off.** Original diagram created for this edition based on the architecture discussion in SC6019 Lecture 06.

Rights Checklist

Every figure in the manuscript now uses original artwork stored as editable SVG. Before a release, verify that each referenced figure exists, matches its caption, and remains covered by the book's license. A concept derived from a

paper or lecture should cite that source even when the artwork is original. Generated build files and superseded source images should not ship as part of the rights inventory.

Threat-Model Worksheets

A scalability design distributes work across more roles, queues, contracts, and networks. The threat model should follow those interfaces. This appendix provides reusable worksheets for design reviews and incident exercises.

Start With Assets and Outcomes

List what the system protects. Examples include base-layer escrow, L2 balances, ordering fairness, confidential transaction content, data needed for exit, governance authority, proof-signing keys, and the ability to make progress before a deadline.

For each asset, name an unacceptable outcome:

Asset	Unacceptable outcome	Detection	Recovery
Bridge escrow	unbacked withdrawal	supply and message reconciliation	pause, contain, and repair under published process
Rollup state	invalid root accepted	independent execution or proof checks	challenge or reject before finality
Transaction data	accepted state cannot be reconstructed	retrieval and sampling failure	reject/halt; repair from independent peers
Channel balance	old state settles	breach monitor sees revoked commitment	submit newer state or penalty before deadline
Ordering	censor or reorder beyond policy	compare mempool/inbox and block evidence	forced inclusion, alternate sequencer, penalty
Upgrade authority	unauthorized or immediate rule change	key and timelock monitoring	cancel, exit, rotate under governance rules

Do not label every outcome "fund loss." Liveness loss, indefinite lock, privacy disclosure, unfair ordering, and unaffordable recovery are distinct harms with different mitigations.

Role Worksheet

Create one row for every actor and automated service:

Role	Inputs observed	Action controlled	Can violate safety?	Can halt progress?	Replacement path
Sequencer	user transactions, order flow	inclusion and ordering	only if proof/contract permits invalid state	yes	L1 inbox or another sequencer
Prover	witness and execution trace	proof delivery	cannot forge under sound verifier	yes, by withholding	redundant permissionless prover
DA operator	batch shares	storage and delivery	may break recovery assumptions	yes	reconstruction, repair, alternate DA policy
Relayer	source events and proofs	message transport	no if destination fully verifies	yes if exclusive	permissionless replacement
Upgrade signer	proposed code and parameters	verifier/bridge changes	often yes	often yes	timelock, cancellation, user exit

A role that cannot forge state may still create a severe user loss by blocking a time-sensitive exit. Evaluate safety, liveness, privacy, and ordering separately.

Trust Boundary Worksheet

For each message crossing a boundary, record:

```

producer and verifier
source and destination domains
canonical encoding and version
freshness: nonce, height, epoch, or expiry
commitment and inclusion proof
source finality rule
data availability requirement
replay and idempotence key
timeout and recovery
upgrade authority for both ends

```

Then mutate one field at a time. Try a valid message from the wrong chain, an old epoch, a duplicate nonce, a proof against a reorganized root, an unknown version, an expired promise, and a payload whose bytes do not match the committed hash.

A verifier should reject for a specific reason. "Invalid proof" is too broad for incident response when the real problem is an unavailable header or unsupported version.

Adversary Worksheet

Describe capabilities rather than using only labels such as honest or malicious:

- controls less than, equal to, or more than a consensus threshold;
- delays, drops, reorders, duplicates, or selectively reveals network messages;
- corrupts participants before execution or adaptively after observing state;
- compromises one key, a signing quorum, a client implementation, or a cloud region;
- submits valid but expensive transactions to exhaust proving, storage, or retries;
- observes private order flow and trades before public inclusion;
- withholds data while answering selected samplers;
- exploits upgrade, pause, or recovery controls;
- causes correlated failures through shared libraries, hardware, RPC, or time sources.

For every capability, state which property still holds and which can fail. If safety requires fewer than one-third Byzantine weight and liveness requires eventual synchrony, write both conditions.

Economic Worksheet

Security penalties must exceed plausible gain and remain enforceable. Record:

```
value at risk
maximum gain from one violation
collateral available for penalty
time until collateral can exit
who proves the violation
who executes the penalty
correlated violator set
cost imposed on honest users during defense
```

A \$1 million bond does not secure \$100 million of extractable value when the violator can withdraw the bond before evidence finalizes. A challenge reward does not create an honest challenger when proof generation costs more than the reward or access is permissioned.

Rate limits bound loss per time but can extend honest withdrawals. Emergency pauses contain damage but place power in pause keys. Show both sides of every control.

Availability and Deadline Worksheet

Time-sensitive protocols require explicit clocks and margins:

Deadline	Starts at	Evidence of expiry	Congestion assumption	Miss consequence
Channel dispute	commitment inclusion	finalized block height	remedy fits before window closes	outdated state settles
Forced inclusion	L1 inbox acceptance	block/time rule	L1 has capacity under mass use	censorship continues
Fault proof	state proposal	settlement rule	challenger can retrieve data and submit	invalid state finalizes
Cross-domain refund	escrow finality	source timestamp/height	claim/refund ordering defined	double claim or long lock
Preconfirmation	signed promise	target slot/finality evidence	signer and evidence remain available	penalty or broken promise

Test deadline paths with fee spikes, reorganization, clock skew, delayed evidence, and many users acting at once. Average inclusion time is not a safe deadline margin.

Upgrade Worksheet

Inventory everything an upgrade can change: VM rules, circuit and verifier, bridge decoder, DA network, sequencer set, fee accounting, message domains, pause behavior, and escape paths.

For each change, record proposer, approver threshold, timelock, cancellation, code hash, audit evidence, activation boundary, in-flight message handling, state migration, old-version exit, and rollback policy.

An upgrade that changes a message format needs compatibility rules for already-open channels, escrows, deposits, and withdrawals. An escape hatch controlled by the same immediate key as the main verifier is not independent protection against that key.

Test-to-Claim Matrix

Connect every external claim to evidence:

Claim	Required test	Passing evidence
Invalid state cannot finalize	generate invalid roots/proofs and exercise dispute	verifier rejection or successful challenge before deadline
Sequencer cannot permanently censor	stop sequencer and use forced path	transaction or exit completes within stated bound

Claim	Required test	Passing evidence
Data is available	shut publisher and selected peers	independent reconstruction from authenticated commitment
Messages execute once	duplicate and reorder valid proofs	one state transition and stable consumed identifier
Validator-set transition is safe	overlap epoch change with timeout/restart	no conflicting commit; light client authenticates new set
Proving keeps up	replay measured job distribution with worker loss	bounded queue and recovery within service-level objective (SLO)
Users can mass exit	invoke recovery at modeled scale under congestion	required population exits within window and budget

If a test demonstrates only the normal path, narrow the claim. A successful proof verification does not demonstrate prover availability. A successful upload does not demonstrate data reconstruction. One user's exit does not demonstrate mass-exit capacity.

Review Output

A completed threat model should produce:

1. architecture and trust-boundary diagrams;
2. asset/outcome and role matrices;
3. adversary and timing assumptions;
4. protocol invariants and domain-separated message formats;
5. economic and key-control analysis;
6. test-to-claim matrix with reproducible evidence;
7. residual risks, launch limits, and user-visible statuses;
8. incident runbooks and reassessment triggers.

Threat modeling is not complete when every risk has a mitigation. It is complete enough to act when assumptions, remaining exposure, detection, recovery, and ownership are explicit and testable.

Benchmark Reporting Template

Use this appendix to publish a blockchain scalability result another team can reproduce. Replace every prompt; do not leave a field blank without explaining why it does not apply.

1. Claim

State one falsifiable result:

System ____ at commit/version ____ sustained ____ completed operations per second for ____ minutes under workload ____ while p99 ____ latency remained below ____, with ____ validators/operators and the following injected faults: ____.

Name the completion boundary: sequencer acceptance, block inclusion, execution, certification, proof acceptance, settlement finality, destination execution, or another precisely defined event.

2. System Under Test

Field	Value
Repository and commit	
Protocol/config version	
VM and compiler	
State/database backend	
Consensus implementation	
Proof/fault system	
DA path and retention	
Bridge/messaging version	
Build flags and dependencies	
Date tested	

Attach configuration files and a machine-readable dependency lock. Identify local patches not present in the named commit.

3. Topology and Control

Role	Count	Independent operators	Regions/providers	Client diversity
Validators				
Sequencers				
Provers/challengers				
DA nodes/retrievers				
Relayers				
RPC/indexing nodes				

List upgrade, pause, bridge, and emergency keys separately. A process count is not an operator count.

4. Hardware and Network

For every machine class, report:

```
CPU model, sockets, physical/logical cores
memory capacity and speed
storage device, filesystem, capacity, and measured IOPS/bandwidth
network interface and measured bandwidth
operating system and kernel
container or virtual-machine limits
power or accelerator settings
```

Report network shaping between regions: round-trip latency distribution, bandwidth cap, packet loss, jitter, and clock synchronization. State whether nodes share one host, switch, cloud account, or availability zone.

5. Initial State

Property	Value
Accounts/objects	
Contracts	
Live state bytes	
History bytes	
Working-set bytes	
Snapshot/sync source	
Cache warm-up rule	

Publish a state generator or snapshot commitment. A short in-memory test does not represent a chain whose working set exceeds memory.

6. Workload

Transaction class	Share	Input bytes	Reads	Writes	Execution cost	Success target
Transfer						
Contract call						
Deployment						
Cross-domain message						
Other						

Describe key popularity, account skew, contract contention, value distributions, bursts, invalid transactions, and retries. Publish generator code, seed, and transaction templates.

7. Offered-Load Schedule

Do not run only at one selected rate. Record a sweep:

Phase	Duration	Offered load	Purpose
Warm-up			fill caches and queues
Low load			baseline
Ramp steps			locate capacity knee
Sustained target			validate service-level objective (SLO)
Overload			observe admission/backpressure
Recovery			drain queues and reconcile

State whether rejected client submissions remain in offered load and how open-loop versus closed-loop generation affects pressure.

8. Metrics

Report time series and summary distributions for:

- submitted, accepted, included, executed, successful, proved, and finalized operations;
- p50, p90, p95, p99, and maximum latency at each boundary;
- mempool, batch, publication, proof, message, and withdrawal queue depth and age;
- CPU, memory, storage, network, and accelerator utilization;

- block/batch size, gas or compute units, data bytes, proof time, and state growth;
- leader changes, reorgs, retries, aborted speculative work, and errors;
- fees paid by users and subsidies paid by operators or treasury.

Define aggregation windows and missing-sample handling. Retain raw results.

9. Fault Schedule

Time	Fault	Scope	Duration	Expected invariant
	leader or sequencer loss			no conflicting finality
	network delay/partition			safety preserved
	prover/challenger loss			accepted state remains valid
	DA withholding/retrieval loss			unavailable data not accepted
	database restart			persisted state and nonces survive
	relayer loss/replay			eventual delivery and execute-once
	key/upgrade test			delay, alert, cancellation/exit

Record the actual observed start and end, not only the test script's requested times.

10. Results Table

Offered load	Completed throughput	p50	p95	p99	Error rate	Oldest queue	Limiting resource
--------------	----------------------	-----	-----	-----	------------	--------------	-------------------

Plot offered load against completed throughput, p99 latency, error rate, and queue age. The sustainable capacity is the highest region where objectives remain satisfied and queues do not trend upward.

11. Resource Envelope

For each resource r , compute:

```
utilization_r = demand_r / capacity_r
```

Document how capacity was measured and which headroom policy applies. Identify the first tight constraint under typical, burst, and recovery workloads. Do not sum unrelated utilization percentages.

12. Finality and User Journey

Follow at least one transaction and one withdrawal through all identifiers and timestamps:

```
client submission
sequencer or mempool receipt
block/batch inclusion
canonical data publication
proof/challenge state
settlement inclusion and finality
message relay
withdrawal execution
```

Show what the user interface reported at each boundary and whether retrying was safe.

13. Recovery Results

For every fault, publish:

Fault	Detection time	User impact	Safety result	Recovery time	Manual action	Residual backlog
-------	----------------	-------------	---------------	---------------	---------------	------------------

Verify final state, supply, consumed messages, pending withdrawals, and proof/data queues after recovery. Restarting services is not recovery until end-to-end state reconciles.

14. Reproduction Package

A release artifact should contain:

```
README with exact commands
source commits and dependency lock
configuration and topology
state generator or snapshot commitment
workload generator and seed
fault-injection scripts
raw metrics and logs
analysis notebook or scripts
plots and final report
checksums for every artifact
```

Remove secrets and personal data without removing evidence needed to reproduce the result. Describe redactions.

15. Limitations and Conclusion

State which environments, workloads, and adversaries were not tested. Separate measured, simulated, and projected values. Name the operating envelope, first bottleneck, recovery cost, trust/control assumptions, and next experiment.

A useful conclusion is narrow:

Under configuration __ and workload __, the system sustained __ at p99 __ through fault __; performance became unstable when __ queue/resource saturated. This result does not establish __ and should be reassessed after __.

The template is complete only when a skeptical team can rerun the test and reach the same interpretation, not merely the same headline number.

Index

A

account abstraction 273
 Address 49, 62, 124, 281, **303**, 332
 Admission control 31, 147, 241, 278, **310**
 Application chain (appchain) **305**
 Aptos 226, 249
 Arbitrum 23, 124
 Avail 206

B

Backpressure 79, **310**, 357
 Bitcoin 40, 54, 104, 313
 Blob 23, 57, 75, 122, 178, 205, 272, **307**
 Blobstream 178, 207
 Block 19, 37, 53, 67, 101, 123, 173, 201, 223, 244
 Block header 68, 201, **303**
 Block-STM 225, 273, **312**, 347
 BoLD 25, 124
 Bridge 23, 38, 54, 72, 102, 121, 174, 209, 252, 270
 Byzantine fault 55, 243, **312**

C

Canonical bridge 26, 60, 121, 180, 290, **306**
 Canonical chain 163, **303**
 Capacity 19, 37, 53, 67, 102, 139, 175, 206, 233, 264
 Capacity knee 23, **310**, 357
 Celestia 177, 205, 317, 347
 Censorship resistance 30, 40, 109, 126, 271, **309**
 Challenge period 122, 284, **306**, 322
 Channel 23, 53, 73, 101, 128, 174, **306**, 315, 339, 349
 CometBFT 252
 Commit-chain 105, **306**
 Commitment 20, 43, 57, 73, 101, 121, 174, 201, 224, 245
 Committee 38, 54, 68, 105, 134, 187, 206, 228, 248, 270
 Composability 23, 59, 71, 175, 228, 271, **305**
 Consensus 13, 19, 38, 53, 67, 102, 134, 173, 203, 224

Cosmos 71

Critical path 81, 143, 241, **311**

Cryptographic hash 19, **303**

D

danksharding 75, 272

Data availability (DA) 17, 201, **308**

Data availability committee (DAC) **308**

data availability sampling 22, 71, 202, 308, 347

Data availability sampling (DAS) 202, **308**

Digital signature 19, **303**, 315

Domain separation 151, 174, 216, 281, **309**, 331

E

EigenDA 206

EIP-4844 57, 75, 124, 205, 272, 308

EIP-7594 75, 206

Erasure coding 79, 202, **308**

ERC-4337 281

Escape hatch 123, **309**, 317, 329, 352

Ethereum 15, 20, 40, 54, 75, 105, 124, 173, 205, 225

Execution 13, 19, 38, 53, 67, 102, 121, 173, 205, 223

F

Fast path 13, 25, 124, 178, 226, 247, 298, **313**

Fault proof or fraud proof 306

Finality 17, 20, 59, 109, 176, 244, **303**, 319, 333, 359

Force inclusion 57, 131, 183, 277, **306**, 332

Forced inclusion 63, 91, 123, 197, **309**, 319, 334, 349

fraud proof 132, 212, 306

Full node 68, 213, 229, 250, 304

G

Gasper 43

H

Headroom 31, 161, 264, **311**, 319, 333, 358

Hot state 224, **313**

HotStuff 245, 318, 347

I

Idempotence **311**

K

KZG commitment **308**

L

L2BEAT 25

Layer 1 (L1) 17, 53, **305**

Layer 2 (L2) 17, 53, **305**

Light client 73, 113, 183, 202, 290, **309**, 331, 353

Lightning Network 104

Liveness 13, 21, 39, 75, 107, 124, 178, 208, 243, 281

M

Merkle proof 20, 88, 106, 170, 189, **304**

MEV 48, 148, 277, **313**, 330

MEV-Boost 282, 318

Modular blockchain 173, **305**

Monolithic blockchain 173, **305**

Mysticeti 226, 250, 313, 323

N

Nakamoto consensus 246, **313**

Narwhal 250

Nightshade 71

Node 21, 41, 53, 68, 117, 126, 178, 204, 229, 250

O

Offered load 17, 24, 40, **311**, 320, 329, 357

OP Stack 60, 130, 177

Operating envelope 26, 39, **311**, 332, 360

Optimistic rollup 107, **306**

P

p50, p95, p99 latency **311**

Pacemaker 244, **313**

Parallel execution 13, 22, 40, 76, 223, **313**, 318

PeerDAS 75, 205, 272, **308**

Plasma 13, 56, 101, **307**

Polygon 105

Preconfirmation 149, 270, **309**, 337, 352

Private key and public key **304**

Prover market 143, 285, **313**

Q

Queue stability **311**

Quorum certificate (QC) 244, **313**

Quorum intersection 250, **313**

R

Recovery point objective (RPO) **311**, 325

Recovery time objective (RTO) **311**, 325

Relayer 13, 73, 163, 174, 283, **309**, 329, 350, 358

Remote procedure call (RPC) 24, **310**

Reorganization (reorg) **309**

Rollup 23, 38, 53, 102, 121, 173, 203, 228, 270, **307**

S

Safety 13, 21, 38, 59, 68, 102, 127, 174, 209, 239

Saturation 235, **312**, 320

Scroll 132

Sequencer 26, 56, 124, 183, 301, **307**, 334, 350

Service-level objective (SLO) **312**, 325, 353, 357

Settlement 13, 22, 40, 53, 69, 101, 121, 173, 206, 228

Sharding 27, 54, 70, 228, **314**, 317

shared sequencing 148, 271, 318

Sidechain 29, 53, 102, **307**, 315

Smart contract 20, **304**

SNARK 133, **308**

Social recovery 183, 275, **310**

Solana 42, 54, 226

STARK 133, **308**

Starknet 131

State 13, 19, 39, 53, 67, 101, 121, 173, 201, 223

State channel 108, **307**

State root 14, 68, 105, 121, 174, 206, 227, 293, **306**, 315

Sui 226, 250, 313, 318

Sync HotStuff 247, 318

Synchronous, partially synchronous, asynchronous **314**

T

Tail latency 17, 28, 40, 237, **312**, 329

Throughput 13, 27, 38, 54, 67, 113, 225, 245, 269, **312**

Timelock 101, 160, 198, **310**, 334, 349

Transaction 13, 19, 41, 56, 67, 101, 121, 173, 203, 223

U

Utilization 22, 98, 134, 236, **312**, 319, 333, 357

V

Validator 13, 19, 39, 54, 67, 105, 126, 173, 206, 224

Validity proof 171, **308**

Validity rollup 57, 107, 121, **307**, 317, 333, 347

Validium 60, 134, 181, **307**

View change 150, 251, **314**, 320

Volition 134, **307**

W

Wallet 20, 72, 115, 123, 186, 229, 269, **304**, 315, 325

Watchtower 117, **307**

Weak subjectivity 186, 257, **310**

Witness 68, 122, 183, 234, 288, **308**, 323, 329, 350

Workload mix **312**

Z

Zero knowledge 133, **309**

zero-knowledge proof 122

ZK rollup 307

ZKsync 127